

Sri Lanka Institute of Information Technology



BSc (Hons) in Information Technology Specialized in Software Engineering

SE4040 - Enterprise Applications Development Year 4 Semester 2 2026 Assignment 1

Student Name	Registration Number	Email
Perera W.A.N.I	IT22061348	it22061348@my.sliit.lk
Weerasinghe D.I	IT22200556	it22200556@my.sliit.lk
Wahalathanthri H.C	IT22205674	it22205674@my.sliit.lk
Gunasena R.K.R.M.S.K	IT22125798	it22125798@my.sliit.lk

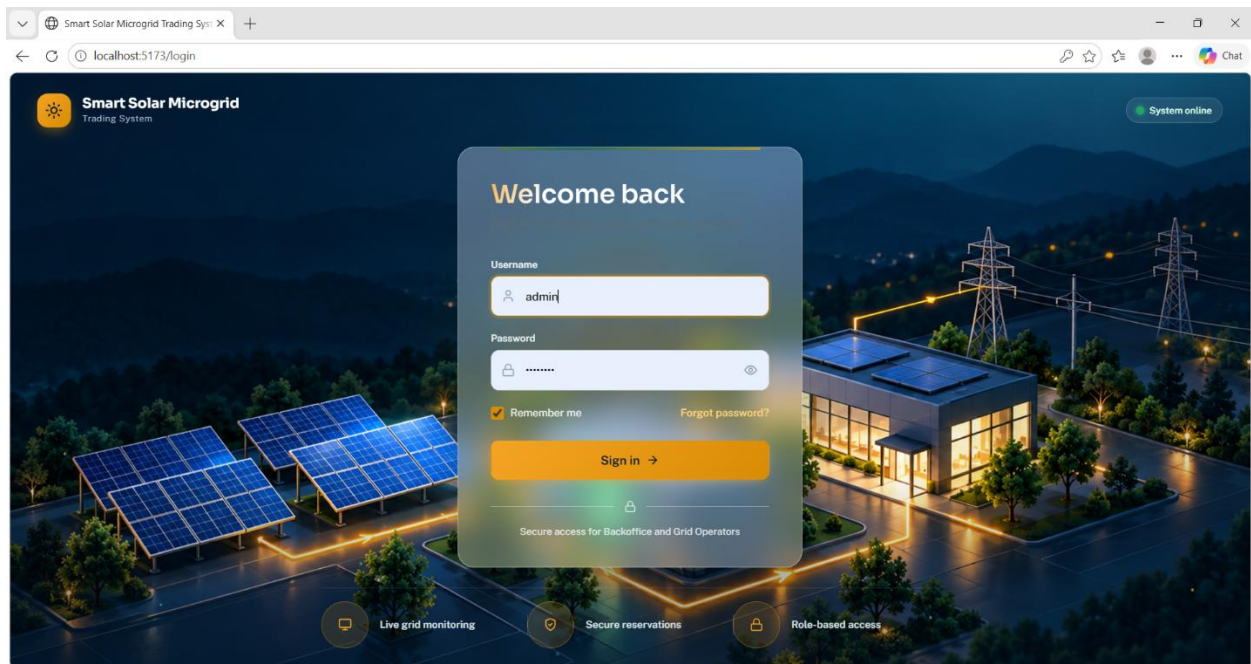
Contents

1 Screen shots of all UI's	3
1.1 Web Application UI's	3
1.1.1 Login and Role-Based Access	3
1.1.2 Backoffice Dashboard.....	3
1.1.3 Prosumer Management	4
1.1.4 Microgrid nodes Management	5
1.1.5 User Management	6
1.1.6 Grid Operator Dashboard.....	7
1.1.7 Grid Operator Reservation Management	8
1.1.8 Grid Operator Prosumers Management	9
1.1.9 Grid Operator Microgrid nodes Management.....	9
1.2 Mobile Application User Interface	10
1.2.1 Login and Authentication	10
1.2.2 Home Dashboard	11
1.2.3 Station Discovery and Search	12
1.2.4 Station Details and Availability	Error! Bookmark not defined.
1.2.5 Energy Slot Selection and Reservation.....	13
1.2.6 Booking Management and Status Tracking.....	14
1.2.7 Booking Details and QR Verification.....	15
2 Architecture and System Diagrams	16
2.1 Application high level diagram.....	16
2.2 Use case Diagram	17
2.3 DFD diagram	18
.....	18
2.4 Database design	19
3 Source code.....	20
4 All the references	204
5 Git Repository link.....	205
6 Individual Contributions	205
7 Challenges.....	206

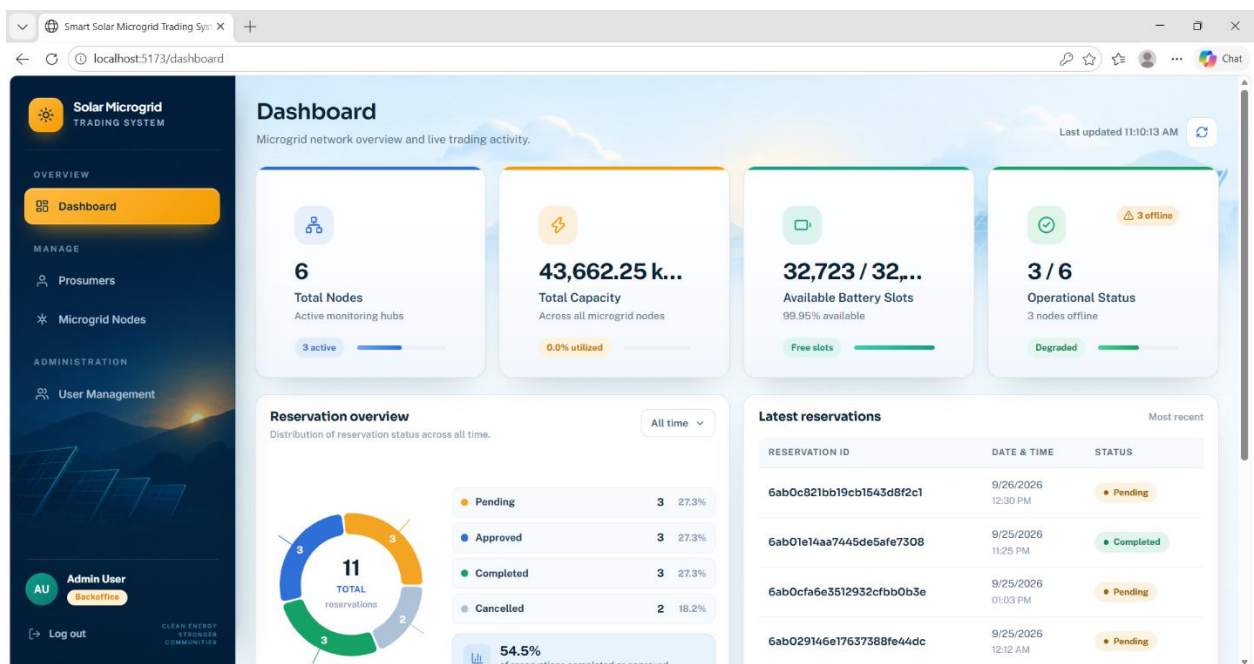
1 Screen shots of all UI's

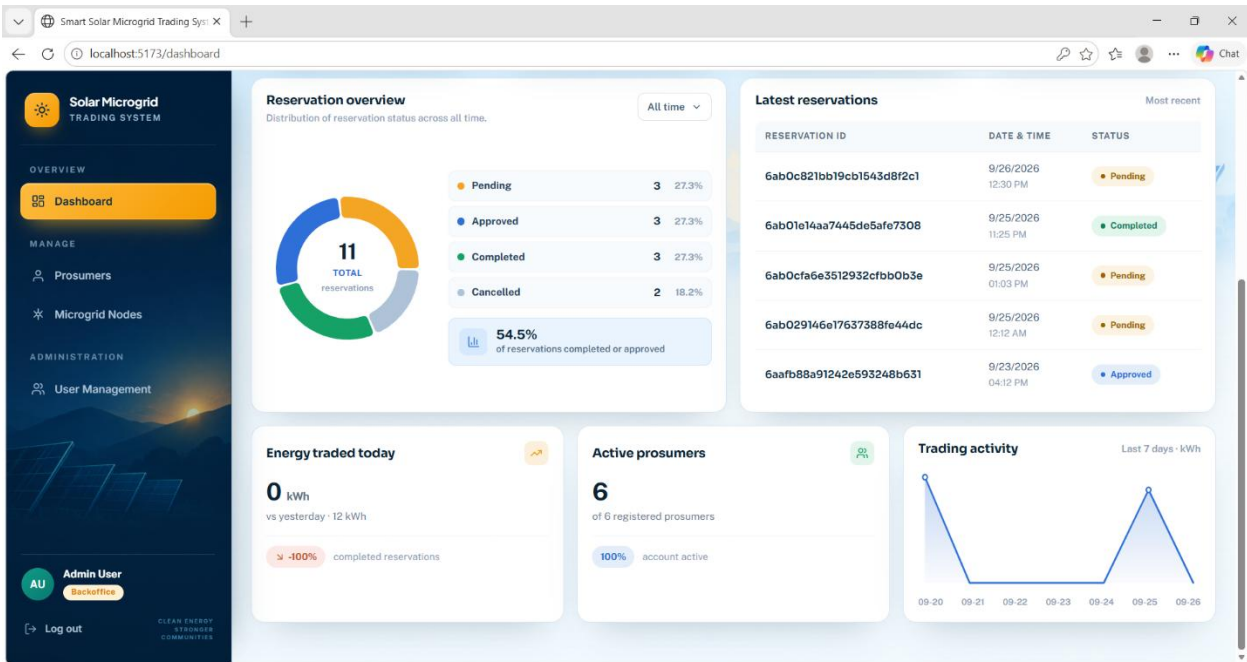
1.1 Web Application UI's

1.1.1 Login and Role-Based Access

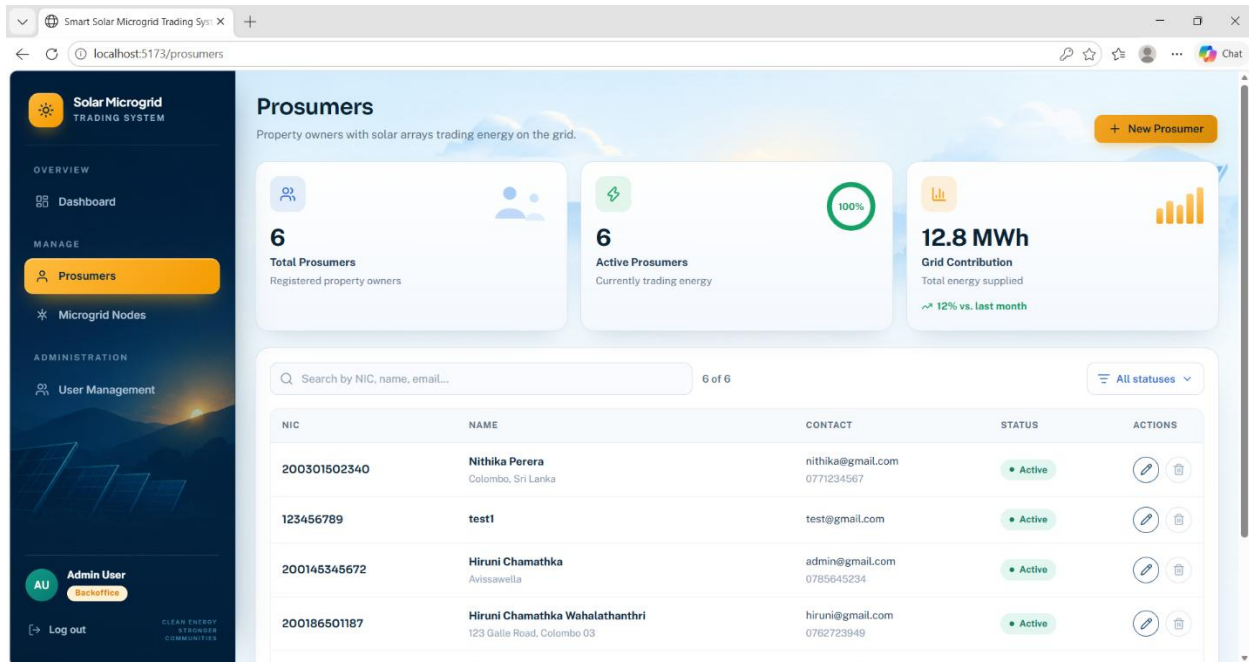


1.1.2 Backoffice Dashboard





1.1.3 Prosumer Management



Register new prosumer

The password lets the prosumer sign in to the mobile app.

NIC (primary key)

Full name

Email

Phone

Address

Mobile app password

[Cancel](#) [Create prosumer](#)

1.1.4 Microgrid nodes Management

Microgrid Nodes

Solar grid hubs — location, capacity and battery storage slots.

[+ Register Node](#)

6 Total Nodes
Active monitoring hubs

43,662.25 kWh Total Capacity
Across all nodes

32,723 / 32,738 Available Battery Slots
100% available

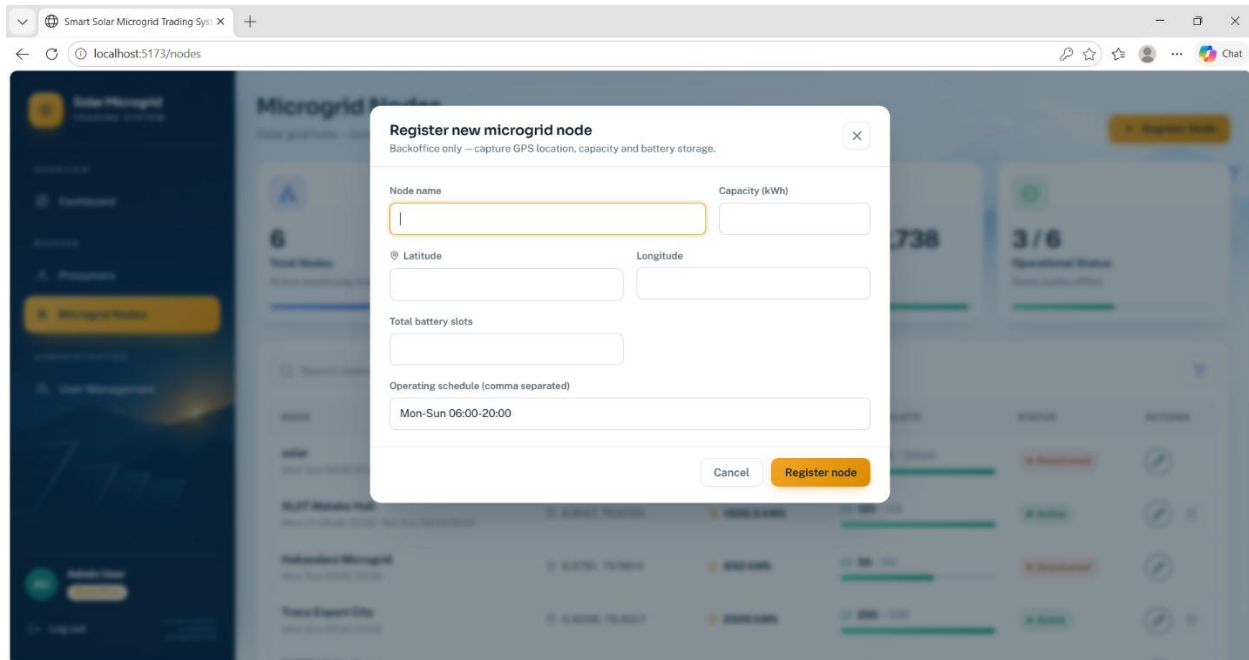
3 / 6 Operational Status
Some nodes offline

Search nodes... 6 of 6

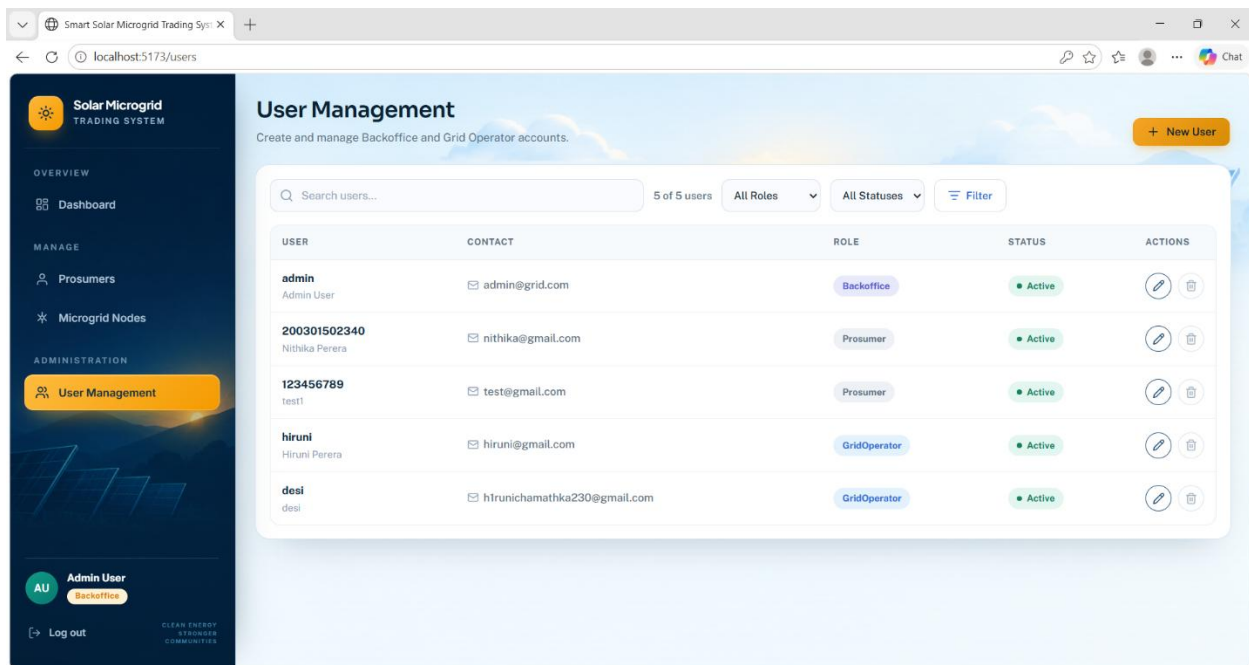
NODE	LOCATION	CAPACITY	BATTERY SLOTS	STATUS	ACTIONS
solar Mon-Sun 06:00-20:00	45.0000, 23.0000	34569 kWh	32000 / 32000	Deactivated	Edit
SLIIT Malabe Hub Mon-Fri 06:00-22:00 Sat-Sun 08:00-18:00	6.9147, 79.9733	1500.5 kWh	120 / 110	Active	Edit Delete
Hokandara Microgrid Mon-Sun 00:00-23:59	6.8781, 79.9614	850 kWh	30 / 50	Deactivated	Edit
Trace Expert City Mon-Sun 05:00-23:00	6.9298, 79.8617	2500 kWh	200 / 200	Active	Edit Delete

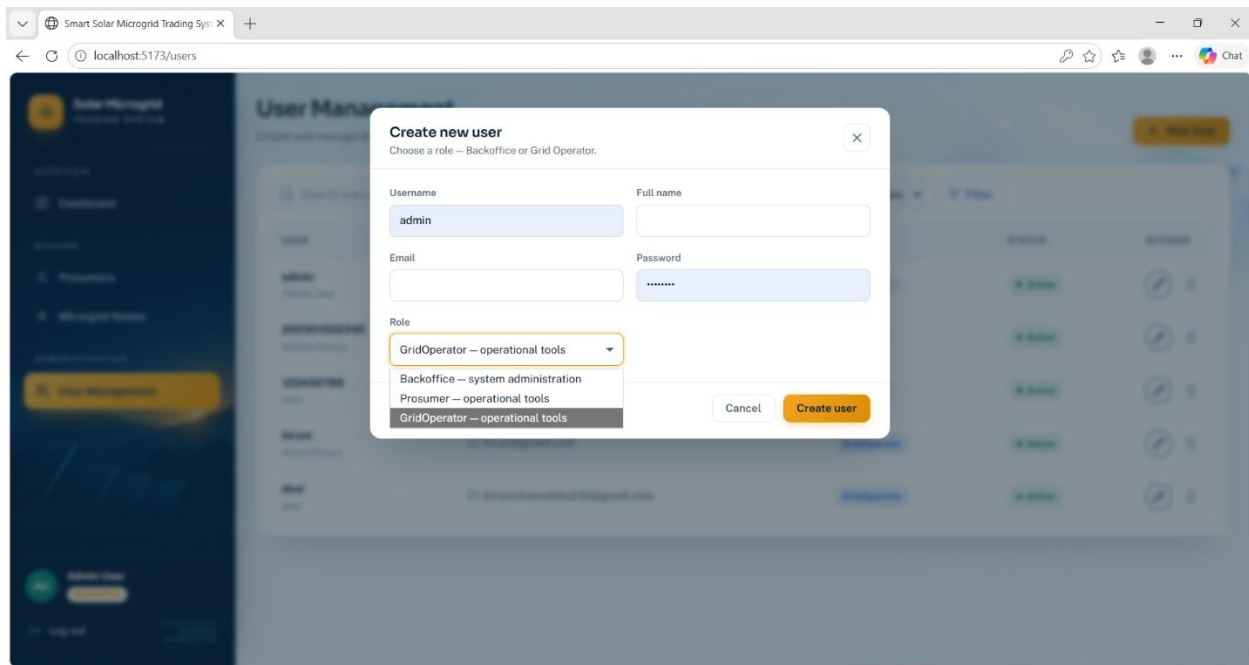
Admin User
Backoffice

[Log out](#)

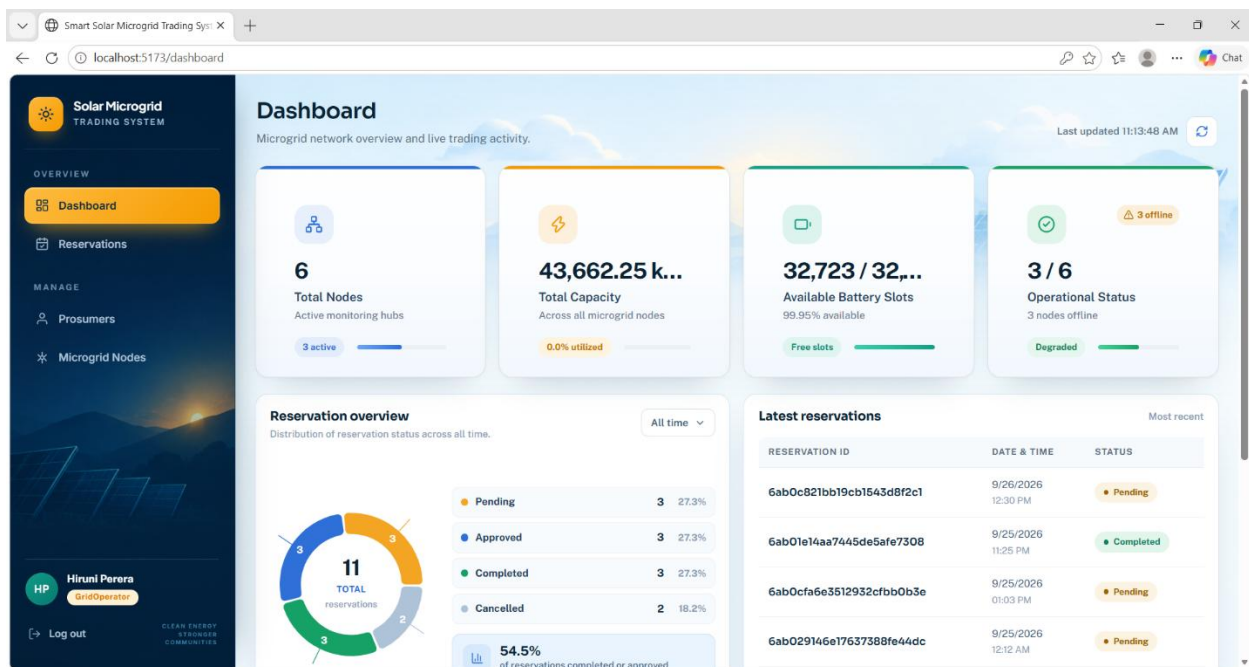


1.1.5 User Management





1.1.6 Grid Operator Dashboard



1.1.7 Grid Operator Reservation Management

Solar Microgrid TRADING SYSTEM

OVERVIEW

- Dashboard
- Reservations**

MANAGE

- Prosumers
- Microgrid Nodes

HP **Hiruni Perera** GridOperator Log out

Reservations

Power trading bookings across all microgrid nodes.

+ New Reservation

All 11 Pending 3 Approved 3 Completed 3 Cancelled 2

Search by prosumer, node, stat 11 of 11 Clear filters

PROSUMER	MICROGRID NODE	SCHEDULED	DURATION	STATUS	ACTIONS
Nithika Perera NIC-200301502340-0771234567	ST-005	Sun, Sep 20, 2026 11:30 PM - now	1h	Cancelled	
Nithika Perera NIC-200301502340-0771234567	ST-001	Sun, Sep 20, 2026 5:30 PM - now	1h	Cancelled	
Nithika Perera NIC-200301502340-0771234567	ST-002	Mon, Sep 21, 2026 2:00 PM - now	1h	Completed	
Nithika Perera NIC-200301502340-0771234567	ST-003	Mon, Sep 21, 2026 3:00 PM - now	1h	Approved 6352265950DFB	
Nithika Perera NIC-200301502340-0771234567	solar 45,000, 23,000	Mon, Sep 21, 2026 3:56 PM - now	2h	Approved ID5616193A85	
Nithika Perera NIC-200301502340-0771234567	solar 45,000, 23,000	Wed, Sep 23, 2026 4:12 PM - now	4h	Approved FCD8263FBACE	
Hiruni Chamathika Wahalathanthri SLIIT Malabe Hub	SLIIT Malabe Hub	Sun, Sep 20, 2026	2h 15m	Completed	

Solar Microgrid TRADING SYSTEM

OVERVIEW

- Dashboard
- Reservations**

MANAGE

- Prosumers
- Microgrid Nodes

HP **Hiruni Perera** GridOperator Log out

Reservations

Power trading bookings across all microgrid nodes.

+ New Reservation

All 11 Pending 3 Approved 3 Completed 3 Cancelled 2

Search by prosumer, node, stat 11 of 11 Clear filters

PROSUMER	MICROGRID NODE	SCHEDULED	DURATION	STATUS	ACTIONS
Nithika Perera NIC-200301502340-0771234567	ST-005	Sun, Sep 20, 2026 11:30 PM - now	1h	Cancelled	
Nithika Perera NIC-200301502340-0771234567	ST-001	Sun, Sep 20, 2026 5:30 PM - now	1h	Cancelled	
Nithika Perera NIC-200301502340-0771234567	ST-002	Mon, Sep 21, 2026 2:00 PM - now	1h	Completed	
Nithika Perera NIC-200301502340-0771234567	ST-003	Mon, Sep 21, 2026 3:00 PM - now	1h	Approved 6352265950DFB	
Nithika Perera NIC-200301502340-0771234567	solar 45,000, 23,000	Mon, Sep 21, 2026 3:56 PM - now	2h	Approved ID5616193A85	
Nithika Perera NIC-200301502340-0771234567	solar 45,000, 23,000	Wed, Sep 23, 2026 4:12 PM - now	4h	Approved FCD8263FBACE	
Hiruni Chamathika Wahalathanthri SLIIT Malabe Hub	SLIIT Malabe Hub	Sun, Sep 20, 2026	2h 15m	Completed	

Create reservation

Must be scheduled within the next 7 days.

Prosumer (NIC) Microgrid node

Select prosumer... Select node...

Date & time Duration (min)

mm/dd/yyyy -- 60

Cancel Create reservation

1.1.8 Grid Operator Prosumers Management

The screenshot shows the 'Prosumers' management interface. The left sidebar contains navigation links: Overview, Dashboard, Reservations, Manage, Prosumers (selected), and Microgrid Nodes. The main content area features a header with the title 'Prosumers' and a subtitle 'Property owners with solar arrays trading energy on the grid.' Below this are three summary cards: '6 Total Prosumers' (Registered property owners), '6 Active Prosumers' (Currently trading energy), and '12.8 MWh Grid Contribution' (Total energy supplied, 12% vs. last month). A search bar and a table with 6 results are displayed. The table columns are NIC, NAME, CONTACT, STATUS, and ACTIONS.

NIC	NAME	CONTACT	STATUS	ACTIONS
200301502340	Nithika Perera Colombo, Sri Lanka	nithika@gmail.com 0771234567	Active	[Edit] [Delete]
123456789	test1	test@gmail.com	Active	[Edit] [Delete]
200145345672	Hiruni Chamathka Avisawella	admin@gmail.com 0785645234	Active	[Edit] [Delete]
200186501187	Hiruni Chamathka Wahalathanthri 123 Galle Road, Colombo 03	hiruni@gmail.com 0762723949	Active	[Edit] [Delete]

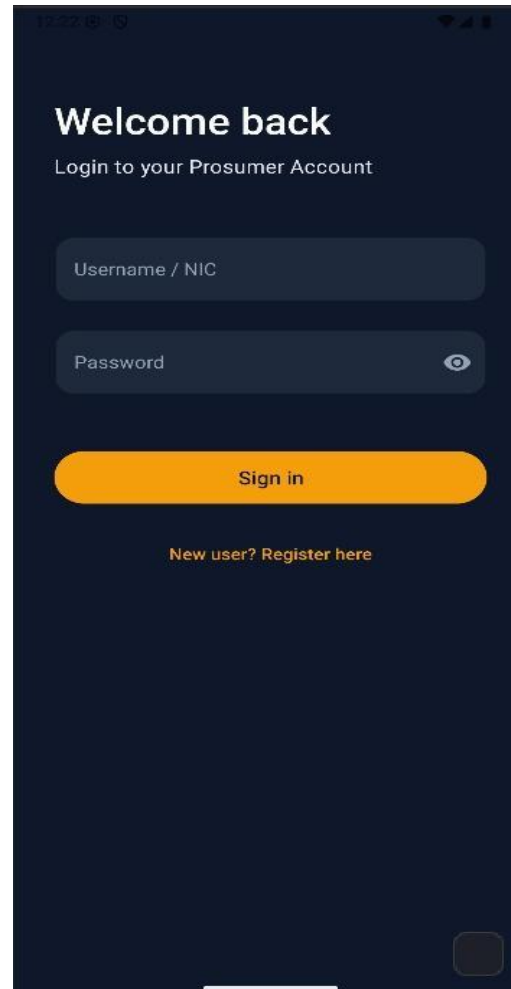
1.1.9 Grid Operator Microgrid nodes Management

The screenshot shows the 'Microgrid Nodes' management interface. The left sidebar contains navigation links: Overview, Dashboard, Reservations, Manage, Prosumers, and Microgrid Nodes (selected). The main content area features a header with the title 'Microgrid Nodes' and a subtitle 'Solar grid hubs – location, capacity and battery storage slots.' Below this are four summary cards: '6 Total Nodes' (Active monitoring hubs), '43,662.25 kWh Total Capacity' (Across all nodes), '32,723 / 32,738 Available Battery Slots' (100% available), and '3 / 6 Operational Status' (Some nodes offline). A search bar and a table with 6 results are displayed. The table columns are NODE, LOCATION, CAPACITY, BATTERY SLOTS, STATUS, and ACTIONS.

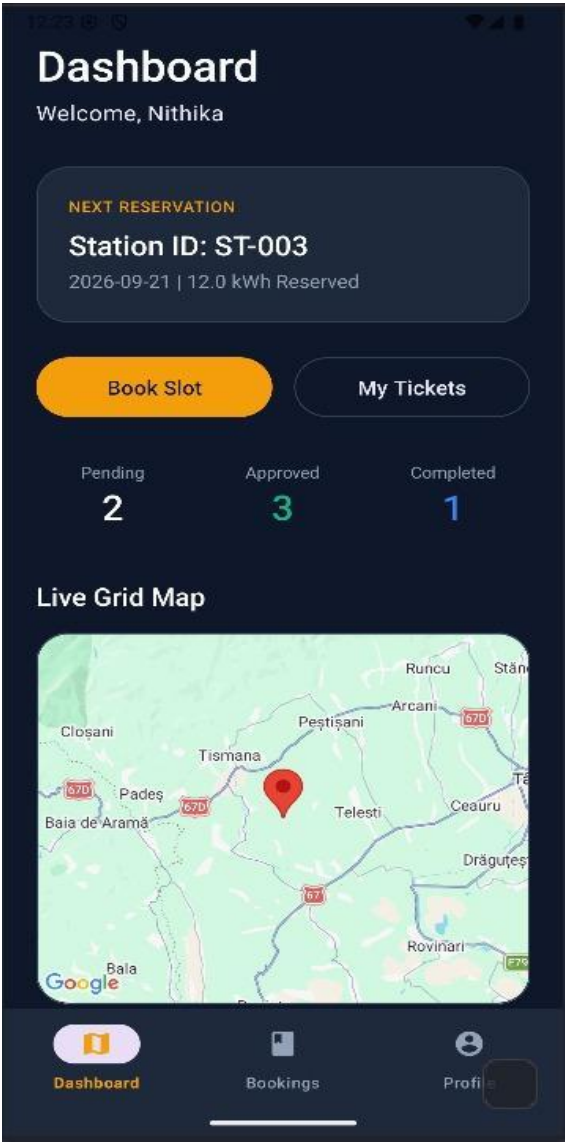
NODE	LOCATION	CAPACITY	BATTERY SLOTS	STATUS	ACTIONS
solar Mon-Sun 06:00-20:00	45.0000, 23.0000	34569 kWh	32000 / 32000	Deactivated	[Edit]
SLIIT Malabe Hub Mon-Fri 06:00-22:00 - Sat-Sun 08:00-18:00	6.9147, 79.9733	1500.5 kWh	120 / 110	Active	[Edit]
Hokandara Microgrid Mon-Sun 00:00-23:59	6.8781, 79.9614	850 kWh	30 / 50	Deactivated	[Edit]
Trace Expert City Mon-Sun 05:00-23:00	6.9298, 79.8617	2500 kWh	200 / 200	Active	[Edit]

1.2 Mobile Application User Interface

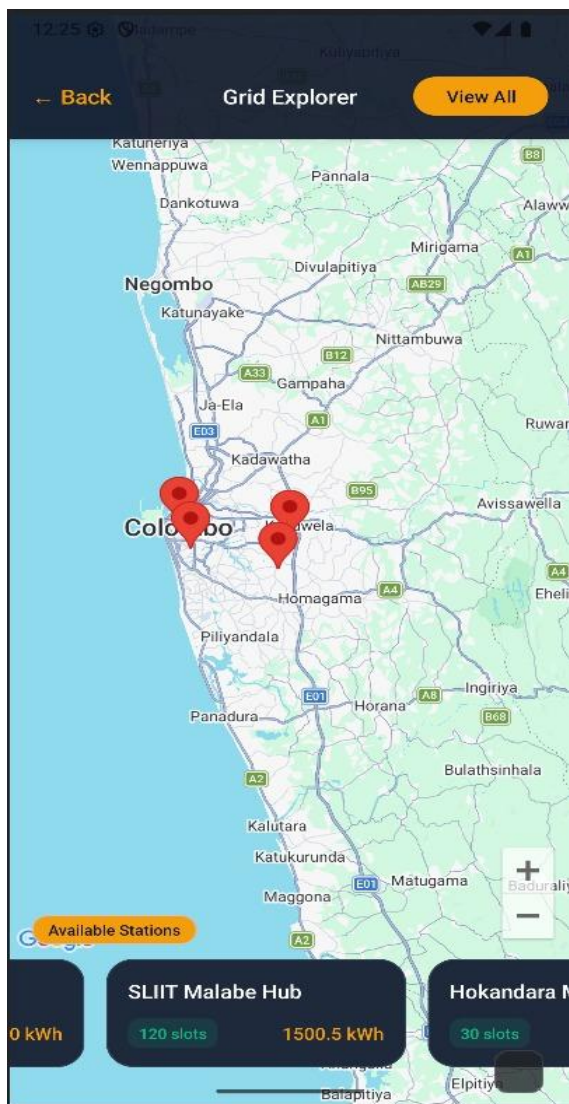
1.2.1 Login and Authentication



1.2.2 Dashboard



1.2.3 Station Discovery and Search



1.2.5 Energy Slot Selection and Reservation

12:24

← Back

Reserve Energy

Book a microgrid slot (max 7 days in advance)

Select Station

solar (32000 slots | 34569.0 kWh) ▼

Reservation Date (YYYY-MM-DD)

Start Time

End Time

Energy Amount (kWh)

Confirm Booking

Dashboard Bookings Profile

1.2.6 Booking Management and Status Tracking

12:24

← Back

Modify Booking

Update time or energy. Station cannot be changed.

Station (Read-Only)

SLIIT Malabe Hub

Reservation Date

2026-09-26

Start Time

12:30

End Time

18:30

Energy Amount (kWh)

40.0

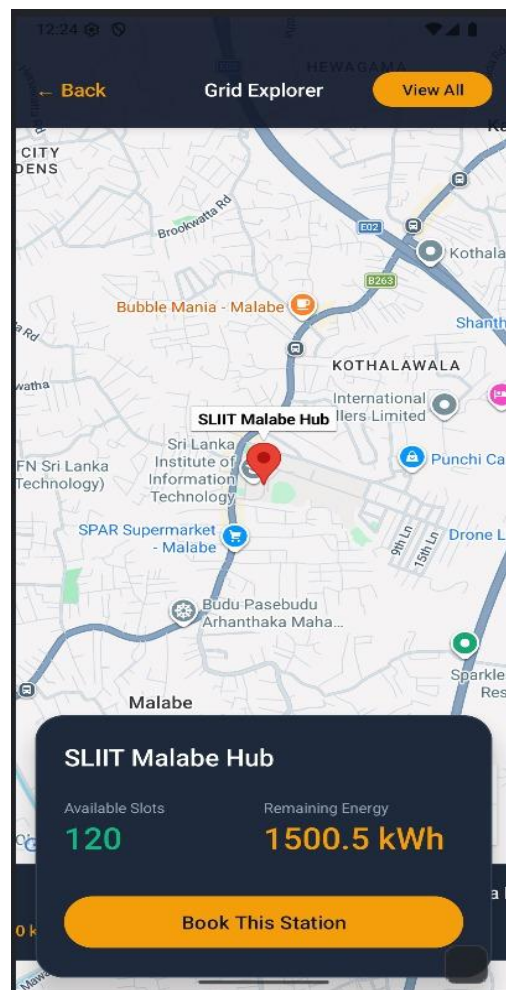
Save Changes

Cancel Booking

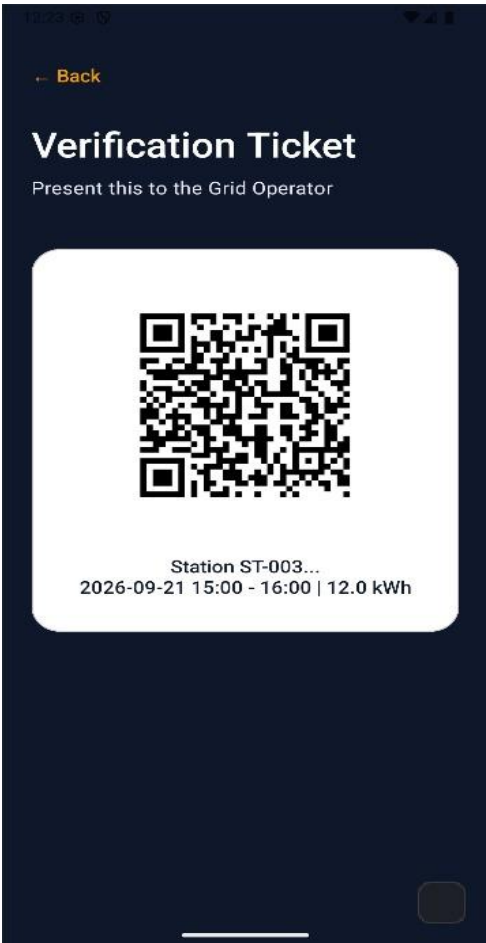
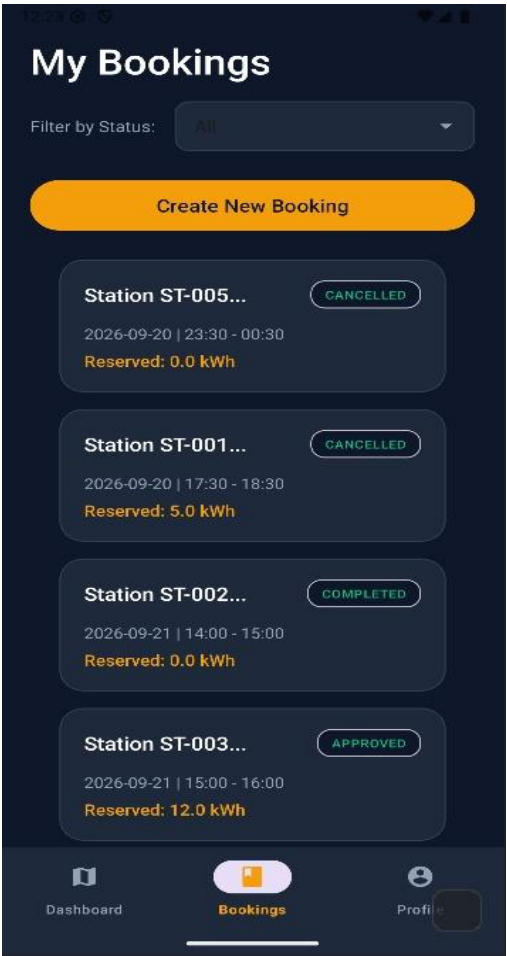
Dashboard

Bookings

Profile

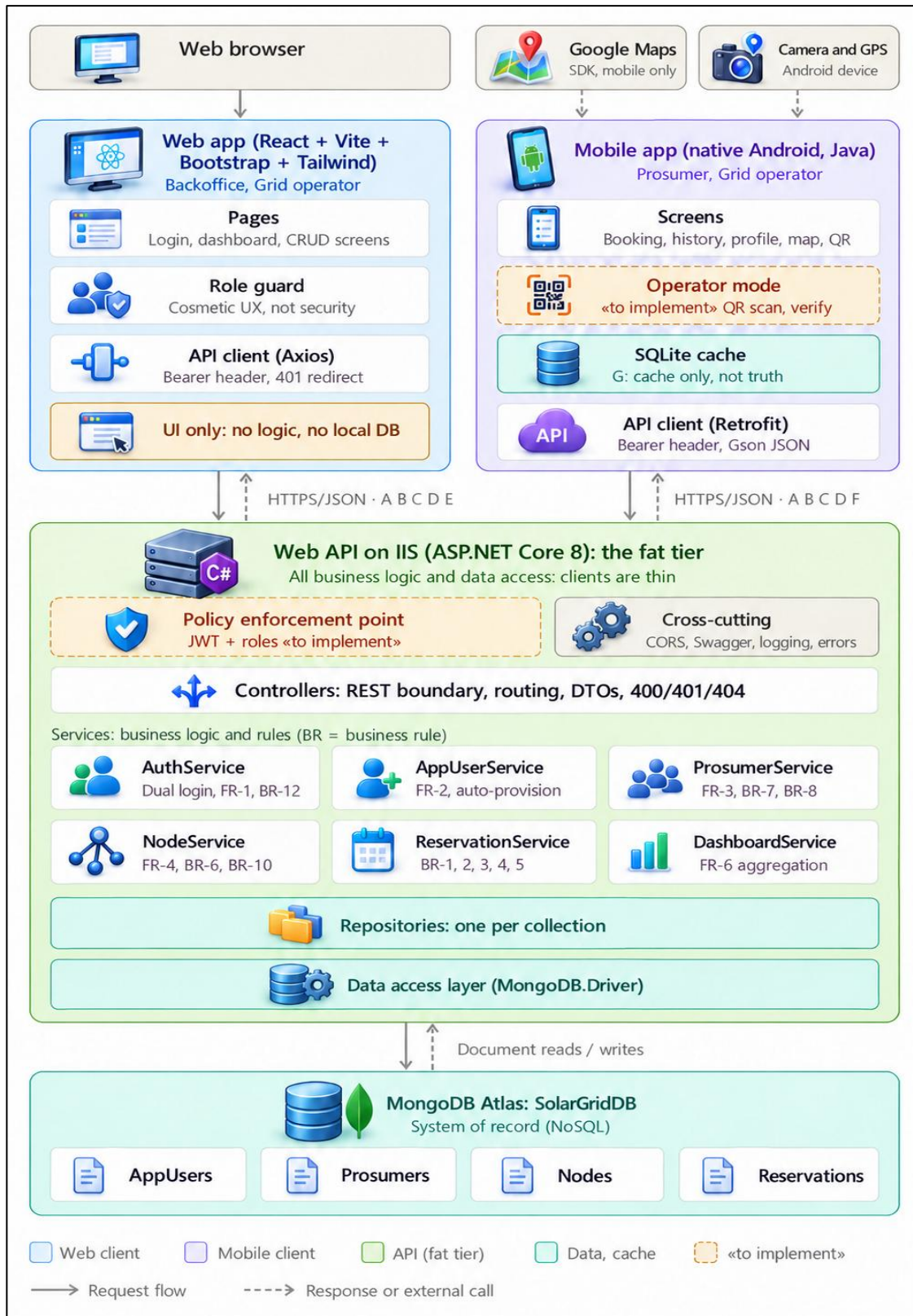


1.2.7 Booking Details and QR Verification

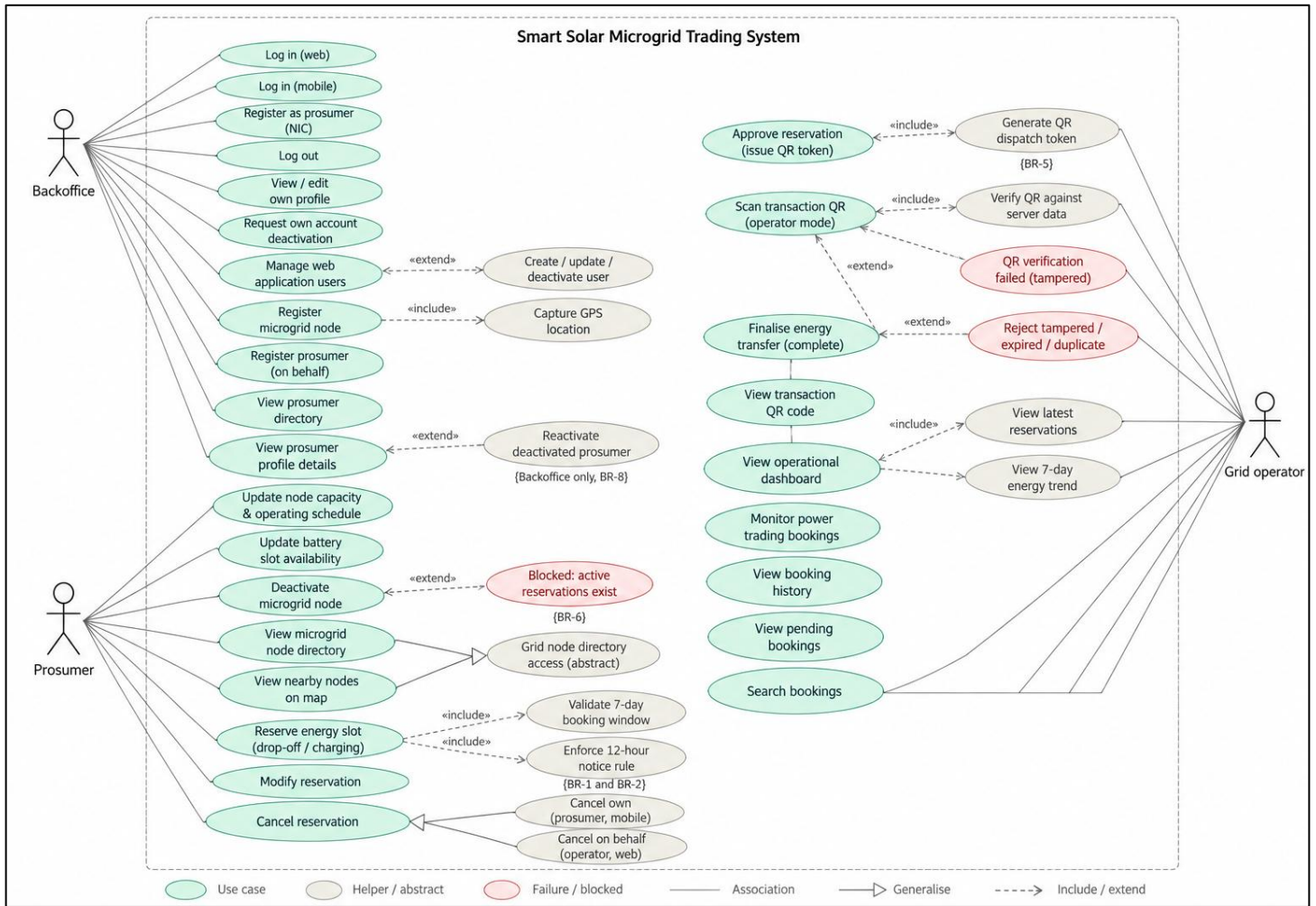


2 Architecture and System Diagrams

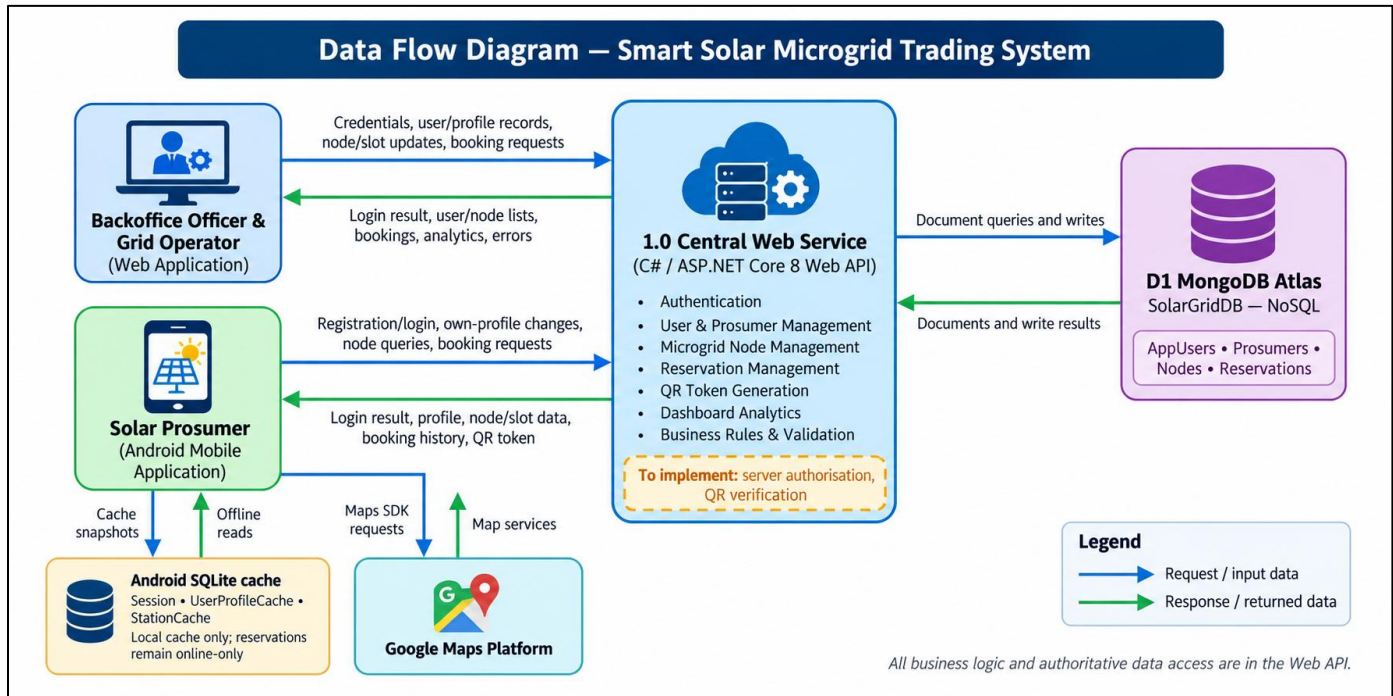
2.1 Application high level diagram



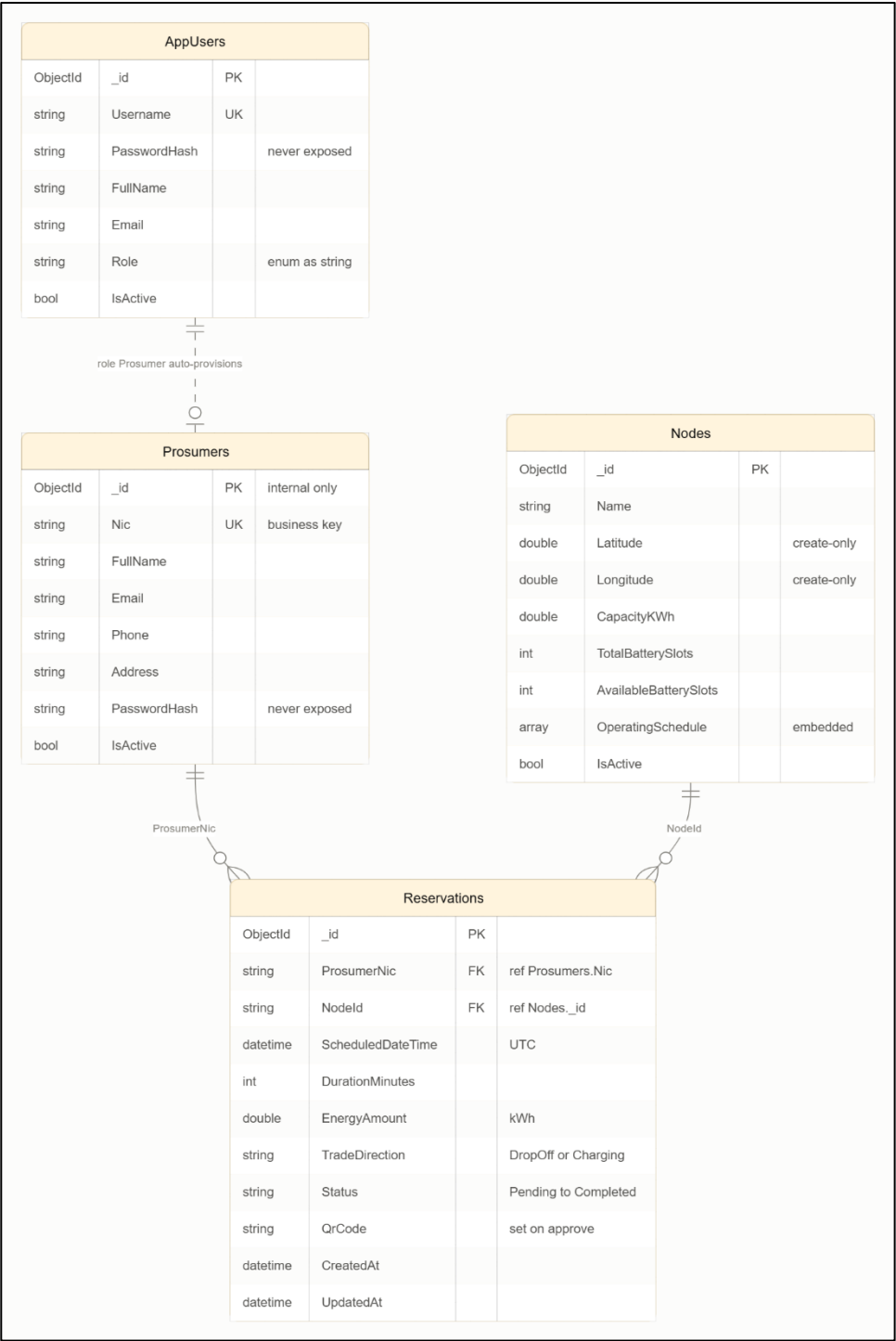
2.2 Use case Diagram



2.3 DFD diagram



2.4 Database design



3 Source code

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\api\api.js

```
1. // -----
2. // File: api.js
3. // Purpose: Thin wrapper functions around axiosClient, one per Web Service
4. // endpoint. Pages import from here rather than calling axios directly, so
5. // the web application stays limited to UI logic (per FAT-service spec).
6. // -----
7. import axiosClient from './axiosClient'
8.
9. // ---- Auth ----
10. export const login = (username, password) =>
11.   axiosClient.post('/auth/login', { username, password })
12.
13. // ---- Users (Backoffice-only administration) ----
14. export const getUsers = () => axiosClient.get('/users')
15. export const getUser = (id) => axiosClient.get(`/users/${id}`)
16. export const createUser = (data) => axiosClient.post('/users', data)
17. export const updateUser = (id, data) => axiosClient.put(`/users/${id}`, data)
18. export const deactivateUser = (id) => axiosClient.patch(`/users/${id}/deactivate`)
19.
20. // ---- Prosumers ----
21. export const getProsumers = () => axiosClient.get('/prosumers')
22. export const getProsumer = (nic) => axiosClient.get(`/prosumers/${nic}`)
23. export const createProsumer = (data) => axiosClient.post('/prosumers', data)
24. export const updateProsumer = (nic, data) => axiosClient.put(`/prosumers/${nic}`, data)
25. export const deactivateProsumer = (nic) => axiosClient.patch(`/prosumers/${nic}/deactivate`)
26. export const reactivateProsumer = (nic) => axiosClient.patch(`/prosumers/${nic}/reactivate`)
27.
28. // ---- Microgrid Nodes ----
29. export const getNodes = () => axiosClient.get('/nodes')
30. export const getNode = (id) => axiosClient.get(`/nodes/${id}`)
31. export const createNode = (data) => axiosClient.post('/nodes', data)
32. export const updateNode = (id, data) => axiosClient.put(`/nodes/${id}`, data)
33. export const deactivateNode = (id) => axiosClient.patch(`/nodes/${id}/deactivate`)
34.
35. // ---- Reservations ----
36. export const getReservations = () => axiosClient.get('/reservations')
37. export const getReservationsByProsumer = (nic) =>
38.   axiosClient.get(`/reservations/prosumer/${nic}`)
39. export const createReservation = (data) => axiosClient.post('/reservations', data)
40. export const updateReservation = (id, data) => axiosClient.put(`/reservations/${id}`, data)
41. export const cancelReservation = (id) => axiosClient.patch(`/reservations/${id}/cancel`)
42. export const approveReservation = (id) => axiosClient.patch(`/reservations/${id}/approve`)
43. export const completeReservation = (id) => axiosClient.patch(`/reservations/${id}/complete`)
44.
45. // ---- Dashboard analytics ----
46. export const getDashboard = (period = 'all') =>
47.   axiosClient.get('/dashboard', { params: { period } })
```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\api\axiosClient.js

```
1. // -----
2. // File: axiosClient.js
3. // Purpose: Single axios instance for all calls to the C# Web Service (the
4. // FAT-service backend). Automatically attaches the JWT and handles 401s by
5. // logging the user out - keeping the web app as a "thin" UI client, per spec.
6. // -----
```

```

7. import axios from 'axios'
8.
9. // Inline comment: point this at your deployed IIS URL in production,
10. // e.g. "https://your-server/api". Defaults to local dev backend.
11. export const API_BASE_URL = 'http://localhost:8080/api'
12.
13. const axiosClient = axios.create({
14.   baseURL: API_BASE_URL,
15.   headers: { 'Content-Type': 'application/json' },
16. })
17.
18. axiosClient.interceptors.request.use((config) => {
19.   const token = localStorage.getItem('ssm_token')
20.   if (token) {
21.     config.headers.Authorization = `Bearer ${token}`
22.   }
23.   return config
24. })
25.
26. axiosClient.interceptors.response.use(
27.   (response) => response,
28.   (error) => {
29.     if (error.response && error.response.status === 401) {
30.       localStorage.removeItem('ssm_token')
31.       localStorage.removeItem('ssm_user')
32.       window.location.href = '/login'
33.     }
34.     return Promise.reject(error)
35.   }
36. )
37.
38. export default axiosClient
39.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\components\ProtectedRoute.jsx

```

1. // -----
2. // File: ProtectedRoute.jsx
3. // Purpose: Route guard that redirects to /login when unauthenticated, and
4. // optionally restricts a route to specific roles (e.g. Backoffice only).
5. // -----
6. import React from 'react'
7. import { Navigate } from 'react-router-dom'
8. import { useAuth } from '../context/AuthContext'
9.
10. export default function ProtectedRoute({ children, allowedRoles }) {
11.   const { user } = useAuth()
12.
13.   if (!user) {
14.     return <Navigate to="/login" replace />
15.   }
16.
17.   // Inline comment: if allowedRoles is provided, block access for roles
18.   // outside that list (e.g. Grid Operators viewing User Management).
19.   if (allowedRoles && !allowedRoles.includes(user.role)) {
20.     return <Navigate to="/dashboard" replace />
21.   }
22.
23.   return children
24. }
25.

```


D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\components\ReservationOverview.jsx

```
1. import React, { useCallback, useEffect, useState } from 'react'
2. import { PieChart as RechartsPieChart, Pie, Cell, ResponsiveContainer } from 'recharts'
3. import { ChartColumn, ChevronDown, Check, TriangleAlert, ChartPie } from 'lucide-react'
4. import { getDashboard } from '../api/api'
5. import { getErrorMessage } from '../utils/errors'
6.
7. const STATUS_COLORS = {
8.   Pending: '#F5A524',
9.   Approved: '#2E6FD8',
10.  Completed: '#16A266',
11.  Cancelled: '#AEC2D8',
12. }
13. const STATUS_ORDER = ['Pending', 'Approved', 'Completed', 'Cancelled']
14. const PERIODS = [
15.   { key: 'all', label: 'All time' },
16.   { key: 'today', label: 'Today' },
17.   { key: 'week', label: 'This week' },
18.   { key: 'month', label: 'This month' },
19. ]
20.
21. const pctOf = (value, total) => (total > 0 ? (value / total) * 100 : 0)
22. const fmtPct = (value) => `${value.toFixed(1)}%`
23.
24. export default function ReservationOverview() {
25.   const [period, setPeriod] = useState('all')
26.   const [menuOpen, setMenuOpen] = useState(false)
27.   const [data, setData] = useState(null)
28.   const [loading, setLoading] = useState(true)
29.   const [error, setError] = useState(null)
30.
31.   const load = useCallback(async (periodKey, asSkeleton = true) => {
32.     if (asSkeleton) setLoading(true)
33.     setError(null)
34.     try {
35.       const res = await getDashboard(periodKey)
36.       setData(res.data?.reservations || null)
37.     } catch (e) {
38.       setError(getErrorMessage(e))
39.     } finally {
40.       setLoading(false)
41.     }
42.   }, [])
43.
44.   useEffect(() => {
45.     load(period, true)
46.   }, [period, load])
47.
48.   const changePeriod = (key) => {
49.     setPeriod(key)
50.     setMenuOpen(false)
51.     load(key, true)
52.   }
53.
54.   const ready = data !== null
55.   const total = ready ? data.total || 0 : 0
56.   const slices = ready
57.     ? STATUS_ORDER.map((s) => ({
58.       status: s,
59.       value: data[s.toLowerCase()] || 0,
60.       color: STATUS_COLORS[s],
```

```

61.         pct: pctOf(data[s.toLowerCase()] || 0, total),
62.     })))
63.     : []
64.     const donutData = slices.filter((s) => s.value > 0)
65.
66.     const successShare = ready
67.       ? pctOf((data.approved || 0) + (data.completed || 0), total)
68.       : 0
69.
70.     const renderInnerLabel = (props) => {
71.       const percent = props?.percent ?? 0
72.       if (percent < 0.07) return null
73.       const RADIANT = Math.PI / 180
74.       const radius = props.innerRadius + (props.outerRadius - props.innerRadius) * 0.55
75.       const x = props.cx + radius * Math.cos(-props.midAngle * RADIANT)
76.       const y = props.cy + radius * Math.sin(-props.midAngle * RADIANT)
77.       return (
78.         <text
79.           x={x}
80.           y={y}
81.           fill="#FFFFFF"
82.           fontSize={12}
83.           fontWeight={700}
84.           textAnchor="middle"
85.           dominantBaseline="central"
86.         >
87.           {Math.round(percent * total)}
88.         </text>
89.       )
90.     }
91.
92.     const periodLabel = PERIODS.find((p) => p.key === period)?.label || 'All time'
93.
94.     return (
95.       <section className="ro-card">
96.         <div className="ro-head">
97.           <div className="ro-title">
98.             <h3>Reservation overview</h3>
99.             <p>Distribution of reservation status across all time.</p>
100.          </div>
101.          <div className="ro-dd-wrap">
102.            <button
103.              className="ro-dd"
104.              type="button"
105.              onClick={() => setMenuOpen(!menuOpen)}
106.              aria-haspopup="menu"
107.              aria-expanded={menuOpen}
108.            >
109.              {periodLabel}
110.              <ChevronDown size={14} />
111.            </button>
112.            {menuOpen && (
113.              <div className="ro-dd-menu" role="menu">
114.                {PERIODS.map((o) => (
115.                  <button
116.                    key={o.key}
117.                    role="menuitem"
118.                    className={period === o.key ? 'active' : ''}
119.                    onClick={() => changePeriod(o.key)}
120.                  >
121.                    <span>{o.label}</span>
122.                    {period === o.key && <Check size={13} />}
123.                  </button>
124.                ))}
125.              </div>

```

```

126.     })
127.   </div>
128. </div>
129.
130. {error ? (
131.   <div className="ro-state ro-error">
132.     <TriangleAlert size={26} />
133.     <h4>Could not load reservations</h4>
134.     <p>{error}</p>
135.     <button type="button" className="btn btn-solar" onClick={() => load(period, true)}>
136.       Retry
137.     </button>
138.   </div>
139. ) : loading && !ready ? (
140.   <div className="ro-loading">
141.     <div className="ro-skel-donut" />
142.     <div className="ro-skel-col">
143.       {[0, 1, 2, 3].map((i) => (
144.         <div className="ro-skel-row" key={i} />
145.       ))}
146.     <div className="ro-skel-sum" />
147.   </div>
148. </div>
149. ) : total === 0 ? (
150.   <div className="ro-state">
151.     <ChartPie size={26} />
152.     <h4>No reservations yet</h4>
153.     <p>Reservations will appear here once bookings are created.</p>
154.   </div>
155. ) : (
156.   <div className="ro-body">
157.     <div className="ro-chart">
158.       <ResponsiveContainer width="100%" height="100%">
159.         <RechartsPieChart>
160.           <Pie
161.             data={donutData}
162.             dataKey="value"
163.             nameKey="status"
164.             cx="50%"
165.             cy="50%"
166.             innerRadius={62}
167.             outerRadius={88}
168.             paddingAngle={2}
169.             cornerRadius={5}
170.             stroke="none"
171.             label={renderInnerLabel}
172.             isAnimationActive={true}
173.           >
174.             {donutData.map((slice) => (
175.               <Cell key={slice.status} fill={slice.color} />
176.             ))}
177.           </Pie>
178.         </RechartsPieChart>
179.       </ResponsiveContainer>
180.       <div className="ro-chart-center">
181.         <span className="ro-center-total">{total.toLocaleString()}</span>
182.         <span className="ro-center-caption">Total</span>
183.         <span className="ro-center-sub">{total === 1 ? 'reservation' :
184.           'reservations'}</span>
185.       </div>
186.     </div>
187.     <div className="ro-side">
188.       <div className="ro-legend">
189.         {slices.map((s) => (

```

```

190.         <div className="ro-legend-row" key={s.status}>
191.             <span className="ro-dot" style={{ background: s.color }} />
192.             <span className="ro-name">{s.status}</span>
193.             <span className="ro-count">{s.value.toLocaleString()}</span>
194.             <span className="ro-pct">{fmtPct(s.pct)}</span>
195.         </div>
196.     )}}
197. </div>
198.
199.     <div className="ro-summary">
200.         <span className="ro-summary-icon">
201.             <ChartColumn size={16} />
202.         </span>
203.         <div className="ro-summary-text">
204.             <span className="ro-summary-value">{fmtPct(successShare)}</span>
205.             <span className="ro-summary-label">of reservations completed or approved</span>
206.         </div>
207.     </div>
208. </div>
209. </div>
210. )}
211. </section>
212. )
213. }
214.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\components\Sidebar.jsx

```

1. import React from 'react'
2. import { useLocation, useNavigate } from 'react-router-dom'
3. import { useAuth } from '../context/AuthContext'
4.
5. // Sidebar background image
6. import sidebarBackground from '../images/sidebar-background.png'
7.
8. import {
9.     IconDashboard,
10.    IconUsers,
11.    IconProsumer,
12.    IconNode,
13.    IconReservation,
14.    IconLogout,
15.    IconSun,
16. } from './ui/Icons'
17.
18. const NAV_SECTIONS = [
19.     {
20.         label: 'Overview',
21.         items: [
22.             {
23.                 path: '/dashboard',
24.                 label: 'Dashboard',
25.                 icon: <IconDashboard size={17} />,
26.                 roles: ['Backoffice', 'GridOperator'],
27.             },
28.             {
29.                 path: '/reservations',
30.                 label: 'Reservations',
31.                 icon: <IconReservation size={17} />,
32.                 roles: ['GridOperator'],
33.             },
34.         ],
35.     },

```

```

36.   {
37.     label: 'Manage',
38.     items: [
39.       {
40.         path: '/prosumers',
41.         label: 'Prosumers',
42.         icon: <IconProsumer size={17} />,
43.         roles: ['Backoffice', 'GridOperator'],
44.       },
45.       {
46.         path: '/nodes',
47.         label: 'Microgrid Nodes',
48.         icon: <IconNode size={17} />,
49.         roles: ['Backoffice', 'GridOperator'],
50.       },
51.     ],
52.   },
53. ]
54.
55. export default function Sidebar() {
56.   const { user, logout } = useAuth()
57.   const location = useLocation()
58.   const navigate = useNavigate()
59.
60.   function handleLogout() {
61.     logout()
62.     navigate('/login')
63.   }
64.
65.   const initials = (user?.fullName || user?.username || '?')
66.     .split(' ')
67.     .filter(Boolean)
68.     .map((name) => name[0])
69.     .slice(0, 2)
70.     .join('')
71.     .toUpperCase()
72.
73.   const sections = [
74.     ...NAV_SECTIONS,
75.
76.     ...(user?.role === 'Backoffice'
77.       ? [
78.         {
79.           label: 'Administration',
80.           items: [
81.             {
82.               path: '/users',
83.               label: 'User Management',
84.               icon: <IconUsers size={17} />,
85.               roles: ['Backoffice'],
86.             },
87.           ],
88.         },
89.       ]
90.       : []),
91.   ]
92.   .map((section) => ({
93.     ...section,
94.     items: section.items.filter((item) =>
95.       item.roles.includes(user?.role)
96.     ),
97.   })))
98.   .filter((section) => section.items.length > 0)
99.
100.  function isActivePath(path) {

```

```

101.     return (
102.         location.pathname === path ||
103.         location.pathname.startsWith(`${path}/`)
104.     )
105. }
106.
107. return (
108.     <aside
109.         className="smg-sideabar"
110.         style={{
111.             '--sidebar-background': `url(${sidebarBackground})`,
112.         }}
113.     >
114.         { /* Brand */ }
115.         <div className="smg-brand">
116.             <div className="smg-brand-logo">
117.                 <IconSun size={20} />
118.             </div>
119.
120.             <div className="smg-brand-text">
121.                 <div className="smg-brand-title">Solar Microgrid</div>
122.                 <div className="smg-brand-sub">Trading System</div>
123.             </div>
124.         </div>
125.
126.         { /* Navigation */ }
127.         <nav className="smg-nav" aria-label="Main navigation">
128.             {sections.map((section) => (
129.                 <React.Fragment key={section.label}>
130.                     <div className="smg-nav-section">{section.label}</div>
131.
132.                     {section.items.map((item) => (
133.                         <button
134.                             key={item.path}
135.                             type="button"
136.                             className={`smg-nav-link ${
137.                                 isActivePath(item.path) ? 'active' : ''
138.                             }}
139.                             onClick={() => navigate(item.path)}
140.                             aria-current={
141.                                 isActivePath(item.path) ? 'page' : undefined
142.                             }
143.                         >
144.                             {item.icon}
145.
146.                             <span>{item.label}</span>
147.                         </button>
148.                     ))}
149.                 </React.Fragment>
150.             ))}
151.         </nav>
152.
153.         { /* User footer */ }
154.         <footer className="smg-foot">
155.             <div className="smg-foot-divider" />
156.
157.             <div className="smg-user">
158.                 <div className="smg-user-avatar">{initials}</div>
159.
160.                 <div className="smg-user-meta">
161.                     <div className="smg-user-name">
162.                         {user?.fullName || user?.username || 'User'}
163.                     </div>
164.
165.                     <span className="smg-user-pill">

```



```

166.         {user?.role || 'User'}
167.     </span>
168. </div>
169. </div>
170.
171.     <div className="smg-foot-bottom">
172.         <button
173.             className="smg-logout"
174.             type="button"
175.             onClick={handleLogout}
176.         >
177.             <IconLogout size={16} />
178.             <span>Log out</span>
179.         </button>
180.
181.         <div className="smg-clean-energy" aria-hidden="true">
182.             <span>Clean Energy</span>
183.             <span>Stronger</span>
184.             <span>Communities</span>
185.         </div>
186.     </div>
187. </footer>
188. </aside>
189. )
190. }
191.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\components\ui\Icons.jsx

```

1. import React from 'react'
2.
3. const S = ({ children, size = 18, ...props }) => (
4.   <svg
5.     width={size}
6.     height={size}
7.     viewBox="0 0 24 24"
8.     fill="none"
9.     stroke="currentColor"
10.    strokeWidth="2"
11.    strokeLinecap="round"
12.    strokeLinejoin="round"
13.    aria-hidden="true"
14.    {...props}
15.  >
16.    {children}
17.  </svg>
18. )
19.
20. export const IconDashboard = (p) => (
21.   <S {...p}><rect x="3" y="3" width="7" height="9" rx="1" /><rect x="14" y="3" width="7"
height="5" rx="1" /><rect x="14" y="12" width="7" height="9" rx="1" /><rect x="3" y="16" width="7"
height="5" rx="1" /></S>
22. )
23. export const IconUser = (p) => <S {...p}><path d="M19 21v-2a4 4 0 0 0-4-4H9a4 4 0 0 0-4 4v2"
/><circle cx="12" cy="7" r="4" /></S>
24. export const IconUsers = (p) => (
25.   <S {...p}><path d="M16 21v-2a4 4 0 0 0-4-4H6a4 4 0 0 0-4 4v2" /><circle cx="9" cy="7" r="4"
/><path d="M22 21v-2a4 4 0 0 0-3-3.87" /><path d="M16 3.13a4 4 0 0 1 0 7.75" /></S>
26. )
27. export const IconProsumer = (p) => (
28.   <S {...p}><path d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2" /><circle cx="12" cy="7" r="4"
/></S>
29. )

```

```

30. export const IconNode = (p) => (
31.   <S {...p}><path d="M12 9v3" /><path d="M12 21v-6" /><circle cx="12" cy="12" r="3" /><path
d="M5 713 3" /><path d="M16 1413 3" /><path d="M19 71-3 3" /><path d="M8 141-3 3" /></S>
32. )
33. export const IconReservation = (p) => (
34.   <S {...p}><rect x="3" y="4" width="18" height="18" rx="2" /><path d="M16 2v4" /><path d="M8
2v4" /><path d="M3 10h18" /><path d="m9 16 2 2 4-4" /></S>
35. )
36. export const IconShield = (p) => (
37.   <S {...p}><path d="M12 22s8-4 8-10V51-8-3-8 3v7c0 6 8 10 8 10" /></S>
38. )
39. export const IconLogout = (p) => (
40.   <S {...p}><path d="M9 21H5a2 2 0 0 1-2-2V5a2 2 0 0 1 2-2h4" /><path d="m16 17 5-5-5-5"
/><path d="M21 12H9" /></S>
41. )
42. export const IconPlus = (p) => <S {...p}><path d="M12 5v14" /><path d="M5 12h14" /></S>
43. export const IconEdit = (p) => (
44.   <S {...p}><path d="M17 3a2.85 2.83 0 1 1 4 4L7.5 20.5 2 2211.5-5.5Z" /><path d="m15 5 4 4"
/></S>
45. )
46. export const IconTrash = (p) => (
47.   <S {...p}><path d="M3 6h18" /><path d="M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6" /><path
d="M8 6V4a2 2 0 0 1 2-2h4a2 2 0 0 1 2 2v2" /><path d="M10 11v6" /><path d="M14 11v6" /></S>
48. )
49. export const IconRefresh = (p) => (
50.   <S {...p}><path d="M3 12a9 9 0 0 1 15-6.7L21 8" /><path d="M21 3v5h-5" /><path d="M21 12a9 9
0 0 1-15 6.7L3 16" /><path d="M3 21v-5h5" /></S>
51. )
52. export const IconSearch = (p) => (
53.   <S {...p}><circle cx="11" cy="11" r="8" /><path d="m21 21-4.35-4.35" /></S>
54. )
55. export const IconCheck = (p) => <S {...p}><path d="M20 6 9 171-5-5" /></S>
56. export const IconCheckCircle = (p) => (
57.   <S {...p}><path d="M22 11.08V12a10 10 0 1 1-5.93-9.14" /><path d="m9 11 3 3L22 4" /></S>
58. )
59. export const IconX = (p) => <S {...p}><path d="M18 6 6 18" /><path d="m6 6 12 12" /></S>
60. export const IconAlert = (p) => (
61.   <S {...p}><path d="M21.73 18-8-14a2 2 0 0 0-3.48 01-8 14A2 2 0 0 0 4 21h16a2 2 0 0 0 1.73-3"
/><path d="M12 9v4" /><path d="M12 17h.01" /></S>
62. )
63. export const IconSun = (p) => (
64.   <S {...p}><circle cx="12" cy="12" r="4" /><path d="M12 2v2" /><path d="M12 20v2" /><path
d="m4.93 4.93 1.41 1.41" /><path d="m17.66 17.66 1.41 1.41" /><path d="M2 12h2" /><path d="M20 12h2"
/><path d="m6.34 17.66-1.41 1.41" /><path d="m19.07 4.93-1.41 1.41" /></S>
65. )
66. export const IconZap = (p) => <S {...p}><path d="M13 2 3 14h91-1 8 10-12h-911-8z" /></S>
67. export const IconBattery = (p) => (
68.   <S {...p}><rect x="1" y="7" width="16" height="10" rx="2" /><path d="M23 11v2" /></S>
69. )
70. export const IconCalendar = (p) => (
71.   <S {...p}><rect x="3" y="4" width="18" height="18" rx="2" /><path d="M16 2v4" /><path d="M8
2v4" /><path d="M3 10h18" /></S>
72. )
73. export const IconClock = (p) => (
74.   <S {...p}><circle cx="12" cy="12" r="10" /><path d="M12 6v614 2" /></S>
75. )
76. export const IconKey = (p) => (
77.   <S {...p}><path d="M21 21-2 2m-7.61 7.61a5.5 5.5 0 1 1-7.778 7.778 5.5 5.5 0 0 1 7.777-
7.777zm0 0L15.5 7.5m0 0 3 3L22 71-3-3m-3.5 3.5L19 4" /></S>
78. )
79. export const IconMapPin = (p) => (
80.   <S {...p}><path d="M20 10c0 6-8 12-8 12s-8-6-8-12a8 8 0 0 1 16 0Z" /><circle cx="12" cy="10"
r="3" /></S>
81. )
82. export const IconEye = (p) => (

```

```

83.   <S {...p}><path d="M2 12s3-7 10-7 10 7 10 7-3 7-10 7-10-7-10-72" /><circle cx="12" cy="12"
r="3" /></S>
84. )
85. export const IconInfo = (p) => (
86.   <S {...p}><circle cx="12" cy="12" r="10" /><path d="M12 16v-4" /><path d="M12 8h.01" /></S>
87. )
88. export const IconQrCode = (p) => (
89.   <S {...p}><rect x="3" y="3" width="7" height="7" rx="1" /><rect x="14" y="3" width="7"
height="7" rx="1" /><rect x="14" y="14" width="7" height="7" rx="1" /><rect x="3" y="14" width="7"
height="7" rx="1" /></S>
90. )
91. export const IconLock = (p) => (
92.   <S {...p}><rect x="3" y="11" width="18" height="11" rx="2" /><path d="M7 11V7a5 5 0 0 1 10
0v4" /></S>
93. )
94. export const IconArrowRight = (p) => <S {...p}><path d="M5 12h14" /><path d="m12 5 7 7-7 7"
/></S>
95. export const IconChevronDown = (p) => <S {...p}><path d="m6 9 6 6 6-6" /></S>
96. export const IconBell = (p) => (
97.   <S {...p}><path d="M6 8a6 6 0 0 1 12 0c0 7 3 9 3 9H3s3-2 3-9" /><path d="M10.3 21a1.94 1.94 0
0 0 3.4 0" /></S>
98. )
99. export const IconMonitor = (p) => (
100.   <S {...p}><rect x="2" y="3" width="20" height="14" rx="2" /><path d="M8 21h8" /><path d="M12
17v4" /></S>
101. )
102. export const IconShieldCheck = (p) => (
103.   <S {...p}><path d="M12 22s8-4 8-10V5l-8-3-8 3v7c0 6 8 10 8 10" /><path d="m9 12 2 2 4-4"
/></S>
104. )
105. export const IconEyeOff = (p) => (
106.   <S {...p}><path d="M9.88 9.88a3 3 0 1 0 4.24 4.24" /><path d="M10.73 5.08A10.43 10.43 0 0 1
12 5c7 0 10 7 10 7a13.16 13.16 0 0 1-1.67 2.68" /><path d="M6.61 6.61A13.526 13.526 0 0 0 2 12s3 7
10 7a9.74 9.74 0 0 0 5.39-1.61" /><path d="M2 21z0 20" /></S>
107. )
108. export const IconSpinner = ({ size = 18 }) => (
109.   <svg width={size} height={size} viewBox="0 0 24 24" fill="none" className="spinner" aria-
hidden="true">
110.     <circle cx="12" cy="12" r="10" stroke="currentColor" strokeOpacity="0.25" strokeWidth="4"
/>
111.     <path d="M22 12a10 10 0 0 0 0-10-10" stroke="currentColor" strokeWidth="4"
strokeLinecap="round" />
112.   </svg>
113. )
114.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\components\ui\Modal.jsx

```

1. import React, { useEffect } from 'react'
2. import { IconX } from './Icons'
3.
4. export default function Modal({ open, onClose, title, subtitle, children, footer, size = 'md' })
{
5.   useEffect(() => {
6.     if (!open) return
7.     const onKey = (e) => {
8.       if (e.key === 'Escape') onClose?.()
9.     }
10.    document.addEventListener('keydown', onKey)
11.    document.body.style.overflow = 'hidden'
12.    return () => {
13.      document.removeEventListener('keydown', onKey)
14.      document.body.style.overflow = ''

```

```

15.     }
16.   }, [open, onClose])
17.
18.   if (!open) return null
19.
20.   return (
21.     <div className="modal-overlay" onMouseDown={(e) => e.target === e.currentTarget &&
onClose?.()}>
22.       <div className={`modal-card modal-${size}`} role="dialog" aria-modal="true">
23.         <div className="modal-header">
24.           <div>
25.             <h3>{title}</h3>
26.             {subtitle && <p>{subtitle}</p>}
27.           </div>
28.           <button className="btn-icon" onClick={onClose} aria-label="Close">
29.             <IconX size={18} />
30.           </button>
31.         </div>
32.         <div className="modal-body">{children}</div>
33.         {footer && <div className="modal-footer">{footer}</div>}
34.       </div>
35.     </div>
36.   )
37. }
38.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\components\ui\Toast.jsx

```

1. import React, { createContext, useContext, useState, useCallbck, useRef } from 'react'
2. import { IconCheckCircle, IconAlert, IconX, IconCheck } from './Icons'
3.
4. const ToastContext = createContext(null)
5.
6. let idCounter = 0
7.
8. export function ToastProvider({ children }) {
9.   const [toasts, setToasts] = useState([])
10.
11.   const dismiss = useCallbck((id) => {
12.     setToasts((prev) => prev.filter((t) => t.id !== id))
13.   }, [])
14.
15.   const push = useCallbck((type, title, description, timeout = 4000) => {
16.     const id = ++idCounter
17.     setToasts((prev) => [...prev, { id, type, title, description }])
18.     if (timeout > 0) {
19.       setTimeout(() => dismiss(id), timeout)
20.     }
21.     return id
22.   }, [dismiss])
23.
24.   const toast = useRef({
25.     success: (title, description) => push('success', title, description),
26.     error: (title, description) => push('error', title, description),
27.     info: (title, description) => push('info', title, description),
28.   }).current
29.
30.   return (
31.     <ToastContext.Provider value={toast}>
32.       {children}
33.       <div className="toast-stack" role="status" aria-live="polite">
34.         {toasts.map((t) => (
35.           <div key={t.id} className={`toast-item toast-${t.type}`}>

```

```

36.         <span className="toast-icon">
37.             {t.type === 'success' && <IconCheckCircle size={18} />}
38.             {t.type === 'error' && <IconAlert size={18} />}
39.             {t.type === 'info' && <IconCheck size={18} />}
40.         </span>
41.         <div className="toast-body">
42.             <div className="toast-title">{t.title}</div>
43.             {t.description && <div className="toast-desc">{t.description}</div>}
44.         </div>
45.         <button className="toast-close" onClick={() => dismiss(t.id)} aria-label="Dismiss">
46.             <IconX size={14} />
47.         </button>
48.     </div>
49.   )}
50. </div>
51. </ToastContext.Provider>
52. )
53. }
54.
55. export function useToast() {
56.   const ctx = useContext(ToastContext)
57.   if (!ctx) throw new Error('useToast must be used within a ToastProvider')
58.   return ctx
59. }
60.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\components\ui\Widgets.jsx

```

1. import React, { useEffect, useState } from 'react'
2. import Modal from './Modal'
3. import { IconSearch, IconSpinner, IconAlert } from './Icons'
4.
5. export function Spinner({ size, label }) {
6.   return (
7.     <span className="inline-spinner">
8.       <IconSpinner size={size} />
9.       {label && <span style={{ marginLeft: 8 }}>{label}</span>}
10.    </span>
11.  )
12. }
13.
14. export function PageHeader({ title, subtitle, actions, className = '' }) {
15.   return (
16.     <div className={`page-header ${className}`}>
17.       <div>
18.         <h1>{title}</h1>
19.         {subtitle && <p>{subtitle}</p>}
20.       </div>
21.       {actions && <div className="page-header-actions">{actions}</div>}
22.     </div>
23.   )
24. }
25.
26. function useCountUp(target, duration = 900) {
27.   const [value, setValue] = useState(0)
28.
29.   useEffect(() => {
30.     if (typeof target !== 'number') {
31.       setValue(target)
32.       return
33.     }
34.     let raf
35.     const start = performance.now()

```

```

36.     const tick = (now) => {
37.         const t = Math.min((now - start) / duration, 1)
38.         const eased = 1 - Math.pow(1 - t, 3)
39.         setValue(Math.round(target * eased))
40.         if (t < 1) raf = requestAnimationFrame(tick)
41.     }
42.     raf = requestAnimationFrame(tick)
43.     return () => cancelAnimationFrame(raf)
44. }, [target, duration])
45.
46. return value
47. }
48.
49. export function StatCard({ icon, label, value, sub, accent = 'navy' }) {
50.     const display = useCountUp(value)
51.     return (
52.         <div className={`stat-card accent-${accent}`}>
53.             <div className="stat-top">
54.                 <span className="stat-icon">{icon}</span>
55.                 {sub} && <span className="stat-sub">{sub}</span>
56.             </div>
57.             <div className="stat-value">{display}</div>
58.             <div className="stat-label">{label}</div>
59.         </div>
60.     )
61. }
62.
63. export function Badge({ children, tone = 'neutral' }) {
64.     return <span className={`badge-status badge-${tone}`}>{children}</span>
65. }
66.
67. export function EmptyState({ icon, title, hint }) {
68.     return (
69.         <div className="empty-state">
70.             {icon} && <div className="empty-icon">{icon}</div>
71.             <div className="empty-title">{title}</div>
72.             {hint} && <div className="empty-hint">{hint}</div>
73.         </div>
74.     )
75. }
76.
77. export function SearchInput({ value, onChange, placeholder }) {
78.     return (
79.         <div className="search-box">
80.             <IconSearch size={16} />
81.             <input
82.                 type="text"
83.                 value={value}
84.                 onChange={(e) => onChange(e.target.value)}
85.                 placeholder={placeholder || 'Search...'}
86.             />
87.         </div>
88.     )
89. }
90.
91. export function ConfirmDialog({ open, title, message, confirmText = 'Confirm', tone = 'danger',
onConfirm, onCancel }) {
92.     const [busy, setBusy] = useState(false)
93.
94.     async function handleConfirm() {
95.         setBusy(true)
96.         try {
97.             await onConfirm()
98.         } finally {
99.             setBusy(false)

```

```

100.     onCancel()
101.   }
102. }
103.
104. return (
105.   <Modal
106.     open={open}
107.     onClose={onCancel}
108.     title={title}
109.     size="sm"
110.     footer={
111.       <>
112.         <button className="btn btn-ghost" onClick={onCancel} disabled={busy}>
113.           Cancel
114.         </button>
115.         <button className={`btn ${tone === 'danger' ? 'btn-danger' : 'btn-solar'}}`
onClick={handleConfirm} disabled={busy}>
116.           {busy ? <Spinner size={15} /> : confirmText}
117.         </button>
118.       </>
119.     }
120.   >
121.     <div className="confirm-body">
122.       <span className="confirm-icon">
123.         <IconAlert size={22} />
124.       </span>
125.       <p>{message}</p>
126.     </div>
127.   </Modal>
128. )
129. }
130.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\context\AuthContext.jsx

```

1. // -----
2. // File: AuthContext.jsx
3. // Purpose: Global auth state (current user, role, JWT) shared across the
4. // app via React Context. Persists to localStorage so a page refresh keeps
5. // the session alive until the token expires.
6. // -----
7. import React, { createContext, useContext, useState } from 'react'
8. import { login as loginApi } from '../api/api'
9.
10. const AuthContext = createContext(null)
11.
12. export function AuthProvider({ children }) {
13.   const [user, setUser] = useState(() => {
14.     const stored = localStorage.getItem('ssm_user')
15.     return stored ? JSON.parse(stored) : null
16.   })
17.
18.   // Inline comment: calls the /auth/login endpoint, stores the JWT + user
19.   // profile, and updates context state so protected routes unlock.
20.   async function login(username, password) {
21.     const response = await loginApi(username, password)
22.     const { token, username: uname, fullName, role } = response.data
23.     localStorage.setItem('ssm_token', token)
24.     const userInfo = { username: uname, fullName, role }
25.     localStorage.setItem('ssm_user', JSON.stringify(userInfo))
26.     setUser(userInfo)
27.     return userInfo
28.   }

```



```

29.
30.   function logout() {
31.     localStorage.removeItem('ssm_token')
32.     localStorage.removeItem('ssm_user')
33.     setUser(null)
34.   }
35.
36.   return (
37.     <AuthContext.Provider value={{ user, login, logout }}>
38.       {children}
39.     </AuthContext.Provider>
40.   )
41. }
42.
43. export function useAuth() {
44.   return useContext(AuthContext)
45. }
46.
47.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\pages\Dashboard.jsx

```

1. import React, { useCallback, useEffect, useState } from 'react'
2. import {
3.   Network,
4.   Zap,
5.   Battery,
6.   CircleCheck,
7.   RefreshCw,
8.   Users,
9.   TriangleAlert,
10.  ArrowUpRight,
11.  ArrowDownRight,
12.  Activity,
13.  TrendingUp,
14. } from 'lucide-react'
15. import { getDashboard } from '../api/api'
16. import { getErrorMessage } from '../utils/errors'
17. import ReservationOverview from '../components/ReservationOverview'
18.
19. const POLL_INTERVAL = 30000
20.
21. const fmtCount = (n) => (n ?? 0).toLocaleString()
22. const fmtKwh = (n) => (n ?? 0).toLocaleString('en-US', { maximumFractionDigits: 2 })
23. const pct = (value, total) => (total > 0 ? (value / total) * 100 : 0)
24. const occupiedPct = (total, available) => (total > 0 ? ((total - available) / total) * 100 : 0)
25.
26. const clampPct = (value) => Math.min(100, Math.max(0, Number(value) || 0))
27.
28. const KPI_TONES = {
29.   blue: {
30.     icon: 'bg-[#E7F0FB] text-[#2E6FD8]',
31.     accent: 'bg-[#2E6FD8]',
32.     bar: 'bg-linear-to-r from-[#6FA7F5] to-[#2E6FD8]',
33.     badge: 'bg-[#E4EDFB] text-[#2E6FD8]',
34.   },
35.   amber: {
36.     icon: 'bg-[#FDF0DC] text-[#F59A05]',
37.     accent: 'bg-[#F59A05]',
38.     bar: 'bg-linear-to-r from-[#FFC766] to-[#F59A05]',
39.     badge: 'bg-[#FFF4E0] text-[#C77300]',
40.   },
41.   teal: {
42.     icon: 'bg-[#DDF4EE] text-[#13A085]',

```

```

43.     accent: 'bg-[#13A085]',
44.     bar: 'bg-linear-to-r from-[#3AD0AF] to-[#13A085]',
45.     badge: 'bg-[#E0F5EF] text-[#0E8F75]',
46.   },
47.   emerald: {
48.     icon: 'bg-[#E2F5EB] text-[#16A266]',
49.     accent: 'bg-[#16A266]',
50.     bar: 'bg-linear-to-r from-[#43D499] to-[#16A266]',
51.     badge: 'bg-[#E5F7EE] text-[#128152]',
52.   },
53. }
54.
55. function Skeleton({ className }) {
56.   return <div className={`dash-skeleton ${className} || ''`} />
57. }
58.
59. function KpiCard({ icon, tone, value, label, sub, pill, barPct, warning }) {
60.   const t = KPI_TONES[tone] || KPI_TONES.blue
61.   return (
62.     <div className="relative flex min-h-0 min-w-0 flex-col overflow-hidden rounded-2xl border
border-[#DBE7F2] bg-linear-to-b from-white via-[#FBFDF2] to-[#F2F8FE] px-5 pt-5 pb-4 shadow-
[0_1px_2px_rgba(13,47,74,0.04),0_22px_40px_-26px_rgba(13,47,74,0.3)]">
63.       <span aria-hidden="true" className={`absolute inset-x-0 top-0 h-1 ${t.accent}`} />
64.       <div className="flex items-start justify-between gap-2.5">
65.         <span className={`grid h-12 w-12 shrink-0 place-items-center rounded-2xl
${t.icon}`}><icon></span>
66.         {warning}
67.       </div>
68.       <div className="mt-3 min-w-0">
69.         <div className="truncate text-[27px] font-extrabold leading-tight tracking-tight text-
[#0A2233]">
70.           {value}
71.         </div>
72.         <div className="mt-1 truncate text-[15px] font-semibold text-[#23405A]">{label}</div>
73.         <div className="mt-0.5 truncate text-[12.5px] font-medium text-[#6F8297]">{sub}</div>
74.       </div>
75.       <div className="mt-auto flex items-center gap-3 pt-3">
76.         <span className={`shrink-0 rounded-full px-2.5 py-1 text-[11px] font-bold
${t.badge}`}>{pill}</span>
77.         <div className="h-1.5 min-w-0 flex-1 overflow-hidden rounded-full bg-[#EBF1F6]">
78.           <span
79.             className={`block h-full rounded-full transition-[width] duration-500 ease-out
${t.bar}`}>
80.             style={{ width: `${clampPct(barPct)}%` }}
81.           />
82.         </div>
83.       </div>
84.     </div>
85.   )
86. }
87.
88. function DeltaPill({ pct: value, suffix = '%' }) {
89.   const v = Number(value) || 0
90.   const up = v > 0
91.   const down = v < 0
92.   return (
93.     <span className={`dash-delta ${down ? 'down' : up ? 'up' : 'flat'}`}>
94.       {down ? <ArrowDownRight size={13} /> : up ? <ArrowUpRight size={13} /> : null}
95.       {up ? '+' : ''}
96.       {fmtKwh(v)}
97.       {suffix}
98.     </span>
99.   )
100. }
101.

```

```

102. function TradingChart({ last7Days }) {
103.   const days = last7Days || []
104.   const maxKwh = Math.max(...days.map((d) => d.kwh || 0), 0)
105.   const W = 400
106.   const H = 150
107.   const pad = 10
108.   const innerW = W - pad * 2
109.   const innerH = H - pad * 2
110.   const hasData = days.some((d) => (d.kwh || 0) > 0)
111.
112.   if (!days.length) {
113.     return (
114.       <div className="dash-empty dash-empty-sm">
115.         <Activity size={22} />
116.         <p>No trading data available.</p>
117.       </div>
118.     )
119.   }
120.
121.   if (!hasData) {
122.     return (
123.       <div className="dash-empty dash-empty-sm">
124.         <Activity size={22} />
125.         <p>No completed trades in the last 7 days yet.</p>
126.         <span>Complete a reservation to see its energy here.</span>
127.       </div>
128.     )
129.   }
130.
131.   const points = days.map((d, i) => {
132.     const x = pad + (days.length === 1 ? innerW / 2 : (innerW * i) / (days.length - 1))
133.     const y = maxKwh > 0 ? pad + innerH - (innerH * (d.kwh || 0)) / maxKwh : pad + innerH
134.     return { x, y, kwh: d.kwh || 0, date: d.date }
135.   })
136.
137.   const line = points.map((p, i) => `${i === 0 ? 'M' :
138.   'L'}${p.x.toFixed(1)},${p.y.toFixed(1)}').join(' ')
139.   const area = `${line} L${points[points.length - 1].x.toFixed(1)},${pad + innerH}
140.   L${points[0].x.toFixed(1)},${pad + innerH} Z`
141.   return (
142.     <div className="dash-chart-wrap">
143.       <svg className="dash-chart" viewBox={`0 0 ${W} ${H}`} preserveAspectRatio="none" aria-
144.       hidden="true">
145.         <defs>
146.           <linearGradient id="dashArea" x1="0" y1="0" x2="0" y2="1">
147.             <stop offset="0%" stopColor="#2E6FD8" stopOpacity="0.28" />
148.             <stop offset="100%" stopColor="#2E6FD8" stopOpacity="0.02" />
149.           </linearGradient>
150.         </defs>
151.         <path d={area} fill="url(#dashArea)" />
152.         <path d={line} fill="none" stroke="#2E6FD8" strokeWidth="2.5" strokeLinecap="round"
153.         strokeLinejoin="round" />
154.         {points.map((p, i) =>
155.           p.kwh > 0 ? (
156.             <circle key={i} cx={p.x} cy={p.y} r="3.2" fill="fff" stroke="#2E6FD8"
157.             strokeWidth="2" />
158.           ) : null,
159.         )}
160.       </svg>
161.       <div className="dash-chart-labels">
162.         {points.map((p, i) => (
163.           <span key={i}>{p.date.slice(5)}</span>
164.         ))}
165.       </div>
166.     </div>

```

```

162.     </div>
163.   )
164. }
165.
166. export default function Dashboard() {
167.   const [data, setData] = useState(null)
168.   const [loading, setLoading] = useState(true)
169.   const [refreshing, setRefreshing] = useState(false)
170.   const [error, setError] = useState(null)
171.   const [lastUpdated, setLastUpdated] = useState(null)
172.
173.   const load = useCallback(async (skeleton = true) => {
174.     if (skeleton) setLoading(true)
175.     else setRefreshing(true)
176.     setError(null)
177.     try {
178.       const res = await getDashboard()
179.       setData(res.data)
180.       setLastUpdated(new Date())
181.     } catch (e) {
182.       setError(getErrorMessage(e))
183.     } finally {
184.       setLoading(false)
185.       setRefreshing(false)
186.     }
187.   }, [])
188.
189.   useEffect(() => {
190.     load(true)
191.     const timer = setInterval(() => load(false), POLL_INTERVAL)
192.     const onFocus = () => load(false)
193.     window.addEventListener('focus', onFocus)
194.     return () => {
195.       clearInterval(timer)
196.       window.removeEventListener('focus', onFocus)
197.     }
198.   }, [load])
199.
200.   const nodes = data?.nodes
201.   const prosumers = data?.prosumers
202.   const reservations = data?.reservations
203.   const energy = data?.energy
204.
205.   return (
206.     <div className="dash-dash">
207.       <div className="dash-container">
208.         <header className="dash-header">
209.           <div className="dash-title">
210.             <h1>Dashboard</h1>
211.             <p>Microgrid network overview and live trading activity.</p>
212.           </div>
213.           <div className="dash-header-actions">
214.             {lastUpdated && (
215.               <span className="dash-updated">
216.                 Last updated {lastUpdated.toLocaleTimeString([], { hour: '2-digit', minute: '2-
217. digit', second: '2-digit' })}
218.               </span>
219.             )}
220.             <button
221.               className={`dash-refresh ${refreshing ? 'is-spinning' : ''}`}
222.               type="button"
223.               title="Refresh dashboard"
224.               onClick={() => load(false)}
225.               disabled={refreshing}

```

```

226.         <RefreshCw size={16} />
227.     </button>
228. </div>
229. </header>
230.
231. {error ? (
232.     <div className="dash-error">
233.         <TriangleAlert size={26} />
234.         <h3>Could not load the dashboard</h3>
235.         <p>{error}</p>
236.         <button type="button" className="btn btn-solar" onClick={() => load(true)}>
237.             Retry
238.         </button>
239.     </div>
240. ) : loading && !data ? (
241.     <div className="dash-skeleton-grid">
242.         <div className="kpi-grid">
243.             {[0, 1, 2, 3].map((i) => (
244.                 <div
245.                     key={i}
246.                     className="flex flex-col overflow-hidden rounded-2xl border border-[#DBE7F2]
bg-white px-5 pt-5 pb-4 shadow-[0_22px_40px_-26px_rgba(13,47,74,0.3)]"
247.                 >
248.                     <div className="dash-skeleton h-12 w-12 rounded-2xl" />
249.                     <div className="mt-3 space-y-2">
250.                         <div className="dash-skeleton h-6 w-3/4" />
251.                         <div className="dash-skeleton h-3.5 w-1/2" />
252.                     </div>
253.                     <div className="mt-auto flex items-center gap-3 pt-3">
254.                         <div className="dash-skeleton h-5 w-2/5 rounded-full" />
255.                         <div className="dash-skeleton h-1.5 w-full rounded-full" />
256.                     </div>
257.                 </div>
258.             )))
259.         </div>
260.         <div className="dash-grid">
261.             <div className="dash-card"><Skeleton className="dash-skel-block" /></div>
262.             <div className="dash-card"><Skeleton className="dash-skel-block" /></div>
263.         </div>
264.     </div>
265. ) : (
266.     <>
267.         <div className="kpi-grid">
268.             <KpiCard
269.                 icon={<Network size={22} />}
270.                 tone="blue"
271.                 value={fmtCount(nodes?.total)}
272.                 label="Total Nodes"
273.                 sub="Active monitoring hubs"
274.                 pill={` ${fmtCount(nodes?.active)} active` }
275.                 barPct={pct(nodes?.active, nodes?.total)}
276.             />
277.             <KpiCard
278.                 icon={<Zap size={22} />}
279.                 tone="amber"
280.                 value={` ${fmtKwh(nodes?.capacityKwh)} kwh` }
281.                 label="Total Capacity"
282.                 sub="Across all microgrid nodes"
283.                 pill={` ${occupiedPct(nodes?.totalBatterySlots,
nodes?.availableBatterySlots).toFixed(1)}% utilized` }
284.                 barPct={occupiedPct(nodes?.totalBatterySlots, nodes?.availableBatterySlots)}
285.             />
286.             <KpiCard
287.                 icon={<Battery size={22} />}
288.                 tone="teal"

```

```

289.         value={` ${fmtCount(nodes?.availableBatterySlots)} /
${fmtCount(nodes?.totalBatterySlots)} `}
290.         label="Available Battery Slots"
291.         sub={` ${nodes?.availabilityPct?.toFixed(2)}% available`}
292.         pill="Free slots"
293.         barPct={nodes?.availabilityPct}
294.     />
295.     <KpiCard
296.         icon={<CircleCheck size={22} />}
297.         tone="emerald"
298.         value={` ${fmtCount(nodes?.active)} / ${fmtCount(nodes?.total)} `}
299.         label="Operational Status"
300.         sub={` ${fmtCount(nodes?.offline)} node${nodes?.offline === 1 ? '' : 's'}
offline`}
301.         pill={nodes?.offline > 0 ? 'Degraded' : 'Operational'}
302.         barPct={pct(nodes?.active, nodes?.total)}
303.         warning={
304.             nodes?.offline > 0 ? (
305.                 <span className="inline-flex items-center gap-1 rounded-full bg-[#FDF0DA]
px-2 py-1 text-[11px] font-extrabold text-[#B87104]">
306.                     <TriangleAlert size={13} /> {fmtCount(nodes?.offline)} offline
307.                 </span>
308.             ) : null
309.         }
310.     />
311. </div>
312.
313. <div className="dash-grid">
314.     <ReservationOverview />
315.
316.     <section className="dash-card">
317.         <div className="dash-card-head">
318.             <h3>Latest reservations</h3>
319.             <span className="dash-card-note">Most recent</span>
320.         </div>
321.         {reservations?.latest?.length ? (
322.             <div className="dash-table-wrap">
323.                 <table className="dash-table">
324.                     <thead>
325.                         <tr>
326.                             <th>Reservation ID</th>
327.                             <th>Date & Time</th>
328.                             <th>Status</th>
329.                         </tr>
330.                     </thead>
331.                     <tbody>
332.                         {reservations.latest.map((r) => {
333.                             const status = r.status || 'Pending'
334.                             const t = new Date(r.scheduledDateTime)
335.                             return (
336.                                 <tr key={r.id}>
337.                                     <td className="dash-id">{r.id}</td>
338.                                     <td className="dash-time">
339.                                         {t.toLocaleDateString()}
340.                                         <span>{t.toLocaleTimeString([], { hour: '2-digit', minute: '2-
digit' })}</span>
341.                                     </td>
342.                                     <td>
343.                                         <span className={ `dash-badge dash-badge-
${status.toLowerCase()} `}>{status}</span>
344.                                     </td>
345.                                 </tr>
346.                             )
347.                         })}
348.                     </tbody>

```

```

349.         </table>
350.     </div>
351.     ) : (
352.         <div className="dash-empty">
353.             <CircleCheck size={22} />
354.             <p>No reservations yet.</p>
355.         </div>
356.     )}
357. </section>
358. </div>
359.
360. <div className="dash-bottom">
361.     <section className="dash-card">
362.         <div className="dash-card-head">
363.             <h3>Energy traded today</h3>
364.             <span className="dash-card-icon amber"><TrendingUp size={15} /></span>
365.         </div>
366.         <div className="dash-bottom-value">{fmtKwh(energy?.todayKWh)}
367.         <small>kWh</small></div>
368.         <div className="dash-bottom-sub">
369.             vs yesterday · {fmtKwh(energy?.yesterdayKWh)} kWh
370.         </div>
371.         <div className="dash-bottom-footer">
372.             <DeltaPill pct={energy?.todayDeltaPct} />
373.             <span className="dash-bottom-hint">completed reservations</span>
374.         </div>
375.     </section>
376.
377.     <section className="dash-card">
378.         <div className="dash-card-head">
379.             <h3>Active prosumers</h3>
380.             <span className="dash-card-icon green"><Users size={15} /></span>
381.         </div>
382.         <div className="dash-bottom-value">{fmtCount(prosumers?.active)}</div>
383.         <div className="dash-bottom-sub">of {fmtCount(prosumers?.total)} registered
384.         prosumers</div>
385.         <div className="dash-bottom-footer">
386.             <span className="dash-pct">{fmtKwh(prosumers?.activePct)}%</span>
387.             <span className="dash-bottom-hint">account active</span>
388.         </div>
389.     </section>
390.
391.     <section className="dash-card dash-card-chart">
392.         <div className="dash-card-head">
393.             <h3>Trading activity</h3>
394.             <span className="dash-card-note">Last 7 days · kWh</span>
395.         </div>
396.         <TradingChart last7Days={energy?.last7Days} />
397.     </section>
398. </div>
399. )}
400. </div>
401. )
402. }
403.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\pages>Login.jsx

```

1. import React, { useRef, useState } from 'react'
2. import { useNavigate } from 'react-router-dom'
3. import { useAuth } from '../context/AuthContext'
4. import {

```



```

5.   IconSun,
6.   IconUser,
7.   IconLock,
8.   IconEye,
9.   IconEyeOff,
10.  IconArrowRight,
11.  IconMonitor,
12.  IconShieldCheck,
13.  IconSpinner,
14. } from '../components/ui/Icons'
15.
16. const PARTICLES = Array.from({ length: 16 })
17.
18. export default function Login() {
19.   const [username, setUsername] = useState('')
20.   const [password, setPassword] = useState('')
21.   const [showPassword, setShowPassword] = useState(false)
22.   const [remember, setRemember] = useState(true)
23.   const [forgotHint, setForgotHint] = useState(false)
24.   const [error, setError] = useState('')
25.   const [loading, setLoading] = useState(false)
26.   const { login } = useAuth()
27.   const navigate = useNavigate()
28.   const passwordRef = useRef(null)
29.
30.   async function handleSubmit(e) {
31.     e.preventDefault()
32.     setError('')
33.     setLoading(true)
34.     try {
35.       await login(username, password)
36.       navigate('/dashboard')
37.     } catch (err) {
38.       setError(err.response?.data?.message || 'Login failed. Please try again.')
39.     } finally {
40.       setLoading(false)
41.     }
42.   }
43.
44.   function togglePassword() {
45.     setShowPassword((s) => !s)
46.     if (passwordRef.current) {
47.       passwordRef.current.focus()
48.     }
49.   }
50.
51.   function handleForgot(e) {
52.     e.preventDefault()
53.     setForgotHint(true)
54.   }
55.
56.   return (
57.     <div className="login-page">
58.       <div className="banner-particles" aria-hidden="true">
59.         {PARTICLES.map((_, i) => (
60.           <span key={i} className="particle"></span>
61.         ))}
62.       </div>
63.
64.       <header className="login-header">
65.         <div className="brand anim-drop">
66.           <div className="brand-logo">
67.             <IconSun size={22} />
68.           </div>
69.           <div>

```

```

70.         <div className="brand-title">Smart Solar Microgrid</div>
71.         <div className="brand-sub">Trading System</div>
72.     </div>
73. </div>
74.
75.     <div className="status-badge anim-pop" role="status">
76.         <span className="status-dot-wrap"><span className="status-dot"></span></span>
77.         System online
78.     </div>
79. </header>
80.
81. <main className="login-center">
82.     <div className="login-card anim-card">
83.         <h1>Welcome back</h1>
84.         <p className="login-sub">Sign in to manage your microgrid network.</p>
85.
86.         {error && (
87.             <div className="error-banner shake" key={error} role="alert">{error}</div>
88.         )}
89.
90.         <form onSubmit={handleSubmit} noValidate>
91.             <div className="login-field">
92.                 <label className="form-label-sm" htmlFor="login-username">Username</label>
93.                 <div className="login-input-wrap">
94.                     <IconUser size={17} className="field-icon" />
95.                     <input
96.                         id="login-username"
97.                         type="text"
98.                         value={username}
99.                         onChange={(e) => setUsername(e.target.value)}
100.                        placeholder="Enter your username"
101.                        required
102.                        autoComplete="username"
103.                        autoFocus
104.                    />
105.                </div>
106.            </div>
107.
108.            <div className="login-field">
109.                <label className="form-label-sm" htmlFor="login-password">Password</label>
110.                <div className="login-input-wrap">
111.                    <IconLock size={17} className="field-icon" />
112.                    <input
113.                        id="login-password"
114.                        ref={passwordRef}
115.                        type={showPassword ? 'text' : 'password'}
116.                        value={password}
117.                        onChange={(e) => setPassword(e.target.value)}
118.                        placeholder="Enter your password"
119.                        required
120.                        autoComplete="current-password"
121.                    />
122.                    <button
123.                        type="button"
124.                        className="password-toggle"
125.                        onClick={togglePassword}
126.                        aria-label={showPassword ? 'Hide password' : 'Show password'}
127.                        tabIndex="0"
128.                    >
129.                        {showPassword ? <IconEyeOff size={17} /> : <IconEye size={17} />}
130.                    </button>
131.                </div>
132.            </div>
133.
134.            <div className="login-options">

```

```

135.         <label className="checkbox-row">
136.             <input
137.                 type="checkbox"
138.                 checked={remember}
139.                 onChange={(e) => setRemember(e.target.checked)}
140.             />
141.             <span>Remember me</span>
142.         </label>
143.         <button
144.             type="button"
145.             className="forgot-link"
146.             onClick={handleForgot}
147.             aria-label="Forgot password - not yet available"
148.         >
149.             Forgot password?
150.         </button>
151.     </div>
152.
153.     {forgotHint && (
154.         <div className="forgot-hint" role="note">
155.             Password recovery is not available yet. Contact your system
156.             administrator to reset your password.
157.         </div>
158.     )}
159.
160.     <button
161.         type="submit"
162.         className="btn btn-solar login-btn"
163.         disabled={loading}
164.     >
165.         {loading ? (
166.             <>
167.                 <IconSpinner size={16} /> Signing in...
168.             </>
169.         ) : (
170.             <>
171.                 Sign in <IconArrowRight size={16} />
172.             </>
173.         )}
174.     </button>
175. </form>
176.
177. <div className="login-divider">
178.     <span className="divider-line"></span>
179.     <IconLock size={15} className="divider-lock" />
180.     <span className="divider-line"></span>
181. </div>
182. <p className="login-secure">
183.     Secure access for Backoffice and Grid Operators
184. </p>
185. </div>
186. </main>
187.
188. <footer className="login-footer">
189.     <div className="login-features anim-rise delay-4">
190.         <div className="feature-item">
191.             <span className="feature-icon"><IconMonitor size={17} /></span>
192.             <span>Live grid monitoring</span>
193.         </div>
194.         <div className="feature-item">
195.             <span className="feature-icon"><IconShieldCheck size={17} /></span>
196.             <span>Secure reservations</span>
197.         </div>
198.         <div className="feature-item">
199.             <span className="feature-icon"><IconLock size={17} /></span>

```

```

200.         <span>Role-based access</span>
201.     </div>
202. </div>
203. </footer>
204. </div>
205. )
206. }
207.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\pages\Nodes.jsx

```

1. import React, { useEffect, useState } from 'react'
2. import {
3.   Network,
4.   Zap,
5.   Battery,
6.   CircleCheck,
7.   Search,
8.   ListFilter,
9.   MapPin,
10.  Pencil,
11.  Plus,
12.  Trash,
13. } from 'lucide-react'
14. import { useAuth } from '../context/AuthContext'
15. import { getNodes, createNode, updateNode, deactivateNode } from '../api/api'
16. import { getErrorMessage } from '../utils/errors'
17. import Modal from '../components/ui/Modal'
18. import { EmptyState, Spinner, ConfirmDialog } from '../components/ui/Widgets'
19. import { useToast } from '../components/ui/Toast'
20.
21. const EMPTY_FORM = {
22.   name: '',
23.   latitude: '',
24.   longitude: '',
25.   capacityKWh: '',
26.   totalBatterySlots: '',
27.   availableBatterySlots: '',
28.   operatingSchedule: 'Mon-Sun 06:00-20:00',
29. }
30.
31. const fmtCap = (value) =>
32.   new Intl.NumberFormat('en-US', { maximumFractionDigits: 2 }).format(Number(value) || 0)
33.
34. export default function Nodes() {
35.   const { user } = useAuth()
36.   const toast = useToast()
37.   const [nodes, setNodes] = useState([])
38.   const [search, setSearch] = useState('')
39.   const [statusFilter, setStatusFilter] = useState('all')
40.   const [loading, setLoading] = useState(true)
41.   const [formOpen, setFormOpen] = useState(false)
42.   const [editing, setEditing] = useState(null)
43.   const [form, setForm] = useState(EMPTY_FORM)
44.   const [confirmDeactivate, setConfirmDeactivate] = useState(null)
45.   const [saving, setSaving] = useState(false)
46.
47.   useEffect(() => {
48.     loadNodes()
49.   }, [])
50.
51.   async function loadNodes() {
52.     setLoading(true)
53.     try {

```

```

54.     const res = await getNodes()
55.     setNodes(res.data)
56.   } catch (err) {
57.     toast.error('Could not load nodes', getErrorMessage(err))
58.   } finally {
59.     setLoading(false)
60.   }
61. }
62.
63. function openCreate() {
64.   setEditing(null)
65.   setForm(EMPTY_FORM)
66.   setFormOpen(true)
67. }
68.
69. function openEdit(node) {
70.   setEditing(node)
71.   setForm({
72.     name: node.name,
73.     latitude: node.latitude,
74.     longitude: node.longitude,
75.     capacityKWh: node.capacityKWh,
76.     totalBatterySlots: node.totalBatterySlots,
77.     availableBatterySlots: node.availableBatterySlots,
78.     operatingSchedule: node.operatingSchedule?.join(',') || '',
79.   })
80.   setFormOpen(true)
81. }
82.
83. async function handleSubmit(e) {
84.   e.preventDefault()
85.   setSaving(true)
86.   const scheduleList = form.operatingSchedule
87.     .split(',')
88.     .map((s) => s.trim())
89.     .filter(Boolean)
90.
91.   try {
92.     if (editing) {
93.       await updateNode(editing.id, {
94.         name: form.name,
95.         capacityKWh: Number(form.capacityKWh),
96.         totalBatterySlots: Number(form.totalBatterySlots),
97.         availableBatterySlots: Number(form.availableBatterySlots),
98.         operatingSchedule: scheduleList,
99.       })
100.      toast.success('Node updated', `${form.name} was updated.`)
101.    } else {
102.      await createNode({
103.        name: form.name,
104.        latitude: Number(form.latitude),
105.        longitude: Number(form.longitude),
106.        capacityKWh: Number(form.capacityKWh),
107.        totalBatterySlots: Number(form.totalBatterySlots),
108.        operatingSchedule: scheduleList,
109.      })
110.      toast.success('Node registered', `${form.name} is now online.`)
111.    }
112.    setFormOpen(false)
113.    await loadNodes()
114.  } catch (err) {
115.    toast.error('Save failed', getErrorMessage(err))
116.  } finally {
117.    setSaving(false)
118.  }

```

```

119.   }
120.
121.   async function handleDeactivate(node) {
122.     try {
123.       await deactivateNode(node.id)
124.       toast.success('Node deactivated', `${node.name} was taken offline.`)
125.       await loadNodes()
126.     } catch (err) {
127.       toast.error('Could not deactivate node', getErrorMessage(err))
128.     }
129.   }
130.
131.   const q = search.trim().toLowerCase()
132.   const filtered = nodes.filter(
133.     (n) =>
134.     (statusFilter === 'all' || n.isActive) &&
135.     (!q ||
136.      n.name?.toLowerCase().includes(q) ||
137.      n.operatingSchedule?.join(' ').toLowerCase().includes(q) ||
138.      String(n.capacityKWh).includes(q)),
139.   )
140.
141.   const totalCapacity = nodes.reduce((sum, node) => sum + (Number(node.capacityKWh) || 0), 0)
142.   const availableBatterySlots = nodes.reduce((sum, node) => sum +
143.   (Number(node.availableBatterySlots) || 0), 0)
144.   const totalBatterySlots = nodes.reduce((sum, node) => sum + (Number(node.totalBatterySlots)
145.   || 0), 0)
146.   const activeNodes = nodes.filter((node) => node.isActive).length
147.   const batteryPct = totalBatterySlots ? (availableBatterySlots / totalBatterySlots) * 100 : 0
148.   const statusPct = nodes.length ? (activeNodes / nodes.length) * 100 : 0
149.
150.   return (
151.     <div className="nodes-dash">
152.       <div className="nodes-container">
153.         <div className="nodes-header">
154.           <div className="nodes-title">
155.             <h1>Microgrid Nodes</h1>
156.             <p>Solar grid hubs – location, capacity and battery storage slots.</p>
157.           </div>
158.           {user.role === 'Backoffice' ? (
159.             <button className="btn btn-solar" onClick={openCreate}>
160.               <Plus size={16} /> Register Node
161.             </button>
162.           ) : (
163.             <span className="cell-muted" style={{ fontSize: 13 }}>
164.               You can update battery slots on each node below.
165.             </span>
166.           )}
167.         </div>
168.
169.         <div className="nodes-cards">
170.           <div className="node-card">
171.             <div className="node-card-icon icon-royal">
172.               <Network size={20} />
173.             </div>
174.             <div className="node-card-value">{nodes.length}</div>
175.             <div className="node-card-label">Total Nodes</div>
176.             <div className="node-card-sub">Active monitoring hubs</div>
177.             <div className="node-card-bar">
178.               <div className="track">
179.                 <span className="fill fill-blue" style={{ width: `${Math.max(statusPct, 4)}%`
180.

```

```

181.
182.     <div className="node-card">
183.         <div className="node-card-icon icon-amber">
184.             <Zap size={20} />
185.         </div>
186.         <div className="node-card-value">{fmtCap(totalCapacity)} kWh</div>
187.         <div className="node-card-label">Total Capacity</div>
188.         <div className="node-card-sub">Across all nodes</div>
189.         <div className="node-card-bar">
190.             <div className="track">
191.                 <span className="fill fill-amber" style={{ width: '76%' }} />
192.             </div>
193.         </div>
194.     </div>
195.
196.     <div className="node-card">
197.         <div className="node-card-icon icon-teal">
198.             <Battery size={20} />
199.         </div>
200.         <div className="node-card-value">
201.             {fmtCap(availableBatterySlots)} / {fmtCap(totalBatterySlots)}
202.         </div>
203.         <div className="node-card-label">Available Battery Slots</div>
204.         <div className="node-card-sub">{Math.round(batteryPct)}% available</div>
205.         <div className="node-card-bar">
206.             <div className="track">
207.                 <span className="fill fill-teal" style={{ width: `${Math.max(batteryPct, 4)}%`
}} />
208.             </div>
209.         </div>
210.     </div>
211.
212.     <div className="node-card">
213.         <div className="node-card-icon icon-emerald">
214.             <CircleCheck size={20} />
215.         </div>
216.         <div className="node-card-value">
217.             {activeNodes} / {nodes.length || 0}
218.         </div>
219.         <div className="node-card-label">Operational Status</div>
220.         <div className="node-card-sub">
221.             {activeNodes === nodes.length && nodes.length > 0 ? 'All nodes active' : 'Some
nodes offline'}
222.         </div>
223.         <div className="node-card-bar">
224.             <div className="track">
225.                 <span className="fill fill-green" style={{ width: `${Math.max(statusPct, 4)}%`
}} />
226.             </div>
227.         </div>
228.     </div>
229. </div>
230.
231. <div className="nodes-panel">
232.     <div className="nodes-toolbar">
233.         <div className="nodes-search">
234.             <Search size={17} />
235.             <input
236.                 value={search}
237.                 onChange={(e) => setSearch(e.target.value)}
238.                 placeholder="Search nodes..."
239.             />
240.         </div>
241.         <span className="nodes-count">
242.             {filtered.length} of {nodes.length}

```



```

243.         </span>
244.         <button
245.             className={`nodes-filter${statusFilter === 'active' ? ' is-on' : ''}`}
246.             title={statusFilter === 'active' ? 'Showing active nodes – click to show all' :
'Filter nodes'}
247.             onClick={() => setStatusFilter(statusFilter === 'active' ? 'all' : 'active')}
248.         >
249.             <ListFilter size={17} />
250.         </button>
251.     </div>
252.
253.     {loading ? (
254.         <div className="loading-block">
255.             <Spinner size={20} /> Loading nodes...
256.         </div>
257.     ) : filtered.length === 0 ? (
258.         <EmptyState
259.             icon={<Network size={26} />}
260.             title={nodes.length === 0 ? 'No microgrid nodes registered' : 'No matching nodes'}
261.             hint="Register solar grid hubs to begin accepting energy bookings."
262.         />
263.     ) : (
264.         <div className="nodes-table-wrap">
265.             <table className="nodes-table">
266.                 <thead>
267.                     <tr>
268.                         <th>Node</th>
269.                         <th>Location</th>
270.                         <th>Capacity</th>
271.                         <th>Battery slots</th>
272.                         <th>Status</th>
273.                         <th style={{ width: 96 }}>Actions</th>
274.                     </tr>
275.                 </thead>
276.                 <tbody>
277.                     {filtered.map((n) => {
278.                         const slotsPct = n.totalBatterySlots
279.                         ? Math.round((n.availableBatterySlots / n.totalBatterySlots) * 100)
280.                         : 0
281.                         return (
282.                             <tr key={n.id}>
283.                                 <td data-label="Node">
284.                                     <div className="node-id">
285.                                         <div>
286.                                             <div className="node-name">{n.name}</div>
287.                                             <div className="node-sched">{n.operatingSchedule?.join(' · ')} ||
'No schedule set'</div>
288.                                         </div>
289.                                     </div>
290.                                 </td>
291.                                 <td data-label="Location">
292.                                     <span className="node-loc">
293.                                         <MapPin size={14} />
294.                                         {n.latitude?.toFixed(4)}, {n.longitude?.toFixed(4)}
295.                                     </span>
296.                                 </td>
297.                                 <td data-label="Capacity">
298.                                     <span className="node-cap">
299.                                         <Zap size={14} />
300.                                         {n.capacityKwh} kWh
301.                                     </span>
302.                                 </td>
303.                                 <td data-label="Battery slots">
304.                                     <div className="node-slots">
305.                                         <div className="node-slots-num">

```

```

306.                 <Battery size={14} />
307.                 {n.availableBatterySlots} <span className="node-slots-total">/
{n.totalBatterySlots}</span>
308.             </div>
309.             <div className="track">
310.                 <span
311.                     className={`fill fill-teal${slotsPct === 0 ? ' off' : ''}`}
312.                     style={{ width: `${Math.min(Math.max(slotsPct, 0), 100)}%` }}
313.                 />
314.             </div>
315.         </div>
316.     </td>
317.     <td data-label="Status">
318.         <span className={`pill ${n.isActive ? 'pill-on' : 'pill-off'}`}>
319.             {n.isActive ? 'Active' : 'Deactivated'}
320.         </span>
321.     </td>
322.     <td data-label="Actions">
323.         <div className="node-actions">
324.             <button className="node-action-btn" title="Edit node" onClick={() =>
openEdit(n)}>
325.                 <Pencil size={15} />
326.             </button>
327.             {n.isActive && user.role === 'Backoffice' && (
328.                 <button
329.                     className="node-action-btn danger"
330.                     title="Deactivate node"
331.                     onClick={() => setConfirmDeactivate(n)}
332.                 >
333.                     <Trash size={15} />
334.                 </button>
335.             )}
336.         </div>
337.     </td>
338. </tr>
339. )
340. }}}
341. </tbody>
342. </table>
343. </div>
344. )}
345. </div>
346.
347. <Modal
348.     open={formOpen}
349.     onClose={() => setFormOpen(false)}
350.     title={editing ? 'Edit microgrid node' : 'Register new microgrid node'}
351.     subtitle={
352.         editing
353.         ? user.role === 'Backoffice'
354.         ? 'Update capacity, battery slots and the operating schedule.'
355.         : 'Grid operators can update battery slot availability and schedules.'
356.         : 'Backoffice only – capture GPS location, capacity and battery storage.'
357.     }
358.     footer={
359.         <>
360.             <button type="button" className="btn btn-ghost" onClick={() => setFormOpen(false)}
disabled={saving}>
361.                 Cancel
362.             </button>
363.             <button type="submit" form="node-form" className="btn btn-solar"
disabled={saving}>
364.                 {saving ? <Spinner size={15} /> : editing ? 'Save changes' : 'Register node'}
365.             </button>
366.         </>

```

```

367.     }
368.   >
369.   <form id="node-form" onSubmit={handleSubmit}>
370.     <div className="row g-3">
371.       <div className="col-md-8">
372.         <label className="form-label-sm">Node name</label>
373.         <input
374.           className="form-control"
375.           value={form.name}
376.           onChange={(e) => setForm({ ...form, name: e.target.value })}
377.           required
378.           autoFocus
379.         />
380.       </div>
381.       <div className="col-md-4">
382.         <label className="form-label-sm">Capacity (kWh)</label>
383.         <input
384.           type="number"
385.           step="any"
386.           className="form-control"
387.           value={form.capacityKWh}
388.           onChange={(e) => setForm({ ...form, capacityKWh: e.target.value })}
389.           required
390.         />
391.       </div>
392.
393.       {!editing && (
394.         <>
395.           <div className="col-md-6">
396.             <label className="form-label-sm">
397.               <span style={{ display: 'inline-flex', alignItems: 'center', gap: 5 }}>
398.                 <MapPin size={12} /> Latitude
399.               </span>
400.             </label>
401.             <input
402.               type="number"
403.               step="any"
404.               className="form-control"
405.               value={form.latitude}
406.               onChange={(e) => setForm({ ...form, latitude: e.target.value })}
407.               required
408.             />
409.           </div>
410.           <div className="col-md-6">
411.             <label className="form-label-sm">Longitude</label>
412.             <input
413.               type="number"
414.               step="any"
415.               className="form-control"
416.               value={form.longitude}
417.               onChange={(e) => setForm({ ...form, longitude: e.target.value })}
418.               required
419.             />
420.           </div>
421.         </>
422.       )}
423.
424.       <div className="col-md-6">
425.         <label className="form-label-sm">Total battery slots</label>
426.         <input
427.           type="number"
428.           className="form-control"
429.           value={form.totalBatterySlots}
430.           onChange={(e) => setForm({ ...form, totalBatterySlots: e.target.value })}
431.           required

```

```

432.         />
433.     </div>
434.     {editing && (
435.         <div className="col-md-6">
436.             <label className="form-label-sm">Available battery slots</label>
437.             <input
438.                 type="number"
439.                 className="form-control"
440.                 value={form.availableBatterySlots}
441.                 onChange={(e) => setForm({ ...form, availableBatterySlots: e.target.value
}}}
442.             required
443.         />
444.     </div>
445. )}
446. <div className="col-12">
447.     <label className="form-label-sm">Operating schedule (comma separated)</label>
448.     <input
449.         className="form-control"
450.         value={form.operatingSchedule}
451.         onChange={(e) => setForm({ ...form, operatingSchedule: e.target.value }}}
452.         placeholder="e.g. Mon-Fri 06:00-20:00, Sat-Sun 08:00-18:00"
453.     />
454. </div>
455. </div>
456. </form>
457. </Modal>
458.
459. <ConfirmDialog
460.     open={!confirmDeactivate}
461.     title="Deactivate node"
462.     message={`"${confirmDeactivate?.name}" will be taken offline. Deactivation is blocked
if it has active energy reservations.`}
463.     confirmText="Deactivate"
464.     onCancel={() => setConfirmDeactivate(null)}
465.     onConfirm={() => (confirmDeactivate ? handleDeactivate(confirmDeactivate) :
Promise.resolve())}
466.     />
467. </div>
468. </div>
469. )
470. }
471. D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\pages\Prosumers.jsx
472. import React, { useEffect, useState } from 'react'
473. import {
474.     Users,
475.     Zap,
476.     ChartColumn,
477.     Search,
478.     ListFilter,
479.     Pencil,
480.     Trash,
481.     Check,
482.     Key,
483.     Plus,
484.     ChevronDown,
485.     TrendingUp,
486. } from 'lucide-react'
487. import { useAuth } from '../context/AuthContext'
488. import { getProsumers, createProsumer, updateProsumer, deactivateProsumer, reactivateProsumer
} from '../api/api'
489. import { getErrorMessage } from '../utils/errors'
490. import Modal from '../components/ui/Modal'
491. import { EmptyState, Spinner, ConfirmDialog } from '../components/ui/Widgets'
492. import { useToast } from '../components/ui/Toast'

```

```

493.
494. const EMPTY_FORM = { nic: '', fullName: '', email: '', phone: '', address: '', password: '' }
495.
496. export default function Prosumers() {
497.   const { user } = useAuth()
498.   const toast = useToast()
499.   const [prosumers, setProsumers] = useState([])
500.   const [search, setSearch] = useState('')
501.   const [statusFilter, setStatusFilter] = useState('all')
502.   const [filterOpen, setFilterOpen] = useState(false)
503.   const [loading, setLoading] = useState(true)
504.   const [formOpen, setFormOpen] = useState(false)
505.   const [editingNic, setEditingNic] = useState(null)
506.   const [form, setForm] = useState(EMPTY_FORM)
507.   const [confirmDeactivate, setConfirmDeactivate] = useState(null)
508.   const [saving, setSaving] = useState(false)
509.
510.   useEffect(() => {
511.     loadProsumers()
512.   }, [])
513.
514.   async function loadProsumers() {
515.     setLoading(true)
516.     try {
517.       const res = await getProsumers()
518.       setProsumers(res.data)
519.     } catch (err) {
520.       toast.error('Could not load prosumers', getErrorMessage(err))
521.     } finally {
522.       setLoading(false)
523.     }
524.   }
525.
526.   function openCreate() {
527.     setEditingNic(null)
528.     setForm(EMPTY_FORM)
529.     setFormOpen(true)
530.   }
531.
532.   function openEdit(p) {
533.     setEditingNic(p.nic)
534.     setForm({
535.       nic: p.nic,
536.       fullName: p.fullName,
537.       email: p.email,
538.       phone: p.phone,
539.       address: p.address,
540.       password: '',
541.     })
542.     setFormOpen(true)
543.   }
544.
545.   async function handleSubmit(e) {
546.     e.preventDefault()
547.     setSaving(true)
548.     try {
549.       if (editingNic) {
550.         await updateProsumer(editingNic, {
551.           fullName: form.fullName,
552.           email: form.email,
553.           phone: form.phone,
554.           address: form.address,
555.         })
556.         toast.success('Prosumer updated', `${form.fullName}'s profile was updated.`)
557.       } else {

```

```

558.         await createProsumer(form)
559.         toast.success('Prosumer registered', `${form.fullName} (${form.nic}) can now use the
mobile app.`)
560.     }
561.     setFormOpen(false)
562.     await loadProsumers()
563.   } catch (err) {
564.     toast.error('Save failed', getErrorMessage(err))
565.   } finally {
566.     setSaving(false)
567.   }
568. }
569.
570. async function handleDeactivate(p) {
571.   try {
572.     await deactivateProsumer(p.nic)
573.     toast.success('Prosumer deactivated', `${p.fullName} can no longer trade energy.`)
574.     await loadProsumers()
575.   } catch (err) {
576.     toast.error('Could not deactivate prosumer', getErrorMessage(err))
577.   }
578. }
579.
580. async function handleReactivate(p) {
581.   try {
582.     await reactivateProsumer(p.nic)
583.     toast.success('Prosumer reactivated', `${p.fullName} can trade energy again.`)
584.     await loadProsumers()
585.   } catch (err) {
586.     toast.error('Could not reactivate prosumer', getErrorMessage(err))
587.   }
588. }
589.
590. const q = search.trim().toLowerCase()
591. const filtered = prosumers.filter(
592.   (p) =>
593.     (statusFilter === 'all' || (statusFilter === 'active') === !p.isActive) &&
594.     (!q ||
595.       p.nic?.toLowerCase().includes(q) ||
596.       p.fullName?.toLowerCase().includes(q) ||
597.       p.email?.toLowerCase().includes(q) ||
598.       p.phone?.toLowerCase().includes(q)),
599. )
600.
601. const activeCount = prosumers.filter((p) => p.isActive).length
602.
603. const FILTER_OPTIONS = [
604.   { value: 'all', label: 'All statuses' },
605.   { value: 'active', label: 'Active' },
606.   { value: 'deactivated', label: 'Deactivated' },
607. ]
608.
609. return (
610.   <div className="pros-dash">
611.     <div className="pros-container">
612.       <div className="pros-header">
613.         <div className="pros-title">
614.           <h1>Prosumers</h1>
615.           <p>Property owners with solar arrays trading energy on the grid.</p>
616.         </div>
617.         <button className="btn btn-solar" onClick={openCreate}>
618.           <Plus size={16} /> New Prosumer
619.         </button>
620.       </div>
621.

```

```

622. <div className="pros-cards">
623.   <div className="pros-card">
624.     <div className="pros-card-body">
625.       <div className="pros-card-icon icon-blue">
626.         <Users size={20} />
627.       </div>
628.       <div className="pros-card-value">{prosumers.length}</div>
629.       <div className="pros-card-label">Total Prosumers</div>
630.       <div className="pros-card-sub">Registered property owners</div>
631.     </div>
632.     <div className="pros-users-illu" aria-hidden="true">
633.       <svg viewBox="0 0 76 48" fill="none">
634.         <g opacity="0.85">
635.           <circle cx="24" cy="13" r="8" fill="#9CC3F5" />
636.           <path d="M6 44c1.8-9 9.4-12.5 18-12.5S42 35 43.8 44H6Z" fill="#B8D4F8" />
637.           <circle cx="54" cy="19" r="5.5" fill="#C8E0FA" />
638.           <path d="M43 46.5c1.2-6.4 6.2-9 12-9s10.8 2.6 12 9H43Z" fill="#DCECFD" />
639.         </g>
640.       </svg>
641.     </div>
642.   </div>
643.
644.   <div className="pros-card">
645.     <div className="pros-card-body">
646.       <div className="pros-card-icon icon-green">
647.         <Zap size={20} />
648.       </div>
649.       <div className="pros-card-value">{activeCount}</div>
650.       <div className="pros-card-label">Active Prosumers</div>
651.       <div className="pros-card-sub">Currently trading energy</div>
652.     </div>
653.     <div className="pros-ring" aria-hidden="true">
654.       <svg viewBox="0 0 64 64">
655.         <circle className="pros-ring-track" cx="32" cy="32" r="26" />
656.         <circle className="pros-ring-fill" cx="32" cy="32" r="26" />
657.       </svg>
658.       <span>{Math.round((prosumers.length ? activeCount / prosumers.length : 0) *
100)}%</span>
659.     </div>
660.   </div>
661.
662.   <div className="pros-card">
663.     <div className="pros-card-body">
664.       <div className="pros-card-icon icon-amber">
665.         <ChartColumn size={20} />
666.       </div>
667.       <div className="pros-card-value">12.8 MWh</div>
668.       <div className="pros-card-label">Grid Contribution</div>
669.       <div className="pros-card-sub">Total energy supplied</div>
670.       <div className="pros-card-delta">
671.         <TrendingUp size={14} /> 12% vs. last month
672.       </div>
673.     </div>
674.     <div className="pros-bars" aria-hidden="true">
675.       <span style={{ height: '36%' }} />
676.       <span style={{ height: '54%' }} />
677.       <span style={{ height: '44%' }} />
678.       <span style={{ height: '70%' }} />
679.       <span style={{ height: '86%' }} />
680.     </div>
681.   </div>
682. </div>
683.
684. <div className="pros-panel">
685.   <div className="pros-toolbar">

```

```

686.     <div className="pros-search">
687.         <Search size={17} />
688.         <input
689.             value={search}
690.             onChange={(e) => setSearch(e.target.value)}
691.             placeholder="Search by NIC, name, email..."
692.         />
693.     </div>
694.     <span className="pros-count">
695.         {filtered.length} of {prosumers.length}
696.     </span>
697.     <div className="pros-filter-wrap">
698.         <button
699.             className={`pros-filter${filterOpen ? ' is-on' : ''}`}
700.             onClick={() => setFilterOpen(!filterOpen)}
701.         >
702.             <ListFilter size={16} />
703.             <span className="pros-filter-label">
704.                 {FILTER_OPTIONS.find((o) => o.value === statusFilter)?.label}
705.             </span>
706.             <ChevronDown size={15} />
707.         </button>
708.         {filterOpen && (
709.             <div className="pros-filter-menu">
710.                 {FILTER_OPTIONS.map((o) => (
711.                     <button
712.                         key={o.value}
713.                         className={statusFilter === o.value ? 'active' : ''}
714.                         onClick={() => {
715.                             setStatusFilter(o.value)
716.                             setFilterOpen(false)
717.                         }}
718.                     >
719.                         <span>{o.label}</span>
720.                         {statusFilter === o.value && <Check size={14} />}
721.                     </button>
722.                 )}}
723.             </div>
724.         )}
725.     </div>
726. </div>
727.
728. {loading ? (
729.     <div className="loading-block">
730.         <Spinner size={20} /> Loading prosumers...
731.     </div>
732. ) : filtered.length === 0 ? (
733.     <EmptyState
734.         icon={<Users size={26} />}
735.         title={prosumers.length === 0 ? 'No prosumers registered yet' : 'No matching
prosusers'}
736.         hint="Register a prosumer with their NIC to grant mobile app access."
737.     />
738. ) : (
739.     <div className="pros-table-wrap">
740.         <table className="pros-table">
741.             <thead>
742.                 <tr>
743.                     <th>NIC</th>
744.                     <th>Name</th>
745.                     <th>Contact</th>
746.                     <th>Status</th>
747.                     <th style={{ width: 96 }}>Actions</th>
748.                 </tr>
749.             </thead>

```



```

750.         <tbody>
751.             {filtered.map((p) => (
752.                 <tr key={p.nic}>
753.                     <td data-label="NIC">
754.                         <span className="pros-nic">{p.nic}</span>
755.                     </td>
756.                     <td data-label="Name">
757.                         <div className="pros-name">{p.fullName}</div>
758.                         <div className="pros-muted">{p.address}</div>
759.                     </td>
760.                     <td data-label="Contact">
761.                         <div className="pros-contact-main">{p.email}</div>
762.                         <div className="pros-muted">{p.phone}</div>
763.                     </td>
764.                     <td data-label="Status">
765.                         <span className={`pill ${p.isActive ? 'pill-on' : 'pill-off'}`}>
766.                             {p.isActive ? 'Active' : 'Deactivated'}
767.                         </span>
768.                     </td>
769.                     <td data-label="Actions">
770.                         <div className="pros-actions">
771.                             <button className="pros-action-btn" title="Edit prosumer" onClick={()
=> openEdit(p)}>
772.                                 <Pencil size={15} />
773.                             </button>
774.                             {p.isActive ? (
775.                                 <button
776.                                     className="pros-action-btn danger"
777.                                     title="Deactivate prosumer"
778.                                     onClick={() => setConfirmDeactivate(p)}
779.                                 >
780.                                     <Trash size={15} />
781.                                 </button>
782.                             ) : (
783.                                 user.role === 'Backoffice' && (
784.                                     <button
785.                                         className="pros-action-btn success"
786.                                         title="Reactivate prosumer"
787.                                         onClick={() => handleReactivate(p)}
788.                                     >
789.                                         <Check size={15} />
790.                                     </button>
791.                                 )
792.                             )}
793.                             </div>
794.                         </td>
795.                     </tr>
796.                 )}
797.             </tbody>
798.         </table>
799.     </div>
800. )}
801. </div>
802.
803. <Modal
804.     open={formOpen}
805.     onClose={() => setFormOpen(false)}
806.     title={editingNic ? 'Edit prosumer ${editingNic}` : 'Register new prosumer'}
807.     subtitle={
808.         editingNic
809.         ? 'NIC is the primary key and cannot be changed.'
810.         : 'The password lets the prosumer sign in to the mobile app.'
811.     }
812.     footer={
813.         <>

```

```

814.         <button type="button" className="btn btn-ghost" onClick={() => setFormOpen(false)}
disabled={saving}>
815.             Cancel
816.         </button>
817.         <button type="submit" form="prosumer-form" className="btn btn-solar"
disabled={saving}>
818.             {saving ? <Spinner size={15} /> : editingNic ? 'Save changes' : 'Create
prosumer'}
819.         </button>
820.     </>
821. }
822. >
823. <form id="prosumer-form" onSubmit={handleSubmit}>
824.     <div className="row g-3">
825.         <div className="col-md-4">
826.             <label className="form-label-sm">NIC (primary key)</label>
827.             <input
828.                 className="form-control"
829.                 value={form.nic}
830.                 disabled={!editingNic}
831.                 onChange={(e) => setForm({ ...form, nic: e.target.value })}
832.                 required
833.                 autoFocus
834.             />
835.         </div>
836.         <div className="col-md-8">
837.             <label className="form-label-sm">Full name</label>
838.             <input
839.                 className="form-control"
840.                 value={form.fullName}
841.                 onChange={(e) => setForm({ ...form, fullName: e.target.value })}
842.                 required
843.             />
844.         </div>
845.         <div className="col-md-6">
846.             <label className="form-label-sm">Email</label>
847.             <input
848.                 type="email"
849.                 className="form-control"
850.                 value={form.email}
851.                 onChange={(e) => setForm({ ...form, email: e.target.value })}
852.                 required
853.             />
854.         </div>
855.         <div className="col-md-6">
856.             <label className="form-label-sm">Phone</label>
857.             <input
858.                 className="form-control"
859.                 value={form.phone}
860.                 onChange={(e) => setForm({ ...form, phone: e.target.value })}
861.                 required
862.             />
863.         </div>
864.         <div className="col-12">
865.             <label className="form-label-sm">Address</label>
866.             <input
867.                 className="form-control"
868.                 value={form.address}
869.                 onChange={(e) => setForm({ ...form, address: e.target.value })}
870.                 required
871.             />
872.         </div>
873.         {!editingNic && (
874.             <div className="col-md-6">
875.                 <label className="form-label-sm">

```

```

876.         <span style={{ display: 'inline-flex', alignItems: 'center', gap: 5 }}>
877.             <Key size={12} /> Mobile app password
878.         </span>
879.     </label>
880.     <input
881.         type="password"
882.         className="form-control"
883.         value={form.password}
884.         onChange={(e) => setForm({ ...form, password: e.target.value })}
885.         required
886.         minLength={6}
887.     />
888. </div>
889. )}
890. </div>
891. </form>
892. </Modal>
893.
894. <ConfirmDialog
895.     open={!confirmDeactivate}
896.     title="Deactivate prosumer"
897.     message={`"${confirmDeactivate?.fullName}" (${confirmDeactivate?.nic}) will no longer
be able to reserve energy slots. Only a Backoffice officer can reactivate this account.`}
898.     confirmText="Deactivate"
899.     onCancel={() => setConfirmDeactivate(null)}
900.     onConfirm={() => (confirmDeactivate ? handleDeactivate(confirmDeactivate) :
Promise.resolve())}
901. />
902. </div>
903. </div>
904. )
905. }
906. D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\pages\Reservations.jsx
907. import React, { useEffect, useMemo, useState } from 'react'
908. import {
909.     CalendarDays,
910.     Clock3,
911.     MapPin,
912.     Eye,
913.     Pencil,
914.     Check,
915.     Trash,
916.     Search,
917.     ListFilter,
918.     Plus,
919.     QrCode,
920.     User,
921. } from 'lucide-react'
922. import {
923.     getReservations,
924.     getProsumers,
925.     getNodes,
926.     createReservation,
927.     updateReservation,
928.     cancelReservation,
929.     approveReservation,
930.     completeReservation,
931. } from '../api/api'
932. import { getErrorMessage } from '../utils/errors'
933. import Modal from '../components/ui/Modal'
934. import { EmptyState, Spinner, ConfirmDialog } from '../components/ui/Widgets'
935. import { useToast } from '../components/ui/Toast'
936.
937. const STATUS_CLASS = {
938.     Pending: 'res-pill-amber',

```

```

939.   Approved: 'res-pill-green',
940.   Completed: 'res-pill-blue',
941.   Cancelled: 'res-pill-red',
942. }
943.
944. const STATUS_BY_INDEX = { 0: 'Pending', 1: 'Approved', 2: 'Completed', 3: 'Cancelled' }
945.
946. const STATUS_ORDER = ['Pending', 'Approved', 'Completed', 'Cancelled']
947.
948. function normalizeStatus(status) {
949.   if (typeof status === 'string') return status
950.   return STATUS_BY_INDEX[status] || String(status ?? '')
951. }
952.
953. function formatDate(value) {
954.   if (!value) return '-'
955.   const d = new Date(value)
956.   return d.toLocaleDateString(undefined, { weekday: 'short', month: 'short', day: 'numeric',
year: 'numeric' })
957. }
958.
959. function formatTime(value) {
960.   if (!value) return '-'
961.   return new Date(value).toLocaleTimeString(undefined, { hour: 'numeric', minute: '2-digit' })
962. }
963.
964. function timeFromNow(value) {
965.   if (!value) return ''
966.   const diffMs = new Date(value).getTime() - Date.now()
967.   const mins = Math.round(diffMs / 60000)
968.   if (mins <= 0) return 'now'
969.   if (mins < 60) return `in ${mins}m`
970.   const hrs = Math.floor(mins / 60)
971.   if (hrs < 24) return `in ${hrs}h ${mins % 60}m`
972.   const days = Math.floor(hrs / 24)
973.   return `in ${days}d ${hrs % 24}h`
974. }
975.
976. function formatDuration(mins) {
977.   if (!mins) return '-'
978.   const h = Math.floor(mins / 60)
979.   const m = mins % 60
980.   if (h && m) return `${h}h ${m}m`
981.   if (h) return `${h}h`
982.   return `${m}m`
983. }
984.
985. const EMPTY_FORM = { prosumerNic: '', nodeId: '', scheduledDateTime: '', durationMinutes: 60 }
986.
987. export default function Reservations() {
988.   const toast = useToast()
989.   const [reservations, setReservations] = useState([])
990.   const [prosumers, setProsumers] = useState([])
991.   const [nodes, setNodes] = useState([])
992.   const [search, setSearch] = useState('')
993.   const [statusFilter, setStatusFilter] = useState('All')
994.   const [filterOpen, setFilterOpen] = useState(false)
995.   const [loading, setLoading] = useState(true)
996.   const [createOpen, setCreateOpen] = useState(false)
997.   const [editReservation, setEditReservation] = useState(null)
998.   const [detailReservation, setDetailReservation] = useState(null)
999.   const [form, setForm] = useState(EMPTY_FORM)
1000.   const [editForm, setEditForm] = useState({ scheduledDateTime: '', durationMinutes: 60 })
1001.   const [confirmCancel, setConfirmCancel] = useState(null)
1002.   const [saving, setSaving] = useState(false)

```

```

1003.
1004.   useEffect(() => {
1005.     loadAll()
1006.   }, [])
1007.
1008.   async function loadAll() {
1009.     setLoading(true)
1010.     try {
1011.       const [resRes, prosumerRes, nodeRes] = await Promise.all([getReservations(),
getProsumers(), getNodes()])
1012.       setReservations(resRes.data)
1013.       setProsumers(prosumerRes.data)
1014.       setNodes(nodeRes.data)
1015.     } catch (err) {
1016.       toast.error('Could not load reservations', getErrorMessage(err))
1017.     } finally {
1018.       setLoading(false)
1019.     }
1020.   }
1021.
1022.   const activeProsumers = useMemo(() => prosumers.filter((p) => p.isActive), [prosumers])
1023.   const activeNodes = useMemo(() => nodes.filter((n) => n.isActive), [nodes])
1024.
1025.   const prosumerMap = useMemo(() => {
1026.     const m = {}
1027.     prosumers.forEach((p) => (m[p.nic] = p))
1028.     return m
1029.   }, [prosumers])
1030.
1031.   const nodeMap = useMemo(() => {
1032.     const m = {}
1033.     nodes.forEach((n) => (m[n.id] = n))
1034.     return m
1035.   }, [nodes])
1036.
1037.   function prosumerOf(r) {
1038.     return r ? prosumerMap[r.prosumerNic] : undefined
1039.   }
1040.
1041.   function nodeOf(r) {
1042.     return r ? nodeMap[r.nodeId] : undefined
1043.   }
1044.
1045.   function nodeName(nodeId) {
1046.     return nodeMap[nodeId]?.name || nodeId
1047.   }
1048.
1049.   function openCreate() {
1050.     setForm(EMPTY_FORM)
1051.     setCreateOpen(true)
1052.   }
1053.
1054.   function openEdit(r) {
1055.     setEditReservation(r)
1056.     setEditForm({
1057.       scheduledDateTime: new Date(r.scheduledDateTime).toISOString().slice(0, 16),
1058.       durationMinutes: r.durationMinutes,
1059.     })
1060.   }
1061.
1062.   async function handleCreate(e) {
1063.     e.preventDefault()
1064.     setSaving(true)
1065.     try {
1066.       await createReservation({

```

```

1067.         ...form,
1068.         scheduledDateTime: new Date(form.scheduledDateTime).toISOString(),
1069.         durationMinutes: Number(form.durationMinutes),
1070.     })
1071.     toast.success('Reservation created', 'The energy slot has been booked.')
1072.     setCreateOpen(false)
1073.     await loadAll()
1074.   } catch (err) {
1075.     toast.error('Could not create reservation', getErrorMessage(err))
1076.   } finally {
1077.     setSaving(false)
1078.   }
1079. }
1080.
1081. async function handleUpdate(e) {
1082.   e.preventDefault()
1083.   setSaving(true)
1084.   try {
1085.     await updateReservation(editReservation.id, {
1086.       scheduledDateTime: new Date(editForm.scheduledDateTime).toISOString(),
1087.       durationMinutes: Number(editForm.durationMinutes),
1088.     })
1089.     toast.success('Reservation updated', 'The booking was rescheduled.')
1090.     setEditReservation(null)
1091.     await loadAll()
1092.   } catch (err) {
1093.     toast.error('Could not update reservation', getErrorMessage(err))
1094.   } finally {
1095.     setSaving(false)
1096.   }
1097. }
1098.
1099. async function handleAction(actionFn, r, successMsg) {
1100.   try {
1101.     await actionFn(r.id)
1102.     toast.success(successMsg)
1103.     await loadAll()
1104.   } catch (err) {
1105.     toast.error('Action failed', getErrorMessage(err))
1106.   }
1107. }
1108.
1109. const maxDateTime = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000).toISOString().slice(0,
1110. 16)
1111. const minDateTime = new Date(Date.now() + 5 * 60 * 1000).toISOString().slice(0, 16)
1112.
1113. const counts = useMemo(() => {
1114.   const c = { All: reservations.length }
1115.   STATUS_ORDER.forEach((s) => (c[s] = reservations.filter((r) => normalizeStatus(r.status)
1116.   === s).length))
1117.   return c
1118. }, [reservations])
1119.
1120. const q = search.trim().toLowerCase()
1121. const filtered = useMemo(
1122.   () =>
1123.     reservations.filter((r) => {
1124.       const status = normalizeStatus(r.status)
1125.       if (statusFilter !== 'All' && status !== statusFilter) return false
1126.       const prosumer = prosumerOf(r)
1127.       const node = nodeOf(r)
1128.       const haystack = [
1129.         r.prosumerNic,
1130.         prosumer?.fullName,
1131.         prosumer?.email,

```

```

1130.         node?.name,
1131.         status,
1132.         formatDate(r.scheduledDateTime),
1133.         r.qrCode,
1134.     ]
1135.     .filter(Boolean)
1136.     .join(' ')
1137.     .toLowerCase()
1138.     return !q || haystack.includes(q)
1139.   }},
1140.   [reservations, statusFilter, q, prosumers, nodes],
1141. )
1142.
1143. const nodeOptions = activeNodes.map((n) => (
1144.   <option key={n.id} value={n.id}>
1145.     {n.name} ({n.availableBatterySlots} slots free)
1146.   </option>
1147. ))
1148. const prosumerOptions = activeProsumers.map((p) => (
1149.   <option key={p.nic} value={p.nic}>
1150.     {p.fullName} ({p.nic})
1151.   </option>
1152. ))
1153.
1154. function DetailRow({ label, children }) {
1155.   return (
1156.     <li>
1157.       <div className="dt">
1158.         <div className="dl">{label}</div>
1159.       </div>
1160.       <div className="dv">{children}</div>
1161.     </li>
1162.   )
1163. }
1164.
1165. return (
1166.   <div className="res-dash">
1167.     <div className="res-container">
1168.       <div className="res-header">
1169.         <div className="res-title">
1170.           <h1>Reservations</h1>
1171.           <p>Power trading bookings across all microgrid nodes.</p>
1172.         </div>
1173.         <button className="btn btn-solar" onClick={openCreate}>
1174.           <Plus size={16} /> New Reservation
1175.         </button>
1176.       </div>
1177.
1178.       <div className="res-panel">
1179.         <div className="res-filters">
1180.           {[ 'All', ...STATUS_ORDER ].map((s) => (
1181.             <button
1182.               key={s}
1183.               className={ `res-filter res-filter-${s.toLowerCase()}${statusFilter === s ? '
selected' : '' }` }
1184.               onClick={() => setStatusFilter(s)}
1185.             >
1186.               {s}
1187.               <span className="res-badge">{counts[s] || 0}</span>
1188.             </button>
1189.           ))}
1190.         </div>
1191.
1192.         <div className="res-toolbar">
1193.           <div className="res-search">

```

```

1194.         <Search size={17} />
1195.         <input
1196.             value={search}
1197.             onChange={(e) => setSearch(e.target.value)}
1198.             placeholder="Search by prosumer, node, stat"
1199.         />
1200.     </div>
1201.     <span className="res-count">
1202.         {filtered.length} of {reservations.length}
1203.     </span>
1204.     <div className="res-filter-wrap">
1205.         <button
1206.             className={`res-filter-btn${filterOpen ? ' is-on' : ''}`}
1207.             title="Filter by status"
1208.             onClick={() => setFilterOpen(!filterOpen)}
1209.         >
1210.             <ListFilter size={16} />
1211.         </button>
1212.         {filterOpen && (
1213.             <div className="res-filter-menu">
1214.                 {[ 'All', ...STATUS_ORDER].map((s) => (
1215.                     <button
1216.                         key={s}
1217.                         className={statusFilter === s ? 'active' : ''}
1218.                         onClick={() => {
1219.                             setStatusFilter(s)
1220.                             setFilterOpen(false)
1221.                         }}
1222.                     >
1223.                         <span>{s}</span>
1224.                         {statusFilter === s && <Check size={13} />}
1225.                     </button>
1226.                 ))}
1227.             </div>
1228.         )}
1229.     </div>
1230.     <button
1231.         className="res-clear"
1232.         onClick={() => {
1233.             setStatusFilter('All')
1234.             setSearch('')
1235.         }}
1236.     >
1237.         Clear filters
1238.     </button>
1239. </div>
1240.
1241. {loading ? (
1242.     <div className="loading-block">
1243.         <Spinner size={20} /> Loading reservations...
1244.     </div>
1245. ) : filtered.length === 0 ? (
1246.     <EmptyState
1247.         icon={<CalendarDays size={26} />}
1248.         title={reservations.length === 0 ? 'No reservations yet' : 'No matching
reservations'}
1249.         hint="Create a booking on a prosumer's behalf, or wait for bookings from the
mobile app."
1250.     />
1251. ) : (
1252.     <div className="res-table-wrap">
1253.         <table className="res-table">
1254.             <thead>
1255.                 <tr>
1256.                     <th>Prosumer</th>

```



```

1252.         <th>Microgrid node</th>
1253.         <th>Scheduled</th>
1254.         <th>Duration</th>
1255.         <th>Status</th>
1256.         <th style={ { width: 168 } }>Actions</th>
1257.     </tr>
1258. </thead>
1259. <tbody>
1260.     {filtered.map((r) => {
1261.         const status = normalizeStatus(r.status)
1262.         const prosumer = prosumerOf(r)
1263.         const node = nodeOf(r)
1264.         return (
1265.             <tr key={r.id}>
1266.                 <td data-label="Prosumer">
1267.                     <div className="res-strong">{prosumer?.fullName ||
1268.
r.prosumerNic}</div>
1269.
1270.                     <div className="res-muted">
1271.                         NIC · {r.prosumerNic}
1272.                         {prosumer?.phone ? ` · ${prosumer.phone}` : ''}
1273.                     </div>
1274.                 </td>
1275.                 <td data-label="Microgrid node">
1276.                     <div className="res-strong res-node">{nodeName(r.nodeId)}</div>
1277.                     {node && (
1278.                         <div className="res-muted res-coords">
1279.                             <MapPin size={12} />
1280.                             {node.latitude?.toFixed(3)}, {node.longitude?.toFixed(3)}
1281.                         </div>
1282.                     )}
1283.                 </td>
1284.                 <td data-label="Scheduled">
1285.                     <div className="res-strong res-date">
1286.                         <CalendarDays size={14} /> {formatDate(r.scheduledDateTime)}
1287.                     </div>
1288.                     <div className="res-muted res-time">
1289.                         <Clock3 size={13} /> {formatTime(r.scheduledDateTime)} ·
1290. {timeFromNow(r.scheduledDateTime)}
1291.                     </div>
1292.                 </td>
1293.                 <td data-label="Duration">
1294.                     <span className="res-
1295. duration">{formatDuration(r.durationMinutes)}</span>
1296.                 </td>
1297.                 <td data-label="Status">
1298.                     <span className={ `res-pill ${STATUS_CLASS[status]} || 'res-pill-
1299. blue'` }>
1300.
<span className="dot" /> {status}
1301.                     </span>
1302.                     {r.qrCode && status === 'Approved' && (
1303.                         <div className="res-muted res-dispatch">
1304.                             <QrCode size={11} /> {r.qrCode}
1305.                         </div>
1306.                     )}
1307.                 </td>
1308.                 <td data-label="Actions">
1309.                     <div className="res-actions">
1310.                         <button className="res-action-btn" title="View details" onClick={()
1311. => setDetailReservation(r)}>
1312.
<Eye size={15} />
1313.                         </button>
1314.                         {(status === 'Pending' || status === 'Approved') && (
1315.                             <button className="res-action-btn" title="Reschedule" onClick={()

```

```

1316.           </button>
1317.       )}
1318.   {status === 'Pending' && (
1319.     <button
1320.       className="res-action-btn success"
1321.       title="Approve"
1322.       onClick={() => handleAction(approveReservation, r, 'Reservation
approved.')}
1323.     >
1324.       <Check size={15} />
1325.     </button>
1326.   )}
1327.   {status === 'Approved' && (
1328.     <button
1329.       className="res-action-btn success"
1330.       title="Complete"
1331.       onClick={() => handleAction(completeReservation, r, 'Marked as
completed.')}
1332.     >
1333.       <Check size={15} />
1334.     </button>
1335.   )}
1336.   {(status === 'Pending' || status === 'Approved') && (
1337.     <button className="res-action-btn danger" title="Cancel"
onClick={() => setConfirmCancel(r)}>
1338.       <Trash size={15} />
1339.     </button>
1340.   )}
1341. </div>
1342. </td>
1343. </tr>
1344. )
1345. }}
1346. </tbody>
1347. </table>
1348. </div>
1349. ))
1350. </div>
1351.
1352. {/* ---- Details modal ---- */}
1353. <Modal
1354.   open={!detailReservation}
1355.   onClose={() => setDetailReservation(null)}
1356.   size="md"
1357.   title="Reservation details"
1358.   subtitle={detailReservation ? `Booking #${detailReservation.id}` : undefined}
1359.   footer={
1360.     detailReservation &&
1361.     (() => {
1362.       const r = detailReservation
1363.       const status = normalizeStatus(r.status)
1364.       return (
1365.         <>
1366.           <button
1367.             type="button"
1368.             className="btn btn-ghost"
1369.             onClick={() => setDetailReservation(null)}
1370.           >
1371.             Close
1372.           </button>
1373.           {(status === 'Pending' || status === 'Approved') && (
1374.             <>
1375.               <button
1376.                 type="button"
1377.                 className="btn btn-outline-grid"

```

```

1378.         onClick={() => {
1379.             openEdit(r)
1380.             setDetailReservation(null)
1381.         }}
1382.     >
1383.         <Pencil size={15} /> Reschedule
1384.     </button>
1385.     <button
1386.         type="button"
1387.         className="btn btn-danger"
1388.         onClick={() => {
1389.             setConfirmCancel(r)
1390.             setDetailReservation(null)
1391.         }}
1392.     >
1393.         <Trash size={15} /> Cancel
1394.     </button>
1395. </>
1396.     )}
1397. </>
1398. )
1399. }}()
1400. }
1401. >
1402. {detailReservation &&
1403.     (() => {
1404.         const r = detailReservation
1405.         const status = normalizeStatus(r.status)
1406.         const prosumer = prosumerOf(r)
1407.         const node = nodeOf(r)
1408.         return (
1409.             <>
1410.                 <div className="mb-3">
1411.                     <span className={`res-pill res-pill-lg ${STATUS_CLASS[status]} || 'res-pill-blue'}`>
1412.                         <span className="dot" /> {status}
1413.                     </span>
1414.                 </div>
1415.
1416.                 {r.qrCode && (
1417.                     <div className="mb-3">
1418.                         <div className="qr-token">
1419.                             <QrCode size={16} /> {r.qrCode}
1420.                         </div>
1421.                         <div style={{ fontSize: 11.5, color: 'var(--ink-faint)', marginTop: 5 }}>
1422.                             Share this dispatch code with the grid operator to complete the energy
1423.                             </div>
1424.                     </div>
1425.                 )}
1426.
1427.                 <ul className="detail-list">
1428.                     <DetailRow label="Prosumer">
1429.                         <div className="cell-sub">
1430.                             <User size={14} style={{ color: 'var(--ink-faint)' }} />
1431.                             <span className="cell-strong">{prosumer?.fullName || 'Unknown
1432.                             prosumer'}</span>
1433.                         </div>
1434.                         <div className="cell-muted" style={{ fontSize: 12.5 }}>
1435.                             NIC · {r.prosumerNic}
1436.                             {prosumer?.email ? ` · ${prosumer.email}` : ''}
1437.                             {prosumer?.phone ? ` · ${prosumer.phone}` : ''}
1438.                         </div>
1439.                         {prosumer?.address && (
1440.                             <div className="cell-muted" style={{ fontSize: 12.5 }}>

```

```

1440.         {prosumer.address}
1441.     </div>
1442. })
1443.     <div className="cell-muted" style={{ fontSize: 12 }}>
1444.         {prosumer?.isActive ? 'Account active' : 'Account deactivated'}
1445.     </div>
1446. </DetailRow>
1447.
1448.     <DetailRow label="Microgrid node">
1449.         <div className="cell-strong">{node?.name || r.nodeId}</div>
1450.         {node && (
1451.             <div className="cell-muted" style={{ fontSize: 12.5 }}>
1452.                 <MapPin size={12} style={{ verticalAlign: '-1px' }} />
1453.                 {node.latitude}, {node.longitude} · {node.capacityKWh} kWh ·
1454.                 {node.availableBatterySlots}/{node.totalBatterySlots} slots
1455.             </div>
1456.         )}
1457.     </DetailRow>
1458.
1459.     <DetailRow label="Scheduled">
1460.         <div className="cell-sub">
1461.             <CalendarDays size={14} style={{ color: 'var(--ink-faint)' }} />
1462.             <span style={{ fontSize: 14, fontWeight: 600 }}>
1463.                 {formatDate(r.scheduledDateTime)} · {formatTime(r.scheduledDateTime)}
1464.             </span>
1465.             <span className="cell-muted" style={{ fontSize: 12.5 }}>
1466.                 ({timeFromNow(r.scheduledDateTime)})
1467.             </span>
1468.         </div>
1469.     </DetailRow>
1470.
1471.     <DetailRow label="Duration">
1472.         <span style={{ fontSize: 14 }}>{formatDuration(r.durationMinutes)}
1473.         ({r.durationMinutes} minutes)</span>
1474.     </DetailRow>
1475.
1476.     <DetailRow label="History">
1477.         <div className="cell-muted" style={{ fontSize: 12.5 }}>
1478.             Created {new Date(r.createdAt).toLocaleString()}
1479.             {r.updatedAt ? ` · last updated ${new
Date(r.updatedAt).toLocaleString()}` : ''}
1480.         </div>
1481.     </DetailRow>
1482. </ul>
1483. </>
1484. )
1485. }}()}}
1486. </Modal>
1487.
1488. { /* ----- Create modal ----- */}
1489. <Modal
1490.     open={createOpen}
1491.     onClose={() => setCreateOpen(false)}
1492.     title="Create reservation"
1493.     subtitle="Must be scheduled within the next 7 days."
1494.     footer={
1495.         <div>
1496.             <button type="button" className="btn btn-ghost" onClick={() =>
setCreateOpen(false)} disabled={saving}>
1497.                 Cancel
1498.             </button>
1499.             <button type="submit" form="create-form" className="btn btn-solar"
disabled={saving}>
1500.                 {saving ? <Spinner size={15} /> : 'Create reservation'}

```

```

1499.         </button>
1500.     </>
1501. }
1502. >
1503. <form id="create-form" onSubmit={handleCreate}>
1504.   <div className="row g-3">
1505.     <div className="col-md-6">
1506.       <label className="form-label-sm">Prosumer (NIC)</label>
1507.       <select
1508.         className="form-select"
1509.         value={form.prosumerNic}
1510.         onChange={(e) => setForm({ ...form, prosumerNic: e.target.value })}
1511.         required
1512.       >
1513.         <option value="">Select prosumer...</option>
1514.         {prosumerOptions}
1515.       </select>
1516.     </div>
1517.     <div className="col-md-6">
1518.       <label className="form-label-sm">Microgrid node</label>
1519.       <select
1520.         className="form-select"
1521.         value={form.nodeId}
1522.         onChange={(e) => setForm({ ...form, nodeId: e.target.value })}
1523.         required
1524.       >
1525.         <option value="">Select node...</option>
1526.         {nodeOptions}
1527.       </select>
1528.     </div>
1529.     <div className="col-md-7">
1530.       <label className="form-label-sm">Date & time</label>
1531.       <input
1532.         type="datetime-local"
1533.         className="form-control"
1534.         min={minDateTime}
1535.         max={maxDateTime}
1536.         value={form.scheduledDateTime}
1537.         onChange={(e) => setForm({ ...form, scheduledDateTime: e.target.value })}
1538.         required
1539.       />
1540.     </div>
1541.     <div className="col-md-5">
1542.       <label className="form-label-sm">Duration (min)</label>
1543.       <input
1544.         type="number"
1545.         className="form-control"
1546.         value={form.durationMinutes}
1547.         onChange={(e) => setForm({ ...form, durationMinutes: e.target.value })}
1548.         min="15"
1549.         step="15"
1550.         required
1551.       />
1552.     </div>
1553.   </div>
1554. </form>
1555. </Modal>
1556.
1557. {/* ---- Reschedule modal ---- */}
1558. <Modal
1559.   open={!editReservation}
1560.   onClose={() => setEditReservation(null)}
1561.   title="Reschedule reservation"
1562.   subtitle={
1563.     editReservation

```

```

1564.         ? `Booking for ${prosumerOf(editReservation)?.fullName ||
editReservation.prosumerNic} · ${nodeName(editReservation.nodeId)}`
1565.         : undefined
1566.     }
1567.     footer={
1568.         <>
1569.             <button type="button" className="btn btn-ghost" onClick={() =>
setEditReservation(null)} disabled={saving}>
1570.                 Cancel
1571.             </button>
1572.             <button type="submit" form="edit-form" className="btn btn-solar"
disabled={saving}>
1573.                 {saving ? <Spinner size={15} /> : 'Save changes'}
1574.             </button>
1575.         </>
1576.     }
1577. >
1578.     <form id="edit-form" onSubmit={handleUpdate}>
1579.         <div className="row g-3">
1580.             <div className="col-md-7">
1581.                 <label className="form-label-sm">New date & time</label>
1582.                 <input
1583.                     type="datetime-local"
1584.                     className="form-control"
1585.                     min={minDateTime}
1586.                     max={maxDateTime}
1587.                     value={editForm.scheduledDateTime}
1588.                     onChange={(e) => setEditForm({ ...editForm, scheduledDateTime: e.target.value
}}}
1589.                     required
1590.                 />
1591.             </div>
1592.             <div className="col-md-5">
1593.                 <label className="form-label-sm">Duration (min)</label>
1594.                 <input
1595.                     type="number"
1596.                     className="form-control"
1597.                     value={editForm.durationMinutes}
1598.                     onChange={(e) => setEditForm({ ...editForm, durationMinutes: e.target.value
}}}
1599.                     min="15"
1600.                     step="15"
1601.                     required
1602.                 />
1603.             </div>
1604.         </div>
1605.         <div className="mt-3" style={{ fontSize: 12.5, color: 'var(--ink-soft)' }}>
1606.             Reschedules require at least 12 hours' notice before the current scheduled time.
1607.         </div>
1608.     </form>
1609. </Modal>
1610.
1611.     <ConfirmDialog
1612.         open={!confirmCancel}
1613.         title="Cancel reservation"
1614.         message={`The booking for ${prosumerOf(confirmCancel)?.fullName ||
confirmCancel?.prosumerNic} will be cancelled and its battery slot released. Cancellations require
at least 12 hours' notice before the scheduled time.`}
1615.         confirmText="Cancel reservation"
1616.         onCancel={() => setConfirmCancel(null)}
1617.         onConfirm={() => (confirmCancel ? handleAction(cancelReservation, confirmCancel,
'Reservation cancelled.') : Promise.resolve())}
1618.     />
1619. </div>
1620. </div>

```

```
1621. )
1622. }
1623.
```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\pages\Users.jsx

```
1. import React, { useEffect, useMemo, useState } from 'react'
2. import { getUsers, createUser, updateUser, deactivateUser } from '../api/api'
3. import { getErrorMessage } from '../utils/errors'
4. import Modal from '../components/ui/Modal'
5. import { EmptyState, Spinner, ConfirmDialog } from '../components/ui/Widgets'
6. import { IconPlus } from '../components/ui/Icons'
7. import { useToast } from '../components/ui/Toast'
8. import {
9.   Search,
10.  ListFilter,
11.  Pencil,
12.  Trash,
13.  Mail,
14.  RefreshCw,
15.  Users as UsersIcon,
16.  TriangleAlert,
17. } from 'lucide-react'
18.
19. const EMPTY_FORM = { username: '', password: '', fullName: '', email: '', role: 'GridOperator'
20. }
21. const ALL_ROLES = 'All Roles'
22. const ALL_STATUSES = 'All Statuses'
23.
24. const ROLE_BADGE = {
25.   Backoffice: 'um-role-indigo',
26.   GridOperator: 'um-role-sky',
27. }
28.
29. const DEFAULT_BADGE = 'um-role-slate'
30.
31. export default function Users() {
32.   const toast = useToast()
33.   const [users, setUsers] = useState([])
34.   const [search, setSearch] = useState('')
35.   const [draftRole, setDraftRole] = useState(ALL_ROLES)
36.   const [draftStatus, setDraftStatus] = useState(ALL_STATUSES)
37.   const [appliedRole, setAppliedRole] = useState(ALL_ROLES)
38.   const [appliedStatus, setAppliedStatus] = useState(ALL_STATUSES)
39.   const [loading, setLoading] = useState(true)
40.   const [error, setError] = useState(null)
41.   const [formOpen, setFormOpen] = useState(false)
42.   const [editing, setEditing] = useState(null)
43.   const [form, setForm] = useState(EMPTY_FORM)
44.   const [confirm, setConfirm] = useState(null)
45.   const [saving, setSaving] = useState(false)
46.
47.   useEffect(() => {
48.     loadUsers()
49.   }, [])
50.
51.   async function loadUsers() {
52.     setLoading(true)
53.     setError(null)
54.     try {
55.       const res = await getUsers()
56.       setUsers(res.data)
57.     } catch (err) {
58.       setError(getErrorMessage(err))
59.     } finally {
```

```

60.     setLoading(false)
61.   }
62. }
63.
64. const roleOptions = useMemo(
65.   () => [...new Set(users.map((u) => u.role).filter(Boolean))],
66.   [users],
67. )
68.
69. function openCreate() {
70.   setEditing(null)
71.   setForm(EMPTY_FORM)
72.   setFormOpen(true)
73. }
74.
75. function openEdit(u) {
76.   setEditing(u)
77.   setForm({ username: u.username, password: '', fullName: u.fullName, email: u.email, role:
u.role })
78.   setFormOpen(true)
79. }
80.
81. async function handleSubmit(e) {
82.   e.preventDefault()
83.   setSaving(true)
84.   try {
85.     if (editing) {
86.       await updateUser(editing.id, {
87.         fullName: form.fullName,
88.         email: form.email,
89.         role: form.role,
90.         isActive: true,
91.       })
92.       toast.success('User updated', `${form.username} has been updated.`)
93.     } else {
94.       await createUser(form)
95.       toast.success('User created', `${form.username} can now sign in.`)
96.     }
97.     setFormOpen(false)
98.     await loadUsers()
99.   } catch (err) {
100.    toast.error('Save failed', getErrorMessage(err))
101.   } finally {
102.     setSaving(false)
103.   }
104. }
105.
106. async function handleDeactivate(u) {
107.   try {
108.     await deactivateUser(u.id)
109.     toast.success('User deactivated', `${u.username} can no longer sign in.`)
110.     await loadUsers()
111.   } catch (err) {
112.     toast.error('Could not deactivate user', getErrorMessage(err))
113.   }
114. }
115.
116. const q = search.trim().toLowerCase()
117. const filtered = users.filter((u) => {
118.   if (
119.     q &&
120.     ![u.username, u.fullName, u.email, u.role].some((v) => (v ||
'').toLowerCase().includes(q))
121.   ) {
122.     return false

```



```

123.     }
124.     if (appliedRole !== ALL_ROLES && u.role !== appliedRole) return false
125.     if (appliedStatus === 'Active' && !u.isActive) return false
126.     if (appliedStatus === 'Deactivated' && u.isActive) return false
127.     return true
128. })
129.
130. const filtersActive = appliedRole !== ALL_ROLES || appliedStatus !== ALL_STATUSES
131.
132. function applyFilters() {
133.     setAppliedRole(draftRole)
134.     setAppliedStatus(draftStatus)
135. }
136.
137. function clearFilters() {
138.     setSearch('')
139.     setDraftRole(ALL_ROLES)
140.     setDraftStatus(ALL_STATUSES)
141.     setAppliedRole(ALL_ROLES)
142.     setAppliedStatus(ALL_STATUSES)
143. }
144.
145. return (
146.     <div className="um-dash">
147.         <div className="um-container">
148.             <div className="um-header">
149.                 <div className="um-title">
150.                     <h1>User Management</h1>
151.                     <p>Create and manage Backoffice and Grid Operator accounts.</p>
152.                 </div>
153.                 <button className="btn btn-solar" onClick={openCreate}>
154.                     <IconPlus size={16} /> New User
155.                 </button>
156.             </div>
157.
158.             <div className="um-panel">
159.                 <div className="um-toolbar">
160.                     <div className="um-search">
161.                         <Search size={17} />
162.                         <input
163.                             value={search}
164.                             onChange={(e) => setSearch(e.target.value)}
165.                             placeholder="Search users..."
166.                         />
167.                     </div>
168.                     <span className="um-count">
169.                         {filtered.length} of {users.length} users
170.                     </span>
171.
172.                     <select
173.                         className="um-select"
174.                         value={draftRole}
175.                         onChange={(e) => setDraftRole(e.target.value)}
176.                     >
177.                         <option value={ALL_ROLES}>{ALL_ROLES}</option>
178.                         {roleOptions.map((r) => (
179.                             <option key={r} value={r}>
180.                                 {r}
181.                             </option>
182.                         ))}
183.                     </select>
184.
185.                     <select
186.                         className="um-select"
187.                         value={draftStatus}

```

```

188.         onChange={(e) => setDraftStatus(e.target.value)}
189.     >
190.         <option value={ALL_STATUSES}>{ALL_STATUSES}</option>
191.         <option value="Active">Active</option>
192.         <option value="Deactivated">Deactivated</option>
193.     </select>
194.
195.     <button
196.         className={`um-filter${filtersActive ? ' is-on' : ''}`}
197.         title="Apply role and status filters"
198.         onClick={applyFilters}
199.     >
200.         <ListFilter size={16} /> Filter
201.     </button>
202. </div>
203.
204. {loading ? (
205.     <div className="loading-block">
206.         <Spinner size={20} /> Loading users...
207.     </div>
208. ) : error ? (
209.     <div className="um-error">
210.         <span className="um-error-icon">
211.             <TriangleAlert size={20} />
212.         </span>
213.         <div className="um-error-body">
214.             <strong>Could not load users</strong>
215.             <span>{error}</span>
216.         </div>
217.         <button className="btn btn-ghost" onClick={() => loadUsers()}>
218.             <RefreshCw size={15} /> Retry
219.         </button>
220.     </div>
221. ) : filtered.length === 0 ? (
222.     <div className="um-empty">
223.         <EmptyState
224.             icon={<UsersIcon size={26} />}
225.             title={users.length === 0 ? 'No users yet' : 'No matching users'}
226.             hint={
227.                 users.length === 0
228.                 ? 'Create web application accounts from the New User button.'
229.                 : 'Adjust or clear the search and filters to see more results.'
230.             }
231.         />
232.         {users.length > 0 && filtersActive && (
233.             <button className="btn btn-ghost" onClick={clearFilters}>
234.                 Clear filters
235.             </button>
236.         )}
237.     </div>
238. ) : (
239.     <div className="um-table-wrap">
240.         <table className="um-table">
241.             <thead>
242.                 <tr>
243.                     <th>User</th>
244.                     <th>Contact</th>
245.                     <th>Role</th>
246.                     <th>Status</th>
247.                     <th style={{ width: 96 }}>Actions</th>
248.                 </tr>
249.             </thead>
250.             <tbody>
251.                 {filtered.map((u) => (
252.                     <tr key={u.id}>

```

```

253. <td data-label="User">
254.     <div>
255.         <div className="um-name">{u.username}</div>
256.         <div className="um-fullname">{u.fullName}</div>
257.     </div>
258. </td>
259. <td data-label="Contact">
260.     <span className="um-email">
261.         <Mail size={14} /> {u.email}
262.     </span>
263. </td>
264. <td data-label="Role">
265.     <span className={`um-role ${ROLE_BADGE[u.role] || DEFAULT_BADGE}`}>
266.         {u.role}
267.     </span>
268. </td>
269. <td data-label="Status">
270.     <span className={`pill ${u.isActive ? 'pill-on' : 'pill-off'}`}>
271.         {u.isActive ? 'Active' : 'Deactivated'}
272.     </span>
273. </td>
274. <td data-label="Actions">
275.     <div className="um-actions">
276.         <button className="um-action-btn" title="Edit user" onClick={() =>
openEdit(u)}>
277.             <Pencil size={15} />
278.         </button>
279.         {u.isActive && (
280.             <button
281.                 className="um-action-btn danger"
282.                 title="Deactivate user"
283.                 onClick={() => setConfirm(u)}
284.             >
285.                 <Trash size={15} />
286.             </button>
287.         )}
288.     </div>
289. </td>
290. </tr>
291. </tbody>
292. </table>
293. </div>
294. </div>
295. </div>
296. </div>
297. </div>
298.
299. <Modal
300.     open={formOpen}
301.     onClose={() => setFormOpen(false)}
302.     title={editing ? 'Edit user' : 'Create new user'}
303.     subtitle={editing ? form.username : 'Choose a role – Backoffice or Grid Operator.'}
304.     footer={
305.         <
306.             <button type="button" className="btn btn-ghost" onClick={() => setFormOpen(false)}
disabled={saving}>
307.                 Cancel
308.             </button>
309.             <button type="submit" form="user-form" className="btn btn-solar" disabled={saving}>
310.                 {saving ? <Spinner size={15} /> : editing ? 'Save changes' : 'Create user'}
311.             </button>
312.         </>
313.     }
314. >
315.     <form id="user-form" onSubmit={handleSubmit}>

```

```

316. <div className="row g-3">
317.   <div className="col-md-6">
318.     <label className="form-label-sm">Username</label>
319.     <input
320.       className="form-control"
321.       value={form.username}
322.       disabled={!editing}
323.       onChange={(e) => setForm({ ...form, username: e.target.value })}
324.       required
325.       autoFocus
326.     />
327.   </div>
328.   <div className="col-md-6">
329.     <label className="form-label-sm">Full name</label>
330.     <input
331.       className="form-control"
332.       value={form.fullName}
333.       onChange={(e) => setForm({ ...form, fullName: e.target.value })}
334.       required
335.     />
336.   </div>
337.   <div className="col-md-6">
338.     <label className="form-label-sm">Email</label>
339.     <input
340.       type="email"
341.       className="form-control"
342.       value={form.email}
343.       onChange={(e) => setForm({ ...form, email: e.target.value })}
344.       required
345.     />
346.   </div>
347.   {!editing && (
348.     <div className="col-md-6">
349.       <label className="form-label-sm">Password</label>
350.       <input
351.         type="password"
352.         className="form-control"
353.         value={form.password}
354.         onChange={(e) => setForm({ ...form, password: e.target.value })}
355.         required
356.         minLength={6}
357.       />
358.     </div>
359.   )}
360.   <div className="col-md-6">
361.     <label className="form-label-sm">Role</label>
362.     <select
363.       className="form-select"
364.       value={form.role}
365.       onChange={(e) => setForm({ ...form, role: e.target.value })}
366.     >
367.       {roleOptions.length > 0
368.         ? roleOptions.map((r) => (
369.           <option key={r} value={r}>
370.             {r === 'Backoffice' ? 'Backoffice - system administration' : `${r} -
operational tools`}
371.           </option>
372.         ))
373.         : (
374.           <>
375.             <option value="Backoffice">Backoffice - system administration</option>
376.             <option value="GridOperator">Grid Operator - operational tools</option>
377.           </>
378.         )}
379.     </select>

```

```

380.         </div>
381.     </div>
382. </form>
383. </Modal>
384.
385.     <ConfirmDialog
386.         open={!confirm}
387.         title="Deactivate user"
388.         message={`The account "${confirm?.username}" will no longer be able to sign in. You can
update it later to restore access.`}
389.         confirmText="Deactivate"
390.         onCancel={() => setConfirm(null)}
391.         onConfirm={() => (confirm ? handleDeactivate(confirm) : Promise.resolve())}
392.     />
393. </div>
394. )
395. }
396.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\utils\errors.js

```

1. export function getErrorMessage(err, fallback = 'Something went wrong.') {
2.     return err?.response?.data?.message || err?.response?.data?.title || err?.message || fallback
3. }
4.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\App.jsx

```

1. import React from 'react'
2. import { Routes, Route, Navigate } from 'react-router-dom'
3. import Sidebar from './components/Sidebar'
4. import ProtectedRoute from './components/ProtectedRoute'
5. import Login from './pages/Login'
6. import Dashboard from './pages/Dashboard'
7. import Prosumers from './pages/Prosumers'
8. import Nodes from './pages/Nodes'
9. import Reservations from './pages/Reservations'
10. import Users from './pages/Users'
11.
12. function AuthenticatedLayout({ children, flush }) {
13.     return (
14.         <div className="app-shell">
15.             <Sidebar />
16.             <div className="main-wrap">
17.                 <main className={`main-content ${flush ? 'main-content-flush' : ''}`}>{children}</main>
18.             </div>
19.         </div>
20.     )
21. }
22.
23. export default function App() {
24.     return (
25.         <Routes>
26.             <Route path="/login" element={<Login />} />
27.
28.             <Route
29.                 path="/dashboard"
30.                 element={
31.                     <ProtectedRoute>
32.                         <AuthenticatedLayout flush>
33.                             <Dashboard />
34.                         </AuthenticatedLayout>
35.                     </ProtectedRoute>
36.                 }
37.             />
38.
39.             <Route

```

```

40.     path="/prosumers"
41.     element={
42.         <ProtectedRoute>
43.             <AuthenticatedLayout>
44.                 <Prosumers />
45.             </AuthenticatedLayout>
46.         </ProtectedRoute>
47.     }
48. />
49.
50. <Route
51.     path="/nodes"
52.     element={
53.         <ProtectedRoute>
54.             <AuthenticatedLayout>
55.                 <Nodes />
56.             </AuthenticatedLayout>
57.         </ProtectedRoute>
58.     }
59. />
60.
61. <Route
62.     path="/reservations"
63.     element={
64.         <ProtectedRoute allowedRoles={['GridOperator']}>
65.             <AuthenticatedLayout>
66.                 <Reservations />
67.             </AuthenticatedLayout>
68.         </ProtectedRoute>
69.     }
70. />
71.
72. <Route
73.     path="/users"
74.     element={
75.         <ProtectedRoute allowedRoles={['Backoffice']}>
76.             <AuthenticatedLayout>
77.                 <Users />
78.             </AuthenticatedLayout>
79.         </ProtectedRoute>
80.     }
81. />
82.
83. <Route path="/" element={<Navigate to="/dashboard" replace />} />
84. <Route path="*" element={<Navigate to="/dashboard" replace />} />
85. </Routes>
86. )
87. }
88.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\src\main.jsx

```

1. import React from 'react'
2. import ReactDOM from 'react-dom/client'
3. import { BrowserRouter } from 'react-router-dom'
4. import App from './App.jsx'
5. import 'bootstrap/dist/css/bootstrap.min.css'
6. import './index.css'
7. import { AuthProvider } from './context/AuthContext.jsx'
8. import { ToastProvider } from './components/ui/Toast.jsx'
9.
10. ReactDOM.createRoot(document.getElementById('root')).render(
11.     <React.StrictMode>
12.         <BrowserRouter>
13.             <AuthProvider>
14.                 <ToastProvider>

```

```

15.         <App />
16.     </ToastProvider>
17. </AuthProvider>
18. </BrowserRouter>
19. </React.StrictMode>,
20. )
21.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\Frontend\vite.config.js

```

1. import { defineConfig } from 'vite'
2. import react from '@vitejs/plugin-react'
3. import tailwindcss from '@tailwindcss/vite'
4.
5. // Vite config for the Smart Solar Microgrid Trading System web application.
6. export default defineConfig({
7.   base: './',
8.   plugins: [react(), tailwindcss()],
9.   server: {
10.     port: 5173
11.   }
12. })
13.

```

Web-service

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Configuration\SolarGridDatabaseSettings.cs

```

1. // -----
2. // File: SolarGridDatabaseSettings.cs
3. // Purpose: Strongly typed options class bound from the "Database" section of
4. // appsettings, holding the MongoDB connection string plus the collection names
5. // used across the API (users, prosumers, nodes, reservations).
6. // -----
7.
8. namespace SolarGrid.Api.Configuration;
9.
10. public class SolarGridDatabaseSettings
11. {
12.     public string ConnectionString { get; set; } = null!;
13.     public string DatabaseName { get; set; } = null!;
14.     public string AppUsersCollectionName { get; set; } = null!;
15.     public string ProsumersCollectionName { get; set; } = null!;
16.     public string NodesCollectionName { get; set; } = null!;
17.     public string ReservationsCollectionName { get; set; } = null!;
18. }
19.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Controllers\AuthController.cs

```

1. // -----
2. // File: AuthController.cs
3. // Purpose: Exposes the authentication endpoint (POST api/auth/login) used by
4. // Backoffice officers and Grid Operators. Delegates credential validation to
5. // AuthService and returns the JWT response or 401 on failure.
6. // -----
7.
8. using Microsoft.AspNetCore.Mvc;
9. using SolarGrid.Api.Models;

```

```

10. using SolarGrid.Api.Services;
11.
12. namespace SolarGrid.Api.Controllers;
13.
14. [ApiController]
15. [Route("api/auth")]
16. public class AuthController : ControllerBase
17. {
18.     private readonly AuthService _authService;
19.
20.     public AuthController(AuthService authService)
21.     {
22.         _authService = authService;
23.     }
24.
25.     /// <summary>Login for Backoffice officers and Grid Operators.</summary>
26.     [HttpPost("login")]
27.     public async Task<IActionResult> Login([FromBody] LoginRequest request)
28.     {
29.         var response = await _authService.LoginAsync(request);
30.         if (response == null)
31.             return Unauthorized(new { message = "Invalid credentials or account is deactivated."
});
32.
33.         return Ok(response);
34.     }
35. }

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Controllers\DashboardController.cs

```

1. // -----
2. // File: DashboardController.cs
3. // Purpose: Serves live analytics for the Backoffice dashboard (GET api/dashboard).
4. // Normalises the requested period filter and delegates the aggregation work to
5. // DashboardService, which computes all values from the current database state.
6. // -----
7.
8. using Microsoft.AspNetCore.Mvc;
9. using SolarGrid.Api.Services;
10.
11. namespace SolarGrid.Api.Controllers;
12.
13. [ApiController]
14. [Route("api/dashboard")]
15. public class DashboardController : ControllerBase
16. {
17.     private readonly DashboardService _dashboardService;
18.
19.     public DashboardController(DashboardService dashboardService)
20.     {
21.         _dashboardService = dashboardService;
22.     }
23.
24.     /// <summary>
25.     /// Live analytics for the Backoffice dashboard. All values are computed
26.     /// from the current database state.
27.     /// </summary>
28.     /// <param name="period">all | today | 7d – scopes the reservation counts.</param>
29.     [HttpGet]
30.     public async Task<IActionResult> Get([FromQuery] string period = "all")
31.     {
32.         var normalized = period.ToLowerInvariant() switch

```



```

33.         {
34.             "today" => "today",
35.             "7d" or "7" or "week" or "thisweek" => "week",
36.             "month" or "thismonth" => "month",
37.             _ => "all",
38.         };
39.         return Ok(await _dashboardService.GetAnalyticsAsync(normalized));
40.     }
41. }
42.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Controllers\NodesController.cs

```

1. // -----
2. // File: NodesController.cs
3. // Purpose: CRUD endpoints for microgrid nodes (api/nodes): list, fetch by id,
4. // create, update and soft-deactivate. All business logic lives in NodeService;
5. // the controller only maps requests to service calls and HTTP results.
6. // -----
7.
8. using Microsoft.AspNetCore.Mvc;
9. using SolarGrid.Api.Models;
10. using SolarGrid.Api.Services;
11.
12. namespace SolarGrid.Api.Controllers;
13.
14. [ApiController]
15. [Route("api/nodes")]
16. public class NodesController : ControllerBase
17. {
18.     private readonly NodeService _nodeService;
19.
20.     public NodesController(NodeService nodeService)
21.     {
22.         _nodeService = nodeService;
23.     }
24.
25.     [HttpGet]
26.     public async Task<IActionResult> GetAll() =>
27.         Ok(await _nodeService.GetAllAsync());
28.
29.     [HttpGet("{id}")]
30.     public async Task<IActionResult> GetById(string id)
31.     {
32.         var node = await _nodeService.GetByIdAsync(id);
33.         if (node == null)
34.             return NotFound(new { message = "Node not found." });
35.
36.         return Ok(node);
37.     }
38.
39.     [HttpPost]
40.     public async Task<IActionResult> Create([FromBody] CreateNodeRequest request)
41.     {
42.         var node = await _nodeService.CreateAsync(request);
43.         return CreatedAtAction(nameof(GetById), new { id = node.Id }, node);
44.     }
45.
46.     [HttpPut("{id}")]
47.     public async Task<IActionResult> Update(string id, [FromBody] UpdateNodeRequest request)

```

```

48.     {
49.         var node = await _nodeService.UpdateAsync(id, request);
50.         if (node == null)
51.             return NotFound(new { message = "Node not found." });
52.
53.         return Ok(node);
54.     }
55.
56.     [HttpPatch("{id}/deactivate")]
57.     public async Task<IActionResult> Deactivate(string id)
58.     {
59.         var success = await _nodeService.DeactivateAsync(id);
60.         if (!success)
61.             return NotFound(new { message = "Node not found." });
62.
63.         return Ok(new { message = "Node deactivated successfully." });
64.     }
65. }
66.
67.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Controllers\ProsumersController.cs

```

1.  // -----
2.  // File: ProsumersController.cs
3.  // Purpose: Endpoints for managing prosumers (api/prosumers), keyed by NIC.
4.  // Handles listing, registration, profile updates and deactivate/reactivate
5.  // toggles, delegating all business rules to ProsumerService.
6.  // -----
7.
8.  using Microsoft.AspNetCore.Mvc;
9.  using SolarGrid.Api.Models;
10. using SolarGrid.Api.Services;
11.
12. namespace SolarGrid.Api.Controllers;
13.
14. [ApiController]
15. [Route("api/prosumers")]
16. public class ProsumersController : ControllerBase
17. {
18.     private readonly ProsumerService _prosumerService;
19.
20.     public ProsumersController(ProsumerService prosumerService)
21.     {
22.         _prosumerService = prosumerService;
23.     }
24.
25.     [HttpGet]
26.     public async Task<IActionResult> GetAll() =>
27.         Ok(await _prosumerService.GetAllAsync());
28.
29.     [HttpPost]
30.     public async Task<IActionResult> Create([FromBody] CreateProsumerRequest request)
31.     {
32.         var prosumer = await _prosumerService.CreateAsync(request);
33.         if (prosumer == null)
34.             return BadRequest(new { message = "A prosumer with this NIC is already registered." });
35.
36.         return CreatedAtAction(nameof(GetAll), prosumer);
37.     }
38.

```

```

39.     [HttpPut("{nic}")]
40.     public async Task<IActionResult> Update(string nic, [FromBody] UpdateProsumerRequest
request)
41.     {
42.         var prosumer = await _prosumerService.UpdateAsync(nic, request);
43.         if (prosumer == null)
44.             return NotFound(new { message = "Prosumer not found." });
45.
46.         return Ok(prosumer);
47.     }
48.
49.     [HttpPatch("{nic}/deactivate")]
50.     public async Task<IActionResult> Deactivate(string nic)
51.     {
52.         var success = await _prosumerService.DeactivateAsync(nic);
53.         if (!success)
54.             return NotFound(new { message = "Prosumer not found." });
55.
56.         return Ok(new { message = "Prosumer deactivated successfully." });
57.     }
58.
59.     [HttpPatch("{nic}/reactivate")]
60.     public async Task<IActionResult> Reactivate(string nic)
61.     {
62.         var success = await _prosumerService.ReactivateAsync(nic);
63.         if (!success)
64.             return NotFound(new { message = "Prosumer not found." });
65.
66.         return Ok(new { message = "Prosumer reactivated successfully." });
67.     }
68. }
69.
70.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Controllers\ReservationsController.cs

```

1.  // -----
2.  // File: ReservationsController.cs
3.  // Purpose: Reservation lifecycle endpoints (api/reservations): list, filter by
4.  // prosumer, create, update, cancel, approve and complete. Each action delegates
5.  // to ReservationService and translates its ArgumentException into HTTP 400.
6.  // -----
7.
8.  using Microsoft.AspNetCore.Mvc;
9.  using SolarGrid.Api.Models;
10. using SolarGrid.Api.Services;
11.
12. namespace SolarGrid.Api.Controllers;
13.
14. [ApiController]
15. [Route("api/reservations")]
16. public class ReservationsController : ControllerBase
17. {
18.     private readonly ReservationService _reservationService;
19.
20.     public ReservationsController(ReservationService reservationService)
21.     {
22.         _reservationService = reservationService;
23.     }
24.
25.     [HttpGet]
26.     public async Task<IActionResult> GetAll() =>

```

```

27.         Ok(await _reservationService.GetAllAsync());
28.
29.     [HttpGet("prosumer/{nic}")]
30.     public async Task<IActionResult> GetByProsumer(string nic) =>
31.         Ok(await _reservationService.GetByProsumerNicAsync(nic));
32.
33.     [HttpPost]
34.     public async Task<IActionResult> Create([FromBody] CreateReservationRequest request)
35.     {
36.         try
37.         {
38.             var reservation = await _reservationService.CreateAsync(request);
39.             return CreatedAtAction(nameof(GetAll), reservation);
40.         }
41.         catch (ArgumentException ex)
42.         {
43.             return BadRequest(new { message = ex.Message });
44.         }
45.     }
46.
47.     [HttpPut("{id}")]
48.     public async Task<IActionResult> Update(string id, [FromBody] UpdateReservationRequest
request)
49.     {
50.         try
51.         {
52.             var reservation = await _reservationService.UpdateAsync(id, request);
53.             if (reservation == null)
54.                 return NotFound(new { message = "Reservation not found." });
55.
56.             return Ok(reservation);
57.         }
58.         catch (ArgumentException ex)
59.         {
60.             return BadRequest(new { message = ex.Message });
61.         }
62.     }
63.
64.     [HttpPatch("{id}/cancel")]
65.     public async Task<IActionResult> Cancel(string id)
66.     {
67.         try
68.         {
69.             var success = await _reservationService.CancelAsync(id);
70.             if (!success)
71.                 return NotFound(new { message = "Reservation not found." });
72.
73.             return Ok(new { message = "Reservation cancelled successfully." });
74.         }
75.         catch (ArgumentException ex)
76.         {
77.             return BadRequest(new { message = ex.Message });
78.         }
79.     }
80.
81.     [HttpPatch("{id}/approve")]
82.     public async Task<IActionResult> Approve(string id)
83.     {
84.         try
85.         {
86.             var reservation = await _reservationService.ApproveAsync(id);
87.             if (reservation == null)
88.                 return NotFound(new { message = "Reservation not found." });
89.
90.             return Ok(reservation);

```

```

91.         }
92.         catch (ArgumentException ex)
93.         {
94.             return BadRequest(new { message = ex.Message });
95.         }
96.     }
97.
98.     [HttpPatch("{id}/complete")]
99.     public async Task<IActionResult> Complete(string id)
100.    {
101.        try
102.        {
103.            var reservation = await _reservationService.CompleteAsync(id);
104.            if (reservation == null)
105.                return NotFound(new { message = "Reservation not found." });
106.
107.            return Ok(reservation);
108.        }
109.        catch (ArgumentException ex)
110.        {
111.            return BadRequest(new { message = ex.Message });
112.        }
113.    }
114. }
115.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Controllers\UsersController.cs

```

1. // -----
2. // File: UsersController.cs
3. // Purpose: Backoffice user administration endpoints (api/users): list staff
4. // accounts, create new ones, edit details and deactivate them. Persistence and
5. // validation are handled by AppUserService.
6. // -----
7.
8. using Microsoft.AspNetCore.Mvc;
9. using SolarGrid.Api.Models;
10. using SolarGrid.Api.Services;
11.
12. namespace SolarGrid.Api.Controllers;
13.
14. [ApiController]
15. [Route("api/users")]
16. public class UsersController : ControllerBase
17. {
18.     private readonly AppUserService _userService;
19.
20.     public UsersController(AppUserService userService)
21.     {
22.         _userService = userService;
23.     }
24.
25.     [HttpGet]
26.     public async Task<IActionResult> GetAll() =>
27.         Ok(await _userService.GetAllAsync());
28.
29.     [HttpPost]
30.     public async Task<IActionResult> Create([FromBody] CreateUserRequest request)
31.     {
32.         var user = await _userService.CreateAsync(request);
33.         if (user == null)
34.             return BadRequest(new { message = "Username is already taken." });

```

```

35.
36.     return CreatedAtAction(nameof(GetAll), user);
37. }
38.
39. [HttpPut("{id}")]
40. public async Task<IActionResult> Update(string id, [FromBody] UpdateUserRequest request)
41. {
42.     var user = await _userService.UpdateAsync(id, request);
43.     if (user == null)
44.         return NotFound(new { message = "User not found." });
45.
46.     return Ok(user);
47. }
48.
49. [HttpPatch("{id}/deactivate")]
50. public async Task<IActionResult> Deactivate(string id)
51. {
52.     var success = await _userService.DeactivateAsync(id);
53.     if (!success)
54.         return NotFound(new { message = "User not found." });
55.
56.     return Ok(new { message = "User deactivated successfully." });
57. }
58. }
59.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Models\ [AppUser.cs](#)

```

1. // -----
2. // File: AppUser.cs
3. // Purpose: MongoDB document representing a web application user (Backoffice
4. // officer or Grid Operator). Maps the document to the camelCase JSON shape the
5. // React client consumes and keeps the password hash out of API responses.
6. // -----
7.
8. using MongoDB.Bson;
9. using MongoDB.Bson.Serialization.Attributes;
10. using System.Text.Json.Serialization;
11.
12. namespace SolarGrid.Api.Models;
13.
14. /// <summary>
15. /// Web application user – Backoffice officers and Grid Operators who log in
16. /// to this React dashboard. Distinct from Prosumers (mobile-app users).
17. /// </summary>
18. public class AppUser
19. {
20.     [BsonId]
21.     [BsonRepresentation(BsonType.ObjectId)]
22.     [JsonPropertyName("id")]
23.     public string? Id { get; set; }
24.
25.     [JsonPropertyName("username")]
26.     public string Username { get; set; } = null!;
27.
28.     /// <summary>SHA-256 password hash – never serialised to the API response.</summary>
29.     [JsonIgnore]
30.     public string PasswordHash { get; set; } = null!;
31.
32.     [JsonPropertyName("fullName")]
33.     public string FullName { get; set; } = null!;
34.

```

```

35.     [JsonPropertyName("email")]
36.     public string Email { get; set; } = null!;
37.
38.     /// <summary>Backoffice | GridOperator</summary>
39.     [JsonPropertyName("role")]
40.     public string Role { get; set; } = "GridOperator";
41.
42.     [JsonPropertyName("isActive")]
43.     public bool IsActive { get; set; } = true;
44. }
45.
46.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Models\DashboardDTOs.cs

```

1.  // -----
2.  // File: DashboardDTOs.cs
3.  // Purpose: Response contracts for the Backoffice dashboard endpoint. Groups the
4.  // live node, prosumer, reservation and energy analytics into the camelCase JSON
5.  // shape rendered by the React dashboard, all computed by DashboardService.
6.  // -----
7.
8.  using System.Text.Json.Serialization;
9.
10. namespace SolarGrid.Api.Models;
11.
12. /// <summary>
13. /// Aggregated analytics for the Backoffice dashboard. All values are computed
14. /// from the live Nodes, Prosumers and Reservations collections – nothing is
15. /// hard-coded. Energy figures are derived from completed reservations, using
16. /// each node's real per-slot storage capacity.
17. /// </summary>
18. public class DashboardAnalytics
19. {
20.     [JsonPropertyName("generatedAt")]
21.     public DateTime GeneratedAt { get; set; }
22.
23.     [JsonPropertyName("period")]
24.     public string Period { get; set; } = "all";
25.
26.     [JsonPropertyName("nodes")]
27.     public DashboardNodeStats Nodes { get; set; } = new();
28.
29.     [JsonPropertyName("prosumers")]
30.     public DashboardProsumerStats Prosumers { get; set; } = new();
31.
32.     [JsonPropertyName("reservations")]
33.     public DashboardReservationStats Reservations { get; set; } = new();
34.
35.     [JsonPropertyName("energy")]
36.     public DashboardEnergyStats Energy { get; set; } = new();
37. }
38.
39. public class DashboardNodeStats
40. {
41.     [JsonPropertyName("total")]
42.     public int Total { get; set; }
43.
44.     [JsonPropertyName("active")]
45.     public int Active { get; set; }
46.
47.     [JsonPropertyName("offline")]

```

```

48.     public int Offline { get; set; }
49.
50.     [JsonPropertyName("capacityKWh")]
51.     public double CapacityKWh { get; set; }
52.
53.     /// <summary>Total storage capacity held by online nodes, in kWh.</summary>
54.     [JsonPropertyName("activeCapacityKWh")]
55.     public double ActiveCapacityKWh { get; set; }
56.
57.     [JsonPropertyName("totalBatterySlots")]
58.     public int TotalBatterySlots { get; set; }
59.
60.     [JsonPropertyName("availableBatterySlots")]
61.     public int AvailableBatterySlots { get; set; }
62.
63.     [JsonPropertyName("availabilityPct")]
64.     public double AvailabilityPct { get; set; }
65. }
66.
67. public class DashboardProsumerStats
68. {
69.     [JsonPropertyName("total")]
70.     public int Total { get; set; }
71.
72.     [JsonPropertyName("active")]
73.     public int Active { get; set; }
74.
75.     [JsonPropertyName("activePct")]
76.     public double ActivePct { get; set; }
77. }
78.
79. public class DashboardReservationStats
80. {
81.     [JsonPropertyName("total")]
82.     public int Total { get; set; }
83.
84.     [JsonPropertyName("pending")]
85.     public int Pending { get; set; }
86.
87.     [JsonPropertyName("approved")]
88.     public int Approved { get; set; }
89.
90.     [JsonPropertyName("completed")]
91.     public int Completed { get; set; }
92.
93.     [JsonPropertyName("cancelled")]
94.     public int Cancelled { get; set; }
95.
96.     [JsonPropertyName("latest")]
97.     public List<DashboardLatestReservation> Latest { get; set; } = new();
98. }
99.
100. public class DashboardLatestReservation
101. {
102.     [JsonPropertyName("id")]
103.     public string Id { get; set; } = null!;
104.
105.     [JsonPropertyName("scheduledDateTime")]
106.     public DateTime ScheduledDateTime { get; set; }
107.
108.     [JsonPropertyName("status")]
109.     public string Status { get; set; } = "Pending";
110. }
111.
112. public class DashboardEnergyStats

```



```

113. {
114.     /// <summary>Energy from completed reservations scheduled today, in kWh.</summary>
115.     [JsonPropertyName("todayKWh")]
116.     public double TodayKWh { get; set; }
117.
118.     /// <summary>Energy from completed reservations scheduled yesterday, in kWh.</summary>
119.     [JsonPropertyName("yesterdayKWh")]
120.     public double YesterdayKWh { get; set; }
121.
122.     [JsonPropertyName("todayDeltaPct")]
123.     public double TodayDeltaPct { get; set; }
124.
125.     /// <summary>Daily energy totals for the last seven days (oldest first).</summary>
126.     [JsonPropertyName("last7Days")]
127.     public List<DashboardDailyEnergy> Last7Days { get; set; } = new();
128. }
129.
130. public class DashboardDailyEnergy
131. {
132.     /// <summary>UTC day bucket in yyyy-MM-dd format.</summary>
133.     [JsonPropertyName("date")]
134.     public string Date { get; set; } = null!;
135.
136.     [JsonPropertyName("kwh")]
137.     public double Kwh { get; set; }
138. }
139.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Models\Node.cs

```

1. // -----
2. // File: Node.cs
3. // Purpose: MongoDB document describing a microgrid solar hub – its location,
4. // storage capacity, battery slot availability, operating schedule and active
5. // flag. This is the node entity managed through the Grid Operator screens.
6. // -----
7.
8. using MongoDB.Bson;
9. using MongoDB.Bson.Serialization.Attributes;
10. using System.Text.Json.Serialization;
11.
12. namespace SolarGrid.Api.Models;
13.
14. /// <summary>
15. /// A microgrid solar hub – a physical location where prosumers can deposit
16. /// or draw energy. Replaces the old SolarStationInfo model.
17. /// </summary>
18. public class Node
19. {
20.     [BsonId]
21.     [BsonRepresentation(BsonType.ObjectId)]
22.     [JsonPropertyName("id")]
23.     public string? Id { get; set; }
24.
25.     [JsonPropertyName("name")]
26.     public string Name { get; set; } = null!;
27.
28.     [JsonPropertyName("latitude")]
29.     public double Latitude { get; set; }
30.
31.     [JsonPropertyName("longitude")]
32.     public double Longitude { get; set; }

```

```

33.
34.     /// <summary>Total storage capacity of the node in kilowatt-hours.</summary>
35.     [JsonPropertyName("capacityKWh")]
36.     public double CapacityKWh { get; set; }
37.
38.     /// <summary>Total number of battery storage slots at this node.</summary>
39.     [JsonPropertyName("totalBatterySlots")]
40.     public int TotalBatterySlots { get; set; }
41.
42.     /// <summary>Slots currently free (not reserved). Updated by Grid Operators.</summary>
43.     [JsonPropertyName("availableBatterySlots")]
44.     public int AvailableBatterySlots { get; set; }
45.
46.     /// <summary>Human-readable schedule strings, e.g. "Mon-Fri 06:00-20:00".</summary>
47.     [JsonPropertyName("operatingSchedule")]
48.     public List<string> OperatingSchedule { get; set; } = new();
49.
50.     [JsonPropertyName("isActive")]
51.     public bool IsActive { get; set; } = true;
52. }
53.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Models\NodeDTOs.cs

```

1.  // -----
2.  // File: NodeDTOs.cs
3.  // Purpose: Request contracts for the node endpoints – the payload accepted when
4.  // creating a new microgrid node and the editable fields for updating an
5.  // existing one. Both are validated and mapped by NodeService.
6.  // -----
7.
8.  namespace SolarGrid.Api.Models;
9.
10. public class CreateNodeRequest
11. {
12.     public string Name { get; set; } = null!;
13.     public double Latitude { get; set; }
14.     public double Longitude { get; set; }
15.     public double CapacityKWh { get; set; }
16.     public int TotalBatterySlots { get; set; }
17.     public List<string> OperatingSchedule { get; set; } = new();
18. }
19.
20. public class UpdateNodeRequest
21. {
22.     public string Name { get; set; } = null!;
23.     public double CapacityKWh { get; set; }
24.     public int TotalBatterySlots { get; set; }
25.     public int AvailableBatterySlots { get; set; }
26.     public List<string> OperatingSchedule { get; set; } = new();
27. }
28.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Models\Prosumer.cs

```

1.  // -----
2.  // File: Prosumer.cs
3.  // Purpose: MongoDB document for a solar energy prosumer, keyed by National

```

```

4. // Identity Card number. Holds the mobile-app user's contact details and hides
5. // both the internal Mongo id and the password hash from API responses.
6. // -----
7.
8. using MongoDB.Bson;
9. using MongoDB.Bson.Serialization.Attributes;
10. using System.Text.Json.Serialization;
11.
12. namespace SolarGrid.Api.Models;
13.
14. /// <summary>
15. /// Solar energy prosumer – a property owner with a solar array who trades
16. /// energy via the mobile app. Managed (registered/deactivated) by Backoffice.
17. /// </summary>
18. public class Prosumer
19. {
20.     [BsonId]
21.     [BsonRepresentation(BsonType.ObjectId)]
22.     [JsonIgnore]
23.     public string? InternalId { get; set; }
24.
25.     /// <summary>National Identity Card number – primary business key.</summary>
26.     [JsonPropertyName("nic")]
27.     public string Nic { get; set; } = null!;
28.
29.     [JsonPropertyName("fullName")]
30.     public string FullName { get; set; } = null!;
31.
32.     [JsonPropertyName("email")]
33.     public string Email { get; set; } = null!;
34.
35.     [JsonPropertyName("phone")]
36.     public string Phone { get; set; } = null!;
37.
38.     [JsonPropertyName("address")]
39.     public string Address { get; set; } = null!;
40.
41.     /// <summary>SHA-256 hash of the mobile-app password – never sent to API
42.     consumers.</summary>
43.     [JsonIgnore]
44.     public string PasswordHash { get; set; } = null!;
45.
46.     [JsonPropertyName("isActive")]
47.     public bool IsActive { get; set; } = true;
48. }
49.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Models\ProsumerDTOs.cs

```

1. // -----
2. // File: ProsumerDTOs.cs
3. // Purpose: Request contracts for the prosumer endpoints – the registration
4. // payload (NIC plus initial mobile-app password) and the editable profile
5. // fields. ProsumerService validates these before writing to MongoDB.
6. // -----
7.
8. namespace SolarGrid.Api.Models;
9.
10. public class CreateProsumerRequest
11. {
12.     public string Nic { get; set; } = null!;

```

```

13.     public string FullName { get; set; } = null!;
14.     public string Email { get; set; } = null!;
15.     public string Phone { get; set; } = null!;
16.     public string Address { get; set; } = null!;
17.     public string Password { get; set; } = null!;
18. }
19.
20. public class UpdateProsumerRequest
21. {
22.     public string FullName { get; set; } = null!;
23.     public string Email { get; set; } = null!;
24.     public string Phone { get; set; } = null!;
25.     public string Address { get; set; } = null!;
26. }
27.
28.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Models\Reservation.cs

```

1.  // -----
2.  // File: Reservation.cs
3.  // Purpose: MongoDB document for an energy slot booking made by a prosumer at a
4.  // microgrid node. Tracks the schedule, duration, lifecycle status and the QR
5.  // dispatch code issued on approval (Pending → Approved → Completed | Cancelled).
6.  // -----
7.
8.  using MongoDB.Bson;
9.  using MongoDB.Bson.Serialization.Attributes;
10. using System.Text.Json.Serialization;
11.
12. namespace SolarGrid.Api.Models;
13.
14. /// <summary>
15. /// An energy slot reservation made by a prosumer at a microgrid node.
16. /// Status lifecycle: Pending → Approved (QR issued) → Completed | Cancelled.
17. /// </summary>
18. [BsonIgnoreExtraElements]
19. public class Reservation
20. {
21.     [BsonId]
22.     [BsonRepresentation(BsonType.ObjectId)]
23.     [JsonPropertyName("id")]
24.     public string? Id { get; set; }
25.
26.     [JsonPropertyName("prosumerNic")]
27.     public string ProsumerNic { get; set; } = null!;
28.
29.     [JsonPropertyName("nodeId")]
30.     public string NodeId { get; set; } = null!;
31.
32.     [JsonPropertyName("scheduledDateTime")]
33.     public DateTime ScheduledDateTime { get; set; }
34.
35.     [JsonPropertyName("durationMinutes")]
36.     public int DurationMinutes { get; set; }
37.
38.     /// <summary>Pending | Approved | Completed | Cancelled</summary>
39.     [JsonPropertyName("status")]
40.     public string Status { get; set; } = "Pending";

```

```

41.
42.     /// <summary>Short QR dispatch code generated when a Grid Operator approves the
booking.</summary>
43.     [JsonPropertyName("qrCode")]
44.     public string? QrCode { get; set; }
45.
46.     [JsonPropertyName("createdAt")]
47.     public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
48.
49.     [JsonPropertyName("updatedAt")]
50.     public DateTime? UpdatedAt { get; set; }
51. }
52.
53.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Models\ReservationDTOs.cs

```

1. // -----
2. // File: ReservationDTOs.cs
3. // Purpose: Request contracts for the reservation endpoints – the booking payload
4. // (prosumer, node, schedule and duration) and the editable slot fields.
5. // ReservationService validates availability before persisting them.
6. // -----
7.
8. namespace SolarGrid.Api.Models;
9.
10. public class CreateReservationRequest
11. {
12.     public string ProsumerNic { get; set; } = null!;
13.     public string NodeId { get; set; } = null!;
14.     public DateTime ScheduledDateTime { get; set; }
15.     public int DurationMinutes { get; set; }
16. }
17.
18. public class UpdateReservationRequest
19. {
20.     public DateTime ScheduledDateTime { get; set; }
21.     public int DurationMinutes { get; set; }
22. }
23.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Models\UserDTOs.cs

```

1. // -----
2. // File: UserDTOs.cs
3. // Purpose: Contracts for authentication and staff account management: the login
4. // credentials in, the JWT plus user details out, and the create/update payloads
5. // accepted by UsersController for Backoffice and Grid Operator accounts.
6. // -----
7.
8. namespace SolarGrid.Api.Models;
9.
10. public class LoginRequest
11. {
12.     public string Username { get; set; } = null!;
13.     public string Password { get; set; } = null!;
14. }

```

```

15.
16. public class LoginResponse
17. {
18.     public string Token { get; set; } = null!;
19.     public string Username { get; set; } = null!;
20.     public string FullName { get; set; } = null!;
21.     public string Role { get; set; } = null!;
22. }
23.
24. public class CreateUserRequest
25. {
26.     public string Username { get; set; } = null!;
27.     public string Password { get; set; } = null!;
28.     public string FullName { get; set; } = null!;
29.     public string Email { get; set; } = null!;
30.     public string Role { get; set; } = "GridOperator";
31. }
32.
33. public class UpdateUserRequest
34. {
35.     public string FullName { get; set; } = null!;
36.     public string Email { get; set; } = null!;
37.     public string Role { get; set; } = null!;
38.     public bool IsActive { get; set; } = true;
39. }
40.
41.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Properties\launchSettings.json

```

1. // -----
2. // File: launchSettings.json
3. // Purpose: Local debug launch profiles for the SolarGrid API. Defines the http,
4. // https and IIS Express profiles (ports, Swagger launch URL and the Development
5. // environment) used by "dotnet run" and Visual Studio. Comments are valid here –
6. // the ASP.NET Core host skips them when parsing this file.
7. // -----
8. {
9.     "$schema": "http://json.schemastore.org/launchsettings.json",
10.    "iisSettings": {
11.        "windowsAuthentication": false,
12.        "anonymousAuthentication": true,
13.        "iisExpress": {
14.            "applicationUrl": "http://localhost:20860",
15.            "sslPort": 44336
16.        }
17.    },
18.    "profiles": {
19.        "http": {
20.            "commandName": "Project",
21.            "dotnetRunMessages": true,
22.            "launchBrowser": true,
23.            "launchUrl": "swagger",
24.            "applicationUrl": "http://localhost:8080",
25.            "environmentVariables": {
26.                "ASPNETCORE_ENVIRONMENT": "Development"
27.            }
28.        },
29.        "https": {
30.            "commandName": "Project",
31.            "dotnetRunMessages": true,
32.            "launchBrowser": true,

```

```

33.     "launchUrl": "swagger",
34.     "applicationUrl": "https://localhost:7040;http://localhost:8080",
35.     "environmentVariables": {
36.         "ASPNETCORE_ENVIRONMENT": "Development"
37.     }
38. },
39. "IIS Express": {
40.     "commandName": "IISExpress",
41.     "launchBrowser": true,
42.     "launchUrl": "swagger",
43.     "environmentVariables": {
44.         "ASPNETCORE_ENVIRONMENT": "Development"
45.     }
46. }
47. }
48. }
49.
50.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Repositories[AppUserRepository.cs](#)

```

1.  // -----
2.  // File: AppUserRepository.cs
3.  // Purpose: MongoDB data-access layer for the AppUser collection. Resolves the
4.  // target database and collection from the configured settings, then provides
5.  // lookup by id or username plus insert and replace operations for staff users.
6.  // -----
7.
8.  using Microsoft.Extensions.Options;
9.  using MongoDB.Driver;
10. using SolarGrid.Api.Configuration;
11. using SolarGrid.Api.Models;
12.
13. namespace SolarGrid.Api.Repositories;
14.
15. public class AppUserRepository : IAppUserRepository
16. {
17.     private readonly IMongoCollection<AppUser> _collection;
18.
19.     public AppUserRepository(IOptions<SolarGridDatabaseSettings> settings, IMongoClient
mongoClient)
20.     {
21.         var db = mongoClient.GetDatabase(settings.Value.DatabaseName);
22.         _collection = db.GetCollection<AppUser>(settings.Value.AppUsersCollectionName);
23.     }
24.
25.     public async Task<List<AppUser>> GetAllAsync() =>
26.         await _collection.Find(_ => true).ToListAsync();
27.
28.     public async Task<AppUser?> GetByIdAsync(string id) =>
29.         await _collection.Find(x => x.Id == id).FirstOrDefaultAsync();
30.
31.     public async Task<AppUser?> GetByUsernameAsync(string username) =>
32.         await _collection.Find(x => x.Username == username).FirstOrDefaultAsync();
33.
34.     public async Task CreateAsync(AppUser user) =>
35.         await _collection.InsertOneAsync(user);
36.
37.     public async Task UpdateAsync(string id, AppUser user) =>
38.         await _collection.ReplaceOneAsync(x => x.Id == id, user);
39. }
40.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Repositories\IAppUserRepository.cs

```

1. // -----
2. // File: IAppUserRepository.cs
3. // Purpose: Persistence contract for AppUser records. Declares the read
4. // operations (all, by id, by username) and the write operations (create,
5. // replace) that AppUserRepository implements against MongoDB.
6. // -----
7.
8. using SolarGrid.Api.Models;
9.
10. namespace SolarGrid.Api.Repositories;
11.
12. public interface IAppUserRepository
13. {
14.     Task<List<AppUser>> GetAllAsync();
15.     Task<AppUser?> GetByIdAsync(string id);
16.     Task<AppUser?> GetByUsernameAsync(string username);
17.     Task CreateAsync(AppUser user);
18.     Task UpdateAsync(string id, AppUser user);
19. }
20.
21.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Repositories\INodeRepository.cs

```

1. // -----
2. // File: INodeRepository.cs
3. // Purpose: Persistence contract for microgrid Node records. Declares the list
4. // and by-id lookups plus the create and replace operations that
5. // NodeRepository implements against the MongoDB nodes collection.
6. // -----
7. using SolarGrid.Api.Models;
8.
9. namespace SolarGrid.Api.Repositories;
10.
11. public interface INodeRepository
12. {
13.     Task<List<Node>> GetAllAsync();
14.     Task<Node?> GetByIdAsync(string id);
15.     Task CreateAsync(Node node);
16.     Task UpdateAsync(string id, Node node);
17. }
18.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Repositories\IProsumerRepository.cs

```

1. // -----
2. // File: IProsumerRepository.cs
3. // Purpose: Persistence contract for Prosumer records, keyed by National Identity
4. // Card number. Declares the list and by-NIC lookups plus the create and
5. // replace operations that ProsumerRepository implements against MongoDB.
6. // -----

```



```

7.
8. using SolarGrid.Api.Models;
9.
10. namespace SolarGrid.Api.Repositories;
11.
12. public interface IProsumerRepository
13. {
14.     Task<List<Prosumer>> GetAllAsync();
15.     Task<Prosumer?> GetByNicAsync(string nic);
16.     Task CreateAsync(Prosumer prosumer);
17.     Task UpdateAsync(string nic, Prosumer prosumer);
18. }
19.
20.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Repositories\IReservationRepository.cs

```

1. // -----
2. // File: IReservationRepository.cs
3. // Purpose: Persistence contract for Reservation records. Declares the list,
4. // by-prosumer and by-id lookups plus the create and replace operations that
5. // ReservationRepository implements against the MongoDB reservations collection.
6. // -----
7.
8. using SolarGrid.Api.Models;
9.
10. namespace SolarGrid.Api.Repositories;
11.
12. public interface IReservationRepository
13. {
14.     Task<List<Reservation>> GetAllAsync();
15.     Task<List<Reservation>> GetByProsumerNicAsync(string nic);
16.     Task<Reservation?> GetByIdAsync(string id);
17.     Task CreateAsync(Reservation reservation);
18.     Task UpdateAsync(string id, Reservation reservation);
19. }
20.
21.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Repositories\NodeRepository.cs

```

1. // -----
2. // File: NodeRepository.cs
3. // Purpose: MongoDB data access for Node (microgrid solar hub) documents.
4. // Resolves the collection from configuration and exposes fetch-by-all,
5. // fetch-by-id, insert and full-document replace operations.
6. // -----
7.
8.
9.
10. using Microsoft.Extensions.Options;
11. using MongoDB.Driver;
12. using SolarGrid.Api.Configuration;
13. using SolarGrid.Api.Models;
14.
15. namespace SolarGrid.Api.Repositories;

```

```

16.
17. public class NodeRepository : INodeRepository
18. {
19.     private readonly IMongoCollection<Node> _collection;
20.
21.     public NodeRepository(IOptions<SolarGridDatabaseSettings> settings, IMongoClient
mongoClient)
22.     {
23.         var db = mongoClient.GetDatabase(settings.Value.DatabaseName);
24.         _collection = db.GetCollection<Node>(settings.Value.NodesCollectionName);
25.     }
26.
27.     public async Task<List<Node>> GetAllAsync() =>
28.         await _collection.Find(_ => true).ToListAsync();
29.
30.     public async Task<Node?> GetByIdAsync(string id) =>
31.         await _collection.Find(x => x.Id == id).FirstOrDefaultAsync();
32.
33.     public async Task CreateAsync(Node node) =>
34.         await _collection.InsertOneAsync(node);
35.
36.     public async Task UpdateAsync(string id, Node node) =>
37.         await _collection.ReplaceOneAsync(x => x.Id == id, node);
38. }
39.

```

**D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-
service\SolarGrid.Api\Repositories**[ProsumerRepository.cs](#)

```

1. // -----
2. // File: ProsumerRepository.cs
3. // Purpose: MongoDB data access for Prosumer documents. Resolves the collection
4. // from configuration and exposes fetch-by-all, lookup by NIC, insert and
5. // NIC-keyed full-document replace operations.
6. // -----
7.
8.
9. using Microsoft.Extensions.Options;
10. using MongoDB.Driver;
11. using SolarGrid.Api.Configuration;
12. using SolarGrid.Api.Models;
13.
14. namespace SolarGrid.Api.Repositories;
15.
16. public class ProsumerRepository : IProsumerRepository
17. {
18.     private readonly IMongoCollection<Prosumer> _collection;
19.
20.     public ProsumerRepository(IOptions<SolarGridDatabaseSettings> settings, IMongoClient
mongoClient)
21.     {
22.         var db = mongoClient.GetDatabase(settings.Value.DatabaseName);
23.         _collection = db.GetCollection<Prosumer>(settings.Value.ProsumersCollectionName);
24.     }
25.
26.     public async Task<List<Prosumer>> GetAllAsync() =>
27.         await _collection.Find(_ => true).ToListAsync();
28.
29.     public async Task<Prosumer?> GetByNicAsync(string nic) =>
30.         await _collection.Find(x => x.Nic == nic).FirstOrDefaultAsync();
31.
32.     public async Task CreateAsync(Prosumer prosumer) =>
33.         await _collection.InsertOneAsync(prosumer);
34.

```

```

35.     public async Task UpdateAsync(string nic, Prosumer prosumer) =>
36.         await _collection.ReplaceOneAsync(x => x.Nic == nic, prosumer);
37. }
38.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Repositories\ReservationRepository.cs

```

1. // -----
2. // File: ReservationRepository.cs
3. // Purpose: MongoDB data access for Reservation documents. Resolves the
4. // collection from configuration and exposes fetch-by-all, fetch-by-prosumer
5. // NIC, fetch-by-id, insert and full-document replace operations.
6. // -----
7.
8.
9. using Microsoft.Extensions.Options;
10. using MongoDB.Driver;
11. using SolarGrid.Api.Configuration;
12. using SolarGrid.Api.Models;
13.
14. namespace SolarGrid.Api.Repositories;
15.
16. public class ReservationRepository : IReservationRepository
17. {
18.     private readonly IMongoCollection<Reservation> _collection;
19.
20.     public ReservationRepository(IOptions<SolarGridDatabaseSettings> settings, IMongoClient
mongoClient)
21.     {
22.         var db = mongoClient.GetDatabase(settings.Value.DatabaseName);
23.         _collection = db.GetCollection<Reservation>(settings.Value.ReservationsCollectionName);
24.     }
25.
26.     public async Task<List<Reservation>> GetAllAsync() =>
27.         await _collection.Find(_ => true).ToListAsync();
28.
29.     public async Task<List<Reservation>> GetByProsumerNicAsync(string nic) =>
30.         await _collection.Find(x => x.ProsumerNic == nic).ToListAsync();
31.
32.     public async Task<Reservation?> GetByIdAsync(string id) =>
33.         await _collection.Find(x => x.Id == id).FirstOrDefaultAsync();
34.
35.     public async Task CreateAsync(Reservation reservation) =>
36.         await _collection.InsertOneAsync(reservation);
37.
38.     public async Task UpdateAsync(string id, Reservation reservation) =>
39.         await _collection.ReplaceOneAsync(x => x.Id == id, reservation);
40. }
41.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Services\AppUserService.cs

```

1. // -----
2. // File: AppUserService.cs
3. // Purpose: CRUD service for web application users (Backoffice officers and
4. // Grid Operators). Enforces username uniqueness, stores passwords as SHA-256
5. // hashes, and supports profile, role and activation updates.
6. // -----
7.
8. using System.Security.Cryptography;
9. using System.Text;

```

```

10. using SolarGrid.Api.Models;
11. using SolarGrid.Api.Repositories;
12.
13. namespace SolarGrid.Api.Services;
14.
15. /// <summary>
16. /// Full CRUD service for web-application users (Backoffice / Grid Operators).
17. /// Managed exclusively by Backoffice officers through the Users page.
18. /// </summary>
19. public class AppUserService
20. {
21.     private readonly IAppUserRepository _userRepository;
22.
23.     public AppUserService(IAppUserRepository userRepository)
24.     {
25.         _userRepository = userRepository;
26.     }
27.
28.     public async Task<List<AppUser>> GetAllAsync() =>
29.         await _userRepository.GetAllAsync();
30.
31.     /// <summary>Returns null if the username is already taken.</summary>
32.     public async Task<AppUser?> CreateAsync(CreateUserRequest request)
33.     {
34.         var existing = await _userRepository.GetByUsernameAsync(request.Username);
35.         if (existing != null) return null;
36.
37.         var user = new AppUser
38.         {
39.             Username = request.Username,
40.             PasswordHash = HashPassword(request.Password),
41.             FullName = request.FullName,
42.             Email = request.Email,
43.             Role = request.Role,
44.             IsActive = true
45.         };
46.
47.         await _userRepository.CreateAsync(user);
48.         return user;
49.     }
50.
51.     /// <summary>Returns null if the user id is not found.</summary>
52.     public async Task<AppUser?> UpdateAsync(string id, UpdateUserRequest request)
53.     {
54.         var user = await _userRepository.GetByIdAsync(id);
55.         if (user == null) return null;
56.
57.         user.FullName = request.FullName;
58.         user.Email = request.Email;
59.         user.Role = request.Role;
60.         user.IsActive = request.IsActive;
61.
62.         await _userRepository.UpdateAsync(id, user);
63.         return user;
64.     }
65.
66.     public async Task<bool> DeactivateAsync(string id)
67.     {
68.         var user = await _userRepository.GetByIdAsync(id);
69.         if (user == null) return false;
70.
71.         user.IsActive = false;
72.         await _userRepository.UpdateAsync(id, user);
73.         return true;
74.     }

```

```

75.
76.     private static string HashPassword(string password)
77.     {
78.         using var sha256 = SHA256.Create();
79.         var bytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
80.         return Convert.ToBase64String(bytes);
81.     }
82. }
83.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Services\AuthService.cs

```

1. using System.Security.Cryptography;
2. using System.Text;
3. using SolarGrid.Api.Models;
4. using SolarGrid.Api.Repositories;
5.
6. namespace SolarGrid.Api.Services;
7.
8. /// <summary>
9. /// Handles login for web-application users (Backoffice and Grid Operators).
10. /// Returns a session token on success; null on bad credentials or deactivated account.
11. /// </summary>
12. public class AuthService
13. {
14.     private readonly IAppUserRepository _userRepository;
15.
16.     public AuthService(IAppUserRepository userRepository)
17.     {
18.         _userRepository = userRepository;
19.     }
20.
21.     public async Task<LoginResponse?> LoginAsync(LoginRequest request)
22.     {
23.         var user = await _userRepository.GetByUsernameAsync(request.Username);
24.
25.         if (user == null || !user.IsActive)
26.             return null;
27.
28.         if (user.PasswordHash != HashPassword(request.Password))
29.             return null;
30.
31.         // Assignment note: using a GUID token rather than a signed JWT to keep
32.         // the dependency surface small. In production, replace with JWT Bearer.
33.         return new LoginResponse
34.         {
35.             Token = Guid.NewGuid().ToString("N"),
36.             Username = user.Username,
37.             FullName = user.FullName,
38.             Role = user.Role
39.         };
40.     }
41.
42.     private static string HashPassword(string password)
43.     {
44.         using var sha256 = SHA256.Create();
45.         var bytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
46.         return Convert.ToBase64String(bytes);
47.     }
48.
49. }

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Services\DashboardService.cs

```

1. // -----
2. // File: DashboardService.cs
3. // Purpose: Server-side analytics for the Backoffice dashboard. Aggregates live
4. // node, prosumer and reservation data into totals, status breakdowns and a
5. // 7-day energy chart. Uses safe division so empty datasets yield 0.
6. // -----
7.
8.
9. using SolarGrid.Api.Models;
10. using SolarGrid.Api.Repositories;
11.
12. namespace SolarGrid.Api.Services;
13.
14. /// <summary>
15. /// Server-side aggregation for the Backoffice dashboard. Every figure is
16. /// derived from live repository data (Nodes, Prosumers, Reservations) so the
17. /// dashboard always reflects the current database state and never hard-codes
18. /// values. Safe division is used throughout so empty datasets return 0.
19. /// </summary>
20. public class DashboardService
21. {
22.     private const int LatestCount = 5;
23.     private const int ChartDays = 7;
24.
25.     private readonly INodeRepository _nodeRepository;
26.     private readonly IProsumerRepository _prosumerRepository;
27.     private readonly IReservationRepository _reservationRepository;
28.
29.     public DashboardService(
30.         INodeRepository nodeRepository,
31.         IProsumerRepository prosumerRepository,
32.         IReservationRepository reservationRepository)
33.     {
34.         _nodeRepository = nodeRepository;
35.         _prosumerRepository = prosumerRepository;
36.         _reservationRepository = reservationRepository;
37.     }
38.
39.     public async Task<DashboardAnalytics> GetAnalyticsAsync(string period)
40.     {
41.         var nodes = await _nodeRepository.GetAllAsync();
42.         var prosumers = await _prosumerRepository.GetAllAsync();
43.         var reservations = await _reservationRepository.GetAllAsync();
44.
45.         var analytics = new DashboardAnalytics
46.         {
47.             GeneratedAt = DateTime.UtcNow,
48.             Period = period,
49.             Nodes = BuildNodeStats(nodes),
50.             Prosumers = BuildProsumerStats(prosumers),
51.             Reservations = BuildReservationStats(reservations, period),
52.             Energy = BuildEnergyStats(reservations, nodes),
53.         };
54.
55.         return analytics;
56.     }
57.
58.     private static DashboardNodeStats BuildNodeStats(List<Node> nodes)
59.     {
60.         double capacity = 0;
61.         double activeCapacity = 0;
62.         var totalSlots = 0;
63.         var availSlots = 0;

```

```

64.         var active = 0;
65.
66.         foreach (var node in nodes)
67.         {
68.             capacity += node.CapacityKWh;
69.             totalSlots += node.TotalBatterySlots;
70.             availSlots += node.AvailableBatterySlots;
71.
72.             if (node.IsActive)
73.             {
74.                 active++;
75.                 activeCapacity += node.CapacityKWh;
76.             }
77.         }
78.
79.         return new DashboardNodeStats
80.         {
81.             Total = nodes.Count,
82.             Active = active,
83.             Offline = nodes.Count - active,
84.             CapacityKWh = Round(capacity),
85.             ActiveCapacityKWh = Round(activeCapacity),
86.             TotalBatterySlots = totalSlots,
87.             AvailableBatterySlots = availSlots,
88.             AvailabilityPct = Round(SafePercent(availSlots, totalSlots)),
89.         };
90.     }
91.
92.     private static DashboardProsumerStats BuildProsumerStats(List<Prosumer> prosumers)
93.     {
94.         var active = prosumers.Count(p => p.IsActive);
95.         return new DashboardProsumerStats
96.         {
97.             Total = prosumers.Count,
98.             Active = active,
99.             ActivePct = Round(SafePercent(active, prosumers.Count)),
100.        };
101.    }
102.
103.     private static DashboardReservationStats BuildReservationStats(List<Reservation>
reservations, string period)
104.     {
105.         var start = PeriodStart(period, DateTime.UtcNow);
106.         var inPeriod = reservations.Where(r => r.ScheduledDateTime >= start).ToList();
107.
108.         var stats = new DashboardReservationStats
109.         {
110.             Total = inPeriod.Count,
111.             Pending = inPeriod.Count(r => r.Status == "Pending"),
112.             Approved = inPeriod.Count(r => r.Status == "Approved"),
113.             Completed = inPeriod.Count(r => r.Status == "Completed"),
114.             Cancelled = inPeriod.Count(r => r.Status == "Cancelled"),
115.             Latest = reservations
116.                 .OrderByDescending(r => r.ScheduledDateTime)
117.                 .Take(LatestCount)
118.                 .Select(r => new DashboardLatestReservation
119.                 {
120.                     Id = r.Id ?? string.Empty,
121.                     ScheduledDateTime = r.ScheduledDateTime,
122.                     Status = r.Status,
123.                 })
124.                 .ToList(),
125.        };
126.
127.         return stats;

```

```

128.     }
129.
130.     private static DashboardEnergyStats BuildEnergyStats(List<Reservation> reservations,
131. List<Node> nodes)
132.     {
133.         var today = DateTime.UtcNow.Date;
134.         var yesterday = today.AddDays(-1);
135.
136.         var buckets = new Dictionary<DateTime, double>();
137.         for (var i = ChartDays - 1; i >= 0; i--)
138.             buckets[today.AddDays(-i)] = 0;
139.
140.         var todayKwh = 0d;
141.         var yesterdayKwh = 0d;
142.
143.         foreach (var r in reservations)
144.         {
145.             if (r.Status != "Completed")
146.                 continue;
147.
148.             var kwh = SlotCapacityKwh(nodes, r.NodeId);
149.             var day = r.ScheduledDateTime.Date;
150.
151.             if (day == today)
152.                 todayKwh += kwh;
153.
154.             if (day == yesterday)
155.                 yesterdayKwh += kwh;
156.
157.             if (buckets.ContainsKey(day))
158.                 buckets[day] += kwh;
159.         }
160.
161.         // Safe difference: when yesterday had no energy and today has some,
162.         // report a 100% increase; otherwise report the true percentage or 0.
163.         double delta;
164.         if (todayKwh > 0 && yesterdayKwh <= 0)
165.             delta = 100;
166.         else if (yesterdayKwh <= 0)
167.             delta = 0;
168.         else
169.             delta = (todayKwh - yesterdayKwh) / yesterdayKwh * 100;
170.
171.         return new DashboardEnergyStats
172.         {
173.             TodayKWh = Round(todayKwh),
174.             YesterdayKWh = Round(yesterdayKwh),
175.             TodayDeltaPct = Round(delta),
176.             Last7Days = buckets
177.                 .OrderBy(b => b.Key)
178.                 .Select(b => new DashboardDailyEnergy
179.                 {
180.                     Date = b.Key.ToString("yyyy-MM-dd"),
181.                     Kwh = Round(b.Value),
182.                 })
183.                 .ToList(),
184.         };
185.     }
186.
187.     /// <summary>
188.     /// Energy attributed to one completed reservation at a node – the node's
189.     /// total storage capacity divided by its real battery-slot count.
190.     /// </summary>
191.     private static double SlotCapacityKwh(List<Node> nodes, string nodeId)
192.     {

```



```

192.         var node = nodes.FirstOrDefault(n => n.Id == nodeId);
193.         if (node == null || node.TotalBatterySlots <= 0)
194.             return 0;
195.         return node.CapacityKWh / node.TotalBatterySlots;
196.     }
197.
198.     private static DateTime PeriodStart(string period, DateTime now) =>
199.     {
200.         period switch
201.         {
202.             "today" => now.Date,
203.             "7d" => now.Date.AddDays(-(ChartDays - 1)),
204.             "week" => StartOfWeek(now, DayOfWeek.Monday),
205.             "month" => new DateTime(now.Year, now.Month, 1, 0, 0, 0, DateTimeKind.Utc),
206.             _ => DateTime.MinValue.ToUniversalTime(),
207.         };
208.
209.         /// <summary>
210.         /// Start of the calendar week that contains <paramref name="date"/>,
211.         /// aligned to the requested first day of week (e.g. Monday).
212.         /// </summary>
213.         private static DateTime StartOfWeek(DateTime date, DayOfWeek firstDay)
214.         {
215.             var daysSinceFirstDay = ((int)date.DayOfWeek - (int)firstDay + 7) % 7;
216.             return date.Date.AddDays(-daysSinceFirstDay).ToUniversalTime();
217.         }
218.
219.         private static double SafePercent(int value, int total) =>
220.             total > 0 ? value * 100.0 / total : 0;
221.
222.         private static double Round(double value) => Math.Round(value, 2);
223.     }

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Services\NodeService.cs

```

1.  // -----
2.  // File: NodeService.cs
3.  // Purpose: Microgrid node (solar hub) management – registration, location and
4.  // capacity details, battery slot counts, operating schedule, and activation.
5.  // New nodes start with every battery slot available.
6.  // -----
7.
8.
9.  using SolarGrid.Api.Models;
10. using SolarGrid.Api.Repositories;
11.
12. namespace SolarGrid.Api.Services;
13.
14. /// <summary>
15. /// Manages microgrid node (solar hub) records.
16. /// Backoffice creates nodes; Grid Operators can update battery slot counts.
17. /// </summary>
18. public class NodeService
19. {
20.     private readonly INodeRepository _nodeRepository;
21.
22.     public NodeService(INodeRepository nodeRepository)
23.     {
24.         _nodeRepository = nodeRepository;
25.     }
26.
27.     public async Task<List<Node>> GetAllAsync() =>
28.         await _nodeRepository.GetAllAsync();

```

```

29.
30.     public async Task<Node?> GetByIdAsync(string id) =>
31.         await _nodeRepository.GetByIdAsync(id);
32.
33.     public async Task<Node> CreateAsync(CreateNodeRequest request)
34.     {
35.         var node = new Node
36.         {
37.             Name                = request.Name,
38.             Latitude            = request.Latitude,
39.             Longitude           = request.Longitude,
40.             CapacityKWh         = request.CapacityKWh,
41.             TotalBatterySlots   = request.TotalBatterySlots,
42.             // All slots are available when first registered
43.             AvailableBatterySlots = request.TotalBatterySlots,
44.             OperatingSchedule   = request.OperatingSchedule,
45.             IsActive            = true
46.         };
47.
48.         await _nodeRepository.CreateAsync(node);
49.         return node;
50.     }
51.
52.     public async Task<Node?> UpdateAsync(string id, UpdateNodeRequest request)
53.     {
54.         var node = await _nodeRepository.GetByIdAsync(id);
55.         if (node == null) return null;
56.
57.         node.Name                = request.Name;
58.         node.CapacityKWh         = request.CapacityKWh;
59.         node.TotalBatterySlots   = request.TotalBatterySlots;
60.         node.AvailableBatterySlots = request.AvailableBatterySlots;
61.         node.OperatingSchedule   = request.OperatingSchedule;
62.
63.         await _nodeRepository.UpdateAsync(id, node);
64.         return node;
65.     }
66.
67.     public async Task<bool> DeactivateAsync(string id)
68.     {
69.         var node = await _nodeRepository.GetByIdAsync(id);
70.         if (node == null) return false;
71.
72.         node.IsActive = false;
73.         await _nodeRepository.UpdateAsync(id, node);
74.         return true;
75.     }
76. }
77.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Services[ProsumerService.cs](#)

```

1.  // -----
2.  // File: ProsumerService.cs
3.  // Purpose: Prosumer account management – registration, profile updates and
4.  // activate/deactivate status changes for solar owners trading on the grid.
5.  // Enforces NIC uniqueness and stores passwords as SHA-256 hashes.
6.  // -----
7.
8.
9.  using System.Security.Cryptography;
10. using System.Text;
11. using SolarGrid.Api.Models;

```

```

12. using SolarGrid.Api.Repositories;
13.
14. namespace SolarGrid.Api.Services;
15.
16. /// <summary>
17. /// Manages prosumer accounts – solar property owners who trade energy on the
18. /// grid via the mobile app. Registered and managed by Backoffice officers.
19. /// </summary>
20. public class ProsumerService
21. {
22.     private readonly IProsumerRepository _prosumerRepository;
23.
24.     public ProsumerService(IProsumerRepository prosumerRepository)
25.     {
26.         _prosumerRepository = prosumerRepository;
27.     }
28.
29.     public async Task<List<Prosumer>> GetAllAsync() =>
30.         await _prosumerRepository.GetAllAsync();
31.
32.     /// <summary>Returns null if a prosumer with that NIC already exists.</summary>
33.     public async Task<Prosumer?> CreateAsync(CreateProsumerRequest request)
34.     {
35.         var existing = await _prosumerRepository.GetByNicAsync(request.Nic);
36.         if (existing != null) return null;
37.
38.         var prosumer = new Prosumer
39.         {
40.             Nic          = request.Nic,
41.             FullName     = request.FullName,
42.             Email        = request.Email,
43.             Phone        = request.Phone,
44.             Address      = request.Address,
45.             PasswordHash = HashPassword(request.Password),
46.             IsActive     = true
47.         };
48.
49.         await _prosumerRepository.CreateAsync(prosumer);
50.         return prosumer;
51.     }
52.
53.     public async Task<Prosumer?> UpdateAsync(string nic, UpdateProsumerRequest request)
54.     {
55.         var prosumer = await _prosumerRepository.GetByNicAsync(nic);
56.         if (prosumer == null) return null;
57.
58.         prosumer.FullName = request.FullName;
59.         prosumer.Email    = request.Email;
60.         prosumer.Phone    = request.Phone;
61.         prosumer.Address  = request.Address;
62.
63.         await _prosumerRepository.UpdateAsync(nic, prosumer);
64.         return prosumer;
65.     }
66.
67.     public async Task<bool> DeactivateAsync(string nic)
68.     {
69.         var prosumer = await _prosumerRepository.GetByNicAsync(nic);
70.         if (prosumer == null) return false;
71.
72.         prosumer.IsActive = false;
73.         await _prosumerRepository.UpdateAsync(nic, prosumer);
74.         return true;
75.     }
76.

```

```

77.     /// <summary>Reactivation is a Backoffice-only action.</summary>
78.     public async Task<bool> ReactivateAsync(string nic)
79.     {
80.         var prosumer = await _prosumerRepository.GetByNicAsync(nic);
81.         if (prosumer == null) return false;
82.
83.         prosumer.IsActive = true;
84.         await _prosumerRepository.UpdateAsync(nic, prosumer);
85.         return true;
86.     }
87.
88.     private static string HashPassword(string password)
89.     {
90.         using var sha256 = SHA256.Create();
91.         var bytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
92.         return Convert.ToBase64String(bytes);
93.     }
94. }
95.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Services\ReservationService.cs

```

// -----
// File: ReservationService.cs
// Purpose: Reservation lifecycle logic (create / update / cancel / approve /
// complete) for prosumer energy slot bookings. Enforces the 7-day scheduling
// window and 12-hour notice rules, and issues a hashed QR code on approval.
// -----
1. using System.Security.Cryptography;
2. using System.Text;
3. using SolarGrid.Api.Models;
4. using SolarGrid.Api.Repositories;
5.
6. namespace SolarGrid.Api.Services;
7.
8. /// <summary>
9. /// Manages energy slot reservations with the full business lifecycle:
10. /// Pending → Approved (QR issued) → Completed | Cancelled.
11. ///
12. /// Business rules preserved from original BookingService:
13. /// - Must be scheduled within the next 7 days
14. /// - Modification/cancellation requires at least 12 hours' notice
15. /// </summary>
16. public class ReservationService
17. {
18.     private readonly IReservationRepository _reservationRepository;
19.
20.     public ReservationService(IReservationRepository reservationRepository)
21.     {
22.         _reservationRepository = reservationRepository;
23.     }
24.
25.     public async Task<List<Reservation>> GetAllAsync() =>
26.         await _reservationRepository.GetAllAsync();
27.
28.     public async Task<List<Reservation>> GetByProsumerNicAsync(string nic) =>
29.         await _reservationRepository.GetByProsumerNicAsync(nic);
30.
31.     public async Task<Reservation> CreateAsync(CreateReservationRequest request)
32.     {
33.         if (request.ScheduledDateTime < DateTime.UtcNow ||
34.             request.ScheduledDateTime > DateTime.UtcNow.AddDays(7))
35.         {

```

```

36.         throw new ArgumentException("Reservation must be scheduled within the next 7
days.");
37.     }
38.
39.     var reservation = new Reservation
40.     {
41.         ProsumerNic      = request.ProsumerNic,
42.         NodeId           = request.NodeId,
43.         ScheduledDateTime = request.ScheduledDateTime,
44.         DurationMinutes  = request.DurationMinutes,
45.         Status            = "Pending",
46.         CreatedAt         = DateTime.UtcNow
47.     };
48.
49.     await _reservationRepository.CreateAsync(reservation);
50.     return reservation;
51. }
52.
53. public async Task<Reservation?> UpdateAsync(string id, UpdateReservationRequest request)
54. {
55.     var reservation = await _reservationRepository.GetByIdAsync(id);
56.     if (reservation == null) return null;
57.
58.     if (reservation.Status is "Completed" or "Cancelled")
59.         throw new ArgumentException("Cannot modify a completed or cancelled reservation.");
60.
61.     if (DateTime.UtcNow > reservation.ScheduledDateTime.AddHours(-12))
62.         throw new ArgumentException("Reservations can only be modified at least 12 hours
before the scheduled time.");
63.
64.     if (request.ScheduledDateTime < DateTime.UtcNow ||
65.         request.ScheduledDateTime > DateTime.UtcNow.AddDays(7))
66.     {
67.         throw new ArgumentException("New scheduled time must be within the next 7 days.");
68.     }
69.
70.     reservation.ScheduledDateTime = request.ScheduledDateTime;
71.     reservation.DurationMinutes   = request.DurationMinutes;
72.     reservation.UpdatedAt         = DateTime.UtcNow;
73.
74.     await _reservationRepository.UpdateAsync(id, reservation);
75.     return reservation;
76. }
77.
78. public async Task<bool> CancelAsync(string id)
79. {
80.     var reservation = await _reservationRepository.GetByIdAsync(id);
81.     if (reservation == null) return false;
82.
83.     if (reservation.Status is "Completed" or "Cancelled")
84.         throw new ArgumentException("This reservation is already completed or cancelled.");
85.
86.     if (DateTime.UtcNow > reservation.ScheduledDateTime.AddHours(-12))
87.         throw new ArgumentException("Reservations can only be cancelled at least 12 hours
before the scheduled time.");
88.
89.     reservation.Status      = "Cancelled";
90.     reservation.UpdatedAt   = DateTime.UtcNow;
91.
92.     await _reservationRepository.UpdateAsync(id, reservation);
93.     return true;
94. }
95.
96. /// <summary>
97. /// Approving a reservation generates a short QR dispatch code so the prosumer

```

```

98.     /// can prove identity at the node terminal.
99.     /// </summary>
100.    public async Task<Reservation?> ApproveAsync(string id)
101.    {
102.        var reservation = await _reservationRepository.GetByIdAsync(id);
103.        if (reservation == null) return null;
104.
105.        if (reservation.Status != "Pending")
106.            throw new ArgumentException("Only Pending reservations can be approved.");
107.
108.        reservation.Status = "Approved";
109.        reservation.QrCode = GenerateQrCode($"{reservation.ProsumerNic}-{reservation.NodeId}-{reservation.ScheduledDateTime:0}");
110.        reservation.UpdatedAt = DateTime.UtcNow;
111.
112.        await _reservationRepository.UpdateAsync(id, reservation);
113.        return reservation;
114.    }
115.
116.    public async Task<Reservation?> CompleteAsync(string id)
117.    {
118.        var reservation = await _reservationRepository.GetByIdAsync(id);
119.        if (reservation == null) return null;
120.
121.        if (reservation.Status != "Approved")
122.            throw new ArgumentException("Only Approved reservations can be completed.");
123.
124.        reservation.Status = "Completed";
125.        reservation.UpdatedAt = DateTime.UtcNow;
126.
127.        await _reservationRepository.UpdateAsync(id, reservation);
128.        return reservation;
129.    }
130.
131.    /// <summary>Returns a short uppercase hex string suitable for display as a QR code value.</summary>
132.    private static string GenerateQrCode(string input)
133.    {
134.        using var sha256 = SHA256.Create();
135.        var bytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(input));
136.        return Convert.ToHexString(bytes)[..12];
137.    }
138. }
139.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\Program.cs

```

// -----
// File: Program.cs
// Purpose: API host entry point. Builds the DI container (MongoDB client,
// repositories, services), enables Swagger and the CORS policy for the Vite
// dev server, then maps the controllers and starts the web host.
// -----

1. using MongoDB.Driver;
2. using SolarGrid.Api.Configuration;
3. using SolarGrid.Api.Repositories;
4. using SolarGrid.Api.Services;
5.
6. var builder = WebApplication.CreateBuilder(args);
7.
8. // — Database —————
9. builder.Services.Configure<SolarGridDatabaseSettings>(
10.     builder.Configuration.GetSection("SolarGridDatabase"));

```

```

11.
12. builder.Services.AddSingleton<IMongoClient>(s =>
13. {
14.     var settings = builder.Configuration
15.         .GetSection("SolarGridDatabase")
16.         .Get<SolarGridDatabaseSettings>();
17.     return new MongoClient(settings!.ConnectionString);
18. });
19.
20. // — CORS — allow the Vite dev server —————
21. builder.Services.AddCors(options =>
22. {
23.     options.AddPolicy("FrontendPolicy", policy =>
24.     {
25.         policy.WithOrigins("http://localhost:5173")
26.             .AllowAnyHeader()
27.             .AllowAnyMethod();
28.     });
29. });
30.
31. // — Repositories —————
32. builder.Services.AddScoped<IAppUserRepository, AppUserRepository>();
33. builder.Services.AddScoped<IProsumerRepository, ProsumerRepository>();
34. builder.Services.AddScoped<INodeRepository, NodeRepository>();
35. builder.Services.AddScoped<IReservationRepository, ReservationRepository>();
36.
37. // — Services —————
38. builder.Services.AddScoped<AuthService>();
39. builder.Services.AddScoped<AppUserService>();
40. builder.Services.AddScoped<ProsumerService>();
41. builder.Services.AddScoped<NodeService>();
42. builder.Services.AddScoped<ReservationService>();
43. builder.Services.AddScoped<DashboardService>();
44.
45. // — MVC & Swagger —————
46. builder.Services.AddControllers();
47. builder.Services.AddEndpointsApiExplorer();
48. builder.Services.AddSwaggerGen();
49.
50. var app = builder.Build();
51.
52. app.UseSwagger();
53. app.UseSwaggerUI();
54.
55. // CORS must come before routing/auth middleware
56. app.UseCors("FrontendPolicy");
57.
58. app.UseAuthorization();
59. app.MapControllers();
60.
61. app.Run();
62.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\web-service\SolarGrid.Api\appsettings.json

```

1. {
2.     "Logging": {
3.         "LogLevel": {
4.             "Default": "Information",
5.             "Microsoft.AspNetCore": "Warning"
6.         }
7.     },
8.     "AllowedHosts": "*",

```

```

9.     "Urls": "http://*:8080",
10.    "SolarGridDatabase": {
11.        "ConnectionString":
12.        "mongodb+srv://kavindugunasena_db_user:XuDpkjkPyXNAxiH6@cluster0.a4wqads.mongodb.net/",
13.        "DatabaseName": "SolarGridDB",
14.        "AppUsersCollectionName": "AppUsers",
15.        "ProsumersCollectionName": "Prosumers",
16.        "NodesCollectionName": "Nodes",
17.        "ReservationsCollectionName": "Reservations"
18.    }
19. }

```

Mobile application Source code (not screen shots).

android\app\src\main\java\com\team\smartsolar\adapters\BookingAdapter.java

```

1. // -----
2. // File: BookingAdapter.java
3. // Purpose: Binds Prosumer reservation data to the RecyclerView in the MyBookings
4. // activity. Handles click routing to the QR generator or modification screens.
5. // -----
6. //
7.
8. package com.team.smartsolar.adapters;
9.
10. import android.view.LayoutInflater;
11. import android.view.View;
12. import android.view.ViewGroup;
13. import android.widget.TextView;
14. import androidx.annotation.NonNull;
15. import androidx.recyclerview.widget.RecyclerView;
16. import com.team.smartsolar.R;
17. import com.team.smartsolar.models.Booking;
18. import java.util.List;
19.
20. public class BookingAdapter extends RecyclerView.Adapter<BookingAdapter.BookingViewHolder> {
21.
22.     private List<Booking> bookingList;
23.
24.     public BookingAdapter(List<Booking> bookingList) {
25.         this.bookingList = bookingList;
26.     }
27.
28.     @NonNull
29.     @Override
30.     public BookingViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
31.         View view = LayoutInflater.from(parent.getContext()).inflate(R.layout.item_booking,
32.         parent, false);
33.         return new BookingViewHolder(view);
34.     }
35.
36.     @Override
37.     public void onBindViewHolder(@NonNull BookingViewHolder holder, int position) {
38.         Booking booking = bookingList.get(position);
39.         holder.txtItemStation.setText(booking.getStationName());
40.         holder.txtItemDateTime.setText(booking.getDate() + " | " + booking.getTime());
41.         holder.txtItemEnergy.setText("Reserved: " + booking.getEnergyAmount() + " kWh");

```



```

41.         holder.txtItemStatus.setText(booking.getStatus().toUpperCase());
42.
43.         // Make the entire card clickable
44.         holder.itemView.setOnClickListener(v -> {
45.             if (booking.getStatus().equalsIgnoreCase("Approved")) {
46.                 // Open the QR ticket
47.                 android.content.Intent intent = new android.content.Intent(v.getContext(),
com.team.smartsolar.ViewQrActivity.class);
48.                 intent.putExtra("STATION", booking.getStationName());
49.                 intent.putExtra("DATE", booking.getDate() + " " + booking.getTime());
50.                 intent.putExtra("ENERGY", booking.getEnergyAmount());
51.                 v.getContext().startActivity(intent);
52.
53.             } else if (booking.getStatus().equalsIgnoreCase("Pending")) {
54.                 // Open the Modify/Cancel screen
55.                 android.content.Intent intent = new android.content.Intent(v.getContext(),
com.team.smartsolar.ModifyBookingActivity.class);
56.                 intent.putExtra("BOOKING_ID", booking.getId());
57.                 intent.putExtra("STATION", booking.getStationName());
58.                 intent.putExtra("DATE", booking.getDate());
59.                 intent.putExtra("TIME", booking.getTime());
60.                 intent.putExtra("ENERGY", booking.getEnergyAmount());
61.                 v.getContext().startActivity(intent);
62.
63.             } else {
64.                 android.widget.Toast.makeText(v.getContext(),
65.                     "This booking is already " + booking.getStatus(),
66.                     android.widget.Toast.LENGTH_SHORT).show();
67.             }
68.         });
69.     }
70.
71.     @Override
72.     public int getItemCount() {
73.         return bookingList.size();
74.     }
75.
76.     static class BookingViewHolder extends RecyclerView.ViewHolder {
77.         TextView txtItemStation, txtItemDateTime, txtItemEnergy, txtItemStatus;
78.
79.         public BookingViewHolder(@NonNull View itemView) {
80.             super(itemView);
81.             txtItemStation = itemView.findViewById(R.id.txtItemStation);
82.             txtItemDateTime = itemView.findViewById(R.id.txtItemDateTime);
83.             txtItemEnergy = itemView.findViewById(R.id.txtItemEnergy);
84.             txtItemStatus = itemView.findViewById(R.id.txtItemStatus);
85.         }
86.     }
87.     // Method to update the list when a search filter is applied
88.     public void filterList(List<Booking> filteredList) {
89.         this.bookingList = filteredList;
90.         notifyDataSetChanged();
91.     }
92. }
93.

```

android\app\src\main\java\com\team\smartsolar\database\DatabaseHelper.java

```

1. // -----
2. // File: DatabaseHelper.java
3. // Purpose: Implements SQLiteOpenHelper for local session and cache persistence,
5. // fulfilling the pure native Android local storage requirement without using Room.
6. // -----

```

```

7.
8. package com.team.smartsolar.database;
9.
10. import android.content.ContentValues;
11. import android.content.Context;
12. import android.database.Cursor;
13. import android.database.sqlite.SQLiteDatabase;
14. import android.database.sqlite.SQLiteOpenHelper;
15.
16. import com.team.smartsolar.models.NodeResponse;
17. import com.team.smartsolar.models.ProsumerProfile;
18.
19. import java.util.ArrayList;
20. import java.util.List;
21.
22. public class DatabaseHelper extends SQLiteOpenHelper {
23.
24.     private static final String DATABASE_NAME = "SmartSolarLocal.db";
25.     // Incremented to 3 to force Android to safely rebuild the tables with the new token
columns
26.     private static final int DATABASE_VERSION = 5;
27.
28.     // Tables
29.     private static final String TABLE_SESSION = "Session";
30.     private static final String TABLE_PROFILE = "UserProfileCache";
31.     private static final String TABLE_STATION = "StationCache";
32.
33.     public DatabaseHelper(Context context) {
34.         super(context, DATABASE_NAME, null, DATABASE_VERSION);
35.     }
36.
37.     @Override
38.     public void onCreate(SQLiteDatabase db) {
39.         // 1. Session Table (Tracks logged-in user, role, and JWT token)
40.         db.execSQL("CREATE TABLE " + TABLE_SESSION + " (" +
41.             "nic TEXT PRIMARY KEY, " +
42.             "role TEXT, " +
43.             "token TEXT)");
44.
45.         // 2. Profile Cache Table (Stores user details for offline viewing)
46.         db.execSQL("CREATE TABLE " + TABLE_PROFILE + " (" +
47.             "nic TEXT PRIMARY KEY, " +
48.             "name TEXT, " +
49.             "email TEXT, " +
50.             "phone TEXT, " +
51.             "address TEXT, " +
52.             "isActive INTEGER, " +
53.             "lastSyncedAt INTEGER)");
54.
55.         // 3. Station Cache Table (Stores grid nodes for the map and booking dropdown)
56.         db.execSQL("CREATE TABLE " + TABLE_STATION + " (" +
57.             "stationId TEXT PRIMARY KEY, " +
58.             "name TEXT, " +
59.             "latitude REAL, " +
60.             "longitude REAL, " +
61.             "capacity REAL, " +
62.             "availableSlots INTEGER, " +
63.             "lastSyncedAt INTEGER)");
64.     }
65.
66.     @Override
67.     public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
68.         db.execSQL("DROP TABLE IF EXISTS " + TABLE_SESSION);
69.         db.execSQL("DROP TABLE IF EXISTS " + TABLE_PROFILE);
70.         db.execSQL("DROP TABLE IF EXISTS " + TABLE_STATION);

```

```

71.         onCreate(db);
72.     }
73.
74.     // =====
75.     // SESSION MANAGEMENT
76.     // =====
77.
78.     public void saveSession(String nic, String role, String token) {
79.         SQLiteDatabase db = this.getWritableDatabase();
80.         db.execSQL("DELETE FROM " + TABLE_SESSION); // Keep only one active session
81.
82.         ContentValues values = new ContentValues();
83.         values.put("nic", nic);
84.         values.put("role", role);
85.         values.put("token", token);
86.         db.insert(TABLE_SESSION, null, values);
87.         db.close();
88.     }
89.
90.     public String getSessionNic() {
91.         SQLiteDatabase db = this.getReadableDatabase();
92.         Cursor cursor = db.rawQuery("SELECT nic FROM " + TABLE_SESSION + " LIMIT 1", null);
93.         String nic = null;
94.         if (cursor.moveToFirst()) {
95.             nic = cursor.getString(0);
96.         }
97.         cursor.close();
98.         db.close();
99.         return nic;
100.    }
101.
102.    public String getSessionToken() {
103.        SQLiteDatabase db = this.getReadableDatabase();
104.        Cursor cursor = db.rawQuery("SELECT token FROM " + TABLE_SESSION + " LIMIT 1", null);
105.        String token = null;
106.        if (cursor.moveToFirst()) {
107.            token = cursor.getString(0);
108.        }
109.        cursor.close();
110.        db.close();
111.        return token;
112.    }
113.
114.    public String getSessionRole() {
115.        SQLiteDatabase db = this.getReadableDatabase();
116.        Cursor cursor = db.rawQuery("SELECT role FROM " + TABLE_SESSION + " LIMIT 1", null);
117.        String role = null;
118.        if (cursor.moveToFirst()) {
119.            role = cursor.getString(0);
120.        }
121.        cursor.close();
122.        db.close();
123.        return role;
124.    }
125.
126.    public void logoutUser() {
127.        SQLiteDatabase db = this.getWritableDatabase();
128.        db.execSQL("DELETE FROM " + TABLE_SESSION);
129.        db.execSQL("DELETE FROM " + TABLE_PROFILE); // Clear sensitive profile cache on logout
130.        db.close();
131.    }
132.
133.    // =====
134.    // PROFILE CACHE
135.    // =====

```

```

136.
137.     public void cacheProfile(String nic, ProsumerProfile profile) {
138.         SQLiteDatabase db = this.getWritableDatabase();
139.         ContentValues values = new ContentValues();
140.         values.put("nic", nic);
141.         values.put("name", profile.getFullName());
142.         values.put("email", profile.getEmail());
143.         values.put("phone", profile.getPhone());
144.         values.put("address", profile.getAddress());
145.         values.put("isActive", profile.isActive() ? 1 : 0);
146.         values.put("lastSyncedAt", System.currentTimeMillis());
147.
148.         // Insert or replace ensures we don't get primary key crashes on updates
149.         db.insertWithOnConflict(TABLE_PROFILE, null, values, SQLiteDatabase.CONFLICT_REPLACE);
150.         db.close();
151.     }
152.
153.     public ProsumerProfile getCacheProfile(String nic) {
154.         SQLiteDatabase db = this.getReadableDatabase();
155.         Cursor cursor = db.rawQuery("SELECT * FROM " + TABLE_PROFILE + " WHERE nic = ?", new
String[]{nic});
156.
157.         ProsumerProfile profile = null;
158.         if (cursor.moveToFirst()) {
159.             String name = cursor.getString(cursor.getColumnIndexOrThrow("name"));
160.             String email = cursor.getString(cursor.getColumnIndexOrThrow("email"));
161.             String phone = cursor.getString(cursor.getColumnIndexOrThrow("phone"));
162.             String address = cursor.getString(cursor.getColumnIndexOrThrow("address"));
163.             boolean isActive = cursor.getInt(cursor.getColumnIndexOrThrow("isActive")) == 1;
164.
165.             profile = new ProsumerProfile(name, email, phone, address);
166.             profile.setActive(isActive);
167.         }
168.         cursor.close();
169.         db.close();
170.         return profile;
171.     }
172.
173.     // =====
174.     // STATION CACHE
175.     // =====
176.
177.     public void cacheStations(List<NodeResponse> stations) {
178.         SQLiteDatabase db = this.getWritableDatabase();
179.         db.beginTransaction();
180.         try {
181.             // Clear old cache to remove deleted stations
182.             db.execSQL("DELETE FROM " + TABLE_STATION);
183.
184.             for (NodeResponse station : stations) {
185.                 ContentValues values = new ContentValues();
186.                 values.put("stationId", station.getId() != null ? station.getId() :
station.getStationId());
187.                 values.put("name", station.getName() != null ? station.getName() :
station.getStationName());
188.                 values.put("latitude", station.getLatitude());
189.                 values.put("longitude", station.getLongitude());
190.                 values.put("capacity", station.getCapacityKWh());
191.                 values.put("availableSlots", station.getAvailableBatterySlots());
192.                 values.put("lastSyncedAt", System.currentTimeMillis());
193.
194.                 db.insert(TABLE_STATION, null, values);
195.             }
196.             db.setTransactionSuccessful();
197.         } finally {

```

```

198.         db.endTransaction();
199.         db.close();
200.     }
201. }
202.
203. public List<NodeResponse> getCachedStations() {
204.     List<NodeResponse> stationList = new ArrayList<>();
205.     SQLiteDatabase db = this.getReadableDatabase();
206.     Cursor cursor = db.rawQuery("SELECT * FROM " + TABLE_STATION, null);
207.
208.     if (cursor.moveToFirst()) {
209.         do {
210.             NodeResponse station = new NodeResponse();
211.
212.             station.setStationId(cursor.getString(cursor.getColumnIndexOrThrow("stationId")));
213.             station.setStationName(cursor.getString(cursor.getColumnIndexOrThrow("name")));
214.             station.setLatitude(cursor.getDouble(cursor.getColumnIndexOrThrow("latitude")));
215.             station.setLongitude(cursor.getDouble(cursor.getColumnIndexOrThrow("longitude")));
216.             station.setCapacityKWh(cursor.getDouble(cursor.getColumnIndexOrThrow("capacity")));
217.             station.setAvailableBatterySlots(cursor.getInt(cursor.getColumnIndexOrThrow("availableSlots")));
218.
219.             stationList.add(station);
220.         } while (cursor.moveToNext());
221.         cursor.close();
222.         db.close();
223.         return stationList;
224.     }
225. }
226.

```

android\app\src\main\java\com\team\smartsolar\models\LoginResponse.java

```

1. // -----
2. // File: LoginResponse.java
3.
4. // Purpose: Data Transfer Object (DTO) for parsing the authentication response
5. // containing the JWT and role from the central C# Web API.
6. // -----
7.
8. package com.team.smartsolar.models;
9.
10. public class LoginResponse {
11.     private String token;
12.     private String role;
13.
14.     public String getToken() { return token; }
15.     public String getRole() { return role; }
16. }
17.

```

android\app\src\main\java\com\team\smartsolar\models\NodeResponse.java

```

1. // -----
2. // File: NodeResponse.java
3.
4. // Purpose: DTO representing a microgrid node's data from the server, mapping

```

```

5. // exact C# JSON keys to local Android variables for map and dropdown rendering.
6. // -----
7.
8. package com.team.smartsolar.models;
9.
10. import com.google.gson.annotations.SerializedName;
11.
12. public class NodeResponse {
13.
14.     @SerializedName("id") // C# returns "id"
15.     private String id;
16.
17.     @SerializedName("name") // C# returns "name"
18.     private String name;
19.
20.     @SerializedName("latitude")
21.     private double latitude;
22.
23.     @SerializedName("longitude")
24.     private double longitude;
25.
26.     @SerializedName("capacityKWh")
27.     private double capacityKWh;
28.
29.     @SerializedName("totalBatterySlots")
30.     private int totalBatterySlots;
31.
32.     @SerializedName("availableBatterySlots")
33.     private int availableBatterySlots;
34.
35.     @SerializedName("isActive")
36.     private boolean isActive;
37.
38.     public String getId() { return id; }
39.     public String getName() { return name; }
40.     public double getLatitude() { return latitude; }
41.     public double getLongitude() { return longitude; }
42.     public double getCapacityKWh() { return capacityKWh; }
43.     public int getTotalBatterySlots() { return totalBatterySlots; }
44.     public int getAvailableBatterySlots() { return availableBatterySlots; }
45.     public boolean isActive() { return isActive; }
46.
47.     // Convenience aliases so existing code does not break
48.     public String getStationId() { return id; }
49.     public String getStationName() { return name != null ? name : "Station " + id; }
50.     public void setStationId(String id) {
51.         this.id = id;
52.     }
53.
54.     public void setStationName(String name) {
55.         this.name = name;
56.     }
57.
58.     public void setLatitude(double latitude) {
59.         this.latitude = latitude;
60.     }
61.
62.     public void setLongitude(double longitude) {
63.         this.longitude = longitude;
64.     }
65.
66.     public void setCapacityKWh(double capacityKWh) {
67.         this.capacityKWh = capacityKWh;
68.     }
69.

```

```

70.     public void setAvailableBatterySlots(int availableBatterySlots) {
71.         this.availableBatterySlots = availableBatterySlots;
72.     }
73. }
74.

```

android\app\src\main\java\com\team\smartsolar\models\ProsumerProfile.java

```

1.  // -----
2.  // File: ProsumerProfile.java
3.  // Author: Nithika Perera
4.  // Purpose: DTO for the Prosumer profile information, used for parsing network
5.  // responses and persisting user data to the local SQLite cache.
6.  // -----
7.
8.  package com.team.smartsolar.models;
9.
10. import com.google.gson.annotations.SerializedName;
11.
12. public class ProsumerProfile {
13.     @SerializedName("nic")
14.     private String nic;
15.
16.     @SerializedName("fullName")
17.     private String fullName;
18.
19.     @SerializedName("email")
20.     private String email;
21.
22.     @SerializedName("phone")
23.     private String phone;
24.
25.     @SerializedName("address")
26.     private String address;
27.
28.     @SerializedName("isActive")
29.     private boolean isActive;
30.
31.     public ProsumerProfile(String fullName, String email, String phone, String address) {
32.         this.fullName = fullName;
33.         this.email = email;
34.         this.phone = phone;
35.         this.address = address;
36.     }
37.
38.     public String getNic() { return nic; }
39.     public String getFullName() { return fullName; }
40.     public String getEmail() { return email; }
41.     public String getPhone() { return phone; }
42.     public String getAddress() { return address; }
43.     public boolean isActive() { return isActive; }
44.     public void setActive(boolean active) {
45.         this.isActive = active;
46.     }
47. }
48.

```

android\app\src\main\java\com\team\smartsolar\models\RegisterRequest.java

```

1.  // -----
2.  // File: RegisterRequest.java

```

```

3.
4. // Purpose: DTO used to package Prosumer registration inputs into a JSON payload
5. // to be sent to the central C# Web API.
6. // -----
7.
8. package com.team.smartsolar.models;
9.
10. public class RegisterRequest {
11.     private String username;
12.     private String password;
13.     private String fullName;
14.     private String email;
15.     private String role;
16.
17.     public RegisterRequest(String username, String password, String fullName, String email,
String role) {
18.         this.username = username;
19.         this.password = password;
20.         this.fullName = fullName;
21.         this.email = email;
22.         this.role = role;
23.     }
24. }
25.

```

android\app\src\main\java\com\team\smartsolar\models\ReservationResponse.java

```

1. // -----
2. // File: ReservationResponse.java
3.
4. // Purpose: DTO for parsing incoming reservation details retrieved from the
5. // central API, including tracking the booking lifecycle status.
6. // -----
7.
8. package com.team.smartsolar.models;
9.
10. import com.google.gson.annotations.SerializedName;
11.
12. public class ReservationResponse {
13.     @SerializedName("id")
14.     private String id;
15.
16.     @SerializedName("nodeId")
17.     private String nodeId;
18.
19.     @SerializedName("scheduledDateTime")
20.     private String scheduledDateTime;
21.
22.     @SerializedName("durationMinutes")
23.     private int durationMinutes;
24.
25.     @SerializedName("energyAmount")
26.     private double energyAmount;
27.
28.
29.     @SerializedName("status")
30.     private String status;
31.
32.     public String getId() { return id; }
33.     public String getNodeId() { return nodeId; }
34.     public String getScheduledDateTime() { return scheduledDateTime; }
35.     public int getDurationMinutes() { return durationMinutes; }
36.     public String getStatus() { return status; }

```



```

37.     public double getEnergyAmount() { return energyAmount; }
38.
39. }
40.

```

android\app\src\main\java\com\team\smartsolar\models\Station.java

```

1. // -----
2. // File: Station.java
3.
4. // Purpose: Lightweight DTO representing basic station telemetry data, mapped to
5. // the server's JSON contract.
6. // -----
7.
8. package com.team.smartsolar.models;
9.
10. import com.google.gson.annotations.SerializedName;
11.
12. public class Station {
13.
14.     // @SerializedName maps the exact C# JSON keys to your Java variables
15.     @SerializedName("id")
16.     private String id;
17.
18.     @SerializedName("name")
19.     private String name;
20.
21.     @SerializedName("address")
22.     private String address;
23.
24.     @SerializedName("latitude")
25.     private double latitude;
26.
27.     @SerializedName("longitude")
28.     private double longitude;
29.
30.     @SerializedName("powerCapacityKw")
31.     private double powerCapacityKw;
32.
33.     @SerializedName("status")
34.     private String status;
35.
36.     // Getters so the Map can read the data
37.     public String getId() { return id; }
38.     public String getName() { return name; }
39.     public String getAddress() { return address; }
40.     public double getLatitude() { return latitude; }
41.     public double getLongitude() { return longitude; }
42.     public double getPowerCapacityKw() { return powerCapacityKw; }
43.     public String getStatus() { return status; }
44. }
45.

```

android\app\src\main\java\com\team\smartsolar\models\UpdateReservationRequest.java

```

1. // -----
2. // File: UpdateReservationRequest.java
3.
4. // Purpose: DTO packaging modified reservation details (date, time, energy) to
5. // send to the API while complying with the 12-hour modification rule.
6. // -----

```

```

7.
8. package com.team.smartsolar.models;
9.
10. import com.google.gson.annotations.SerializedName;
11.
12. public class UpdateReservationRequest {
13.     @SerializedName("scheduledDateTime")
14.     private String scheduledDateTime;
15.
16.     @SerializedName("durationMinutes")
17.     private int durationMinutes;
18.
19.     @SerializedName("energyAmount")
20.     private double energyAmount;
21.
22.     public UpdateReservationRequest(String scheduledDateTime, int durationMinutes, double
energyAmount) {
23.         this.scheduledDateTime = scheduledDateTime;
24.         this.durationMinutes = durationMinutes;
25.         this.energyAmount = energyAmount;
26.     }
27. }
28.

```

android\app\src\main\java\com\team\smartsolar\models\Booking.java

```

1. // -----
2. // File: Booking.java
3.
4. // Purpose: Local UI model used to populate the custom RecyclerView adapter in
5. // the My Bookings history and filtering screen.
6. // -----
7.
8. package com.team.smartsolar.models;
9.
10. public class Booking {
11.     private String id;
12.     private String stationName;
13.     private String date;
14.     private String time;
15.     private String energyAmount;
16.     private String status;
17.
18.     public Booking(String id, String stationName, String date, String time, String energyAmount,
String status) {
19.         this.id = id;
20.         this.stationName = stationName;
21.         this.date = date;
22.         this.time = time;
23.         this.energyAmount = energyAmount;
24.         this.status = status;
25.     }
26.
27.     public String getId(){return id;}
28.     public String getStationName() { return stationName; }
29.     public String getDate() { return date; }
30.     public String getTime() { return time; }
31.     public String getEnergyAmount() { return energyAmount; }
32.     public String getStatus() { return status; }
33. }
34.

```

android\app\src\main\java\com\team\smartsolar\models\CreateReservationRequest.java

```
1. // -----
2. // File: CreateReservationRequest.java
3.
4. // Purpose: DTO packaging new reservation parameters for submission to the
5. // central API, subject to the 7-day advance booking window rule.
6. // -----
7.
8. package com.team.smartsolar.models;
9.
10. import com.google.gson.annotations.SerializedName;
11.
12. public class CreateReservationRequest {
13.     @SerializedName("prosumerNic")
14.     private String prosumerNic;
15.
16.     @SerializedName("nodeId")
17.     private String nodeId;
18.
19.     @SerializedName("scheduledDateTime")
20.     private String scheduledDateTime;
21.
22.     @SerializedName("durationMinutes")
23.     private int durationMinutes;
24.
25.     @SerializedName("energyAmount")
26.     private double energyAmount;
27.
28.     public CreateReservationRequest(String prosumerNic, String nodeId, String scheduledDateTime,
int durationMinutes, double energyAmount) {
29.         this.prosumerNic = prosumerNic;
30.         this.nodeId = nodeId;
31.         this.scheduledDateTime = scheduledDateTime;
32.         this.durationMinutes = durationMinutes;
33.         this.energyAmount = energyAmount;
34.     }
35. }
36.
```

android\app\src\main\java\com\team\smartsolar\models>LoginRequest.java

```
1. // -----
2. // File: LoginRequest.java
3.
4. // Purpose: DTO for securely transmitting Prosumer credentials (NIC and password)
5. // to the authentication endpoint of the central service.
6. // -----
7.
8. package com.team.smartsolar.models;
9.
10. import com.google.gson.annotations.SerializedName;
11.
12. public class LoginRequest {
13.
14.     // Forces Retrofit to send this as "username" in the JSON body
15.     @SerializedName("username")
16.     private String nic;
17.
18.     @SerializedName("password")
19.     private String password;
20. }
```

```

21.     public LoginRequest(String nic, String password) {
22.         this.nic = nic;
23.         this.password = password;
24.     }
25. }
26.

```

android\app\src\main\java\com\team\smartsolar\network\RetrofitClient.java

```

1. // -----
2. // File: RetrofitClient.java
3.
4. // Purpose: Configures and provides the singleton Retrofit instance used to handle
5. // all network communication between the Android client and the C# Web API.
6. // -----
7.
8. package com.team.smartsolar.network;
9.
10. import java.security.cert.CertificateException;
11. import javax.net.ssl.HostnameVerifier;
12. import javax.net.ssl.SSLContext;
13. import javax.net.ssl.SSLSession;
14. import javax.net.ssl.SSLSocketFactory;
15. import javax.net.ssl.TrustManager;
16. import javax.net.ssl.X509TrustManager;
17. import okhttp3.OkHttpClient;
18. import retrofit2.Retrofit;
19. import retrofit2.converter.gson.GsonConverterFactory;
20.
21. public class RetrofitClient {
22.
23.     // Remember to use the port your API is running on!
24.     private static final String BASE_URL = "http://10.0.2.2:8080";
25.
26.     private static Retrofit retrofit = null;
27.
28.     public static Retrofit getClient() {
29.         if (retrofit == null) {
30.             retrofit = new Retrofit.Builder()
31.                 .baseUrl(BASE_URL)
32.                 .client(getUnsafeOkHttpClient()) // Inject the bypassing client here
33.                 .addConverterFactory(GsonConverterFactory.create())
34.                 .build();
35.         }
36.         return retrofit;
37.     }
38.
39.     // --- BYPASS SSL FOR LOCAL DEVELOPMENT ---
40.     private static OkHttpClient getUnsafeOkHttpClient() {
41.         try {
42.             // Create a trust manager that does not validate certificate chains
43.             final TrustManager[] trustAllCerts = new TrustManager[] {
44.                 new X509TrustManager() {
45.                     @Override
46.                     public void checkClientTrusted(java.security.cert.X509Certificate[]
chain, String authType) throws CertificateException {}
47.
48.                     @Override
49.                     public void checkServerTrusted(java.security.cert.X509Certificate[]
chain, String authType) throws CertificateException {}
50.
51.                     @Override
52.                     public java.security.cert.X509Certificate[] getAcceptedIssuers() {

```

```

53.                 return new java.security.cert.X509Certificate[]{};
54.             }
55.         }
56.     };
57.
58.     // Install the all-trusting trust manager
59.     final SSLContext sslContext = SSLContext.getInstance("SSL");
60.     sslContext.init(null, trustAllCerts, new java.security.SecureRandom());
61.
62.     // Create an ssl socket factory with our all-trusting manager
63.     final SSLSocketFactory sslSocketFactory = sslContext.getSocketFactory();
64.
65.     OkHttpClient.Builder builder = new OkHttpClient.Builder();
66.     builder.sslSocketFactory(sslSocketFactory, (X509TrustManager) trustAllCerts[0]);
67.     builder.hostnameVerifier(new HostnameVerifier() {
68.         @Override
69.         public boolean verify(String hostname, SSLSession session) {
70.             return true; // Trust all hostnames
71.         }
72.     });
73.
74.     return builder.build();
75.
76. } catch (Exception e) {
77.     throw new RuntimeException(e);
78. }
79. }
80. }
81.

```

android\app\src\main\java\com\team\smartsolar\network\SolarApi.java

```

1. // -----
2. // File: SolarApi.java
3.
4. // Purpose: Retrofit interface defining all RESTful HTTP calls to the central
5. // C# Web API. Serves as the sole data conduit, ensuring the app remains a thin client.
6. // -----
7.
8. package com.team.smartsolar.network;
9.
10. import com.team.smartsolar.models.CreateReservationRequest;
11. import com.team.smartsolar.models.NodeResponse;
12. import com.team.smartsolar.models.RegisterRequest;
13. import com.team.smartsolar.models.ReservationResponse;
14. import com.team.smartsolar.models.Station;
15. import com.team.smartsolar.models.LoginRequest; // <-- Forces it to use your model
16. import com.team.smartsolar.models.LoginResponse; // <-- Forces it to use your model
17. import com.team.smartsolar.models.UpdateReservationRequest;
18.
19. import java.util.List;
20. import retrofit2.Call;
21. import retrofit2.http.Body;
22. import retrofit2.http.GET;
23. import retrofit2.http.POST;
24. import retrofit2.http.PATCH;
25. import retrofit2.http.PUT;
26. import retrofit2.http.Path;
27.
28. public interface SolarApi {
29.
30.     // --- AUTHENTICATION ---
31.     @POST("api/auth/login")

```

```

32.     Call<LoginResponse> login(@Body LoginRequest request);
33.
34.     // --- NODES (Map Grid) ---
35.     @GET("api/nodes")
36.     Call<List<Station>> getNodes();
37.
38.     // --- RESERVATIONS ---
39.     @POST("api/reservations")
40.     Call<Void> createReservation(@Body CreateReservationRequest request);
41.
42.     @GET("api/reservations/prosumer/{nic}")
43.     Call<java.util.List<ReservationResponse>> getMyReservations(@Path("nic") String nic);
44.
45.     @PUT("api/reservations/{id}")
46.     Call<Void> updateReservation(@Path("id") String id, @Body UpdateReservationRequest request);
47.
48.     @PATCH("api/reservations/{id}/cancel")
49.     Call<Void> cancelReservation(@Path("id") String id);
50.
51.     // --- PROSUMERS ---
52.
53.     @POST("api/prosumers")
54.     Call<Void> registerProsumer(@Body RegisterRequest profile);
55.
56.     @GET("api/prosumers/{nic}")
57.     Call<com.team.smartsolar.models.ProsumerProfile> getProfile(@Path("nic") String nic);
58.
59.     @PUT("api/prosumers/{nic}")
60.     Call<Void> updateProfile(@Path("nic") String nic, @Body
com.team.smartsolar.models.ProsumerProfile profile);
61.
62.     @PATCH("api/prosumers/{nic}/deactivate")
63.     Call<Void> requestDeactivation(@Path("nic") String nic);
64.
65.     @GET("api/nodes")
66.     Call<List<NodeResponse>> getAllStations();
67.
68.
69.
70. }

```

android\app\src\main\java\com\team\smartsolar>LoginActivity.java

```

1. // -----
2. // File: LoginActivity.java
3.
4. // Purpose: Handles Prosumer authentication, capturing credentials and storing
5. // the issued JWT token securely in the local SQLite database for session persistence.
6. // -----
7.
8. package com.team.smartsolar;
9.
10. import android.content.Intent;
11. import android.os.Bundle;
12. import android.view.View;
13. import android.widget.Button;
14. import android.widget.EditText;
15. import android.widget.TextView;
16. import android.widget.Toast;
17. import androidx.appcompat.app.AppCompatActivity;
18.
19. import com.team.smartsolar.database.DatabaseHelper;
20. import com.team.smartsolar.models.LoginRequest;

```

```

21. import com.team.smartsolar.models.LoginResponse;
22. import com.team.smartsolar.network.RetrofitClient;
23. import com.team.smartsolar.network.SolarApi;
24.
25. import retrofit2.Call;
26. import retrofit2.Callback;
27. import retrofit2.Response;
28.
29. public class LoginActivity extends AppCompatActivity {
30.
31.     private EditText inputLoginNic, inputLoginPassword;
32.     private Button btnLogin;
33.     private TextView txtGoToRegister;
34.
35.     @Override
36.     protected void onCreate(Bundle savedInstanceState) {
37.         super.onCreate(savedInstanceState);
38.         setContentView(R.layout.activity_login);
39.
40.         inputLoginNic = findViewById(R.id.inputLoginNic);
41.         inputLoginPassword = findViewById(R.id.inputLoginPassword);
42.         btnLogin = findViewById(R.id.btnLogin);
43.         txtGoToRegister = findViewById(R.id.txtGoToRegister);
44.
45.         btnLogin.setOnClickListener(new View.OnClickListener() {
46.             @Override
47.             public void onClick(View v) {
48.                 String nic = inputLoginNic.getText().toString().trim();
49.                 String password = inputLoginPassword.getText().toString().trim();
50.
51.                 if (nic.isEmpty() || password.isEmpty()) {
52.                     Toast.makeText(LoginActivity.this, "Please enter NIC and Password",
Toast.LENGTH_SHORT).show();
53.                     return;
54.                 }
55.
56.                 // 1. Create the request body
57.                 LoginRequest request = new LoginRequest(nic, password);
58.
59.                 // 2. Call the API
60.                 SolarApi api = RetrofitClient.getClient().create(SolarApi.class);
61.                 api.login(request).enqueue(new Callback<LoginResponse>() {
62.
63.                     @Override
64.                     public void onResponse(Call<LoginResponse> call, Response<LoginResponse>
response) {
65.                         if (response.isSuccessful() && response.body() != null) {
66.                             String token = response.body().getToken();
67.                             String role = response.body().getRole();
68.
69.                             // Save the real JWT token to SQLite
70.                             DatabaseHelper dbHelper = new DatabaseHelper(LoginActivity.this);
71.                             dbHelper.saveSession(nic, role, token);
72.
73.                             Toast.makeText(LoginActivity.this, "Login Successful!",
Toast.LENGTH_SHORT).show();
74.
75.                             // Route to the dashboard
76.                             startActivity(new Intent(LoginActivity.this,
DashboardActivity.class));
77.                             finish(); // Close login screen
78.                         } else {
79.                             Toast.makeText(LoginActivity.this, "Invalid credentials.",
Toast.LENGTH_SHORT).show();
80.                         }

```

```

81.         }
82.
83.         @Override
84.         public void onFailure(Call<LoginResponse> call, Throwable t) {
85.             Toast.makeText(LoginActivity.this, "Network error: " + t.getMessage(),
Toast.LENGTH_LONG).show();
86.         }
87.     });
88. }
89. });
90.
91. // Switches the screen to RegisterActivity
92. txtGoToRegister.setOnClickListener(new View.OnClickListener() {
93.     @Override
94.     public void onClick(View v) {
95.         Intent intent = new Intent(LoginActivity.this, RegisterActivity.class);
96.         startActivity(intent);
97.     }
98. });
99. }
100. }
101.

```

android\app\src\main\java\com\team\smartsolar\ModifyBookingActivity.java

```

1. // -----
2. // File: ModifyBookingActivity.java
3.
4. // Purpose: Allows Prosumers to reschedule or cancel an existing booking.
5. // Validates the 12-hour notice rule locally before submitting to the API.
6. // -----
7.
8. package com.team.smartsolar;
9.
10. import android.app.DatePickerDialog;
11. import android.app.TimePickerDialog;
12. import android.content.Intent;
13. import android.os.Bundle;
14. import android.widget.Button;
15. import android.widget.EditText;
16. import android.widget.Toast;
17.
18. import com.team.smartsolar.models.UpdateReservationRequest;
19. import com.team.smartsolar.network.RetrofitClient;
20. import com.team.smartsolar.network.SolarApi;
21.
22. import java.text.ParseException;
23. import java.text.SimpleDateFormat;
24. import java.util.Calendar;
25. import java.util.Date;
26. import java.util.Locale;
27. import java.util.TimeZone;
28.
29. import retrofit2.Call;
30. import retrofit2.Callback;
31. import retrofit2.Response;
32.
33. public class ModifyBookingActivity extends BaseActivity {
34.
35.     private EditText editStationId, editBookingDate, editStartTime, editEndTime,
editEnergyAmount;
36.     private String originalDate, originalTime, bookingId;
37.

```



```

38.     @Override
39.     protected void onCreate(Bundle savedInstanceState) {
40.         super.onCreate(savedInstanceState);
41.         setContentView(R.layout.activity_modify_booking);
42.
43.         findViewById(R.id.btnBack).setOnClickListener(v -> finish());
44.         editStationId = findViewById(R.id.editStationId);
45.         editBookingDate = findViewById(R.id.editBookingDate);
46.         editStartTime = findViewById(R.id.editStartTime);
47.         editEndTime = findViewById(R.id.editEndTime);
48.         editEnergyAmount = findViewById(R.id.editEnergyAmount);
49.
50.         // 1. Retrieve data passed from the adapter
51.         Intent intent = getIntent();
52.         bookingId = intent.getStringExtra("BOOKING_ID");
53.         editStationId.setText(intent.getStringExtra("STATION"));
54.         originalDate = intent.getStringExtra("DATE");
55.         editBookingDate.setText(originalDate);
56.
57.         originalTime = intent.getStringExtra("TIME");
58.         if (originalTime != null && originalTime.contains("-")) {
59.             String[] times = originalTime.split("-");
60.             editStartTime.setText(times[0].trim());
61.             editEndTime.setText(times[1].trim());
62.         }
63.         editEnergyAmount.setText(intent.getStringExtra("ENERGY"));
64.
65.         // Date Picker
66.         editBookingDate.setOnClickListener(v -> {
67.             Calendar cal = Calendar.getInstance();
68.             new DatePickerDialog(this, (view, year, month, day) ->
69.                 editBookingDate.setText(String.format("%04d-%02d-%02d", year, month + 1,
day)),
70.                 cal.get(Calendar.YEAR), cal.get(Calendar.MONTH),
cal.get(Calendar.DAY_OF_MONTH)).show();
71.         });
72.
73.         // Time Pickers
74.         editStartTime.setOnClickListener(v -> {
75.             Calendar cal = Calendar.getInstance();
76.             new TimePickerDialog(this, (view, h, m) ->
77.                 editStartTime.setText(String.format("%02d:%02d", h, m)),
78.                 cal.get(Calendar.HOUR_OF_DAY), cal.get(Calendar.MINUTE), true).show();
79.         });
80.
81.         editEndTime.setOnClickListener(v -> {
82.             Calendar cal = Calendar.getInstance();
83.             new TimePickerDialog(this, (view, h, m) ->
84.                 editEndTime.setText(String.format("%02d:%02d", h, m)),
85.                 cal.get(Calendar.HOUR_OF_DAY), cal.get(Calendar.MINUTE), true).show();
86.         });
87.
88.         // Setup Buttons connected to Live Network Methods
89.         findViewById(R.id.btnUpdateBooking).setOnClickListener(v -> handleUpdateBooking());
90.         findViewById(R.id.btnCancelBooking).setOnClickListener(v -> handleCancelBooking());
91.
92.         setupBottomNavigation(R.id.nav_booking);
93.     }
94.
95.     private void handleUpdateBooking() {
96.         if (bookingId == null || bookingId.isEmpty()) {
97.             Toast.makeText(this, "Error: Booking ID missing.", Toast.LENGTH_SHORT).show();
98.             return;
99.         }
100.

```

```

101.         if (isWithin12Hours(originalDate, originalTime.split("-")[0].trim())) {
102.             Toast.makeText(this, "Action Denied: Less than 12 hours until original start
time.", Toast.LENGTH_LONG).show();
103.             return;
104.         }
105.
106.         String date = editBookingDate.getText().toString().trim();
107.         String start = editStartTime.getText().toString().trim();
108.         String end = editEndTime.getText().toString().trim();
109.         String amountStr = editEnergyAmount.getText().toString().trim();
110.
111.         if (date.isEmpty() || start.isEmpty() || end.isEmpty() || amountStr.isEmpty()) {
112.             Toast.makeText(this, "Please fill in all details", Toast.LENGTH_SHORT).show();
113.             return;
114.         }
115.
116.         int durationMinutes = 0;
117.         String scheduledDateTime = "";
118.         double energyAmountValue = 0.0;
119.
120.         try {
121.             SimpleDateFormat localFormat = new SimpleDateFormat("yyyy-MM-dd HH:mm",
Locale.getDefault());
122.             Date startDate = localFormat.parse(date + " " + start);
123.             Date endDate = localFormat.parse(date + " " + end);
124.
125.             if (startDate != null && endDate != null) {
126.                 long diffInMillis = endDate.getTime() - startDate.getTime();
127.                 durationMinutes = (int) (diffInMillis / (1000 * 60));
128.
129.                 if (durationMinutes <= 0) {
130.                     Toast.makeText(this, "End time must be after start time",
Toast.LENGTH_SHORT).show();
131.                     return;
132.                 }
133.
134.                 SimpleDateFormat utcFormat = new SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ss'Z'",
Locale.getDefault());
135.                 utcFormat.setTimeZone(TimeZone.getTimeZone("UTC"));
136.                 scheduledDateTime = utcFormat.format(startDate);
137.             }
138.
139.             energyAmountValue = Double.parseDouble(amountStr);
140.         } catch (Exception e) {
141.             Toast.makeText(this, "Invalid time format", Toast.LENGTH_SHORT).show();
142.             return;
143.         }
144.
145.         UpdateReservationRequest request = new UpdateReservationRequest(scheduledDateTime,
durationMinutes, energyAmountValue);
146.         SolarApi api = RetrofitClient.getClient().create(SolarApi.class);
147.
148.         api.updateReservation(bookingId, request).enqueue(new Callback<Void>() {
149.             @Override
150.             public void onResponse(Call<Void> call, Response<Void> response) {
151.                 if (response.isSuccessful()) {
152.                     Toast.makeText(ModifyBookingActivity.this, "Update Saved Live!",
Toast.LENGTH_SHORT).show();
153.                     Intent intent = new Intent(ModifyBookingActivity.this,
MyBookingsActivity.class);
154.                     intent.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP |
Intent.FLAG_ACTIVITY_SINGLE_TOP);
155.                     startActivity(intent);
156.                     finish();
157.                 } else {

```

```

158.             Toast.makeText(ModifyBookingActivity.this, "Update failed: " +
response.code(), Toast.LENGTH_LONG).show();
159.         }
160.     }
161.
162.     @Override
163.     public void onFailure(Call<Void> call, Throwable t) {
164.         Toast.makeText(ModifyBookingActivity.this, "Network error",
Toast.LENGTH_SHORT).show();
165.     }
166. });
167. }
168.
169. private void handleCancelBooking() {
170.     if (bookingId == null || bookingId.isEmpty()) {
171.         Toast.makeText(this, "Error: Booking ID missing.", Toast.LENGTH_SHORT).show();
172.         return;
173.     }
174.
175.     if (isWithin12Hours(originalDate, originalTime.split("-")[0].trim())) {
176.         Toast.makeText(this, "Action Denied: Less than 12 hours until start time.",
Toast.LENGTH_LONG).show();
177.         return;
178.     }
179.
180.     SolarApi api = RetrofitClient.getClient().create(SolarApi.class);
181.     api.cancelReservation(bookingId).enqueue(new Callback<Void>() {
182.         @Override
183.         public void onResponse(Call<Void> call, Response<Void> response) {
184.             if (response.isSuccessful()) {
185.                 Toast.makeText(ModifyBookingActivity.this, "Reservation Cancelled",
Toast.LENGTH_SHORT).show();
186.                 Intent intent = new Intent(ModifyBookingActivity.this,
MyBookingsActivity.class);
187.                 intent.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP |
Intent.FLAG_ACTIVITY_SINGLE_TOP);
188.                 startActivity(intent);
189.                 finish();
190.             } else {
191.                 Toast.makeText(ModifyBookingActivity.this, "Cancellation failed: " +
response.code(), Toast.LENGTH_LONG).show();
192.             }
193.         }
194.     }
195.
196.     @Override
197.     public void onFailure(Call<Void> call, Throwable t) {
198.         Toast.makeText(ModifyBookingActivity.this, "Network error",
Toast.LENGTH_SHORT).show();
199.     }
200. });
201. }
202.
203. private boolean isWithin12Hours(String dateStr, String timeStr) {
204.     try {
205.         SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm",
Locale.getDefault());
206.         Date bookingDate = sdf.parse(dateStr + " " + timeStr);
207.         if (bookingDate == null) return false;
208.
209.         long differenceInMillis = bookingDate.getTime() - System.currentTimeMillis();
210.         long hoursDifference = differenceInMillis / (1000 * 60 * 60);
211.
212.         return hoursDifference < 12;
213.     } catch (ParseException e) {
214.         return false;

```

```

214.     }
215. }
216. }
217.

```

android\app\src\main\java\com\team\smartsolar\MyBookingsActivity.java

```

1. // -----
2. // File: MyBookingsActivity.java
3.
4. // Purpose: Displays a list of the Prosumer's current and past reservations.
5. // Provides local filtering by status and routes to modification or QR views.
6. // -----
7. package com.team.smartsolar;
8.
9. import android.content.Intent;
10. import android.os.Bundle;
11. import android.view.View;
12. import android.widget.AdapterView;
13. import android.widget.AdapterView.OnItemClickListener;
14. import android.widget.Button;
15. import android.widget.Spinner;
16. import android.widget.Toast;
17. import androidx.recyclerview.widget.LinearLayoutManager;
18. import androidx.recyclerview.widget.RecyclerView;
19.
20. import com.team.smartsolar.adapters.BookingAdapter;
21. import com.team.smartsolar.database.DatabaseHelper;
22. import com.team.smartsolar.models.Booking;
23. import com.team.smartsolar.models.ReservationResponse;
24. import com.team.smartsolar.network.RetrofitClient;
25. import com.team.smartsolar.network.SolarApi;
26.
27. import java.text.SimpleDateFormat;
28. import java.util.ArrayList;
29. import java.util.Calendar;
30. import java.util.Date;
31. import java.util.List;
32. import java.util.Locale;
33. import java.util.TimeZone;
34.
35. import retrofit2.Call;
36. import retrofit2.Callback;
37. import retrofit2.Response;
38.
39. public class MyBookingsActivity extends BaseActivity {
40.
41.     private RecyclerView recyclerView;
42.     private BookingAdapter adapter;
43.     private Spinner spinnerFilterStatus;
44.     private java.util.Map<String, String> stationMap = new java.util.HashMap<>();
45.
46.     // Master list from the server
47.     private List<ReservationResponse> allServerReservations = new ArrayList<>();
48.
49.     @Override
50.     protected void onCreate(Bundle savedInstanceState) {
51.         super.onCreate(savedInstanceState);
52.         setContentView(R.layout.activity_my_bookings);
53.
54.         Button btnGoToCreateBooking = findViewById(R.id.btnGoToCreateBooking);
55.         btnGoToCreateBooking.setOnClickListener(v -> {
56.             startActivity(new Intent(MyBookingsActivity.this, CreateBookingActivity.class));

```

```

57.     });
58.
59.     recyclerView = findViewById(R.id.recyclerViewBookings);
60.     recyclerView.setLayoutManager(new LinearLayoutManager(this));
61.
62.     spinnerFilterStatus = findViewById(R.id.spinnerFilterStatus);
63.     setupFilterSpinner();
64.
65.     // Change nav_booking to match your bottom nav ID if necessary
66.     setupBottomNavigation(R.id.nav_booking);
67. }
68.
69. @Override
70. protected void onResume() {
71.     super.onResume();
72.     loadBookingsFromDatabase();
73. }
74.
75. private void setupFilterSpinner() {
76.     String[] statuses = new String[]{"All", "Pending", "Approved", "Completed",
"Cancelled"};
77.     ArrayAdapter<String> spinnerAdapter = new ArrayAdapter<>(this,
android.R.layout.simple_spinner_dropdown_item, statuses);
78.     spinnerFilterStatus.setAdapter(spinnerAdapter);
79.
80.     spinnerFilterStatus.setOnItemClickListener(new AdapterView.OnItemClickListener()
{
81.         @Override
82.         public void onItemClick(AdapterView<?> parent, View view, int position, long id)
{
83.             String selectedStatus = statuses[position];
84.             applyFilter(selectedStatus);
85.         }
86.
87.         @Override
88.         public void onNothingSelected(AdapterView<?> parent) {}
89.     });
90. }
91.
92. private void loadBookingsFromDatabase() {
93.     DatabaseHelper db = new DatabaseHelper(this);
94.     String nic = db.getSessionNic();
95.
96.     if (nic == null) {
97.         Toast.makeText(this, "Session expired. Please log in.", Toast.LENGTH_SHORT).show();
98.         return;
99.     }
100.
101.     // 1. Load cached stations and map their IDs to their readable names
102.     stationMap.clear();
103.     for (com.team.smartsolar.models.NodeResponse node : db.getCachedStations()) {
104.         String id = node.getId() != null ? node.getId() : node.getStationId();
105.         String name = node.getName() != null ? node.getName() : node.getStationName();
106.         if (id != null && name != null) {
107.             stationMap.put(id, name);
108.         }
109.     }
110.
111.     // 2. Fetch the user's reservations
112.     SolarApi api = RetrofitClient.getClient().create(SolarApi.class);
113.     api.getMyReservations(nic).enqueue(new Callback<List<ReservationResponse>>() {
114.         @Override
115.         public void onResponse(Call<List<ReservationResponse>> call,
Response<List<ReservationResponse>> response) {
116.             if (response.isSuccessful() && response.body() != null) {

```

```

117.         allServerReservations = response.body();
118.         applyFilter(spinnerFilterStatus.getSelectedItem().toString());
119.     } else {
120.         Toast.makeText(MyBookingsActivity.this, "Failed to fetch bookings.",
121.             Toast.LENGTH_SHORT).show();
122.     }
123.
124.     @Override
125.     public void onFailure(Call<List<ReservationResponse>> call, Throwable t) {
126.         Toast.makeText(MyBookingsActivity.this, "Network error: " + t.getMessage(),
127.             Toast.LENGTH_SHORT).show();
128.     }
129. }
130.
131. // Filters the master list locally and updates the RecyclerView
132. private void applyFilter(String statusFilter) {
133.     List<Booking> filteredBookings = new ArrayList<>();
134.
135.     for (ReservationResponse res : allServerReservations) {
136.         if (statusFilter.equals("All") || statusFilter.equalsIgnoreCase(res.getStatus())) {
137.             String displayDate = "";
138.             String displayTime = "";
139.
140.             try {
141.                 String serverTime = res.getScheduledDateTime();
142.                 Date parsedDate = null;
143.
144.                 try {
145.                     SimpleDateFormat isoFormatMs = new SimpleDateFormat("yyyy-MM-
146.                         dd'T'HH:mm:ss.SSS'Z'", Locale.getDefault());
147.                     isoFormatMs.setTimeZone(TimeZone.getTimeZone("UTC"));
148.                     parsedDate = isoFormatMs.parse(serverTime);
149.                 } catch (Exception e1) {
150.                     SimpleDateFormat isoFormat = new SimpleDateFormat("yyyy-MM-
151.                         dd'T'HH:mm:ss'Z'", Locale.getDefault());
152.                     isoFormat.setTimeZone(TimeZone.getTimeZone("UTC"));
153.                     parsedDate = isoFormat.parse(serverTime);
154.                 }
155.
156.                 SimpleDateFormat dateFormat = new SimpleDateFormat("yyyy-MM-dd",
157.                     Locale.getDefault());
158.                 dateFormat.setTimeZone(TimeZone.getDefault());
159.                 displayDate = dateFormat.format(parsedDate);
160.
161.                 SimpleDateFormat timeFormat = new SimpleDateFormat("HH:mm",
162.                     Locale.getDefault());
163.                 timeFormat.setTimeZone(TimeZone.getDefault());
164.                 String startTime = timeFormat.format(parsedDate);
165.
166.                 java.util.Calendar cal = java.util.Calendar.getInstance();
167.                 cal.setTime(parsedDate);
168.                 cal.add(java.util.Calendar.MINUTE, res.getDurationMinutes());
169.                 String endTime = timeFormat.format(cal.getTime());
170.
171.                 displayTime = startTime + " - " + endTime;
172.
173.             } catch (Exception e) {
174.                 displayDate = "Format Error";
175.                 displayTime = "Format Error";
176.             }
177.
178.             // Look up the real name from the dictionary, fallback to a shortened ID if not
179.             found

```

```

175.         String realStationName = stationMap.containsKey(res.getNodeId())
176.             ? stationMap.get(res.getNodeId())
177.             : "Station " + res.getNodeId().substring(0, 6) + "...";
178.
179.         filteredBookings.add(new Booking(
180.             res.getId(),
181.             realStationName, // <-- This passes the readable name instead of
res.getNodeId()
182.             displayDate,
183.             displayTime,
184.             String.valueOf(res.getEnergyAmount()),
185.             res.getStatus()
186.         ));
187.     }
188. }
189.
190. adapter = new BookingAdapter(filteredBookings);
191. recyclerView.setAdapter(adapter);
192. }
193. }
194.

```

android\app\src\main\java\com\team\smartsolar\ProfileActivity.java

```

1. // -----
2. // File: ProfileActivity.java
3.
4. // Purpose: Allows Prosumers to view and update their profile details. Sends
5. // updates to the API and syncs with the local SQLite cache for offline access.
6. // Includes functionality to request account deactivation.
7. // -----
8.
9. package com.team.smartsolar;
10.
11. import android.content.Intent;
12. import android.graphics.Color;
13. import android.os.Bundle;
14. import android.widget.Button;
15. import android.widget.TextView;
16. import android.widget.Toast;
17.
18. import com.google.android.material.textfield.TextInputEditText;
19. import com.team.smartsolar.database.DatabaseHelper;
20. import com.team.smartsolar.models.ProsumerProfile;
21. import com.team.smartsolar.network.RetrofitClient;
22. import com.team.smartsolar.network.SolarApi;
23.
24. import retrofit2.Call;
25. import retrofit2.Callback;
26. import retrofit2.Response;
27.
28. public class ProfileActivity extends BaseActivity {
29.
30.     private TextView txtProfileNic, txtProfileStatus;
31.     private TextInputEditText inputProfileName, inputProfileEmail, inputProfilePhone,
inputProfileAddress;
32.     private Button btnUpdateProfile, btnDeactivateAccount, btnLogout;
33.     private String sessionNic;
34.
35.     @Override
36.     protected void onCreate(Bundle savedInstanceState) {
37.         super.onCreate(savedInstanceState);
38.         setContentView(R.layout.activity_profile);

```

```

39.
40.     txtProfileNic = findViewById(R.id.txtProfileNic);
41.     txtProfileStatus = findViewById(R.id.txtProfileStatus);
42.
43.     inputProfileName = findViewById(R.id.inputProfileName);
44.     inputProfileEmail = findViewById(R.id.inputProfileEmail);
45.     inputProfilePhone = findViewById(R.id.inputProfilePhone);
46.     inputProfileAddress = findViewById(R.id.inputProfileAddress);
47.
48.     btnUpdateProfile = findViewById(R.id.btnUpdateProfile);
49.     btnDeactivateAccount = findViewById(R.id.btnDeactivateAccount);
50.     btnLogout = findViewById(R.id.btnLogout);
51.
52.     DatabaseHelper db = new DatabaseHelper(this);
53.     sessionNic = db.getSessionNic();
54.
55.     if (sessionNic == null) {
56.         Toast.makeText(this, "Session expired.", Toast.LENGTH_SHORT).show();
57.         finish();
58.         return;
59.     }
60.
61.     txtProfileNic.setText("NIC ID: " + sessionNic);
62.
63.     loadLiveProfileData();
64.
65.     btnUpdateProfile.setOnClickListener(v -> handleProfileUpdate());
66.     btnDeactivateAccount.setOnClickListener(v -> handleDeactivationRequest());
67.
68.     btnLogout.setOnClickListener(v -> {
69.         db.logoutUser();
70.         Intent intent = new Intent(ProfileActivity.this, LoginActivity.class);
71.         intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
72.         startActivity(intent);
73.         finish();
74.     });
75.
76.     setupBottomNavigation(R.id.nav_profile);
77. }
78.
79. private void loadLiveProfileData() {
80.     SolarApi api = RetrofitClient.getClient().create(SolarApi.class);
81.     DatabaseHelper db = new DatabaseHelper(this);
82.
83.     api.getProfile(sessionNic).enqueue(new Callback<ProsumerProfile>() {
84.         @Override
85.         public void onResponse(Call<ProsumerProfile> call, Response<ProsumerProfile>
response) {
86.             if (response.isSuccessful() && response.body() != null) {
87.                 ProsumerProfile profile = response.body();
88.                 db.cacheProfile(sessionNic, profile); // 1. Save fresh data to local SQLite
89.                 populateProfileUI(profile);
90.             } else {
91.                 loadOfflineProfile(db);
92.             }
93.         }
94.
95.         @Override
96.         public void onFailure(Call<ProsumerProfile> call, Throwable t) {
97.             loadOfflineProfile(db); // 2. Trigger fallback on network error
98.         }
99.     });
100. }
101.
102. private void loadOfflineProfile(DatabaseHelper db) {

```



```

103.         ProsumerProfile cachedProfile = db.getCachedProfile(sessionNic);
104.         if (cachedProfile != null) {
105.             Toast.makeText(this, "Offline Mode: Showing cached profile",
106. Toast.LENGTH_SHORT).show();
106.             populateProfileUI(cachedProfile);
107.         } else {
108.             Toast.makeText(this, "Network Error: No offline data available",
109. Toast.LENGTH_SHORT).show();
109.         }
110.     }
111.
112.     private void populateProfileUI(ProsumerProfile profile) {
113.         if (inputProfileName != null) inputProfileName.setText(profile.getFullName());
114.         if (inputProfileEmail != null) inputProfileEmail.setText(profile.getEmail());
115.         if (inputProfilePhone != null) inputProfilePhone.setText(profile.getPhone());
116.         if (inputProfileAddress != null) inputProfileAddress.setText(profile.getAddress());
117.         updateStatusBadge(profile.isActive());
118.     }
119.
120.     private void updateStatusBadge(boolean isActive) {
121.         if (isActive) {
122.             txtProfileStatus.setText("Status: ACTIVE");
123.             txtProfileStatus.setTextColor(Color.parseColor("#10B981")); // Green
124.             txtProfileStatus.setBackgroundColor(Color.parseColor("#2010B981"));
125.         } else {
126.             txtProfileStatus.setText("Status: DEACTIVATED");
127.             txtProfileStatus.setTextColor(Color.parseColor("#EF4444")); // Red
128.             txtProfileStatus.setBackgroundColor(Color.parseColor("#20EF4444"));
129.         }
130.     }
131.
132.     private void handleProfileUpdate() {
133.         String name = inputProfileName.getText() != null ?
inputProfileName.getText().toString().trim() : "";
134.         String email = inputProfileEmail.getText() != null ?
inputProfileEmail.getText().toString().trim() : "";
135.         String phone = inputProfilePhone.getText() != null ?
inputProfilePhone.getText().toString().trim() : "";
136.         String address = inputProfileAddress.getText() != null ?
inputProfileAddress.getText().toString().trim() : "";
137.
138.         if (name.isEmpty() || email.isEmpty() || phone.isEmpty() || address.isEmpty()) {
139.             Toast.makeText(this, "All fields are required", Toast.LENGTH_SHORT).show();
140.             return;
141.         }
142.
143.         ProsumerProfile updateRequest = new ProsumerProfile(name, email, phone, address);
144.         SolarApi api = RetrofitClient.getClient().create(SolarApi.class);
145.
146.         api.updateProfile(sessionNic, updateRequest).enqueue(new Callback<Void>() {
147.             @Override
148.             public void onResponse(Call<Void> call, Response<Void> response) {
149.                 if (response.isSuccessful()) {
150.                     Toast.makeText(ProfileActivity.this, "Profile updated successfully!",
151. Toast.LENGTH_SHORT).show();
152.                 } else {
153.                     Toast.makeText(ProfileActivity.this, "Update failed: " + response.code(),
154. Toast.LENGTH_SHORT).show();
155.                 }
156.             }
157.             @Override
158.             public void onFailure(Call<Void> call, Throwable t) {
159.                 Toast.makeText(ProfileActivity.this, "Network error",
160. Toast.LENGTH_SHORT).show();

```

```

159.     }
160.     });
161. }
162.
163.     private void handleDeactivationRequest() {
164.         SolarApi api = RetrofitClient.getClient().create(SolarApi.class);
165.         api.requestDeactivation(sessionNic).enqueue(new Callback<Void>() {
166.             @Override
167.             public void onResponse(Call<Void> call, Response<Void> response) {
168.                 if (response.isSuccessful()) {
169.                     Toast.makeText(ProfileActivity.this, "Account deactivated",
170. Toast.LENGTH_LONG).show();
171.                     updateStatusBadge(false);
172.                 } else {
173.                     Toast.makeText(ProfileActivity.this, "Failed to deactivate",
174. Toast.LENGTH_SHORT).show();
175.                 }
176.             }
177.             @Override
178.             public void onFailure(Call<Void> call, Throwable t) {
179.                 Toast.makeText(ProfileActivity.this, "Network error",
180. Toast.LENGTH_SHORT).show();
181.             }
182.         });
183.     }

```

android\app\src\main\java\com\team\smartsolar\BaseActivity.java

```

1. // -----
2. // File: BaseActivity.java
3.
4. // Purpose: Core activity extended by all primary screens to manage the global
5. // bottom navigation bar state and seamless tab switching.
6. // -----
7.
8. package com.team.smartsolar;
9.
10. import android.content.Intent;
11. import androidx.appcompat.app.AppCompatActivity;
12. import com.google.android.material.bottomnavigation.BottomNavigationView;
13.
14. public class BaseActivity extends AppCompatActivity {
15.
16.     /**
17.      * Call this method from any activity's onCreate to activate the bottom navigation bar.
18.      * @param currentMenuItemId The menu item ID corresponding to the current activity (e.g.,
19.      * R.id.nav_dashboard)
20.      */
21.     protected void setupBottomNavigation(int currentMenuItemId) {
22.         BottomNavigationView bottomNav = findViewById(R.id.bottomNavigation);
23.
24.         if (bottomNav != null) {
25.             bottomNav.setSelectedItemId(currentMenuItemId);
26.
27.             bottomNav.setOnItemSelectedListener(item -> {
28.                 int itemId = item.getItemId();
29.
30.                 // If user clicks the tab they are already on, do nothing
31.                 if (itemId == currentMenuItemId) {

```

```

32.         }
33.
34.         Intent intent = null;
35.         if (itemId == R.id.nav_dashboard) {
36.             intent = new Intent(this, DashboardActivity.class);
37.         } else if (itemId == R.id.nav_booking) {
38.             intent = new Intent(this, MyBookingsActivity.class);
39.         } else if (itemId == R.id.nav_profile) {
40.             intent = new Intent(this, ProfileActivity.class);
41.         }
42.
43.         if (intent != null) {
44.             // Brings an existing instance to the front instead of re-creating
duplicates
45.             intent.setFlags(Intent.FLAG_ACTIVITY_REORDER_TO_FRONT);
46.             startActivity(intent);
47.             overridePendingTransition(0, 0); // Disable transition animation
48.             return true;
49.         }
50.         return false;
51.     });
52. }
53. }
54. }
55.

```

android\app\src\main\java\com\team\smartsolar\BookingSummaryActivity.java

```

1.  // -----
2.  // File: BookingSummaryActivity.java
3.
4.  // Purpose: Displays a static confirmation view of the booking details immediately
5.  // after a successful reservation creation.
6.  // -----
7.
8.  package com.team.smartsolar;
9.
10. import android.content.Intent;
11. import android.os.Bundle;
12. import android.widget.Button;
13. import android.widget.TextView;
14.
15. public class BookingSummaryActivity extends BaseActivity {
16.
17.     @Override
18.     protected void onCreate(Bundle savedInstanceState) {
19.         super.onCreate(savedInstanceState);
20.         setContentView(R.layout.activity_booking_summary);
21.
22.         TextView txtSummaryStation = findViewById(R.id.txtSummaryStation);
23.         TextView txtSummaryDate = findViewById(R.id.txtSummaryDate);
24.         TextView txtSummaryTime = findViewById(R.id.txtSummaryTime);
25.         TextView txtSummaryAmount = findViewById(R.id.txtSummaryAmount);
26.         Button btnBackToDashboard = findViewById(R.id.btnBackToDashboard);
27.
28.         // 1. Retrieve the data passed from CreateBookingActivity
29.         Intent intent = getIntent();
30.         String station = intent.getStringExtra("STATION");
31.         String date = intent.getStringExtra("DATE");
32.         String time = intent.getStringExtra("TIME");
33.         String amount = intent.getStringExtra("AMOUNT");
34.
35.         // 2. Display the data

```

```

36.         txtSummaryStation.setText("Station: " + (station != null ? station : "N/A"));
37.         txtSummaryDate.setText("Date: " + (date != null ? date : "N/A"));
38.         txtSummaryTime.setText("Time: " + (time != null ? time : "N/A"));
39.         txtSummaryAmount.setText("Energy Reserved: " + (amount != null ? amount : "0") + "
kWh");
40.
41.         // 3. Navigate safely back to the Dashboard
42.         btnBackToDashboard.setOnClickListener(v -> {
43.             Intent dashboardIntent = new Intent(BookingSummaryActivity.this,
DashboardActivity.class);
44.             // This flag clears the backstack so the user doesn't accidentally hit 'back' into a
completed booking
45.             dashboardIntent.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP |
Intent.FLAG_ACTIVITY_SINGLE_TOP);
46.             startActivity(dashboardIntent);
47.             finish();
48.         });
49.
50.         // --- Setup Bottom Navigation ---
51.         setupBottomNavigation(R.id.nav_booking);
52.     }
53. }
54.

```

android\app\src\main\java\com\team\smartsolar\CreateBookingActivity.java

```

1. // -----
2. // File: CreateBookingActivity.java
3.
4. // Purpose: Provides the form for Prosumers to create new energy reservations.
5. // Validates node capacity and enforces the 7-day advance booking constraint locally.
6. // -----
7.
8. package com.team.smartsolar;
9.
10. import android.content.Intent;
11. import android.os.Bundle;
12. import android.view.View;
13. import android.widget.AdapterView;
14. import android.widget.Button;
15. import android.widget.EditText;
16. import android.widget.Spinner;
17. import android.widget.Toast;
18.
19. import com.team.smartsolar.database.DatabaseHelper;
20. import com.team.smartsolar.models.CreateReservationRequest;
21. import com.team.smartsolar.models.NodeResponse;
22. import com.team.smartsolar.network.RetrofitClient;
23. import com.team.smartsolar.network.SolarApi;
24.
25. import java.text.SimpleDateFormat;
26. import java.util.ArrayList;
27. import java.util.Date;
28. import java.util.List;
29. import java.util.Locale;
30. import java.util.TimeZone;
31.
32. import retrofit2.Call;
33. import retrofit2.Callback;
34. import retrofit2.Response;
35.
36. public class CreateBookingActivity extends BaseActivity {
37.

```

```

38.     private Spinner spinnerStation;
39.     private EditText inputBookingDate, inputStartTimeSlot, inputEndTimeSlot, inputEnergyAmount;
40.     private Button btnConfirmBooking;
41.
42.     // Store the live list of stations so we can look up the ID later
43.     private List<NodeResponse> liveStations = new ArrayList<>();
44.
45.     @Override
46.     protected void onCreate(Bundle savedInstanceState) {
47.         super.onCreate(savedInstanceState);
48.         setContentView(R.layout.activity_create_booking);
49.
50.         findViewById(R.id.btnBack).setOnClickListener(v -> finish());
51.
52.         spinnerStation = findViewById(R.id.spinnerStation);
53.         inputBookingDate = findViewById(R.id.inputBookingDate);
54.         inputStartTimeSlot = findViewById(R.id.inputStartTime);
55.         inputEndTimeSlot = findViewById(R.id.inputEndTime);
56.         inputEnergyAmount = findViewById(R.id.inputEnergyAmount);
57.         btnConfirmBooking = findViewById(R.id.btnConfirmBooking);
58.
59.         // Fetch the active nodes to populate the dropdown
60.         loadStationsIntoSpinner();
61.
62.         // Date Picker
63.         inputBookingDate.setOnClickListener(v -> {
64.             java.util.Calendar calendar = java.util.Calendar.getInstance();
65.             new android.app.DatePickerDialog(CreateBookingActivity.this,
66.                 (view, year, month, dayOfMonth) -> {
67.                     String selectedDate = String.format(Locale.getDefault(), "%04d-%02d-%02d", year, month + 1, dayOfMonth);
68.                     inputBookingDate.setText(selectedDate);
69.                 },
70.                 calendar.get(java.util.Calendar.YEAR),
71.                 calendar.get(java.util.Calendar.MONTH),
72.                 calendar.get(java.util.Calendar.DAY_OF_MONTH)
73.             ).show();
74.         });
75.
76.         // Start Time Picker
77.         inputStartTimeSlot.setOnClickListener(v -> {
78.             java.util.Calendar calendar = java.util.Calendar.getInstance();
79.             new android.app.TimePickerDialog(CreateBookingActivity.this,
80.                 (view, hourOfDay, minute) -> {
81.                     String selectedTime = String.format(Locale.getDefault(), "%02d:%02d", hourOfDay, minute);
82.                     inputStartTimeSlot.setText(selectedTime);
83.                 },
84.                 calendar.get(java.util.Calendar.HOUR_OF_DAY),
85.                 calendar.get(java.util.Calendar.MINUTE),
86.                 true
87.             ).show();
88.         });
89.
90.         // End Time Picker
91.         inputEndTimeSlot.setOnClickListener(v -> {
92.             java.util.Calendar calendar = java.util.Calendar.getInstance();
93.             new android.app.TimePickerDialog(CreateBookingActivity.this,
94.                 (view, hourOfDay, minute) -> {
95.                     String selectedTime = String.format(Locale.getDefault(), "%02d:%02d", hourOfDay, minute);
96.                     inputEndTimeSlot.setText(selectedTime);
97.                 },
98.                 calendar.get(java.util.Calendar.HOUR_OF_DAY),
99.                 calendar.get(java.util.Calendar.MINUTE),

```

```

100.         true
101.     ).show();
102. });
103.
104.     btnConfirmBooking.setOnClickListener(v -> handleBookingSubmission());
105.
106.     setupBottomNavigation(R.id.nav_booking);
107. }
108.
109.     private void loadStationsIntoSpinner() {
110.         SolarApi api = RetrofitClient.getClient().create(SolarApi.class);
111.         api.getAllStations().enqueue(new Callback<List<NodeResponse>>() {
112.             @Override
113.             public void onResponse(Call<List<NodeResponse>> call, Response<List<NodeResponse>>
response) {
114.                 if (response.isSuccessful() && response.body() != null) {
115.                     liveStations = response.body();
116.                     List<String> stationNames = new ArrayList<>();
117.
118.                     for (NodeResponse station : liveStations) {
119.                         String cleanName = station.getName() != null ? station.getName().trim()
: "Unknown";
120.
121.                         if (station.getAvailableBatterySlots() > 0 && station.getCapacityKWh()
> 0) {
122.                             stationNames.add(cleanName + "(" +
station.getAvailableBatterySlots() + " slots | " + station.getCapacityKWh() + " kWh)");
123.                         } else {
124.                             stationNames.add(cleanName + " (STATION FULL)");
125.                         }
126.                     }
127.
128.                     ArrayAdapter<String> adapter = new
ArrayAdapter<>(CreateBookingActivity.this,
129.                         android.R.layout.simple_spinner_dropdown_item, stationNames);
130.                     spinnerStation.setAdapter(adapter);
131.
132.                     // NEW: Auto-select the station if the user came from the Grid Explorer Map
133.                     String preselectedId =
getIntent().getStringExtra("PRESELECTED_STATION_ID");
134.                     if (preselectedId != null) {
135.                         for (int i = 0; i < liveStations.size(); i++) {
136.                             if (preselectedId.equals(liveStations.get(i).getStationId())) {
137.                                 spinnerStation.setSelection(i);
138.                                 break; // Stop looping once we find it
139.                             }
140.                         }
141.                     }
142.
143.                 } else {
144.                     Toast.makeText(CreateBookingActivity.this, "Failed to load stations",
Toast.LENGTH_SHORT).show();
145.                 }
146.             }
147.
148.             @Override
149.             public void onFailure(Call<List<NodeResponse>> call, Throwable t) {
150.                 Toast.makeText(CreateBookingActivity.this, "Network error loading stations",
Toast.LENGTH_SHORT).show();
151.             }
152.         });
153.     }
154.
155.     private void handleBookingSubmission() {
156.         if (liveStations.isEmpty() || spinnerStation.getSelectedItemPosition() < 0) {

```

```

157.         Toast.makeText(this, "No valid station selected", Toast.LENGTH_SHORT).show();
158.         return;
159.     }
160.
161.     int selectedIndex = spinnerStation.getSelectedItemId();
162.     NodeResponse selectedStation = liveStations.get(selectedIndex);
163.     String selectedStationId = selectedStation.getStationId();
164.     String selectedStationName = selectedStation.getStationName().trim();
165.
166.     // Prevent booking if the station is physically full
167.     if (selectedStation.getAvailableBatterySlots() <= 0) {
168.         Toast.makeText(this, "This station has no available slots.",
169. Toast.LENGTH_SHORT).show();
170.         return;
171.     }
172.
173.     String date = inputBookingDate.getText().toString().trim();
174.     String start = inputStartTimeSlot.getText().toString().trim();
175.     String end = inputEndTimeSlot.getText().toString().trim();
176.     String amountStr = inputEnergyAmount.getText().toString().trim();
177.
178.     if (date.isEmpty() || start.isEmpty() || end.isEmpty() || amountStr.isEmpty()) {
179.         Toast.makeText(this, "Please fill in all booking details",
180. Toast.LENGTH_SHORT).show();
181.         return;
182.     }
183.
184.     int durationMinutes = 0;
185.     String scheduledDateTime = "";
186.
187.     try {
188.         java.text.SimpleDateFormat localFormat = new java.text.SimpleDateFormat("yyyy-MM-dd
189. HH:mm", java.util.Locale.getDefault());
190.         java.util.Date startDate = localFormat.parse(date + " " + start);
191.         java.util.Date endDate = localFormat.parse(date + " " + end);
192.
193.         if (startDate != null && endDate != null) {
194.             long diffInMillis = endDate.getTime() - startDate.getTime();
195.             durationMinutes = (int) (diffInMillis / (1000 * 60));
196.
197.             if (durationMinutes <= 0) {
198.                 Toast.makeText(this, "End time must be after start time",
199. Toast.LENGTH_SHORT).show();
200.                 return;
201.             }
202.
203.             java.text.SimpleDateFormat utcFormat = new java.text.SimpleDateFormat("yyyy-MM-
204. dd'T'HH:mm:ss'Z'", java.util.Locale.getDefault());
205.             utcFormat.setTimeZone(java.util.TimeZone.getTimeZone("UTC"));
206.             scheduledDateTime = utcFormat.format(startDate);
207.         }
208.     } catch (Exception e) {
209.         Toast.makeText(this, "Invalid time format", Toast.LENGTH_SHORT).show();
210.         return;
211.     }
212.
213.     double energyAmountValue = 0.0;
214.     try {
215.         energyAmountValue = Double.parseDouble(amountStr);
216.     } catch (NumberFormatException e) {
217.         Toast.makeText(this, "Invalid energy amount", Toast.LENGTH_SHORT).show();
218.         return;
219.     }
220.
221.     // Prevent booking if they request more energy than the station has left

```

```

217.         if (energyAmountValue > selectedStation.getCapacityKWh()) {
218.             Toast.makeText(this, "Station only has " + selectedStation.getCapacityKWh() + " kWh
available.", Toast.LENGTH_SHORT).show();
219.             return;
220.         }
221.
222.         DatabaseHelper db = new DatabaseHelper(this);
223.         String prosumerNic = db.getSessionNic();
224.
225.         if (prosumerNic == null) {
226.             Toast.makeText(this, "Session expired. Please log in again.",
Toast.LENGTH_SHORT).show();
227.             return;
228.         }
229.
230.         CreateReservationRequest request = new CreateReservationRequest(
231.             prosumerNic,
232.             selectedStationId,
233.             scheduledDateTime,
234.             durationMinutes,
235.             energyAmountValue
236.         );
237.
238.         SolarApi api = RetrofitClient.getClient().create(SolarApi.class);
239.
240.         api.createReservation(request).enqueue(new Callback<Void>() {
241.             @Override
242.             public void onResponse(Call<Void> call, Response<Void> response) {
243.                 if (response.isSuccessful()) {
244.                     Toast.makeText(CreateBookingActivity.this, "Reservation Confirmed!",
Toast.LENGTH_SHORT).show();
245.
246.                     Intent intent = new Intent(CreateBookingActivity.this,
BookingSummaryActivity.class);
247.                     intent.putExtra("STATION", selectedStationName);
248.                     intent.putExtra("DATE", date);
249.                     intent.putExtra("TIME", start + " - " + end);
250.                     intent.putExtra("AMOUNT", amountStr);
251.                     startActivity(intent);
252.                     finish();
253.                 } else {
254.                     Toast.makeText(CreateBookingActivity.this, "Failed to create booking: " +
response.code(), Toast.LENGTH_LONG).show();
255.                 }
256.             }
257.
258.             @Override
259.             public void onFailure(Call<Void> call, Throwable t) {
260.                 Toast.makeText(CreateBookingActivity.this, "Network Error: " + t.getMessage(),
Toast.LENGTH_LONG).show();
261.             }
262.         });
263.     }
264. }
265.

```

android/app/src/main/java/com/team/smartsolar/StationMapActivity.java

```

1. // -----
2. // File: StationMapActivity.java
3.
4. // Purpose: Prosumer Grid Explorer map. Fetches microgrid nodes from the API
5. // (or local SQLite cache) and renders them on Google Maps. Includes a bottom

```



```

6. // sheet to view capacity and jump straight into booking.
7. // -----
8.
9. package com.team.smartsolar;
10.
11. import android.content.Intent;
12. import android.os.Bundle;
13. import android.view.LayoutInflater;
14. import android.view.View;
15. import android.view.ViewGroup;
16. import android.widget.TextView;
17. import android.widget.Toast;
18.
19. import androidx.annotation.NonNull;
20. import androidx.appcompat.app.AppCompatActivity;
21. import androidx.recyclerview.widget.LinearLayoutManager;
22. import androidx.recyclerview.widget.RecyclerView;
23.
24. import com.google.android.gms.maps.CameraUpdateFactory;
25. import com.google.android.gms.maps.GoogleMap;
26. import com.google.android.gms.maps.OnMapReadyCallback;
27. import com.google.android.gms.maps.SupportMapFragment;
28. import com.google.android.gms.maps.UiSettings;
29. import com.google.android.gms.maps.model.LatLng;
30. import com.google.android.gms.maps.model.LatLngBounds;
31. import com.google.android.gms.maps.model.Marker;
32. import com.google.android.gms.maps.model.MarkerOptions;
33. import com.google.android.material.card.MaterialCardView;
34. import com.team.smartsolar.database.DatabaseHelper;
35. import com.team.smartsolar.models.NodeResponse;
36. import com.team.smartsolar.network.RetrofitClient;
37. import com.team.smartsolar.network.SolarApi;
38.
39. import java.util.ArrayList;
40. import java.util.HashMap;
41. import java.util.List;
42. import java.util.Map;
43.
44. import retrofit2.Call;
45. import retrofit2.Callback;
46. import retrofit2.Response;
47.
48. public class StationMapActivity extends AppCompatActivity implements OnMapReadyCallback {
49.
50.     private GoogleMap mMap;
51.     private final Map<Marker, NodeResponse> markerStationMap = new HashMap<>();
52.     private final Map<String, Marker> stationIdMarkerMap = new HashMap<>();
53.     private final List<NodeResponse> stationList = new ArrayList<>();
54.
55.     private MaterialCardView cardStationDetails;
56.     private TextView txtSheetStationName, txtSheetSlots, txtSheetEnergy;
57.     private RecyclerView recyclerStations;
58.     private StationCardAdapter adapter;
59.     private NodeResponse currentlySelectedStation = null;
60.
61.     @Override
62.     protected void onCreate(Bundle savedInstanceState) {
63.         super.onCreate(savedInstanceState);
64.         setContentView(R.layout.activity_station_map);
65.
66.         findViewById(R.id.btnBackFromMap).setOnClickListener(v -> finish());
67.
68.         cardStationDetails = findViewById(R.id.cardStationDetails);
69.         txtSheetStationName = findViewById(R.id.txtSheetStationName);
70.         txtSheetSlots = findViewById(R.id.txtSheetSlots);

```

```

71.         txtSheetEnergy = findViewById(R.id.txtSheetEnergy);
72.
73.         findViewById(R.id.btnSheetBook).setOnClickListener(v -> {
74.             Intent intent = new Intent(StationMapActivity.this, CreateBookingActivity.class);
75.             if (currentlySelectedStation != null) {
76.                 // Pass the ID to the booking page
77.                 intent.putExtra("PRESELECTED_STATION_ID",
currentlySelectedStation.getStationId());
78.             }
79.             startActivity(intent);
80.             finish();
81.         });
82.
83.         findViewById(R.id.btnZoomToStations).setOnClickListener(v -> zoomToAllStations());
84.
85.         recyclerStations = findViewById(R.id.recyclerStations);
86.         recyclerStations.setLayoutManager(new LinearLayoutManager(this,
LinearLayoutManager.HORIZONTAL, false));
87.         adapter = new StationCardAdapter(stationList, this::focusOnStation);
88.         recyclerStations.setAdapter(adapter);
89.
90.         SupportMapFragment mapFragment = (SupportMapFragment)
getSupportFragmentManager().findFragmentById(R.id.fullMap);
91.         if (mapFragment != null) {
92.             mapFragment.getMapAsync(this);
93.         }
94.     }
95.
96.     @Override
97.     public void onMapReady(@NonNull GoogleMap googleMap) {
98.         mMap = googleMap;
99.
100.        // Configure UI controls
101.        UiSettings ui = mMap.getUiSettings();
102.        ui.setZoomControlsEnabled(true);           // Visual + / - buttons on the right edge
103.        ui.setCompassEnabled(true);                 // Compass displays automatically when rotated
104.        ui.setRotateGesturesEnabled(true);          // Two-finger twist to rotate map
105.        ui.setTiltGesturesEnabled(true);             // Two-finger slide to tilt 3D perspective
106.        ui.setScrollGesturesEnabled(true);
107.        ui.setZoomGesturesEnabled(true);
108.
109.        // Adjust map control margins so buttons don't clip into the top header or cards
110.        mMap.setPadding(0, 220, 0, 320);
111.
112.        mMap.setOnMapClickListener(latLng -> cardStationDetails.setVisibility(View.GONE));
113.
114.        mMap.setOnMarkerClickListener(marker -> {
115.            NodeResponse station = markerStationMap.get(marker);
116.            if (station != null) {
117.                displayStationDetails(station);
118.            }
119.            return false;
120.        });
121.
122.        loadStations();
123.    }
124.
125.    private void loadStations() {
126.        SolarApi api = RetrofitClient.getClient().create(SolarApi.class);
127.        DatabaseHelper db = new DatabaseHelper(this);
128.
129.        api.getAllStations().enqueue(new Callback<List<NodeResponse>>() {
130.            @Override
131.            public void onResponse(Call<List<NodeResponse>> call, Response<List<NodeResponse>>
response) {

```

```

132.         if (response.isSuccessful() && response.body() != null) {
133.             db.cacheStations(response.body()); // 1. Save grid nodes to local SQLite
134.             populateMapAndList(response.body());
135.         } else {
136.             loadOfflineStations(db);
137.         }
138.     }
139.
140.     @Override
141.     public void onFailure(Call<List<NodeResponse>> call, Throwable t) {
142.         loadOfflineStations(db); // 2. Trigger fallback on network error
143.     }
144. });
145. }
146.
147. private void loadOfflineStations(DatabaseHelper db) {
148.     List<NodeResponse> cachedStations = db.getCachedStations();
149.     if (!cachedStations.isEmpty()) {
150.         Toast.makeText(this, "Offline Mode: Showing cached stations",
151.             Toast.LENGTH_SHORT).show();
152.         populateMapAndList(cachedStations);
153.     } else {
154.         Toast.makeText(this, "Network error loading stations", Toast.LENGTH_SHORT).show();
155.     }
156. }
157. private void populateMapAndList(List<NodeResponse> stations) {
158.     stationList.clear();
159.     markerStationMap.clear();
160.     stationIdMarkerMap.clear();
161.
162.     if (mMap != null) mMap.clear(); // Clear old pins
163.
164.     stationList.addAll(stations);
165.     adapter.notifyDataSetChanged();
166.
167.     for (NodeResponse station : stationList) {
168.         LatLng loc = new LatLng(station.getLatitude(), station.getLongitude());
169.         String stationName = station.getStationName() != null ? station.getStationName() :
170. station.getName();
171.         String stationId = station.getStationId() != null ? station.getStationId() :
172. station.getId();
173.         if (mMap != null) {
174.             Marker marker = mMap.addMarker(new
175. MarkerOptions().position(loc).title(stationName));
176.             if (marker != null) {
177.                 markerStationMap.put(marker, station);
178.                 stationIdMarkerMap.put(stationId, marker);
179.             }
180.         }
181.     }
182.     zoomToAllStations();
183. }
184. private void focusOnStation(NodeResponse station) {
185.     if (mMap == null || station == null) return;
186.
187.     LatLng loc = new LatLng(station.getLatitude(), station.getLongitude());
188.     mMap.animateCamera(CameraUpdateFactory.newLatLngZoom(loc, 15f));
189.
190.     Marker marker = stationIdMarkerMap.get(station.getStationId());
191.     if (marker != null) {
192.         marker.showInfoWindow();
193.     }

```

```

193.
194.         displayStationDetails(station);
195.     }
196.
197.     private void displayStationDetails(NodeResponse station) {
198.         currentlySelectedStation = station; // Track for booking
199.
200.         String cleanName = station.getStationName() != null ? station.getStationName().trim() :
"Unknown Station";
201.         txtSheetStationName.setText(cleanName);
202.         txtSheetSlots.setText(String.valueOf(station.getAvailableBatterySlots()));
203.         txtSheetEnergy.setText(station.getCapacityKWh() + " kWh");
204.         cardStationDetails.setVisibility(View.VISIBLE);
205.     }
206.
207.     private void zoomToAllStations() {
208.         if (markerStationMap.isEmpty() || mMap == null) return;
209.
210.         LatLngBounds.Builder builder = new LatLngBounds.Builder();
211.         for (Marker marker : markerStationMap.keySet()) {
212.             builder.include(marker.getPosition());
213.         }
214.
215.         try {
216.             LatLngBounds bounds = builder.build();
217.             mMap.animateCamera(CameraUpdateFactory.newLatLngBounds(bounds, 180));
218.         } catch (Exception e) {
219.             Marker first = markerStationMap.keySet().iterator().next();
220.             mMap.animateCamera(CameraUpdateFactory.newLatLngZoom(first.getPosition(), 11f));
221.         }
222.     }
223.
224.     // Horizontal List Adapter
225.     private static class StationCardAdapter extends
RecyclerView.Adapter<StationCardAdapter.ViewHolder> {
226.         private final List<NodeResponse> list;
227.         private final OnStationClickListener listener;
228.
229.         interface OnStationClickListener {
230.             void onStationClick(NodeResponse station);
231.         }
232.
233.         StationCardAdapter(List<NodeResponse> list, OnStationClickListener listener) {
234.             this.list = list;
235.             this.listener = listener;
236.         }
237.
238.         @NonNull
239.         @Override
240.         public ViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
241.             View v =
LayoutInflater.from(parent.getContext()).inflate(R.layout.item_station_card, parent, false);
242.             return new ViewHolder(v);
243.         }
244.
245.         @Override
246.         public void onBindViewHolder(@NonNull ViewHolder h, int pos) {
247.             NodeResponse st = list.get(pos);
248.             h.txtName.setText(st.getStationName() != null ? st.getStationName().trim() :
"Station");
249.             h.txtSlots.setText(st.getAvailableBatterySlots() + " slots");
250.             h.txtEnergy.setText(st.getCapacityKWh() + " kWh");
251.             h.itemView.setOnClickListener(v -> listener.onStationClick(st));
252.         }
253.

```

```

254.         @Override
255.         public int getItemCount() {
256.             return list.size();
257.         }
258.
259.         static class ViewHolder extends RecyclerView.ViewHolder {
260.             TextView txtName, txtSlots, txtEnergy;
261.             ViewHolder(View v) {
262.                 super(v);
263.                 txtName = v.findViewById(R.id.txtCardStationName);
264.                 txtSlots = v.findViewById(R.id.txtCardSlots);
265.                 txtEnergy = v.findViewById(R.id.txtCardEnergy);
266.             }
267.         }
268.     }
269. }
270.

```

android\app\src\main\java\com\team\smartsolar\ViewQrActivity.java

```

1. // -----
2. // File: ViewQrActivity.java
3.
4. // Purpose: Renders the secure dispatch QR code for approved energy reservations.
5. // Generates the code locally using ZXing based on the payload received from the API.
6. // -----
7.
8. package com.team.smartsolar;
9.
10. import android.graphics.Bitmap;
11. import android.os.Bundle;
12. import android.widget.ImageView;
13. import android.widget.TextView;
14. import android.widget.Toast;
15.
16. import androidx.appcompat.app.AppCompatActivity;
17.
18. import com.google.zxing.BarcodeFormat;
19. import com.journeyapps.barcodescanner.BarcodeEncoder;
20.
21. public class ViewQrActivity extends AppCompatActivity {
22.
23.     @Override
24.     protected void onCreate(Bundle savedInstanceState) {
25.         super.onCreate(savedInstanceState);
26.         setContentView(R.layout.activity_view_qr);
27.
28.         ImageView imgQrCode = findViewById(R.id.imgQrCode);
29.         TextView txtQrDetails = findViewById(R.id.txtQrDetails);
30.
31.         // 1. Retrieve the booking data passed from the list
32.         String station = getIntent().getStringExtra("STATION");
33.         String date = getIntent().getStringExtra("DATE");
34.         String energy = getIntent().getStringExtra("ENERGY");
35.
36.         txtQrDetails.setText(station + "\n" + date + " | " + energy + " kWh");
37.
38.         // 2. Format the payload that Sasmitha's API will verify later
39.         String payload = "SMARTSOLAR|" + station + "|" + date + "|" + energy;
40.
41.         // 3. Generate the QR Code Image
42.         try {
43.             BarcodeEncoder barcodeEncoder = new BarcodeEncoder();

```

```

44.         // Create a 400x400 pixel QR code
45.         Bitmap bitmap = barcodeEncoder.encodeBitmap(payload, BarcodeFormat.QR_CODE, 400,
400);
46.         imgQrCode.setImageBitmap(bitmap);
47.     } catch (Exception e) {
48.         Toast.makeText(this, "Failed to generate QR: " + e.getMessage(),
Toast.LENGTH_LONG).show();
49.     }
50.
51.         // New Back Navigation
52.         findViewById(R.id.btnBack).setOnClickListener(v -> finish());
53.     }
54. }
55.

```

android\app\src\main\java\com\team\smartsolar\DashboardActivity.java

```

1.  // -----
2.  // File: DashboardActivity.java
3.
4.  // Purpose: Prosumer dashboard displaying dynamic counts from the API and a
5.  // preview map of nearby grid nodes. Includes SQLite fallback for offline mode.
6.  // -----
7.
8.  package com.team.smartsolar;
9.
10. import android.content.Intent;
11. import android.os.Bundle;
12. import android.view.View;
13. import android.widget.Button;
14. import android.widget.TextView;
15. import android.widget.Toast;
16.
17. import com.google.android.gms.maps.CameraUpdateFactory;
18. import com.google.android.gms.maps.GoogleMap;
19. import com.google.android.gms.maps.OnMapReadyCallback;
20. import com.google.android.gms.maps.SupportMapFragment;
21. import com.google.android.gms.maps.model.LatLng;
22. import com.google.android.gms.maps.model.MarkerOptions;
23. import com.team.smartsolar.database.DatabaseHelper;
24. import com.team.smartsolar.models.ProsumerProfile;
25. import com.team.smartsolar.models.ReservationResponse;
26. import com.team.smartsolar.network.RetrofitClient;
27. import com.team.smartsolar.network.SolarApi;
28. import com.team.smartsolar.models.NodeResponse;
29.
30. import java.util.List;
31.
32. import retrofit2.Call;
33. import retrofit2.Callback;
34. import retrofit2.Response;
35.
36. public class DashboardActivity extends BaseActivity implements OnMapReadyCallback {
37.
38.     private TextView txtPendingCount, txtApprovedCount, txtCompletedCount, txtWelcome;
39.     private String sessionNic;
40.     private GoogleMap mMap;
41.
42.     // Dictionary to hold live station names for the Next Booking Widget

```

```

43.     private java.util.Map<String, String> stationMap = new java.util.HashMap<>();
44.
45.     @Override
46.     protected void onCreate(Bundle savedInstanceState) {
47.         super.onCreate(savedInstanceState);
48.         setContentView(R.layout.activity_dashboard);
49.
50.         txtWelcome = findViewById(R.id.txtWelcome);
51.         txtPendingCount = findViewById(R.id.txtPendingCount);
52.         txtApprovedCount = findViewById(R.id.txtApprovedCount);
53.         txtCompletedCount = findViewById(R.id.txtCompletedCount);
54.
55.         DatabaseHelper db = new DatabaseHelper(this);
56.         sessionNic = db.getSessionNic();
57.
58.         if (sessionNic != null) {
59.             ProsumerProfile profile = db.getCachedProfile(sessionNic);
60.
61.             if (profile != null && profile.getFullName() != null) {
62.                 // Extract just the first name for a cleaner UI
63.                 String firstName = profile.getFullName().split(" ")[0];
64.                 txtWelcome.setText("Welcome, " + firstName);
65.             } else {
66.                 txtWelcome.setText("Welcome, Prosumer");
67.             }
68.         }
69.
70.         setupBottomNavigation(R.id.nav_dashboard);
71.
72.         // Safely hook up Quick Actions to prevent NullPointerExceptions
73.         Button btnQuickBook = findViewById(R.id.btnQuickBook);
74.         if (btnQuickBook != null) {
75.             btnQuickBook.setOnClickListener(v -> startActivity(new
Intent(DashboardActivity.this, CreateBookingActivity.class)));
76.         }
77.
78.         Button btnQuickHistory = findViewById(R.id.btnQuickHistory);
79.         if (btnQuickHistory != null) {
80.             btnQuickHistory.setOnClickListener(v -> startActivity(new
Intent(DashboardActivity.this, MyBookingsActivity.class)));
81.         }
82.
83.         View mapClickOverlay = findViewById(R.id.mapClickOverlay);
84.         if (mapClickOverlay != null) {
85.             mapClickOverlay.setOnClickListener(v -> {
86.                 startActivity(new Intent(DashboardActivity.this, StationMapActivity.class));
87.             });
88.         }
89.         // Initialize the Map Fragment
90.         SupportMapFragment mapFragment = (SupportMapFragment) getSupportFragmentManager()
91.             .findFragmentById(R.id.mapPlaceholder);
92.         if (mapFragment != null) {
93.             mapFragment.getMapAsync(this);
94.         }
95.
96.         Button btnViewStations = findViewById(R.id.btnViewStations);
97.         if (btnViewStations != null) {
98.             btnViewStations.setOnClickListener(v -> {
99.                 if (mMap != null) {
100.                     mMap.clear();
101.                     onMapReady(mMap);
102.                     Toast.makeText(DashboardActivity.this, "Refreshing grid stations...",
Toast.LENGTH_SHORT).show();
103.                 }
104.             });

```

```

105.     }
106. }
107.
108. @Override
109. protected void onResume() {
110.     super.onResume();
111.     loadDashboardMetrics();
112. }
113.
114. private void loadDashboardMetrics() {
115.     SolarApi api = RetrofitClient.getClient().create(SolarApi.class);
116.     DatabaseHelper db = new DatabaseHelper(this);
117.
118.     // 1. Fetch stations to map their names for the widget
119.     api.getAllStations().enqueue(new Callback<List<NodeResponse>>() {
120.         @Override
121.         public void onResponse(Call<List<NodeResponse>> call, Response<List<NodeResponse>>
response) {
122.             if (response.isSuccessful() && response.body() != null) {
123.                 db.cacheStations(response.body()); // Keep cache fresh
124.                 mapStationsToDictionary(response.body());
125.             } else {
126.                 mapStationsToDictionary(db.getCachedStations()); // Fallback
127.             }
128.             fetchReservationsAndUpdateDashboard(api);
129.         }
130.
131.         @Override
132.         public void onFailure(Call<List<NodeResponse>> call, Throwable t) {
133.             mapStationsToDictionary(db.getCachedStations()); // Fallback on error
134.             fetchReservationsAndUpdateDashboard(api);
135.         }
136.     });
137. }
138.
139. private void mapStationsToDictionary(List<NodeResponse> stations) {
140.     stationMap.clear();
141.     for (NodeResponse node : stations) {
142.         String id = node.getId() != null ? node.getId() : node.getStationId();
143.         String name = node.getName() != null ? node.getName() : node.getStationName();
144.         if (id != null && name != null) {
145.             stationMap.put(id, name);
146.         }
147.     }
148. }
149.
150. private void fetchReservationsAndUpdateDashboard(SolarApi api) {
151.     api.getMyReservations(sessionNic).enqueue(new Callback<List<ReservationResponse>>() {
152.         @Override
153.         public void onResponse(Call<List<ReservationResponse>> call,
Response<List<ReservationResponse>> response) {
154.             if (response.isSuccessful() && response.body() != null) {
155.                 int pending = 0, approved = 0, completed = 0;
156.                 ReservationResponse nextBooking = null;
157.
158.                 for (ReservationResponse res : response.body()) {
159.                     if ("Pending".equalsIgnoreCase(res.getStatus())) {
160.                         pending++;
161.                     } else if ("Approved".equalsIgnoreCase(res.getStatus())) {
162.                         approved++;
163.                         // Grab the first approved booking to feature on the dashboard
164.                         if (nextBooking == null) nextBooking = res;
165.                     } else if ("Completed".equalsIgnoreCase(res.getStatus())) {
166.                         completed++;
167.                     }

```



```

168.         }
169.
170.         // Safely update UI
171.         if (txtPendingCount != null)
txtPendingCount.setText(String.valueOf(pending));
172.         if (txtApprovedCount != null)
txtApprovedCount.setText(String.valueOf(approved));
173.         if (txtCompletedCount != null)
txtCompletedCount.setText(String.valueOf(completed));
174.
175.         // Update the Next Booking Card
176.         TextView txtNextStation = findViewById(R.id.txtNextStation);
177.         TextView txtNextTime = findViewById(R.id.txtNextTime);
178.
179.         if (txtNextStation != null && txtNextTime != null) {
180.             if (nextBooking != null) {
181.                 String realStationName =
stationMap.containsKey(nextBooking.getNodeId())
182.                     ? stationMap.get(nextBooking.getNodeId())
183.                     : "Station ID: " + nextBooking.getNodeId().substring(0, 6);
184.
185.                 txtNextStation.setText(realStationName);
186.
187.                 // Clean up the ISO date string for display (e.g., 2026-09-
25T14:30:00Z -> 2026-09-25)
188.                 try {
189.                     String rawDate = nextBooking.getScheduledDateTime();
190.                     String displayDate = rawDate.contains("T") ?
rawDate.split("T")[0] : rawDate;
191.                     txtNextTime.setText(displayDate + " | " +
nextBooking.getEnergyAmount() + " kWh Reserved");
192.                 } catch (Exception e) {
193.                     txtNextTime.setText(nextBooking.getEnergyAmount() + " kWh
Reserved");
194.                 }
195.
196.             } else {
197.                 txtNextStation.setText("No upcoming bookings");
198.                 txtNextTime.setText("Ready to book a microgrid slot?");
199.             }
200.         }
201.     }
202. }
203.
204. @Override
205. public void onFailure(Call<List<ReservationResponse>> call, Throwable t) {
206.     Toast.makeText(DashboardActivity.this, "Network Error: " + t.getMessage(),
Toast.LENGTH_SHORT).show();
207. }
208. });
209. }
210.
211. @Override
212. public void onMapReady(GoogleMap googleMap) {
213.     mMap = googleMap;
214.     mMap.getUiSettings().setAllGesturesEnabled(false);
215.     SolarApi api = RetrofitClient.getClient().create(SolarApi.class);
216.     api.getAllStations().enqueue(new Callback<List<NodeResponse>>() {
217.         @Override
218.         public void onResponse(Call<List<NodeResponse>> call, Response<List<NodeResponse>>
response) {
219.             if (response.isSuccessful() && response.body() != null) {
220.                 for (NodeResponse station : response.body()) {
221.                     LatLng location = new LatLng(station.getLatitude(),
station.getLongitude());

```

```

222.                mMap.addMarker(new
MarkerOptions().position(location).title(station.getStationName()));
223.            }
224.
225.                if (!response.body().isEmpty()) {
226.                    LatLng firstLoc = new LatLng(response.body().get(0).getLatitude(),
response.body().get(0).getLongitude());
227.                    mMap.moveCamera(CameraUpdateFactory.newLatLngZoom(firstLoc, 10));
228.                }
229.            }
230.        }
231.
232.        @Override
233.        public void onFailure(Call<List<NodeResponse>> call, Throwable t) {
234.            Toast.makeText(DashboardActivity.this, "Failed to load map markers",
Toast.LENGTH_SHORT).show();
235.        }
236.    });
237. }
238. }
239.

```

android\app\src\main\java\com\team\smartsolar\RegisterActivity.java

```

1. // -----
2. // File: RegisterActivity.java
3.
4. // Purpose: Handles the Prosumer account creation process. Keys the user to their
5. // NIC as requested by the assignment specification.
6. // -----
7.
8. package com.team.smartsolar;
9.
10. import android.content.Intent;
11. import android.os.Bundle;
12. import android.view.View;
13. import android.widget.Button;
14. import android.widget.EditText;
15. import android.widget.TextView;
16. import android.widget.Toast;
17. import androidx.appcompat.app.AppCompatActivity;
18.
19. import com.team.smartsolar.models.RegisterRequest;
20. import com.team.smartsolar.network.RetrofitClient;
21. import com.team.smartsolar.network.SolarApi;
22.
23. import retrofit2.Call;
24. import retrofit2.Callback;
25. import retrofit2.Response;
26.
27. public class RegisterActivity extends AppCompatActivity {
28.
29.     private EditText inputNic, inputName, inputEmail, inputPassword;
30.     private Button btnRegister;
31.     private TextView txtGoToLogin;
32.
33.     @Override
34.     protected void onCreate(Bundle savedInstanceState) {
35.         super.onCreate(savedInstanceState);
36.         setContentView(R.layout.activity_register);
37.
38.         inputNic = findViewById(R.id.inputNic);
39.         inputName = findViewById(R.id.inputName);

```

```

40.         inputEmail = findViewById(R.id.inputEmail);
41.         inputPassword = findViewById(R.id.inputPassword);
42.         btnRegister = findViewById(R.id.btnRegister);
43.         txtGoToLogin = findViewById(R.id.txtGoToLogin);
44.
45.         btnRegister.setOnClickListener(new View.OnClickListener() {
46.             @Override
47.             public void onClick(View v) {
48.                 handleRegistration();
49.             }
50.         });
51.
52.         txtGoToLogin.setOnClickListener(new View.OnClickListener() {
53.             @Override
54.             public void onClick(View v) {
55.                 Intent intent = new Intent(RegisterActivity.this, LoginActivity.class);
56.                 startActivity(intent);
57.                 finish();
58.             }
59.         });
60.     }
61.
62.     private void handleRegistration() {
63.         String nic = inputNic.getText().toString().trim();
64.         String name = inputName.getText().toString().trim();
65.         String email = inputEmail.getText().toString().trim();
66.         String password = inputPassword.getText().toString().trim();
67.
68.         if (nic.isEmpty() || name.isEmpty() || email.isEmpty() || password.isEmpty()) {
69.             Toast.makeText(this, "Please fill in all fields", Toast.LENGTH_SHORT).show();
70.             return;
71.         }
72.
73.         if (nic.length() < 9) {
74.             Toast.makeText(this, "Please enter a valid NIC", Toast.LENGTH_SHORT).show();
75.             return;
76.         }
77.
78.         // 1. Package the variables into the JSON model expected by the backend
79.         // We map NIC to username, and hardcode the role as "Prosumer"
80.         RegisterRequest request = new RegisterRequest(nic, password, name, email, "Prosumer");
81.
82.         // 2. Send to C# API endpoint using Retrofit
83.         SolarApi api = RetrofitClient.getClient().create(SolarApi.class);
84.         api.registerProsumer(request).enqueue(new Callback<Void>() {
85.             @Override
86.             public void onResponse(Call<Void> call, Response<Void> response) {
87.                 if (response.isSuccessful()) {
88.                     Toast.makeText(RegisterActivity.this, "Registration Successful! Please log
in.", Toast.LENGTH_LONG).show();
89.
90.                     // Route back to Log in
91.                     Intent intent = new Intent(RegisterActivity.this, LoginActivity.class);
92.                     startActivity(intent);
93.                     finish();
94.                 } else {
95.                     Toast.makeText(RegisterActivity.this, "Registration Failed: " +
response.code(), Toast.LENGTH_LONG).show();
96.                 }
97.             }
98.
99.             @Override
100.            public void onFailure(Call<Void> call, Throwable t) {
101.                Toast.makeText(RegisterActivity.this, "Network Error: " + t.getMessage(),
Toast.LENGTH_LONG).show();

```

```
102.         }
103.     });
104. }
105. }
106.
```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\color\bottom_nav_selector.xml

```
1. <?xml version="1.0" encoding="utf-8"?>
2. <selector xmlns:android="http://schemas.android.com/apk/res/android">
3.     <!-- Selected state: Amber -->
4.     <item android:state_checked="true" android:color="#F59E0B" />
5.     <!-- Default state: Slate Gray -->
6.     <item android:color="#94A3B8" />
7. </selector>
8.
```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\drawable\ic_book.xml

```
1. <!--
2. ~ Copyright (C) 2026 The Android Open Source Project
3. ~
4. ~ Licensed under the Apache License, Version 2.0 (the "License");
5. ~ you may not use this file except in compliance with the License.
6. ~ You may obtain a copy of the License at
7. ~
8. ~     http://www.apache.org/licenses/LICENSE-2.0
9. ~
10. ~ Unless required by applicable law or agreed to in writing, software
11. ~ distributed under the License is distributed on an "AS IS" BASIS,
12. ~ WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13. ~ See the License for the specific language governing permissions and
14. ~ limitations under the License.
15. -->
16. <vector xmlns:android="http://schemas.android.com/apk/res/android" android:height="24dp"
17.     android:tint="#000000" android:viewportHeight="24" android:viewportWidth="24" android:width="24dp">
18.     <path android:fillColor="@android:color/white" android:pathData="M18,2H6c-1.1,0 -2,0.9 -
19.     2,2v16c0,1.1 0.9,2 2,2h12c1.1,0 2,-0.9 2,-2V4c0,-1.1 -0.9,-2 -2,-2zM6,4h5v8l-2.5,-1.5L6,12V4z"/>
20. </vector>
21.
22.
```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\drawable\ic_launcher_background.xml

```
1. <?xml version="1.0" encoding="utf-8"?>
2. <vector xmlns:android="http://schemas.android.com/apk/res/android"
3.     android:width="108dp"
4.     android:height="108dp"
5.     android:viewportWidth="108"
6.     android:viewportHeight="108">
7.     <path
8.         android:fillColor="#3DDC84"
9.         android:pathData="M0,0h108v108h-108z" />
10.     <path
11.         android:fillColor="#00000000"
12.         android:pathData="M9,0L9,108"
13.         android:strokeWidth="0.8"
```

```
14.         android:strokeColor="#33FFFFFF" />
15.     <path
16.         android:fillColor="#00000000"
17.         android:pathData="M19,0L19,108"
18.         android:strokeWidth="0.8"
19.         android:strokeColor="#33FFFFFF" />
20.     <path
21.         android:fillColor="#00000000"
22.         android:pathData="M29,0L29,108"
23.         android:strokeWidth="0.8"
24.         android:strokeColor="#33FFFFFF" />
25.     <path
26.         android:fillColor="#00000000"
27.         android:pathData="M39,0L39,108"
28.         android:strokeWidth="0.8"
29.         android:strokeColor="#33FFFFFF" />
30.     <path
31.         android:fillColor="#00000000"
32.         android:pathData="M49,0L49,108"
33.         android:strokeWidth="0.8"
34.         android:strokeColor="#33FFFFFF" />
35.     <path
36.         android:fillColor="#00000000"
37.         android:pathData="M59,0L59,108"
38.         android:strokeWidth="0.8"
39.         android:strokeColor="#33FFFFFF" />
40.     <path
41.         android:fillColor="#00000000"
42.         android:pathData="M69,0L69,108"
43.         android:strokeWidth="0.8"
44.         android:strokeColor="#33FFFFFF" />
45.     <path
46.         android:fillColor="#00000000"
47.         android:pathData="M79,0L79,108"
48.         android:strokeWidth="0.8"
49.         android:strokeColor="#33FFFFFF" />
50.     <path
51.         android:fillColor="#00000000"
52.         android:pathData="M89,0L89,108"
53.         android:strokeWidth="0.8"
54.         android:strokeColor="#33FFFFFF" />
55.     <path
56.         android:fillColor="#00000000"
57.         android:pathData="M99,0L99,108"
58.         android:strokeWidth="0.8"
59.         android:strokeColor="#33FFFFFF" />
60.     <path
61.         android:fillColor="#00000000"
62.         android:pathData="M0,9L108,9"
63.         android:strokeWidth="0.8"
64.         android:strokeColor="#33FFFFFF" />
65.     <path
66.         android:fillColor="#00000000"
67.         android:pathData="M0,19L108,19"
68.         android:strokeWidth="0.8"
69.         android:strokeColor="#33FFFFFF" />
70.     <path
71.         android:fillColor="#00000000"
72.         android:pathData="M0,29L108,29"
73.         android:strokeWidth="0.8"
74.         android:strokeColor="#33FFFFFF" />
75.     <path
76.         android:fillColor="#00000000"
77.         android:pathData="M0,39L108,39"
78.         android:strokeWidth="0.8"
```

```
79.         android:strokeColor="#33FFFFFF" />
80.     <path
81.         android:fillColor="#00000000"
82.         android:pathData="M0,49L108,49"
83.         android:strokeWidth="0.8"
84.         android:strokeColor="#33FFFFFF" />
85.     <path
86.         android:fillColor="#00000000"
87.         android:pathData="M0,59L108,59"
88.         android:strokeWidth="0.8"
89.         android:strokeColor="#33FFFFFF" />
90.     <path
91.         android:fillColor="#00000000"
92.         android:pathData="M0,69L108,69"
93.         android:strokeWidth="0.8"
94.         android:strokeColor="#33FFFFFF" />
95.     <path
96.         android:fillColor="#00000000"
97.         android:pathData="M0,79L108,79"
98.         android:strokeWidth="0.8"
99.         android:strokeColor="#33FFFFFF" />
100.    <path
101.        android:fillColor="#00000000"
102.        android:pathData="M0,89L108,89"
103.        android:strokeWidth="0.8"
104.        android:strokeColor="#33FFFFFF" />
105.    <path
106.        android:fillColor="#00000000"
107.        android:pathData="M0,99L108,99"
108.        android:strokeWidth="0.8"
109.        android:strokeColor="#33FFFFFF" />
110.    <path
111.        android:fillColor="#00000000"
112.        android:pathData="M19,29L89,29"
113.        android:strokeWidth="0.8"
114.        android:strokeColor="#33FFFFFF" />
115.    <path
116.        android:fillColor="#00000000"
117.        android:pathData="M19,39L89,39"
118.        android:strokeWidth="0.8"
119.        android:strokeColor="#33FFFFFF" />
120.    <path
121.        android:fillColor="#00000000"
122.        android:pathData="M19,49L89,49"
123.        android:strokeWidth="0.8"
124.        android:strokeColor="#33FFFFFF" />
125.    <path
126.        android:fillColor="#00000000"
127.        android:pathData="M19,59L89,59"
128.        android:strokeWidth="0.8"
129.        android:strokeColor="#33FFFFFF" />
130.    <path
131.        android:fillColor="#00000000"
132.        android:pathData="M19,69L89,69"
133.        android:strokeWidth="0.8"
134.        android:strokeColor="#33FFFFFF" />
135.    <path
136.        android:fillColor="#00000000"
137.        android:pathData="M19,79L89,79"
138.        android:strokeWidth="0.8"
139.        android:strokeColor="#33FFFFFF" />
140.    <path
141.        android:fillColor="#00000000"
142.        android:pathData="M29,19L29,89"
143.        android:strokeWidth="0.8"
```

```

144.         android:strokeColor="#33FFFFFF" />
145.     <path
146.         android:fillColor="#00000000"
147.         android:pathData="M39,19L39,89"
148.         android:strokeWidth="0.8"
149.         android:strokeColor="#33FFFFFF" />
150.     <path
151.         android:fillColor="#00000000"
152.         android:pathData="M49,19L49,89"
153.         android:strokeWidth="0.8"
154.         android:strokeColor="#33FFFFFF" />
155.     <path
156.         android:fillColor="#00000000"
157.         android:pathData="M59,19L59,89"
158.         android:strokeWidth="0.8"
159.         android:strokeColor="#33FFFFFF" />
160.     <path
161.         android:fillColor="#00000000"
162.         android:pathData="M69,19L69,89"
163.         android:strokeWidth="0.8"
164.         android:strokeColor="#33FFFFFF" />
165.     <path
166.         android:fillColor="#00000000"
167.         android:pathData="M79,19L79,89"
168.         android:strokeWidth="0.8"
169.         android:strokeColor="#33FFFFFF" />
170. </vector>
171.
172.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\drawable\ic_launcher_foreground.xml

```

1. <vector xmlns:android="http://schemas.android.com/apk/res/android"
2.     xmlns:aapt="http://schemas.android.com/aapt"
3.     android:width="108dp"
4.     android:height="108dp"
5.     android:viewportWidth="108"
6.     android:viewportHeight="108">
7.     <path android:pathData="M31,63.928c0,0 6.4,-11 12.1,-13.1c7.2,-2.6 26,-1.4 26,-
1.4l38.1,38.1L107,108.928l-32,-1L31,63.928z">
8.         <aapt:attr name="android:fillColor">
9.             <gradient
10.                 android:endX="85.84757"
11.                 android:endY="92.4963"
12.                 android:startX="42.9492"
13.                 android:startY="49.59793"
14.                 android:type="linear">
15.                     <item
16.                         android:color="#44000000"
17.                         android:offset="0.0" />
18.                     <item
19.                         android:color="#00000000"
20.                         android:offset="1.0" />
21.                 </gradient>
22.             </aapt:attr>
23.         </path>
24.     <path
25.         android:fillColor="#FFFFFF"
26.         android:fillType="nonZero"
27.         android:pathData="M65.3,45.828l3.8,-6.6c0.2,-0.4 0.1,-0.9 -0.3,-1.1c-0.4,-0.2 -0.9,-0.1
-1.1,0.3l-3.9,6.7c-6.3,-2.8 -13.4,-2.8 -19.7,0l-3.9,-6.7c-0.2,-0.4 -0.7,-0.5 -1.1,-0.3C38.8,38.328
38.7,38.828 38.9,39.228l3.8,6.6C36.2,49.428 31.7,56.028 31,63.928h46C76.3,56.028 71.8,49.428
65.3,45.828z"
28.         android:strokeColor="#33FFFFFF"
29.         android:strokeWidth="0.8" />
30.     </path>
31. </vector>

```

```

2.1,-0.4c0.7,0.3 1.2,1 1.2,1.8C45.3,56.528 44.5,57.328 43.4,57.328L43.4,57.328zM64.6,57.328c-0.8,0 -
1.5,-0.5 -1.8,-1.2s-0.1,-1.5 0.4,-2.1c0.5,-0.5 1.4,-0.7 2.1,-0.4c0.7,0.3 1.2,1 1.2,1.8C66.5,56.528
65.6,57.328 64.6,57.328L64.6,57.328z"
28.         android:strokeWidth="1"
29.         android:strokeColor="#00000000" />
30. </vector>
31.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\drawable\ic_map.xml

```

1. <!--
2.  ~ Copyright (C) 2026 The Android Open Source Project
3.  ~
4.  ~ Licensed under the Apache License, Version 2.0 (the "License");
5.  ~ you may not use this file except in compliance with the License.
6.  ~ You may obtain a copy of the License at
7.  ~
8.  ~     http://www.apache.org/licenses/LICENSE-2.0
9.  ~
10. ~ Unless required by applicable law or agreed to in writing, software
11. ~ distributed under the License is distributed on an "AS IS" BASIS,
12. ~ WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13. ~ See the License for the specific language governing permissions and
14. ~ limitations under the License.
15. -->
16. <vector xmlns:android="http://schemas.android.com/apk/res/android" android:height="24dp"
17. android:tint="#000000" android:viewportHeight="24" android:viewportWidth="24" android:width="24dp">
18.     <path android:fillColor="@android:color/white" android:pathData="M20.5,31-0.16,0.03L15,5.1
19. 9,3 3.36,4.9c-0.21,0.07 -0.36,0.25 -0.36,0.48V20.5c0,0.28 0.22,0.5 0.5,0.510.16,-0.03L9,18.916,2.1
20. 5.64,-1.9c0.21,-0.07 0.36,-0.25 0.36,-0.48V3.5c0,-0.28 -0.22,-0.5 -0.5,-0.5zM15,191-6,-
21. 2.11V516,2.11V19z"/>
22.
23. D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\drawable\ic_profile.xml
24. <!--
25.  ~ Copyright (C) 2026 The Android Open Source Project
26.  ~
27.  ~ Licensed under the Apache License, Version 2.0 (the "License");
28.  ~ you may not use this file except in compliance with the License.
29.  ~ You may obtain a copy of the License at
30.  ~
31.  ~     http://www.apache.org/licenses/LICENSE-2.0
32.  ~
33. ~ Unless required by applicable law or agreed to in writing, software
34. ~ distributed under the License is distributed on an "AS IS" BASIS,
35. ~ WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
36. ~ See the License for the specific language governing permissions and
37. ~ limitations under the License.
38. -->
39. <vector xmlns:android="http://schemas.android.com/apk/res/android" android:height="24dp"
40. android:tint="#000000" android:viewportHeight="24" android:viewportWidth="24" android:width="24dp">
41.     <path android:fillColor="@android:color/white" android:pathData="M12,2C6.48,2 2,6.48
42. 2,12s4.48,10 10,10 -4.48 10,-10S17.52,2 12,2zM12,5c1.66,0 3,1.34 3,3s-1.34,3 -3,3 -3,-1.34 -3,-3
43. 1.34,-3 3,-3zM12,19.2c-2.5,0 -4.71,-1.28 -6,-3.22 0.03,-1.99 4,-3.08 6,-3.08 1.99,0 5.97,1.09 6,3.08
44. -1.29,1.94 -3.5,3.22 -6,3.22z"/>

```


D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\activity_booking_summary.xml

```
1. <?xml version="1.0" encoding="utf-8"?>
2. <androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     android:layout_width="match_parent"
5.     android:layout_height="match_parent"
6.     android:background="#0F172A">
7.
8.     <!-- Content Wrapper for Vertical Centering -->
9.     <androidx.constraintlayout.widget.ConstraintLayout
10.         android:layout_width="0dp"
11.         android:layout_height="0dp"
12.         android:padding="24dp"
13.         app:layout_constraintBottom_toTopOf="@id/bottomNavigation"
14.         app:layout_constraintEnd_toEndOf="parent"
15.         app:layout_constraintStart_toStartOf="parent"
16.         app:layout_constraintTop_toTopOf="parent">
17.
18.         <!-- Title mapped to a packed chain for perfect centering -->
19.         <TextView
20.             android:id="@+id/tvSuccessTitle"
21.             android:layout_width="0dp"
22.             android:layout_height="wrap_content"
23.             android:fontFamily="sans-serif-medium"
24.             android:text="Booking Confirmed!"
25.             android:textAlignment="center"
26.             android:textColor="#10B981"
27.             android:textSize="28sp"
28.             app:layout_constraintBottom_toTopOf="@id/cardSummary"
29.             app:layout_constraintEnd_toEndOf="parent"
30.             app:layout_constraintStart_toStartOf="parent"
31.             app:layout_constraintTop_toTopOf="parent"
32.             app:layout_constraintVertical_chainStyle="packed" />
33.
34.         <!-- Summary Card -->
35.         <com.google.android.material.card.MaterialCardView
36.             android:id="@+id/cardSummary"
37.             android:layout_width="0dp"
38.             android:layout_height="wrap_content"
39.             android:layout_marginTop="40dp"
40.             app:cardBackgroundColor="#1E293B"
41.             app:cardCornerRadius="24dp"
42.             app:cardElevation="0dp"
43.             app:layout_constraintBottom_toTopOf="@id/btnBackToDashboard"
44.             app:layout_constraintEnd_toEndOf="parent"
45.             app:layout_constraintStart_toStartOf="parent"
46.             app:layout_constraintTop_toBottomOf="@id/tvSuccessTitle"
47.             app:strokeColor="#334155"
48.             app:strokeWidth="1dp">
49.
50.             <androidx.constraintlayout.widget.ConstraintLayout
51.                 android:layout_width="match_parent"
52.                 android:layout_height="wrap_content"
53.                 android:padding="24dp">
54.
55.                 <TextView
56.                     android:id="@+id/txtSummaryStation"
57.                     android:layout_width="0dp"
58.                     android:layout_height="wrap_content"
59.                     android:fontFamily="sans-serif-medium"
60.                     android:textColor="FFFFFF"
61.                     android:textSize="20sp"
```

```

62.         app:layout_constraintEnd_toEndOf="parent"
63.         app:layout_constraintStart_toStartOf="parent"
64.         app:layout_constraintTop_toTopOf="parent" />
65.
66.     <TextView
67.         android:id="@+id/txtSummaryDate"
68.         android:layout_width="0dp"
69.         android:layout_height="wrap_content"
70.         android:layout_marginTop="16dp"
71.         android:fontFamily="sans-serif"
72.         android:textColor="#94A3B8"
73.         android:textSize="15sp"
74.         app:layout_constraintEnd_toEndOf="parent"
75.         app:layout_constraintStart_toStartOf="parent"
76.         app:layout_constraintTop_toBottomOf="@id/txtSummaryStation" />
77.
78.     <TextView
79.         android:id="@+id/txtSummaryTime"
80.         android:layout_width="0dp"
81.         android:layout_height="wrap_content"
82.         android:layout_marginTop="4dp"
83.         android:fontFamily="sans-serif"
84.         android:textColor="#94A3B8"
85.         android:textSize="15sp"
86.         app:layout_constraintEnd_toEndOf="parent"
87.         app:layout_constraintStart_toStartOf="parent"
88.         app:layout_constraintTop_toBottomOf="@id/txtSummaryDate" />
89.
90.     <TextView
91.         android:id="@+id/txtSummaryAmount"
92.         android:layout_width="0dp"
93.         android:layout_height="wrap_content"
94.         android:layout_marginTop="24dp"
95.         android:fontFamily="sans-serif-medium"
96.         android:textColor="#F59E0B"
97.         android:textSize="18sp"
98.         app:layout_constraintEnd_toEndOf="parent"
99.         app:layout_constraintStart_toStartOf="parent"
100.        app:layout_constraintTop_toBottomOf="@id/txtSummaryTime" />
101.
102.    </androidx.constraintlayout.widget.ConstraintLayout>
103. </com.google.android.material.card.MaterialCardView>
104.
105. <!-- Pill-shaped CTA -->
106. <com.google.android.material.button.MaterialButton
107.     android:id="@+id/btnBackToDashboard"
108.     android:layout_width="0dp"
109.     android:layout_height="56dp"
110.     android:layout_marginTop="48dp"
111.     android:fontFamily="sans-serif-medium"
112.     android:text="Return to Dashboard"
113.     android:textAllCaps="false"
114.     android:textColor="#0F172A"
115.     android:textSize="17sp"
116.     app:backgroundTint="#F59E0B"
117.     app:cornerRadius="28dp"
118.     app:layout_constraintBottom_toBottomOf="parent"
119.     app:layout_constraintEnd_toEndOf="parent"
120.     app:layout_constraintStart_toStartOf="parent"
121.     app:layout_constraintTop_toBottomOf="@id/cardSummary" />
122.
123. </androidx.constraintlayout.widget.ConstraintLayout>
124.
125. <!-- Bottom Navigation natively anchored to the bottom -->
126. <com.google.android.material.bottomnavigation.BottomNavigationView

```

```

127.         android:id="@+id/bottomNavigation"
128.         android:layout_width="0dp"
129.         android:layout_height="wrap_content"
130.         android:background="#1E293B"
131.         app:itemIconTint="@color/bottom_nav_selector"
132.         app:itemRippleColor="#334155"
133.         app:itemTextColor="@color/bottom_nav_selector"
134.         app:labelVisibilityMode="labeled"
135.         app:layout_constraintBottom_toBottomOf="parent"
136.         app:layout_constraintEnd_toEndOf="parent"
137.         app:layout_constraintStart_toStartOf="parent"
138.         app:menu="@menu/bottom_nav_menu" />
139.
140. </androidx.constraintlayout.widget.ConstraintLayout>
141.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\activity_create_booking.xml

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     android:layout_width="match_parent"
5.     android:layout_height="match_parent"
6.     android:background="#0F172A">
7.
8.     <ScrollView
9.         android:layout_width="0dp"
10.        android:layout_height="0dp"
11.        android:fillViewport="true"
12.        app:layout_constraintBottom_toTopOf="@id/bottomNavigation"
13.        app:layout_constraintEnd_toEndOf="parent"
14.        app:layout_constraintStart_toStartOf="parent"
15.        app:layout_constraintTop_toTopOf="parent">
16.
17.        <androidx.constraintlayout.widget.ConstraintLayout
18.            android:layout_width="match_parent"
19.            android:layout_height="wrap_content"
20.            android:padding="24dp">
21.
22.            <!-- Interactive Navigation Text -->
23.            <TextView
24.                android:id="@+id/btnBack"
25.                android:layout_width="wrap_content"
26.                android:layout_height="wrap_content"
27.                android:layout_marginTop="16dp"
28.                android:background="?attr/selectableItemBackground"
29.                android:clickable="true"
30.                android:focusable="true"
31.                android:fontFamily="sans-serif-medium"
32.                android:paddingVertical="8dp"
33.                android:paddingEnd="16dp"
34.                android:text="← Back"
35.                android:textColor="#F59E0B"
36.                android:textSize="17sp"
37.                app:layout_constraintStart_toStartOf="parent"
38.                app:layout_constraintTop_toTopOf="parent" />
39.
40.            <!-- Standardized iOS Large Header -->
41.            <TextView
42.                android:id="@+id/tvTitle"
43.                android:layout_width="0dp"
44.                android:layout_height="wrap_content"

```

```

45.         android:layout_marginTop="24dp"
46.         android:fontFamily="sans-serif-medium"
47.         android:text="Reserve Energy"
48.         android:textColor="#FFFFFF"
49.         android:textSize="34sp"
50.         app:layout_constraintEnd_toEndOf="parent"
51.         app:layout_constraintStart_toStartOf="parent"
52.         app:layout_constraintTop_toBottomOf="@id/btnBack" />
53.
54.     <TextView
55.         android:id="@+id/tvSubtitle"
56.         android:layout_width="0dp"
57.         android:layout_height="wrap_content"
58.         android:layout_marginTop="8dp"
59.         android:fontFamily="sans-serif"
60.         android:letterSpacing="0.02"
61.         android:text="Book a microgrid slot (max 7 days in advance)"
62.         android:textColor="#EBEBF5"
63.         android:textSize="17sp"
64.         app:layout_constraintEnd_toEndOf="parent"
65.         app:layout_constraintStart_toStartOf="parent"
66.         app:layout_constraintTop_toBottomOf="@id/tvTitle" />
67.
68.     <!-- Station Spinner Integration -->
69.     <TextView
70.         android:id="@+id/tvStationLabel"
71.         android:layout_width="wrap_content"
72.         android:layout_height="wrap_content"
73.         android:layout_marginTop="40dp"
74.         android:layout_marginStart="4dp"
75.         android:fontFamily="sans-serif"
76.         android:text="Select Station"
77.         android:textColor="#94A3B8"
78.         android:textSize="13sp"
79.         app:layout_constraintStart_toStartOf="parent"
80.         app:layout_constraintTop_toBottomOf="@id/tvSubtitle" />
81.
82.     <com.google.android.material.card.MaterialCardView
83.         android:id="@+id/layoutStation"
84.         android:layout_width="0dp"
85.         android:layout_height="60dp"
86.         android:layout_marginTop="8dp"
87.         app:cardBackgroundColor="#1E293B"
88.         app:cardCornerRadius="16dp"
89.         app:cardElevation="0dp"
90.         app:strokeColor="#334155"
91.         app:strokeWidth="1dp"
92.         app:layout_constraintEnd_toEndOf="parent"
93.         app:layout_constraintStart_toStartOf="parent"
94.         app:layout_constraintTop_toBottomOf="@id/tvStationLabel">
95.
96.         <Spinner
97.             android:id="@+id/spinnerStation"
98.             android:layout_width="match_parent"
99.             android:layout_height="match_parent"
100.             android:backgroundTint="#94A3B8"
101.             android:paddingHorizontal="8dp" />
102.     </com.google.android.material.card.MaterialCardView>
103.
104.     <!-- Reservation Date -->
105.     <com.google.android.material.textfield.TextInputLayout
106.         android:id="@+id/layoutDate"
107.         style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
108.         android:layout_width="0dp"
109.         android:layout_height="wrap_content"

```

```

110.         android:layout_marginTop="24dp"
111.         android:textColorHint="#94A3B8"
112.         app:boxBackgroundColor="#1E293B"
113.         app:boxCornerRadiusBottomEnd="16dp"
114.         app:boxCornerRadiusBottomStart="16dp"
115.         app:boxCornerRadiusTopEnd="16dp"
116.         app:boxCornerRadiusTopStart="16dp"
117.         app:boxStrokeColor="#334155"
118.         app:boxStrokeWidth="1dp"
119.         app:boxStrokeWidthFocused="2dp"
120.         app:hintTextColor="#F59E0B"
121.         app:layout_constraintEnd_toEndOf="parent"
122.         app:layout_constraintStart_toStartOf="parent"
123.         app:layout_constraintTop_toBottomOf="@id/layoutStation">
124.
125.         <com.google.android.material.textfield.TextInputEditText
126.             android:id="@+id/inputBookingDate"
127.             android:layout_width="match_parent"
128.             android:layout_height="60dp"
129.             android:focusable="false"
130.             android:hint="Reservation Date (YYYY-MM-DD)"
131.             android:textColor="#FFFFFF"
132.             android:textSize="16sp" />
133.     </com.google.android.material.textfield.TextInputLayout>
134.
135.     <!-- Side-by-side Time Slots -->
136.     <LinearLayout
137.         android:id="@+id/layoutTimeRow"
138.         android:layout_width="0dp"
139.         android:layout_height="wrap_content"
140.         android:layout_marginTop="16dp"
141.         android:baselineAligned="false"
142.         android:orientation="horizontal"
143.         android:weightSum="2"
144.         app:layout_constraintEnd_toEndOf="parent"
145.         app:layout_constraintStart_toStartOf="parent"
146.         app:layout_constraintTop_toBottomOf="@id/layoutDate">
147.
148.         <com.google.android.material.textfield.TextInputLayout
149.             style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
150.             android:layout_width="0dp"
151.             android:layout_height="wrap_content"
152.             android:layout_marginEnd="8dp"
153.             android:layout_weight="1"
154.             android:textColorHint="#94A3B8"
155.             app:boxBackgroundColor="#1E293B"
156.             app:boxCornerRadiusBottomEnd="16dp"
157.             app:boxCornerRadiusBottomStart="16dp"
158.             app:boxCornerRadiusTopEnd="16dp"
159.             app:boxCornerRadiusTopStart="16dp"
160.             app:boxStrokeColor="#334155"
161.             app:boxStrokeWidth="1dp"
162.             app:boxStrokeWidthFocused="2dp"
163.             app:hintTextColor="#F59E0B">
164.
165.             <com.google.android.material.textfield.TextInputEditText
166.                 android:id="@+id/inputStartTime"
167.                 android:layout_width="match_parent"
168.                 android:layout_height="60dp"
169.                 android:focusable="false"
170.                 android:hint="Start Time"
171.                 android:textColor="#FFFFFF"
172.                 android:textSize="16sp" />
173.             </com.google.android.material.textfield.TextInputLayout>
174.

```

```

175.         <com.google.android.material.textfield.TextInputLayout
176.             style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
177.             android:layout_width="0dp"
178.             android:layout_height="wrap_content"
179.             android:layout_marginStart="8dp"
180.             android:layout_weight="1"
181.             android:textColorHint="#94A3B8"
182.             app:boxBackgroundColor="#1E293B"
183.             app:boxCornerRadiusBottomEnd="16dp"
184.             app:boxCornerRadiusBottomStart="16dp"
185.             app:boxCornerRadiusTopEnd="16dp"
186.             app:boxCornerRadiusTopStart="16dp"
187.             app:boxStrokeColor="#334155"
188.             app:boxStrokeWidth="1dp"
189.             app:boxStrokeWidthFocused="2dp"
190.             app:hintTextColor="#F59E0B">
191.
192.             <com.google.android.material.textfield.TextInputEditText
193.                 android:id="@+id/inputEndTime"
194.                 android:layout_width="match_parent"
195.                 android:layout_height="60dp"
196.                 android:focusable="false"
197.                 android:hint="End Time"
198.                 android:textColor="#FFFFFF"
199.                 android:textSize="16sp" />
200.         </com.google.android.material.textfield.TextInputLayout>
201.     </LinearLayout>
202.
203. <!-- Energy Amount -->
204. <com.google.android.material.textfield.TextInputLayout
205.     android:id="@+id/layoutEnergy"
206.     style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
207.     android:layout_width="0dp"
208.     android:layout_height="wrap_content"
209.     android:layout_marginTop="16dp"
210.     android:textColorHint="#94A3B8"
211.     app:boxBackgroundColor="#1E293B"
212.     app:boxCornerRadiusBottomEnd="16dp"
213.     app:boxCornerRadiusBottomStart="16dp"
214.     app:boxCornerRadiusTopEnd="16dp"
215.     app:boxCornerRadiusTopStart="16dp"
216.     app:boxStrokeColor="#334155"
217.     app:boxStrokeWidth="1dp"
218.     app:boxStrokeWidthFocused="2dp"
219.     app:hintTextColor="#F59E0B"
220.     app:layout_constraintEnd_toEndOf="parent"
221.     app:layout_constraintStart_toStartOf="parent"
222.     app:layout_constraintTop_toBottomOf="@id/layoutTimeRow">
223.
224.         <com.google.android.material.textfield.TextInputEditText
225.             android:id="@+id/inputEnergyAmount"
226.             android:layout_width="match_parent"
227.             android:layout_height="60dp"
228.             android:hint="Energy Amount (kWh)"
229.             android:inputType="numberDecimal"
230.             android:textColor="#FFFFFF"
231.             android:textSize="16sp" />
232.     </com.google.android.material.textfield.TextInputLayout>
233.
234. <!-- Pill-shaped Primary Action Button -->
235. <com.google.android.material.button.MaterialButton
236.     android:id="@+id/btnConfirmBooking"
237.     android:layout_width="0dp"
238.     android:layout_height="56dp"
239.     android:layout_marginTop="40dp"

```

```

240.         android:layout_marginBottom="24dp"
241.         android:fontFamily="sans-serif-medium"
242.         android:text="Confirm Booking"
243.         android:textAllCaps="false"
244.         android:textColor="#0F172A"
245.         android:textSize="17sp"
246.         app:backgroundTint="#F59E0B"
247.         app:cornerRadius="28dp"
248.         app:layout_constraintBottom_toBottomOf="parent"
249.         app:layout_constraintEnd_toEndOf="parent"
250.         app:layout_constraintStart_toStartOf="parent"
251.         app:layout_constraintTop_toBottomOf="@id/layoutEnergy" />
252.
253.     </androidx.constraintlayout.widget.ConstraintLayout>
254. </ScrollView>
255.
256. <!-- Bottom Navigation natively anchored to the bottom -->
257. <com.google.android.material.bottomnavigation.BottomNavigationView
258.     android:id="@+id/bottomNavigation"
259.     android:layout_width="0dp"
260.     android:layout_height="wrap_content"
261.     android:background="#1E293B"
262.     app:itemIconTint="@color/bottom_nav_selector"
263.     app:itemRippleColor="#334155"
264.     app:itemTextColor="@color/bottom_nav_selector"
265.     app:labelVisibilityMode="labeled"
266.     app:layout_constraintBottom_toBottomOf="parent"
267.     app:layout_constraintEnd_toEndOf="parent"
268.     app:layout_constraintStart_toStartOf="parent"
269.     app:menu="@menu/bottom_nav_menu" />
270.
271. </androidx.constraintlayout.widget.ConstraintLayout>
272.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\activity_dashboard.xml

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     android:layout_width="match_parent"
5.     android:layout_height="match_parent"
6.     android:background="#0F172A">
7.
8.     <ScrollView
9.         android:layout_width="0dp"
10.        android:layout_height="0dp"
11.        android:fillViewport="true"
12.        app:layout_constraintBottom_toTopOf="@id/bottomNavigation"
13.        app:layout_constraintEnd_toEndOf="parent"
14.        app:layout_constraintStart_toStartOf="parent"
15.        app:layout_constraintTop_toTopOf="parent">
16.
17.        <androidx.constraintlayout.widget.ConstraintLayout
18.            android:layout_width="match_parent"
19.            android:layout_height="wrap_content"
20.            android:padding="24dp">
21.
22.            <!-- Large iOS-style Header -->
23.            <TextView
24.                android:id="@+id/tvDashboardTitle"
25.                android:layout_width="0dp"
26.                android:layout_height="wrap_content"

```



```

27.         android:fontFamily="sans-serif-medium"
28.         android:text="Dashboard"
29.         android:textColor="#FFFFFF"
30.         android:textSize="34sp"
31.         app:layout_constraintEnd_toEndOf="parent"
32.         app:layout_constraintStart_toStartOf="parent"
33.         app:layout_constraintTop_toTopOf="parent" />
34.
35.     <TextView
36.         android:id="@+id/txtWelcome"
37.         android:layout_width="0dp"
38.         android:layout_height="wrap_content"
39.         android:layout_marginTop="4dp"
40.         android:fontFamily="sans-serif"
41.         android:letterSpacing="0.02"
42.         android:text="Welcome"
43.         android:textColor="#EBEBF5"
44.         android:textSize="17sp"
45.         app:layout_constraintEnd_toEndOf="parent"
46.         app:layout_constraintStart_toStartOf="parent"
47.         app:layout_constraintTop_toBottomOf="@id/tvDashboardTitle" />
48.
49.     <!-- Next Upcoming Booking Widget -->
50.     <com.google.android.material.card.MaterialCardView
51.         android:id="@+id/cardNextBooking"
52.         android:layout_width="0dp"
53.         android:layout_height="wrap_content"
54.         android:layout_marginTop="32dp"
55.         app:cardBackgroundColor="#1E293B"
56.         app:cardCornerRadius="24dp"
57.         app:cardElevation="0dp"
58.         app:layout_constraintEnd_toEndOf="parent"
59.         app:layout_constraintStart_toStartOf="parent"
60.         app:layout_constraintTop_toBottomOf="@id/txtWelcome"
61.         app:strokeColor="#334155"
62.         app:strokeWidth="1dp">
63.
64.         <androidx.constraintlayout.widget.ConstraintLayout
65.             android:layout_width="match_parent"
66.             android:layout_height="wrap_content"
67.             android:padding="24dp">
68.
69.             <TextView
70.                 android:id="@+id/tvNextLabel"
71.                 android:layout_width="wrap_content"
72.                 android:layout_height="wrap_content"
73.                 android:fontFamily="sans-serif-medium"
74.                 android:letterSpacing="0.05"
75.                 android:text="NEXT RESERVATION"
76.                 android:textColor="#F59E0B"
77.                 android:textSize="12sp"
78.                 app:layout_constraintStart_toStartOf="parent"
79.                 app:layout_constraintTop_toTopOf="parent" />
80.
81.             <TextView
82.                 android:id="@+id/txtNextStation"
83.                 android:layout_width="0dp"
84.                 android:layout_height="wrap_content"
85.                 android:layout_marginTop="8dp"
86.                 android:fontFamily="sans-serif-medium"
87.                 android:text="No upcoming bookings"
88.                 android:textColor="#FFFFFF"
89.                 android:textSize="22sp"
90.                 app:layout_constraintEnd_toEndOf="parent"
91.                 app:layout_constraintStart_toStartOf="parent"

```



```

92.         app:layout_constraintTop_toBottomOf="@id/tvNextLabel" />
93.
94.         <TextView
95.             android:id="@+id/txtNextTime"
96.             android:layout_width="0dp"
97.             android:layout_height="wrap_content"
98.             android:layout_marginTop="4dp"
99.             android:fontFamily="sans-serif"
100.            android:text="Ready to book a microgrid slot?"
101.            android:textColor="#94A3B8"
102.            android:textSize="15sp"
103.            app:layout_constraintEnd_toEndOf="parent"
104.            app:layout_constraintStart_toStartOf="parent"
105.            app:layout_constraintTop_toBottomOf="@id/txtNextStation" />
106.        </androidx.constraintlayout.widget.ConstraintLayout>
107.    </com.google.android.material.card.MaterialCardView>
108.
109.    <!-- Quick Actions -->
110.    <LinearLayout
111.        android:id="@+id/layoutQuickActions"
112.        android:layout_width="0dp"
113.        android:layout_height="wrap_content"
114.        android:layout_marginTop="24dp"
115.        android:orientation="horizontal"
116.        app:layout_constraintEnd_toEndOf="parent"
117.        app:layout_constraintStart_toStartOf="parent"
118.        app:layout_constraintTop_toBottomOf="@id/cardNextBooking">
119.
120.        <com.google.android.material.button.MaterialButton
121.            android:id="@+id/btnQuickBook"
122.            android:layout_width="0dp"
123.            android:layout_height="56dp"
124.            android:layout_marginEnd="8dp"
125.            android:layout_weight="1"
126.            android:fontFamily="sans-serif-medium"
127.            android:text="Book Slot"
128.            android:textAllCaps="false"
129.            android:textColor="#0F172A"
130.            android:textSize="16sp"
131.            app:backgroundTint="#F59E0B"
132.            app:cornerRadius="28dp" />
133.
134.        <com.google.android.material.button.MaterialButton
135.            android:id="@+id/btnQuickHistory"
136.            android:layout_width="0dp"
137.            android:layout_height="56dp"
138.            android:layout_marginStart="8dp"
139.            android:layout_weight="1"
140.            android:fontFamily="sans-serif-medium"
141.            android:text="My Tickets"
142.            android:textAllCaps="false"
143.            android:textColor="#EBEBF5"
144.            android:textSize="16sp"
145.            app:backgroundTint="@android:color/transparent"
146.            app:cornerRadius="28dp"
147.            app:strokeColor="#334155"
148.            app:strokeWidth="1dp" />
149.    </LinearLayout>
150.
151.    <!-- Metrics Row -->
152.    <LinearLayout
153.        android:id="@+id/layoutMetrics"
154.        android:layout_width="0dp"
155.        android:layout_height="wrap_content"
156.        android:layout_marginTop="32dp"

```

```

157.         android:baselineAligned="false"
158.         android:orientation="horizontal"
159.         android:weightSum="3"
160.         app:layout_constraintEnd_toEndOf="parent"
161.         app:layout_constraintStart_toStartOf="parent"
162.         app:layout_constraintTop_toBottomOf="@id/layoutQuickActions">
163.
164.     <LinearLayout
165.         android:layout_width="0dp"
166.         android:layout_height="wrap_content"
167.         android:layout_marginEnd="8dp"
168.         android:layout_weight="1"
169.         android:gravity="center"
170.         android:orientation="vertical">
171.
172.         <TextView
173.             android:layout_width="wrap_content"
174.             android:layout_height="wrap_content"
175.             android:fontFamily="sans-serif"
176.             android:text="Pending"
177.             android:textColor="#94A3B8"
178.             android:textSize="13sp" />
179.
180.         <TextView
181.             android:id="@+id/txtPendingCount"
182.             android:layout_width="wrap_content"
183.             android:layout_height="wrap_content"
184.             android:fontFamily="sans-serif-medium"
185.             android:text="0"
186.             android:textColor="#FFFFFF"
187.             android:textSize="28sp" />
188.     </LinearLayout>
189.
190.     <LinearLayout
191.         android:layout_width="0dp"
192.         android:layout_height="wrap_content"
193.         android:layout_marginHorizontal="4dp"
194.         android:layout_weight="1"
195.         android:gravity="center"
196.         android:orientation="vertical">
197.
198.         <TextView
199.             android:layout_width="wrap_content"
200.             android:layout_height="wrap_content"
201.             android:fontFamily="sans-serif"
202.             android:text="Approved"
203.             android:textColor="#94A3B8"
204.             android:textSize="13sp" />
205.
206.         <TextView
207.             android:id="@+id/txtApprovedCount"
208.             android:layout_width="wrap_content"
209.             android:layout_height="wrap_content"
210.             android:fontFamily="sans-serif-medium"
211.             android:text="0"
212.             android:textColor="#10B981"
213.             android:textSize="28sp" />
214.     </LinearLayout>
215.
216.     <LinearLayout
217.         android:layout_width="0dp"
218.         android:layout_height="wrap_content"
219.         android:layout_marginStart="8dp"
220.         android:layout_weight="1"
221.         android:gravity="center"

```

```

222.         android:orientation="vertical">
223.
224.         <TextView
225.             android:layout_width="wrap_content"
226.             android:layout_height="wrap_content"
227.             android:fontFamily="sans-serif"
228.             android:text="Completed"
229.             android:textColor="#94A3B8"
230.             android:textSize="13sp" />
231.
232.         <TextView
233.             android:id="@+id/txtCompletedCount"
234.             android:layout_width="wrap_content"
235.             android:layout_height="wrap_content"
236.             android:fontFamily="sans-serif-medium"
237.             android:text="0"
238.             android:textColor="#3B82F6"
239.             android:textSize="28sp" />
240.     </LinearLayout>
241. </LinearLayout>
242.
243. <!-- Map Section -->
244. <TextView
245.     android:id="@+id/tvMapHeader"
246.     android:layout_width="0dp"
247.     android:layout_height="wrap_content"
248.     android:layout_marginTop="40dp"
249.     android:fontFamily="sans-serif-medium"
250.     android:text="Live Grid Map"
251.     android:textColor="#FFFFFF"
252.     android:textSize="20sp"
253.     app:layout_constraintEnd_toEndOf="parent"
254.     app:layout_constraintStart_toStartOf="parent"
255.     app:layout_constraintTop_toBottomOf="@id/layoutMetrics" />
256.
257. <!-- Cleaned up redundant nested MaterialCardViews -->
258. <com.google.android.material.card.MaterialCardView
259.     android:id="@+id/cardMap"
260.     android:layout_width="0dp"
261.     android:layout_height="300dp"
262.     android:layout_marginTop="16dp"
263.     app:cardBackgroundColor="#1E293B"
264.     app:cardCornerRadius="24dp"
265.     app:cardElevation="0dp"
266.     app:layout_constraintEnd_toEndOf="parent"
267.     app:layout_constraintStart_toStartOf="parent"
268.     app:layout_constraintTop_toBottomOf="@id/tvMapHeader"
269.     app:strokeColor="#334155"
270.     app:strokeWidth="1dp">
271.
272.     <RelativeLayout
273.         android:layout_width="match_parent"
274.         android:layout_height="match_parent">
275.
276.         <androidx.fragment.app.FragmentContainerView
277.             android:id="@+id/mapPlaceholder"
278.             android:name="com.google.android.gms.maps.SupportMapFragment"
279.             android:layout_width="match_parent"
280.             android:layout_height="match_parent" />
281.
282.         <View
283.             android:id="@+id/mapClickOverlay"
284.             android:layout_width="match_parent"
285.             android:layout_height="match_parent"
286.             android:background="?attr/selectableItemBackground"

```

```

287.             android:clickable="true"
288.             android:focusable="true" />
289.         </RelativeLayout>
290.     </com.google.android.material.card.MaterialCardView>
291.
292.     <com.google.android.material.button.MaterialButton
293.         android:id="@+id/btnViewStations"
294.         style="@style/Widget.MaterialComponents.Button.TextButton"
295.         android:layout_width="wrap_content"
296.         android:layout_height="wrap_content"
297.         android:layout_marginTop="12dp"
298.         android:layout_marginBottom="16dp"
299.         android:fontFamily="sans-serif-medium"
300.         android:text="Refresh Map"
301.         android:textAllCaps="false"
302.         android:textColor="#F59E0B"
303.         android:textSize="15sp"
304.         app:layout_constraintBottom_toBottomOf="parent"
305.         app:layout_constraintEnd_toEndOf="parent"
306.         app:layout_constraintStart_toStartOf="parent"
307.         app:layout_constraintTop_toBottomOf="@id/cardMap" />
308.
309.     </androidx.constraintlayout.widget.ConstraintLayout>
310. </ScrollView>
311.
312. <!-- Bottom Navigation elevated out of the ScrollView -->
313. <com.google.android.material.bottomnavigation.BottomNavigationView
314.     android:id="@+id/bottomNavigation"
315.     android:layout_width="0dp"
316.     android:layout_height="wrap_content"
317.     android:background="#1E293B"
318.     app:itemIconTint="@color/bottom_nav_selector"
319.     app:itemRippleColor="#334155"
320.     app:itemTextColor="@color/bottom_nav_selector"
321.     app:labelVisibilityMode="labeled"
322.     app:layout_constraintBottom_toBottomOf="parent"
323.     app:layout_constraintEnd_toEndOf="parent"
324.     app:layout_constraintStart_toStartOf="parent"
325.     app:menu="@menu/bottom_nav_menu" />
326.
327. </androidx.constraintlayout.widget.ConstraintLayout>
328.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\activity_login.xml

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     android:layout_width="match_parent"
5.     android:layout_height="match_parent"
6.     android:background="#0F172A"
7.     android:padding="32dp">
8.
9.     <!-- Left-aligned, large title typography typical of modern iOS apps -->
10.    <TextView
11.        android:id="@+id/tvWelcome"
12.        android:layout_width="0dp"
13.        android:layout_height="wrap_content"
14.        android:layout_marginTop="48dp"
15.        android:fontFamily="sans-serif-medium"
16.        android:text="Welcome back"
17.        android:textColor="#FFFFFF"

```

```

18.         android:textSize="34sp"
19.         app:layout_constraintEnd_toEndOf="parent"
20.         app:layout_constraintStart_toStartOf="parent"
21.         app:layout_constraintTop_toTopOf="parent" />
22.
23.     <TextView
24.         android:id="@+id/tvSubTitle"
25.         android:layout_width="0dp"
26.         android:layout_height="wrap_content"
27.         android:layout_marginTop="8dp"
28.         android:fontFamily="sans-serif"
29.         android:letterSpacing="0.02"
30.         android:text="Login to your Prosumer Account"
31.         android:textColor="#EBEBF5"
32.         android:textSize="17sp"
33.         app:layout_constraintEnd_toEndOf="parent"
34.         app:layout_constraintStart_toStartOf="parent"
35.         app:layout_constraintTop_toBottomOf="@id/tvWelcome" />
36.
37.     <!-- Consolidated Label and Input into a modern outlined box with rounded corners -->
38.     <com.google.android.material.textfield.TextInputLayout
39.         android:id="@+id/layoutNic"
40.         style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
41.         android:layout_width="0dp"
42.         android:layout_height="wrap_content"
43.         android:layout_marginTop="48dp"
44.         android:textColorHint="#94A3B8"
45.         app:boxBackgroundColor="#1E293B"
46.         app:boxCornerRadiusBottomEnd="16dp"
47.         app:boxCornerRadiusBottomStart="16dp"
48.         app:boxCornerRadiusTopEnd="16dp"
49.         app:boxCornerRadiusTopStart="16dp"
50.         app:boxStrokeColor="#334155"
51.         app:boxStrokeWidth="1dp"
52.         app:boxStrokeWidthFocused="2dp"
53.         app:hintTextColor="#F59E0B"
54.         app:layout_constraintEnd_toEndOf="parent"
55.         app:layout_constraintStart_toStartOf="parent"
56.         app:layout_constraintTop_toBottomOf="@id/tvSubTitle">
57.
58.         <!-- ID strictly preserved for backend logic -->
59.         <com.google.android.material.textfield.TextInputEditText
60.             android:id="@+id/inputLoginNic"
61.             android:layout_width="match_parent"
62.             android:layout_height="60dp"
63.             android:hint="Username / NIC"
64.             android:inputType="text"
65.             android:textColor="#FFFFFF"
66.             android:textSize="16sp" />
67.     </com.google.android.material.textfield.TextInputLayout>
68.
69.     <com.google.android.material.textfield.TextInputLayout
70.         android:id="@+id/layoutPassword"
71.         style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
72.         android:layout_width="0dp"
73.         android:layout_height="wrap_content"
74.         android:layout_marginTop="20dp"
75.         android:textColorHint="#94A3B8"
76.         app:boxBackgroundColor="#1E293B"
77.         app:boxCornerRadiusBottomEnd="16dp"
78.         app:boxCornerRadiusBottomStart="16dp"
79.         app:boxCornerRadiusTopEnd="16dp"
80.         app:boxCornerRadiusTopStart="16dp"
81.         app:boxStrokeColor="#334155"
82.         app:boxStrokeWidth="1dp"

```

```

83.         app:boxStrokeWidthFocused="2dp"
84.         app:hintTextColor="#F59E0B"
85.         app:passwordToggleEnabled="true"
86.         app:passwordToggleTint="#94A3B8"
87.         app:layout_constraintEnd_toEndOf="parent"
88.         app:layout_constraintStart_toStartOf="parent"
89.         app:layout_constraintTop_toBottomOf="@id/layoutNic">
90.
91.         <!-- ID strictly preserved for backend logic -->
92.         <com.google.android.material.textfield.TextInputEditText
93.             android:id="@+id/inputLoginPassword"
94.             android:layout_width="match_parent"
95.             android:layout_height="60dp"
96.             android:hint="Password"
97.             android:inputType="textPassword"
98.             android:textColor="#FFFFFF"
99.             android:textSize="16sp" />
100.     </com.google.android.material.textfield.TextInputLayout>
101.
102.     <!-- Pill-shaped button matching the Get Started screen -->
103.     <com.google.android.material.button.MaterialButton
104.         android:id="@+id/btnLogin"
105.         android:layout_width="match_parent"
106.         android:layout_height="56dp"
107.         android:layout_marginTop="48dp"
108.         android:fontFamily="sans-serif-medium"
109.         android:text="Sign in"
110.         android:textAllCaps="false"
111.         android:textColor="#0F172A"
112.         android:textSize="17sp"
113.         app:backgroundTint="#F59E0B"
114.         app:cornerRadius="28dp"
115.         app:layout_constraintEnd_toEndOf="parent"
116.         app:layout_constraintStart_toStartOf="parent"
117.         app:layout_constraintTop_toBottomOf="@id/layoutPassword" />
118.
119.     <!-- ID strictly preserved for backend logic -->
120.     <TextView
121.         android:id="@+id/txtGoToRegister"
122.         android:layout_width="wrap_content"
123.         android:layout_height="wrap_content"
124.         android:layout_marginTop="24dp"
125.         android:background="?attr/selectableItemBackground"
126.         android:clickable="true"
127.         android:focusable="true"
128.         android:fontFamily="sans-serif-medium"
129.         android:padding="8dp"
130.         android:text="New user? Register here"
131.         android:textColor="#F59E0B"
132.         android:textSize="15sp"
133.         app:layout_constraintEnd_toEndOf="parent"
134.         app:layout_constraintStart_toStartOf="parent"
135.         app:layout_constraintTop_toBottomOf="@id/btnLogin" />
136.
137. </androidx.constraintlayout.widget.ConstraintLayout>
138.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\activity_main.xml

```
1. <?xml version="1.0" encoding="utf-8"?>
```

```

2. <androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     android:layout_width="match_parent"
5.     android:layout_height="match_parent"
6.     android:background="#0F172A"
7.     android:padding="32dp">
8.
9.     <!-- Slightly reduced size to allow more breathing room (whitespace) -->
10.    <ImageView
11.        android:id="@+id/logoImage"
12.        android:layout_width="240dp"
13.        android:layout_height="240dp"
14.        android:src="@drawable/logo"
15.        app:layout_constraintBottom_toTopOf="@+id/subtitleText"
16.        app:layout_constraintEnd_toEndOf="parent"
17.        app:layout_constraintStart_toStartOf="parent"
18.        app:layout_constraintTop_toTopOf="parent"
19.        app:layout_constraintVertical_chainStyle="packed" />
20.
21.    <!-- Updated to an iOS-standard 17sp with a secondary dark mode text color -->
22.    <TextView
23.        android:id="@+id/subtitleText"
24.        android:layout_width="0dp"
25.        android:layout_height="wrap_content"
26.        android:layout_marginTop="32dp"
27.        android:fontFamily="sans-serif-medium"
28.        android:letterSpacing="0.02"
29.        android:text="Peer-to-Peer Energy Trading System"
30.        android:textAlignment="center"
31.        android:textColor="#EBEBF5"
32.        android:textSize="17sp"
33.        app:layout_constraintBottom_toTopOf="@+id/btnGetStarted"
34.        app:layout_constraintEnd_toEndOf="parent"
35.        app:layout_constraintStart_toStartOf="parent"
36.        app:layout_constraintTop_toBottomOf="@+id/logoImage" />
37.
38.    <!-- Converted to a pill-shaped button with title-case typography -->
39.    <com.google.android.material.button.MaterialButton
40.        android:id="@+id/btnGetStarted"
41.        android:layout_width="match_parent"
42.        android:layout_height="56dp"
43.        android:layout_marginBottom="16dp"
44.        android:fontFamily="sans-serif-medium"
45.        android:text="Get Started"
46.        android:textAllCaps="false"
47.        android:textColor="#0F172A"
48.        android:textSize="17sp"
49.        app:backgroundTint="#F59E0B"
50.        app:cornerRadius="28dp"
51.        app:layout_constraintBottom_toBottomOf="parent"
52.        app:layout_constraintEnd_toEndOf="parent"
53.        app:layout_constraintStart_toStartOf="parent" />
54.
55. </androidx.constraintlayout.widget.ConstraintLayout>
56.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\activity_modify_booking.xml

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"

```

```

4.     android:layout_width="match_parent"
5.     android:layout_height="match_parent"
6.     android:background="#0F172A">
7.
8.     <ScrollView
9.         android:layout_width="0dp"
10.        android:layout_height="0dp"
11.        android:fillViewport="true"
12.        app:layout_constraintBottom_toTopOf="@id/bottomNavigation"
13.        app:layout_constraintEnd_toEndOf="parent"
14.        app:layout_constraintStart_toStartOf="parent"
15.        app:layout_constraintTop_toTopOf="parent">
16.
17.        <androidx.constraintlayout.widget.ConstraintLayout
18.            android:layout_width="match_parent"
19.            android:layout_height="wrap_content"
20.            android:padding="24dp">
21.
22.            <!-- Interactive Navigation Text -->
23.            <TextView
24.                android:id="@+id/btnBack"
25.                android:layout_width="wrap_content"
26.                android:layout_height="wrap_content"
27.                android:layout_marginTop="16dp"
28.                android:background="?attr/selectableItemBackground"
29.                android:clickable="true"
30.                android:focusable="true"
31.                android:fontFamily="sans-serif-medium"
32.                android:paddingVertical="8dp"
33.                android:paddingEnd="16dp"
34.                android:text="← Back"
35.                android:textColor="#F59E0B"
36.                android:textSize="17sp"
37.                app:layout_constraintStart_toStartOf="parent"
38.                app:layout_constraintTop_toTopOf="parent" />
39.
40.            <!-- Standardized iOS Large Header -->
41.            <TextView
42.                android:id="@+id/tvTitle"
43.                android:layout_width="0dp"
44.                android:layout_height="wrap_content"
45.                android:layout_marginTop="24dp"
46.                android:fontFamily="sans-serif-medium"
47.                android:text="Modify Booking"
48.                android:textColor="FFFFFF"
49.                android:textSize="34sp"
50.                app:layout_constraintEnd_toEndOf="parent"
51.                app:layout_constraintStart_toStartOf="parent"
52.                app:layout_constraintTop_toBottomOf="@id/btnBack" />
53.
54.            <TextView
55.                android:id="@+id/tvSubtitle"
56.                android:layout_width="0dp"
57.                android:layout_height="wrap_content"
58.                android:layout_marginTop="8dp"
59.                android:fontFamily="sans-serif"
60.                android:letterSpacing="0.02"
61.                android:text="Update time or energy. Station cannot be changed."
62.                android:textColor="#EBEBF5"
63.                android:textSize="17sp"
64.                app:layout_constraintEnd_toEndOf="parent"
65.                app:layout_constraintStart_toStartOf="parent"
66.                app:layout_constraintTop_toBottomOf="@id/tvTitle" />
67.
68.            <!-- Station (Read-Only) -->

```



```

69.         <com.google.android.material.textfield.TextInputLayout
70.             android:id="@+id/layoutStation"
71.             style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
72.             android:layout_width="0dp"
73.             android:layout_height="wrap_content"
74.             android:layout_marginTop="40dp"
75.             android:textColorHint="#64748B"
76.             app:boxBackgroundColor="#0F172A"
77.             app:boxCornerRadiusBottomEnd="16dp"
78.             app:boxCornerRadiusBottomStart="16dp"
79.             app:boxCornerRadiusTopEnd="16dp"
80.             app:boxCornerRadiusTopStart="16dp"
81.             app:boxStrokeColor="#334155"
82.             app:boxStrokeWidth="1dp"
83.             app:hintTextColor="#64748B"
84.             app:layout_constraintEnd_toEndOf="parent"
85.             app:layout_constraintStart_toStartOf="parent"
86.             app:layout_constraintTop_toBottomOf="@id/tvSubtitle">
87.
88.             <com.google.android.material.textfield.TextInputEditText
89.                 android:id="@+id/editStationId"
90.                 android:layout_width="match_parent"
91.                 android:layout_height="60dp"
92.                 android:clickable="false"
93.                 android:focusable="false"
94.                 android:hint="Station (Read-Only)"
95.                 android:textColor="#94A3B8"
96.                 android:textSize="16sp" />
97.         </com.google.android.material.textfield.TextInputLayout>
98.
99.         <!-- Reservation Date -->
100.        <com.google.android.material.textfield.TextInputLayout
101.            android:id="@+id/layoutDate"
102.            style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
103.            android:layout_width="0dp"
104.            android:layout_height="wrap_content"
105.            android:layout_marginTop="16dp"
106.            android:textColorHint="#94A3B8"
107.            app:boxBackgroundColor="#1E293B"
108.            app:boxCornerRadiusBottomEnd="16dp"
109.            app:boxCornerRadiusBottomStart="16dp"
110.            app:boxCornerRadiusTopEnd="16dp"
111.            app:boxCornerRadiusTopStart="16dp"
112.            app:boxStrokeColor="#334155"
113.            app:boxStrokeWidth="1dp"
114.            app:boxStrokeWidthFocused="2dp"
115.            app:hintTextColor="#F59E0B"
116.            app:layout_constraintEnd_toEndOf="parent"
117.            app:layout_constraintStart_toStartOf="parent"
118.            app:layout_constraintTop_toBottomOf="@id/layoutStation">
119.
120.            <com.google.android.material.textfield.TextInputEditText
121.                android:id="@+id/editBookingDate"
122.                android:layout_width="match_parent"
123.                android:layout_height="60dp"
124.                android:focusable="false"
125.                android:hint="Reservation Date"
126.                android:textColor="FFFFFF"
127.                android:textSize="16sp" />
128.        </com.google.android.material.textfield.TextInputLayout>
129.
130.        <!-- Side-by-side Time Slots -->
131.        <LinearLayout
132.            android:id="@+id/layoutTimeRow"
133.            android:layout_width="0dp"

```

```

134.         android:layout_height="wrap_content"
135.         android:layout_marginTop="16dp"
136.         android:baselineAligned="false"
137.         android:orientation="horizontal"
138.         android:weightSum="2"
139.         app:layout_constraintEnd_toEndOf="parent"
140.         app:layout_constraintStart_toStartOf="parent"
141.         app:layout_constraintTop_toBottomOf="@id/layoutDate">
142.
143.         <com.google.android.material.textfield.TextInputLayout
144.             style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
145.             android:layout_width="0dp"
146.             android:layout_height="wrap_content"
147.             android:layout_marginEnd="8dp"
148.             android:layout_weight="1"
149.             android:textColorHint="#94A3B8"
150.             app:boxBackgroundColor="#1E293B"
151.             app:boxCornerRadiusBottomEnd="16dp"
152.             app:boxCornerRadiusBottomStart="16dp"
153.             app:boxCornerRadiusTopEnd="16dp"
154.             app:boxCornerRadiusTopStart="16dp"
155.             app:boxStrokeColor="#334155"
156.             app:boxStrokeWidth="1dp"
157.             app:boxStrokeWidthFocused="2dp"
158.             app:hintTextColor="#F59E0B">
159.
160.             <com.google.android.material.textfield.TextInputEditText
161.                 android:id="@+id/editStartTime"
162.                 android:layout_width="match_parent"
163.                 android:layout_height="60dp"
164.                 android:focusable="false"
165.                 android:hint="Start Time"
166.                 android:textColor="#FFFFFF"
167.                 android:textSize="16sp" />
168.         </com.google.android.material.textfield.TextInputLayout>
169.
170.         <com.google.android.material.textfield.TextInputLayout
171.             style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
172.             android:layout_width="0dp"
173.             android:layout_height="wrap_content"
174.             android:layout_marginStart="8dp"
175.             android:layout_weight="1"
176.             android:textColorHint="#94A3B8"
177.             app:boxBackgroundColor="#1E293B"
178.             app:boxCornerRadiusBottomEnd="16dp"
179.             app:boxCornerRadiusBottomStart="16dp"
180.             app:boxCornerRadiusTopEnd="16dp"
181.             app:boxCornerRadiusTopStart="16dp"
182.             app:boxStrokeColor="#334155"
183.             app:boxStrokeWidth="1dp"
184.             app:boxStrokeWidthFocused="2dp"
185.             app:hintTextColor="#F59E0B">
186.
187.             <com.google.android.material.textfield.TextInputEditText
188.                 android:id="@+id/editEndTime"
189.                 android:layout_width="match_parent"
190.                 android:layout_height="60dp"
191.                 android:focusable="false"
192.                 android:hint="End Time"
193.                 android:textColor="#FFFFFF"
194.                 android:textSize="16sp" />
195.         </com.google.android.material.textfield.TextInputLayout>
196.     </LinearLayout>
197.
198. <!-- Energy Amount -->

```

```

199. <com.google.android.material.textfield.TextInputLayout
200.     android:id="@+id/layoutEnergy"
201.     style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
202.     android:layout_width="0dp"
203.     android:layout_height="wrap_content"
204.     android:layout_marginTop="16dp"
205.     android:textColorHint="#94A3B8"
206.     app:boxBackgroundColor="#1E293B"
207.     app:boxCornerRadiusBottomEnd="16dp"
208.     app:boxCornerRadiusBottomStart="16dp"
209.     app:boxCornerRadiusTopEnd="16dp"
210.     app:boxCornerRadiusTopStart="16dp"
211.     app:boxStrokeColor="#334155"
212.     app:boxStrokeWidth="1dp"
213.     app:boxStrokeWidthFocused="2dp"
214.     app:hintTextColor="#F59E0B"
215.     app:layout_constraintEnd_toEndOf="parent"
216.     app:layout_constraintStart_toStartOf="parent"
217.     app:layout_constraintTop_toBottomOf="@id/layoutTimeRow">
218.
219.     <com.google.android.material.textfield.TextInputEditText
220.         android:id="@+id/editEnergyAmount"
221.         android:layout_width="match_parent"
222.         android:layout_height="60dp"
223.         android:hint="Energy Amount (kWh)"
224.         android:inputType="numberDecimal"
225.         android:textColor="#FFFFFF"
226.         android:textSize="16sp" />
227. </com.google.android.material.textfield.TextInputLayout>
228.
229. <!-- Pill-shaped Primary Action Button -->
230. <com.google.android.material.button.MaterialButton
231.     android:id="@+id/btnUpdateBooking"
232.     android:layout_width="0dp"
233.     android:layout_height="56dp"
234.     android:layout_marginTop="40dp"
235.     android:fontFamily="sans-serif-medium"
236.     android:text="Save Changes"
237.     android:textAllCaps="false"
238.     android:textColor="#0F172A"
239.     android:textSize="17sp"
240.     app:backgroundTint="#F59E0B"
241.     app:cornerRadius="28dp"
242.     app:layout_constraintEnd_toEndOf="parent"
243.     app:layout_constraintStart_toStartOf="parent"
244.     app:layout_constraintTop_toBottomOf="@id/layoutEnergy" />
245.
246. <!-- Destructive Secondary Action matching Profile screen styling -->
247. <com.google.android.material.button.MaterialButton
248.     android:id="@+id/btnCancelBooking"
249.     style="@style/Widget.MaterialComponents.Button.OutlinedButton"
250.     android:layout_width="0dp"
251.     android:layout_height="56dp"
252.     android:layout_marginTop="16dp"
253.     android:layout_marginBottom="24dp"
254.     android:fontFamily="sans-serif-medium"
255.     android:text="Cancel Booking"
256.     android:textAllCaps="false"
257.     android:textColor="#EF4444"
258.     android:textSize="17sp"
259.     app:cornerRadius="28dp"
260.     app:layout_constraintBottom_toBottomOf="parent"
261.     app:layout_constraintEnd_toEndOf="parent"
262.     app:layout_constraintStart_toStartOf="parent"
263.     app:layout_constraintTop_toBottomOf="@id/btnUpdateBooking"

```

```

264.         app:strokeColor="#EF4444"
265.         app:strokeWidth="1dp" />
266.
267.     </androidx.constraintlayout.widget.ConstraintLayout>
268. </ScrollView>
269.
270. <!-- Bottom Navigation natively anchored to the bottom -->
271. <com.google.android.material.bottomnavigation.BottomNavigationView
272.     android:id="@+id/bottomNavigation"
273.     android:layout_width="0dp"
274.     android:layout_height="wrap_content"
275.     android:background="#1E293B"
276.     app:itemIconTint="@color/bottom_nav_selector"
277.     app:itemRippleColor="#334155"
278.     app:itemTextColor="@color/bottom_nav_selector"
279.     app:labelVisibilityMode="labeled"
280.     app:layout_constraintBottom_toBottomOf="parent"
281.     app:layout_constraintEnd_toEndOf="parent"
282.     app:layout_constraintStart_toStartOf="parent"
283.     app:menu="@menu/bottom_nav_menu" />
284.
285. </androidx.constraintlayout.widget.ConstraintLayout>
286.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\activity_my_bookings.xml

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     android:layout_width="match_parent"
5.     android:layout_height="match_parent"
6.     android:background="#0F172A">
7.
8.     <!-- Modern iOS-style Large Header -->
9.     <TextView
10.         android:id="@+id/tvBookingsTitle"
11.         android:layout_width="0dp"
12.         android:layout_height="wrap_content"
13.         android:layout_marginHorizontal="24dp"
14.         android:layout_marginTop="32dp"
15.         android:fontFamily="sans-serif-medium"
16.         android:text="My Bookings"
17.         android:textColor="#FFFFFF"
18.         android:textSize="34sp"
19.         app:layout_constraintEnd_toEndOf="parent"
20.         app:layout_constraintStart_toStartOf="parent"
21.         app:layout_constraintTop_toTopOf="parent" />
22.
23.     <!-- Filter Row -->
24.     <LinearLayout
25.         android:id="@+id/layoutFilter"
26.         android:layout_width="0dp"
27.         android:layout_height="wrap_content"
28.         android:layout_marginHorizontal="24dp"
29.         android:layout_marginTop="16dp"
30.         android:gravity="center_vertical"
31.         android:orientation="horizontal"
32.         app:layout_constraintEnd_toEndOf="parent"
33.         app:layout_constraintStart_toStartOf="parent"
34.         app:layout_constraintTop_toBottomOf="@id/tvBookingsTitle">
35.
36.         <TextView

```

```

37.         android:layout_width="wrap_content"
38.         android:layout_height="wrap_content"
39.         android:fontFamily="sans-serif"
40.         android:text="Filter by Status: "
41.         android:textColor="#94A3B8"
42.         android:textSize="15sp" />
43.
44.         <!-- Wrapped the stark white Spinner in a styled MaterialCardView to blend with dark
mode -->
45.         <com.google.android.material.card.MaterialCardView
46.             android:layout_width="match_parent"
47.             android:layout_height="48dp"
48.             android:layout_marginStart="12dp"
49.             app:cardBackgroundColor="#1E293B"
50.             app:cardCornerRadius="12dp"
51.             app:cardElevation="0dp"
52.             app:strokeColor="#334155"
53.             app:strokeWidth="1dp">
54.
55.             <Spinner
56.                 android:id="@+id/spinnerFilterStatus"
57.                 android:layout_width="match_parent"
58.                 android:layout_height="match_parent"
59.                 android:backgroundTint="#94A3B8"
60.                 android:paddingHorizontal="8dp" />
61.             </com.google.android.material.card.MaterialCardView>
62.         </LinearLayout>
63.
64.         <!-- Pill-shaped Primary Action Button -->
65.         <com.google.android.material.button.MaterialButton
66.             android:id="@+id/btnGoToCreateBooking"
67.             android:layout_width="0dp"
68.             android:layout_height="56dp"
69.             android:layout_marginHorizontal="24dp"
70.             android:layout_marginTop="24dp"
71.             android:fontFamily="sans-serif-medium"
72.             android:text="Create New Booking"
73.             android:textAllCaps="false"
74.             android:textColor="#0F172A"
75.             android:textSize="17sp"
76.             app:backgroundTint="#F59E0B"
77.             app:cornerRadius="28dp"
78.             app:layout_constraintEnd_toEndOf="parent"
79.             app:layout_constraintStart_toStartOf="parent"
80.             app:layout_constraintTop_toBottomOf="@id/layoutFilter" />
81.
82.         <!-- RecyclerView mapped to occupy dynamic remaining space -->
83.         <androidx.recyclerview.widget.RecyclerView
84.             android:id="@+id/recyclerViewBookings"
85.             android:layout_width="0dp"
86.             android:layout_height="0dp"
87.             android:layout_marginTop="16dp"
88.             android:clipToPadding="false"
89.             android:paddingHorizontal="24dp"
90.             android:paddingBottom="24dp"
91.             app:layout_constraintBottom_toTopOf="@id/bottomNavigation"
92.             app:layout_constraintEnd_toEndOf="parent"
93.             app:layout_constraintStart_toStartOf="parent"
94.             app:layout_constraintTop_toBottomOf="@id/btnGoToCreateBooking" />
95.
96.         <!-- Bottom Navigation natively anchored to the bottom -->
97.         <com.google.android.material.bottomnavigation.BottomNavigationView
98.             android:id="@+id/bottomNavigation"
99.             android:layout_width="0dp"
100.            android:layout_height="wrap_content"

```

```

101.         android:background="#1E293B"
102.         app:itemIconTint="@color/bottom_nav_selector"
103.         app:itemRippleColor="#334155"
104.         app:itemTextColor="@color/bottom_nav_selector"
105.         app:labelVisibilityMode="labeled"
106.         app:layout_constraintBottom_toBottomOf="parent"
107.         app:layout_constraintEnd_toEndOf="parent"
108.         app:layout_constraintStart_toStartOf="parent"
109.         app:menu="@menu/bottom_nav_menu" />
110.
111. </androidx.constraintlayout.widget.ConstraintLayout>
112.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\activity_profile.xml

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     android:layout_width="match_parent"
5.     android:layout_height="match_parent"
6.     android:background="#0F172A">
7.
8.     <ScrollView
9.         android:layout_width="0dp"
10.        android:layout_height="0dp"
11.        android:fillViewport="true"
12.        app:layout_constraintBottom_toTopOf="@id/bottomNavigation"
13.        app:layout_constraintEnd_toEndOf="parent"
14.        app:layout_constraintStart_toStartOf="parent"
15.        app:layout_constraintTop_toTopOf="parent">
16.
17.        <androidx.constraintlayout.widget.ConstraintLayout
18.            android:layout_width="match_parent"
19.            android:layout_height="wrap_content"
20.            android:padding="24dp">
21.
22.            <!-- Premium Avatar Header -->
23.            <com.google.android.material.card.MaterialCardView
24.                android:id="@+id/cardAvatar"
25.                android:layout_width="88dp"
26.                android:layout_height="88dp"
27.                android:layout_marginTop="24dp"
28.                app:cardBackgroundColor="#1E293B"
29.                app:cardCornerRadius="44dp"
30.                app:cardElevation="0dp"
31.                app:layout_constraintEnd_toEndOf="parent"
32.                app:layout_constraintStart_toStartOf="parent"
33.                app:layout_constraintTop_toTopOf="parent"
34.                app:strokeColor="#F59E0B"
35.                app:strokeWidth="2dp">
36.
37.                <TextView
38.                    android:layout_width="match_parent"
39.                    android:layout_height="match_parent"
40.                    android:fontFamily="sans-serif-medium"
41.                    android:gravity="center"
42.                    android:text="N"
43.                    android:textColor="#F59E0B"
44.                    android:textSize="36sp" />
45.            </com.google.android.material.card.MaterialCardView>
46.
47.            <TextView

```

```

48.         android:id="@+id/tvProfileName"
49.         android:layout_width="wrap_content"
50.         android:layout_height="wrap_content"
51.         android:layout_marginTop="16dp"
52.         android:fontFamily="sans-serif-medium"
53.         android:text="Nithika Perera"
54.         android:textColor="#FFFFFF"
55.         android:textSize="24sp"
56.         app:layout_constraintEnd_toEndOf="parent"
57.         app:layout_constraintStart_toStartOf="parent"
58.         app:layout_constraintTop_toBottomOf="@id/cardAvatar" />
59.
60.     <TextView
61.         android:id="@+id/txtProfileNic"
62.         android:layout_width="wrap_content"
63.         android:layout_height="wrap_content"
64.         android:layout_marginTop="4dp"
65.         android:fontFamily="sans-serif"
66.         android:text="NIC: Loading..."
67.         android:textColor="#94A3B8"
68.         android:textSize="15sp"
69.         app:layout_constraintEnd_toEndOf="parent"
70.         app:layout_constraintStart_toStartOf="parent"
71.         app:layout_constraintTop_toBottomOf="@id/tvProfileName" />
72.
73.     <!-- Wrapped Status in a pill-shaped MaterialCardView for native corner rounding -->
74.     <com.google.android.material.card.MaterialCardView
75.         android:id="@+id/cardStatus"
76.         android:layout_width="wrap_content"
77.         android:layout_height="wrap_content"
78.         android:layout_marginTop="12dp"
79.         app:cardBackgroundColor="#2010B981"
80.         app:cardCornerRadius="16dp"
81.         app:cardElevation="0dp"
82.         app:layout_constraintEnd_toEndOf="parent"
83.         app:layout_constraintStart_toStartOf="parent"
84.         app:layout_constraintTop_toBottomOf="@id/txtProfileNic">
85.
86.         <TextView
87.             android:id="@+id/txtProfileStatus"
88.             android:layout_width="wrap_content"
89.             android:layout_height="wrap_content"
90.             android:fontFamily="sans-serif-medium"
91.             android:paddingHorizontal="12dp"
92.             android:paddingVertical="6dp"
93.             android:text="Status: Loading"
94.             android:textColor="#10B981"
95.             android:textSize="13sp" />
96.     </com.google.android.material.card.MaterialCardView>
97.
98.     <!-- Form Section -->
99.     <TextView
100.         android:id="@+id/tvPersonalInfoLabel"
101.         android:layout_width="0dp"
102.         android:layout_height="wrap_content"
103.         android:layout_marginTop="40dp"
104.         android:fontFamily="sans-serif-medium"
105.         android:letterSpacing="0.05"
106.         android:text="PERSONAL INFORMATION"
107.         android:textColor="#94A3B8"
108.         android:textSize="12sp"
109.         app:layout_constraintEnd_toEndOf="parent"
110.         app:layout_constraintStart_toStartOf="parent"
111.         app:layout_constraintTop_toBottomOf="@id/cardStatus" />

```



```

112.
113. <com.google.android.material.textfield.TextInputLayout
114.     android:id="@+id/layoutProfileName"
115.     style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
116.     android:layout_width="0dp"
117.     android:layout_height="wrap_content"
118.     android:layout_marginTop="16dp"
119.     android:textColorHint="#94A3B8"
120.     app:boxBackgroundColor="#1E293B"
121.     app:boxCornerRadiusBottomEnd="16dp"
122.     app:boxCornerRadiusBottomStart="16dp"
123.     app:boxCornerRadiusTopEnd="16dp"
124.     app:boxCornerRadiusTopStart="16dp"
125.     app:boxStrokeColor="#334155"
126.     app:boxStrokeWidth="1dp"
127.     app:boxStrokeWidthFocused="2dp"
128.     app:hintTextColor="#F59E0B"
129.     app:layout_constraintEnd_toEndOf="parent"
130.     app:layout_constraintStart_toStartOf="parent"
131.     app:layout_constraintTop_toBottomOf="@id/tvPersonalInfoLabel">
132.
133.     <com.google.android.material.textfield.TextInputEditText
134.         android:id="@+id/inputProfileName"
135.         android:layout_width="match_parent"
136.         android:layout_height="60dp"
137.         android:hint="Full Name"
138.         android:inputType="textPersonName"
139.         android:textColor="FFFFFF"
140.         android:textSize="16sp" />
141. </com.google.android.material.textfield.TextInputLayout>
142.
143. <com.google.android.material.textfield.TextInputLayout
144.     android:id="@+id/layoutProfileEmail"
145.     style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
146.     android:layout_width="0dp"
147.     android:layout_height="wrap_content"
148.     android:layout_marginTop="16dp"
149.     android:textColorHint="#94A3B8"
150.     app:boxBackgroundColor="#1E293B"
151.     app:boxCornerRadiusBottomEnd="16dp"
152.     app:boxCornerRadiusBottomStart="16dp"
153.     app:boxCornerRadiusTopEnd="16dp"
154.     app:boxCornerRadiusTopStart="16dp"
155.     app:boxStrokeColor="#334155"
156.     app:boxStrokeWidth="1dp"
157.     app:boxStrokeWidthFocused="2dp"
158.     app:hintTextColor="#F59E0B"
159.     app:layout_constraintEnd_toEndOf="parent"
160.     app:layout_constraintStart_toStartOf="parent"
161.     app:layout_constraintTop_toBottomOf="@id/layoutProfileName">
162.
163.     <com.google.android.material.textfield.TextInputEditText
164.         android:id="@+id/inputProfileEmail"
165.         android:layout_width="match_parent"
166.         android:layout_height="60dp"
167.         android:hint="Email Address"
168.         android:inputType="textEmailAddress"
169.         android:textColor="FFFFFF"
170.         android:textSize="16sp" />
171. </com.google.android.material.textfield.TextInputLayout>
172.
173. <com.google.android.material.textfield.TextInputLayout
174.     android:id="@+id/layoutProfilePhone"
175.     style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
176.     android:layout_width="0dp"

```



```

177.         android:layout_height="wrap_content"
178.         android:layout_marginTop="16dp"
179.         android:textColorHint="#94A3B8"
180.         app:boxBackgroundColor="#1E293B"
181.         app:boxCornerRadiusBottomEnd="16dp"
182.         app:boxCornerRadiusBottomStart="16dp"
183.         app:boxCornerRadiusTopEnd="16dp"
184.         app:boxCornerRadiusTopStart="16dp"
185.         app:boxStrokeColor="#334155"
186.         app:boxStrokeWidth="1dp"
187.         app:boxStrokeWidthFocused="2dp"
188.         app:hintTextColor="#F59E0B"
189.         app:layout_constraintEnd_toEndOf="parent"
190.         app:layout_constraintStart_toStartOf="parent"
191.         app:layout_constraintTop_toBottomOf="@id/layoutProfileEmail">
192.
193.         <com.google.android.material.textfield.TextInputEditText
194.             android:id="@+id/inputProfilePhone"
195.             android:layout_width="match_parent"
196.             android:layout_height="60dp"
197.             android:hint="Phone Number"
198.             android:inputType="phone"
199.             android:textColor="#FFFFFF"
200.             android:textSize="16sp" />
201.     </com.google.android.material.textfield.TextInputLayout>
202.
203.     <com.google.android.material.textfield.TextInputLayout
204.         android:id="@+id/layoutProfileAddress"
205.         style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
206.         android:layout_width="0dp"
207.         android:layout_height="wrap_content"
208.         android:layout_marginTop="16dp"
209.         android:textColorHint="#94A3B8"
210.         app:boxBackgroundColor="#1E293B"
211.         app:boxCornerRadiusBottomEnd="16dp"
212.         app:boxCornerRadiusBottomStart="16dp"
213.         app:boxCornerRadiusTopEnd="16dp"
214.         app:boxCornerRadiusTopStart="16dp"
215.         app:boxStrokeColor="#334155"
216.         app:boxStrokeWidth="1dp"
217.         app:boxStrokeWidthFocused="2dp"
218.         app:hintTextColor="#F59E0B"
219.         app:layout_constraintEnd_toEndOf="parent"
220.         app:layout_constraintStart_toStartOf="parent"
221.         app:layout_constraintTop_toBottomOf="@id/layoutProfilePhone">
222.
223.         <com.google.android.material.textfield.TextInputEditText
224.             android:id="@+id/inputProfileAddress"
225.             android:layout_width="match_parent"
226.             android:layout_height="60dp"
227.             android:hint="Billing Address"
228.             android:inputType="textPostalAddress"
229.             android:textColor="#FFFFFF"
230.             android:textSize="16sp" />
231.     </com.google.android.material.textfield.TextInputLayout>
232.
233.     <!-- Action Buttons -->
234.     <com.google.android.material.button.MaterialButton
235.         android:id="@+id/btnUpdateProfile"
236.         android:layout_width="0dp"
237.         android:layout_height="56dp"
238.         android:layout_marginTop="32dp"
239.         android:fontFamily="sans-serif-medium"
240.         android:text="Save Changes"
241.         android:textAllCaps="false"

```

```

242.         android:textColor="#0F172A"
243.         android:textSize="17sp"
244.         app:backgroundTint="#F59E0B"
245.         app:cornerRadius="28dp"
246.         app:layout_constraintEnd_toEndOf="parent"
247.         app:layout_constraintStart_toStartOf="parent"
248.         app:layout_constraintTop_toBottomOf="@id/layoutProfileAddress" />
249.
250.     <com.google.android.material.button.MaterialButton
251.         android:id="@+id/btnDeactivateAccount"
252.         style="@style/Widget.MaterialComponents.Button.OutlinedButton"
253.         android:layout_width="0dp"
254.         android:layout_height="56dp"
255.         android:layout_marginTop="16dp"
256.         android:fontFamily="sans-serif-medium"
257.         android:text="Request Deactivation"
258.         android:textAllCaps="false"
259.         android:textColor="#EF4444"
260.         android:textSize="17sp"
261.         app:cornerRadius="28dp"
262.         app:layout_constraintEnd_toEndOf="parent"
263.         app:layout_constraintStart_toStartOf="parent"
264.         app:layout_constraintTop_toBottomOf="@id/btnUpdateProfile"
265.         app:strokeColor="#EF4444"
266.         app:strokeWidth="1dp" />
267.
268.     <com.google.android.material.button.MaterialButton
269.         android:id="@+id/btnLogout"
270.         style="@style/Widget.MaterialComponents.Button.TextButton"
271.         android:layout_width="wrap_content"
272.         android:layout_height="wrap_content"
273.         android:layout_marginTop="24dp"
274.         android:layout_marginBottom="24dp"
275.         android:fontFamily="sans-serif-medium"
276.         android:text="Sign Out"
277.         android:textAllCaps="false"
278.         android:textColor="#94A3B8"
279.         android:textSize="16sp"
280.         app:layout_constraintBottom_toBottomOf="parent"
281.         app:layout_constraintEnd_toEndOf="parent"
282.         app:layout_constraintStart_toStartOf="parent"
283.         app:layout_constraintTop_toBottomOf="@id/btnDeactivateAccount" />
284.
285.     </androidx.constraintlayout.widget.ConstraintLayout>
286. </ScrollView>
287.
288. <!-- Bottom Navigation natively anchored to the bottom -->
289. <com.google.android.material.bottomnavigation.BottomNavigationView
290.     android:id="@+id/bottomNavigation"
291.     android:layout_width="0dp"
292.     android:layout_height="wrap_content"
293.     android:background="#1E293B"
294.     app:itemIconTint="@color/bottom_nav_selector"
295.     app:itemRippleColor="#334155"
296.     app:itemTextColor="@color/bottom_nav_selector"
297.     app:labelVisibilityMode="labeled"
298.     app:layout_constraintBottom_toBottomOf="parent"
299.     app:layout_constraintEnd_toEndOf="parent"
300.     app:layout_constraintStart_toStartOf="parent"
301.     app:menu="@menu/bottom_nav_menu" />
302.
303. </androidx.constraintlayout.widget.ConstraintLayout>
304.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\activity_register.xml

```
1. <?xml version="1.0" encoding="utf-8"?>
2. <ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     android:layout_width="match_parent"
5.     android:layout_height="match_parent"
6.     android:fillViewport="true"
7.     android:background="#0F172A">
8.
9.     <androidx.constraintlayout.widget.ConstraintLayout
10.         android:layout_width="match_parent"
11.         android:layout_height="wrap_content"
12.         android:padding="32dp">
13.
14.         <TextView
15.             android:id="@+id/tvCreateAccount"
16.             android:layout_width="0dp"
17.             android:layout_height="wrap_content"
18.             android:layout_marginTop="16dp"
19.             android:fontFamily="sans-serif-medium"
20.             android:text="Create Account"
21.             android:textColor="#FFFFFF"
22.             android:textSize="34sp"
23.             app:layout_constraintEnd_toEndOf="parent"
24.             app:layout_constraintStart_toStartOf="parent"
25.             app:layout_constraintTop_toTopOf="parent" />
26.
27.         <TextView
28.             android:id="@+id/tvSubTitle"
29.             android:layout_width="0dp"
30.             android:layout_height="wrap_content"
31.             android:layout_marginTop="8dp"
32.             android:fontFamily="sans-serif"
33.             android:letterSpacing="0.02"
34.             android:text="Register as a Solar Prosumer"
35.             android:textColor="#EBEBF5"
36.             android:textSize="17sp"
37.             app:layout_constraintEnd_toEndOf="parent"
38.             app:layout_constraintStart_toStartOf="parent"
39.             app:layout_constraintTop_toBottomOf="@id/tvCreateAccount" />
40.
41.         <!-- NIC Input -->
42.         <com.google.android.material.textfield.TextInputLayout
43.             android:id="@+id/layoutNic"
44.             style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
45.             android:layout_width="0dp"
46.             android:layout_height="wrap_content"
47.             android:layout_marginTop="40dp"
48.             android:textColorHint="#94A3B8"
49.             app:boxBackgroundColor="#1E293B"
50.             app:boxCornerRadiusBottomEnd="16dp"
51.             app:boxCornerRadiusBottomStart="16dp"
52.             app:boxCornerRadiusTopEnd="16dp"
53.             app:boxCornerRadiusTopStart="16dp"
54.             app:boxStrokeColor="#334155"
55.             app:boxStrokeWidth="1dp"
56.             app:boxStrokeWidthFocused="2dp"
57.             app:hintTextColor="#F59E0B"
58.             app:layout_constraintEnd_toEndOf="parent"
59.             app:layout_constraintStart_toStartOf="parent"
60.             app:layout_constraintTop_toBottomOf="@id/tvSubTitle">
61.
62.             <com.google.android.material.textfield.TextInputEditText
```

```

63.         android:id="@+id/inputNic"
64.         android:layout_width="match_parent"
65.         android:layout_height="60dp"
66.         android:hint="National Identity Card (NIC)"
67.         android:inputType="text"
68.         android:textColor="#FFFFFF"
69.         android:textSize="16sp" />
70.     </com.google.android.material.textfield.TextInputLayout>
71.
72.     <!-- Name Input -->
73.     <com.google.android.material.textfield.TextInputLayout
74.         android:id="@+id/layoutName"
75.         style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
76.         android:layout_width="0dp"
77.         android:layout_height="wrap_content"
78.         android:layout_marginTop="16dp"
79.         android:textColorHint="#94A3B8"
80.         app:boxBackgroundColor="#1E293B"
81.         app:boxCornerRadiusBottomEnd="16dp"
82.         app:boxCornerRadiusBottomStart="16dp"
83.         app:boxCornerRadiusTopEnd="16dp"
84.         app:boxCornerRadiusTopStart="16dp"
85.         app:boxStrokeColor="#334155"
86.         app:boxStrokeWidth="1dp"
87.         app:boxStrokeWidthFocused="2dp"
88.         app:hintTextColor="#F59E0B"
89.         app:layout_constraintEnd_toEndOf="parent"
90.         app:layout_constraintStart_toStartOf="parent"
91.         app:layout_constraintTop_toBottomOf="@id/layoutNic">
92.
93.         <com.google.android.material.textfield.TextInputEditText
94.             android:id="@+id/inputName"
95.             android:layout_width="match_parent"
96.             android:layout_height="60dp"
97.             android:hint="Full Name"
98.             android:inputType="textPersonName"
99.             android:textColor="#FFFFFF"
100.            android:textSize="16sp" />
101.     </com.google.android.material.textfield.TextInputLayout>
102.
103.     <!-- Email Input -->
104.     <com.google.android.material.textfield.TextInputLayout
105.         android:id="@+id/layoutEmail"
106.         style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
107.         android:layout_width="0dp"
108.         android:layout_height="wrap_content"
109.         android:layout_marginTop="16dp"
110.         android:textColorHint="#94A3B8"
111.         app:boxBackgroundColor="#1E293B"
112.         app:boxCornerRadiusBottomEnd="16dp"
113.         app:boxCornerRadiusBottomStart="16dp"
114.         app:boxCornerRadiusTopEnd="16dp"
115.         app:boxCornerRadiusTopStart="16dp"
116.         app:boxStrokeColor="#334155"
117.         app:boxStrokeWidth="1dp"
118.         app:boxStrokeWidthFocused="2dp"
119.         app:hintTextColor="#F59E0B"
120.         app:layout_constraintEnd_toEndOf="parent"
121.         app:layout_constraintStart_toStartOf="parent"
122.         app:layout_constraintTop_toBottomOf="@id/layoutName">
123.
124.         <com.google.android.material.textfield.TextInputEditText
125.             android:id="@+id/inputEmail"
126.             android:layout_width="match_parent"
127.             android:layout_height="60dp"

```

```

128.         android:hint="Email Address"
129.         android:inputType="textEmailAddress"
130.         android:textColor="#FFFFFF"
131.         android:textSize="16sp" />
132. </com.google.android.material.textfield.TextInputLayout>
133. <!-- Phone Input -->
134. <com.google.android.material.textfield.TextInputLayout
135.     android:id="@+id/layoutPhone"
136.     style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
137.     android:layout_width="0dp"
138.     android:layout_height="wrap_content"
139.     android:layout_marginTop="16dp"
140.     android:textColorHint="#94A3B8"
141.     app:boxBackgroundColor="#1E293B"
142.     app:boxCornerRadiusBottomEnd="16dp"
143.     app:boxCornerRadiusBottomStart="16dp"
144.     app:boxCornerRadiusTopEnd="16dp"
145.     app:boxCornerRadiusTopStart="16dp"
146.     app:boxStrokeColor="#334155"
147.     app:boxStrokeWidth="1dp"
148.     app:boxStrokeWidthFocused="2dp"
149.     app:hintTextColor="#F59E0B"
150.     app:layout_constraintEnd_toEndOf="parent"
151.     app:layout_constraintStart_toStartOf="parent"
152.     app:layout_constraintTop_toBottomOf="@id/layoutEmail">
153.
154.     <com.google.android.material.textfield.TextInputEditText
155.         android:id="@+id/inputPhone"
156.         android:layout_width="match_parent"
157.         android:layout_height="60dp"
158.         android:hint="Phone Number"
159.         android:inputType="phone"
160.         android:textColor="#FFFFFF"
161.         android:textSize="16sp" />
162. </com.google.android.material.textfield.TextInputLayout>
163.
164. <!-- Address Input -->
165. <com.google.android.material.textfield.TextInputLayout
166.     android:id="@+id/layoutAddress"
167.     style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
168.     android:layout_width="0dp"
169.     android:layout_height="wrap_content"
170.     android:layout_marginTop="16dp"
171.     android:textColorHint="#94A3B8"
172.     app:boxBackgroundColor="#1E293B"
173.     app:boxCornerRadiusBottomEnd="16dp"
174.     app:boxCornerRadiusBottomStart="16dp"
175.     app:boxCornerRadiusTopEnd="16dp"
176.     app:boxCornerRadiusTopStart="16dp"
177.     app:boxStrokeColor="#334155"
178.     app:boxStrokeWidth="1dp"
179.     app:boxStrokeWidthFocused="2dp"
180.     app:hintTextColor="#F59E0B"
181.     app:layout_constraintEnd_toEndOf="parent"
182.     app:layout_constraintStart_toStartOf="parent"
183.     app:layout_constraintTop_toBottomOf="@id/layoutPhone">
184.
185.     <com.google.android.material.textfield.TextInputEditText
186.         android:id="@+id/inputAddress"
187.         android:layout_width="match_parent"
188.         android:layout_height="60dp"
189.         android:hint="Address"
190.         android:inputType="textPostalAddress"
191.         android:textColor="#FFFFFF"
192.         android:textSize="16sp" />

```

```

193.     </com.google.android.material.textfield.TextInputLayout>
194.
195.     <!-- Password Input -->
196.     <com.google.android.material.textfield.TextInputLayout
197.         android:id="@+id/layoutPassword"
198.         style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox"
199.         android:layout_width="0dp"
200.         android:layout_height="wrap_content"
201.         android:layout_marginTop="16dp"
202.         android:textColorHint="#94A3B8"
203.         app:boxBackgroundColor="#1E293B"
204.         app:boxCornerRadiusBottomEnd="16dp"
205.         app:boxCornerRadiusBottomStart="16dp"
206.         app:boxCornerRadiusTopEnd="16dp"
207.         app:boxCornerRadiusTopStart="16dp"
208.         app:boxStrokeColor="#334155"
209.         app:boxStrokeWidth="1dp"
210.         app:boxStrokeWidthFocused="2dp"
211.         app:hintTextColor="#F59E0B"
212.         app:passwordToggleEnabled="true"
213.         app:passwordToggleTint="#94A3B8"
214.         app:layout_constraintEnd_toEndOf="parent"
215.         app:layout_constraintStart_toStartOf="parent"
216.         app:layout_constraintTop_toBottomOf="@id/layoutAddress">
217.
218.         <com.google.android.material.textfield.TextInputEditText
219.             android:id="@+id/inputPassword"
220.             android:layout_width="match_parent"
221.             android:layout_height="60dp"
222.             android:hint="Password"
223.             android:inputType="textPassword"
224.             android:textColor="#FFFFFF"
225.             android:textSize="16sp" />
226.     </com.google.android.material.textfield.TextInputLayout>
227.
228.     <!-- Register Button -->
229.     <com.google.android.material.button.MaterialButton
230.         android:id="@+id/btnRegister"
231.         android:layout_width="match_parent"
232.         android:layout_height="56dp"
233.         android:layout_marginTop="40dp"
234.         android:fontFamily="sans-serif-medium"
235.         android:text="Register"
236.         android:textAllCaps="false"
237.         android:textColor="#0F172A"
238.         android:textSize="17sp"
239.         app:backgroundTint="#F59E0B"
240.         app:cornerRadius="28dp"
241.         app:layout_constraintEnd_toEndOf="parent"
242.         app:layout_constraintStart_toStartOf="parent"
243.         app:layout_constraintTop_toBottomOf="@id/layoutPassword" />
244.
245.     <!-- Navigate to Login -->
246.     <TextView
247.         android:id="@+id/txtGoToLogin"
248.         android:layout_width="wrap_content"
249.         android:layout_height="wrap_content"
250.         android:layout_marginTop="24dp"
251.         android:layout_marginBottom="24dp"
252.         android:background="?attr/selectableItemBackground"
253.         android:clickable="true"
254.         android:focusable="true"
255.         android:fontFamily="sans-serif-medium"
256.         android:padding="8dp"
257.         android:text="Already have an account? Login here"

```

```

258.         android:textColor="#F59E0B"
259.         android:textSize="15sp"
260.         app:layout_constraintBottom_toBottomOf="parent"
261.         app:layout_constraintEnd_toEndOf="parent"
262.         app:layout_constraintStart_toStartOf="parent"
263.         app:layout_constraintTop_toBottomOf="@id/btnRegister" />
264.
265.     </androidx.constraintlayout.widget.ConstraintLayout>
266. </ScrollView>
267.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\activity_station_map.xml

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     android:layout_width="match_parent"
5.     android:layout_height="match_parent"
6.     android:background="#0F172A">
7.
8.     <!-- Full Screen Map Base -->
9.     <androidx.fragment.app.FragmentContainerView
10.         android:id="@+id/fullMap"
11.         android:name="com.google.android.gms.maps.SupportMapFragment"
12.         android:layout_width="0dp"
13.         android:layout_height="0dp"
14.         app:layout_constraintBottom_toBottomOf="parent"
15.         app:layout_constraintEnd_toEndOf="parent"
16.         app:layout_constraintStart_toStartOf="parent"
17.         app:layout_constraintTop_toTopOf="parent" />
18.
19.     <!-- Glassmorphism-style Top Header -->
20.     <androidx.constraintlayout.widget.ConstraintLayout
21.         android:id="@+id/topHeader"
22.         android:layout_width="0dp"
23.         android:layout_height="wrap_content"
24.         android:background="#E60F172A"
25.         android:elevation="6dp"
26.         android:paddingHorizontal="16dp"
27.         android:paddingTop="48dp"
28.         android:paddingBottom="16dp"
29.         app:layout_constraintEnd_toEndOf="parent"
30.         app:layout_constraintStart_toStartOf="parent"
31.         app:layout_constraintTop_toTopOf="parent">
32.
33.         <TextView
34.             android:id="@+id/btnBackFromMap"
35.             android:layout_width="wrap_content"
36.             android:layout_height="wrap_content"
37.             android:background="?attr/selectableItemBackground"
38.             android:clickable="true"
39.             android:focusable="true"
40.             android:fontFamily="sans-serif-medium"
41.             android:paddingVertical="8dp"
42.             android:paddingEnd="16dp"
43.             android:text="← Back"
44.             android:textColor="#F59E0B"
45.             android:textSize="17sp"
46.             app:layout_constraintBottom_toBottomOf="parent"
47.             app:layout_constraintStart_toStartOf="parent"
48.             app:layout_constraintTop_toTopOf="parent" />
49.

```



```

50.     <TextView
51.         android:layout_width="wrap_content"
52.         android:layout_height="wrap_content"
53.         android:fontFamily="sans-serif-medium"
54.         android:text="Grid Explorer"
55.         android:textColor="#FFFFFF"
56.         android:textSize="17sp"
57.         app:layout_constraintBottom_toBottomOf="parent"
58.         app:layout_constraintEnd_toEndOf="parent"
59.         app:layout_constraintStart_toStartOf="parent"
60.         app:layout_constraintTop_toTopOf="parent" />
61.
62.     <com.google.android.material.button.MaterialButton
63.         android:id="@+id/btnZoomToStations"
64.         android:layout_width="wrap_content"
65.         android:layout_height="40dp"
66.         android:fontFamily="sans-serif-medium"
67.         android:text="View All"
68.         android:textAllCaps="false"
69.         android:textColor="#0F172A"
70.         android:textSize="14sp"
71.         app:backgroundTint="#F59E0B"
72.         app:cornerRadius="20dp"
73.         app:layout_constraintBottom_toBottomOf="parent"
74.         app:layout_constraintEnd_toEndOf="parent"
75.         app:layout_constraintTop_toTopOf="parent" />
76. </androidx.constraintlayout.widget.ConstraintLayout>
77.
78. <!-- Horizontal Station List Drawer -->
79. <LinearLayout
80.     android:id="@+id/layoutStationsDrawer"
81.     android:layout_width="0dp"
82.     android:layout_height="wrap_content"
83.     android:layout_marginBottom="32dp"
84.     android:orientation="vertical"
85.     app:layout_constraintBottom_toBottomOf="parent"
86.     app:layout_constraintEnd_toEndOf="parent"
87.     app:layout_constraintStart_toStartOf="parent">
88.
89. <!-- Converted blocky label to an iOS-style rounded badge -->
90. <com.google.android.material.card.MaterialCardView
91.     android:layout_width="wrap_content"
92.     android:layout_height="wrap_content"
93.     android:layout_marginStart="16dp"
94.     android:layout_marginBottom="12dp"
95.     app:cardBackgroundColor="#F59E0B"
96.     app:cardCornerRadius="12dp"
97.     app:cardElevation="0dp">
98.
99.     <TextView
100.         android:layout_width="wrap_content"
101.         android:layout_height="wrap_content"
102.         android:fontFamily="sans-serif-medium"
103.         android:paddingHorizontal="12dp"
104.         android:paddingVertical="4dp"
105.         android:text="Available Stations"
106.         android:textColor="#0F172A"
107.         android:textSize="12sp" />
108. </com.google.android.material.card.MaterialCardView>
109.
110. <androidx.recyclerview.widget.RecyclerView
111.     android:id="@+id/recyclerStations"
112.     android:layout_width="match_parent"
113.     android:layout_height="wrap_content"
114.     android:clipToPadding="false"

```



```

180.         <TextView
181.             android:id="@+id/txtSheetSlots"
182.             android:layout_width="wrap_content"
183.             android:layout_height="wrap_content"
184.             android:fontFamily="sans-serif-medium"
185.             android:text="0"
186.             android:textColor="#10B981"
187.             android:textSize="28sp" />
188.     </LinearLayout>
189.
190.     <LinearLayout
191.         android:layout_width="0dp"
192.         android:layout_height="wrap_content"
193.         android:layout_weight="1"
194.         android:orientation="vertical">
195.
196.         <TextView
197.             android:layout_width="wrap_content"
198.             android:layout_height="wrap_content"
199.             android:fontFamily="sans-serif"
200.             android:text="Remaining Energy"
201.             android:textColor="#94A3B8"
202.             android:textSize="13sp" />
203.
204.         <TextView
205.             android:id="@+id/txtSheetEnergy"
206.             android:layout_width="wrap_content"
207.             android:layout_height="wrap_content"
208.             android:fontFamily="sans-serif-medium"
209.             android:text="0 kWh"
210.             android:textColor="#F59E0B"
211.             android:textSize="28sp" />
212.     </LinearLayout>
213. </LinearLayout>
214.
215.     <com.google.android.material.button.MaterialButton
216.         android:id="@+id/btnSheetBook"
217.         android:layout_width="0dp"
218.         android:layout_height="56dp"
219.         android:layout_marginTop="32dp"
220.         android:fontFamily="sans-serif-medium"
221.         android:text="Book This Station"
222.         android:textAllCaps="false"
223.         android:textColor="#0F172A"
224.         android:textSize="17sp"
225.         app:backgroundTint="#F59E0B"
226.         app:cornerRadius="28dp"
227.         app:layout_constraintBottom_toBottomOf="parent"
228.         app:layout_constraintEnd_toEndOf="parent"
229.         app:layout_constraintStart_toStartOf="parent"
230.         app:layout_constraintTop_toBottomOf="@id/layoutSheetMetrics" />
231.
232.     </androidx.constraintlayout.widget.ConstraintLayout>
233. </com.google.android.material.card.MaterialCardView>
234.
235. </androidx.constraintlayout.widget.ConstraintLayout>
236.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\activity_view_qr.xml

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"

```

```

3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     android:layout_width="match_parent"
5.     android:layout_height="match_parent"
6.     android:background="#0F172A"
7.     android:padding="24dp">
8.
9.     <!-- Interactive Navigation Text -->
10.    <TextView
11.        android:id="@+id/btnBack"
12.        android:layout_width="wrap_content"
13.        android:layout_height="wrap_content"
14.        android:layout_marginTop="24dp"
15.        android:background="?attr/selectableItemBackground"
16.        android:clickable="true"
17.        android:focusable="true"
18.        android:fontFamily="sans-serif-medium"
19.        android:paddingVertical="8dp"
20.        android:paddingEnd="16dp"
21.        android:text="← Back"
22.        android:textColor="#F59E0B"
23.        android:textSize="17sp"
24.        app:layout_constraintStart_toStartOf="parent"
25.        app:layout_constraintTop_toTopOf="parent" />
26.
27.    <!-- Standardized iOS Large Header -->
28.    <TextView
29.        android:id="@+id/tvTicketTitle"
30.        android:layout_width="0dp"
31.        android:layout_height="wrap_content"
32.        android:layout_marginTop="24dp"
33.        android:fontFamily="sans-serif-medium"
34.        android:text="Verification Ticket"
35.        android:textColor="#FFFFFF"
36.        android:textSize="34sp"
37.        app:layout_constraintEnd_toEndOf="parent"
38.        app:layout_constraintStart_toStartOf="parent"
39.        app:layout_constraintTop_toBottomOf="@id/btnBack" />
40.
41.    <TextView
42.        android:id="@+id/tvTicketSubtitle"
43.        android:layout_width="0dp"
44.        android:layout_height="wrap_content"
45.        android:layout_marginTop="8dp"
46.        android:fontFamily="sans-serif"
47.        android:letterSpacing="0.02"
48.        android:text="Present this to the Grid Operator"
49.        android:textColor="#EBEBF5"
50.        android:textSize="17sp"
51.        app:layout_constraintEnd_toEndOf="parent"
52.        app:layout_constraintStart_toStartOf="parent"
53.        app:layout_constraintTop_toBottomOf="@id/tvTicketTitle" />
54.
55.    <!-- Apple Wallet-style Ticket Card -->
56.    <!-- Retained stark white background (#FFFFFF) as it is required for reliable QR scanning -
57.    <com.google.android.material.card.MaterialCardView
58.        android:id="@+id/cardQrContainer"
59.        android:layout_width="0dp"
60.        android:layout_height="wrap_content"
61.        android:layout_marginTop="48dp"
62.        app:cardBackgroundColor="#FFFFFF"
63.        app:cardCornerRadius="24dp"
64.        app:cardElevation="0dp"
65.        app:layout_constraintEnd_toEndOf="parent"
66.        app:layout_constraintStart_toStartOf="parent"

```

```

67.         app:layout_constraintTop_toBottomOf="@id/tvTicketSubtitle">
68.
69.         <androidx.constraintlayout.widget.ConstraintLayout
70.             android:layout_width="match_parent"
71.             android:layout_height="wrap_content"
72.             android:padding="32dp">
73.
74.             <ImageView
75.                 android:id="@+id/imgQrCode"
76.                 android:layout_width="240dp"
77.                 android:layout_height="240dp"
78.                 app:layout_constraintEnd_toEndOf="parent"
79.                 app:layout_constraintStart_toStartOf="parent"
80.                 app:layout_constraintTop_toTopOf="parent" />
81.
82.             <TextView
83.                 android:id="@+id/txtQrDetails"
84.                 android:layout_width="0dp"
85.                 android:layout_height="wrap_content"
86.                 android:layout_marginTop="24dp"
87.                 android:fontFamily="sans-serif-medium"
88.                 android:gravity="center"
89.                 android:textColor="#0F172A"
90.                 android:textSize="17sp"
91.                 app:layout_constraintBottom_toBottomOf="parent"
92.                 app:layout_constraintEnd_toEndOf="parent"
93.                 app:layout_constraintStart_toStartOf="parent"
94.                 app:layout_constraintTop_toBottomOf="@id/imgQrCode" />
95.
96.         </androidx.constraintlayout.widget.ConstraintLayout>
97.     </com.google.android.material.card.MaterialCardView>
98.
99. </androidx.constraintlayout.widget.ConstraintLayout>
100.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\content_station_map.xml

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     android:layout_width="match_parent"
5.     android:layout_height="match_parent"
6.     app:layout_behavior="@string/appbar_scrolling_view_behavior">
7.
8.     <androidx.fragment.app.FragmentContainerView
9.         android:id="@+id/nav_host_fragment_content_station_map"
10.        android:name="androidx.navigation.fragment.NavHostFragment"
11.        android:layout_width="0dp"
12.        android:layout_height="0dp"
13.        app:defaultNavHost="true"
14.        app:layout_constraintBottom_toBottomOf="parent"
15.        app:layout_constraintEnd_toEndOf="parent"
16.        app:layout_constraintStart_toStartOf="parent"
17.        app:layout_constraintTop_toTopOf="parent"
18.        app:navGraph="@navigation/nav_graph" />
19. </androidx.constraintlayout.widget.ConstraintLayout>
20.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\fragment_first.xml

```

1. <?xml version="1.0" encoding="utf-8"?>

```

```

2. <androidx.core.widget.NestedScrollView
xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     xmlns:tools="http://schemas.android.com/tools"
5.     android:layout_width="match_parent"
6.     android:layout_height="match_parent"
7.     tools:context=".FirstFragment">
8.
9.     <androidx.constraintlayout.widget.ConstraintLayout
10.         android:layout_width="match_parent"
11.         android:layout_height="match_parent"
12.         android:padding="16dp">
13.
14.         <Button
15.             android:id="@+id/button_first"
16.             android:layout_width="wrap_content"
17.             android:layout_height="wrap_content"
18.             android:text="@string/next"
19.             app:layout_constraintBottom_toTopOf="@id/textview_first"
20.             app:layout_constraintEnd_toEndOf="parent"
21.             app:layout_constraintStart_toStartOf="parent"
22.             app:layout_constraintTop_toTopOf="parent" />
23.
24.         <TextView
25.             android:id="@+id/textview_first"
26.             android:layout_width="wrap_content"
27.             android:layout_height="wrap_content"
28.             android:layout_marginTop="16dp"
29.             android:text="@string/lorem_ipsum"
30.             app:layout_constraintBottom_toBottomOf="parent"
31.             app:layout_constraintEnd_toEndOf="parent"
32.             app:layout_constraintStart_toStartOf="parent"
33.             app:layout_constraintTop_toBottomOf="@id/button_first" />
34.     </androidx.constraintlayout.widget.ConstraintLayout>
35. </androidx.core.widget.NestedScrollView>
36.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\fragment_second.xml

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <androidx.core.widget.NestedScrollView
xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     xmlns:tools="http://schemas.android.com/tools"
5.     android:layout_width="match_parent"
6.     android:layout_height="match_parent"
7.     tools:context=".SecondFragment">
8.
9.     <androidx.constraintlayout.widget.ConstraintLayout
10.         android:layout_width="match_parent"
11.         android:layout_height="match_parent"
12.         android:padding="16dp">
13.
14.         <Button
15.             android:id="@+id/button_second"
16.             android:layout_width="wrap_content"
17.             android:layout_height="wrap_content"
18.             android:text="@string/previous"
19.             app:layout_constraintBottom_toTopOf="@id/textview_second"
20.             app:layout_constraintEnd_toEndOf="parent"
21.             app:layout_constraintStart_toStartOf="parent"
22.             app:layout_constraintTop_toTopOf="parent" />
23.

```

```

24.         <TextView
25.             android:id="@+id/textview_second"
26.             android:layout_width="wrap_content"
27.             android:layout_height="wrap_content"
28.             android:layout_marginTop="16dp"
29.             android:text="@string/lorem_ipsum"
30.             app:layout_constraintBottom_toBottomOf="parent"
31.             app:layout_constraintEnd_toEndOf="parent"
32.             app:layout_constraintStart_toStartOf="parent"
33.             app:layout_constraintTop_toBottomOf="@id/button_second" />
34.     </androidx.constraintlayout.widget.ConstraintLayout>
35. </androidx.core.widget.NestedScrollView>
36.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\item_booking.xml

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <com.google.android.material.card.MaterialCardView
xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     android:layout_width="match_parent"
5.     android:layout_height="wrap_content"
6.     android:layout_marginHorizontal="24dp"
7.     android:layout_marginVertical="8dp"
8.     app:cardBackgroundColor="#1E293B"
9.     app:cardCornerRadius="20dp"
10.    app:cardElevation="0dp"
11.    app:strokeColor="#334155"
12.    app:strokeWidth="1dp">
13.
14.    <androidx.constraintlayout.widget.ConstraintLayout
15.        android:layout_width="match_parent"
16.        android:layout_height="wrap_content"
17.        android:padding="20dp">
18.
19.        <!-- Pill-shaped Status Badge container mapped to the top right -->
20.        <com.google.android.material.card.MaterialCardView
21.            android:id="@+id/cardStatusContainer"
22.            android:layout_width="wrap_content"
23.            android:layout_height="wrap_content"
24.            app:cardBackgroundColor="#0F172A"
25.            app:cardCornerRadius="12dp"
26.            app:cardElevation="0dp"
27.            app:layout_constraintEnd_toEndOf="parent"
28.            app:layout_constraintTop_toTopOf="parent">
29.
30.        <!-- ID strictly preserved for backend recycling -->
31.        <TextView
32.            android:id="@+id/txtItemStatus"
33.            android:layout_width="wrap_content"
34.            android:layout_height="wrap_content"
35.            android:fontFamily="sans-serif-medium"
36.            android:letterSpacing="0.05"
37.            android:paddingHorizontal="12dp"
38.            android:paddingVertical="6dp"
39.            android:text="STATUS"
40.            android:textColor="#10B981"
41.            android:textSize="11sp" />
42.        </com.google.android.material.card.MaterialCardView>
43.
44.        <!-- Dynamic width prevents long station names from overlapping the status badge -->
45.        <TextView
46.            android:id="@+id/txtItemStation"

```

```

47.         android:layout_width="0dp"
48.         android:layout_height="wrap_content"
49.         android:layout_marginEnd="16dp"
50.         android:fontFamily="sans-serif-medium"
51.         android:text="Station Name"
52.         android:textColor="#FFFFFF"
53.         android:textSize="18sp"
54.         app:layout_constraintBottom_toBottomOf="@+id/cardStatusContainer"
55.         app:layout_constraintEnd_toStartOf="@id/cardStatusContainer"
56.         app:layout_constraintStart_toStartOf="parent"
57.         app:layout_constraintTop_toTopOf="@+id/cardStatusContainer" />
58.
59.     <TextView
60.         android:id="@+id/txtItemDateTime"
61.         android:layout_width="0dp"
62.         android:layout_height="wrap_content"
63.         android:layout_marginTop="16dp"
64.         android:fontFamily="sans-serif"
65.         android:text="YYYY-MM-DD | 00:00 - 00:00"
66.         android:textColor="#94A3B8"
67.         android:textSize="14sp"
68.         app:layout_constraintEnd_toEndOf="parent"
69.         app:layout_constraintStart_toStartOf="parent"
70.         app:layout_constraintTop_toBottomOf="@id/txtItemStation" />
71.
72.     <TextView
73.         android:id="@+id/txtItemEnergy"
74.         android:layout_width="0dp"
75.         android:layout_height="wrap_content"
76.         android:layout_marginTop="6dp"
77.         android:fontFamily="sans-serif-medium"
78.         android:text="Reserved: 0.0 kWh"
79.         android:textColor="#F59E0B"
80.         android:textSize="15sp"
81.         app:layout_constraintEnd_toEndOf="parent"
82.         app:layout_constraintStart_toStartOf="parent"
83.         app:layout_constraintTop_toBottomOf="@id/txtItemDateTime" />
84.
85. </androidx.constraintlayout.widget.ConstraintLayout>
86. </com.google.android.material.card.MaterialCardView>
87.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\layout\item_station_card.xml

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <com.google.android.material.card.MaterialCardView
xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     android:layout_width="220dp"
5.     android:layout_height="wrap_content"
6.     android:layout_marginEnd="16dp"
7.     android:clickable="true"
8.     android:focusable="true"
9.     android:foreground="?attr/selectableItemBackground"
10.    app:cardBackgroundColor="#1E293B"
11.    app:cardCornerRadius="20dp"
12.    app:cardElevation="0dp"
13.    app:strokeColor="#334155"
14.    app:strokeWidth="1dp">
15.
16.    <androidx.constraintlayout.widget.ConstraintLayout
17.        android:layout_width="match_parent"
18.        android:layout_height="wrap_content"

```

```

19.         android:padding="16dp">
20.
21.         <TextView
22.             android:id="@+id/txtCardStationName"
23.             android:layout_width="0dp"
24.             android:layout_height="wrap_content"
25.             android:ellipsize="end"
26.             android:fontFamily="sans-serif-medium"
27.             android:maxLines="1"
28.             android:text="Station Name"
29.             android:textColor="#FFFFFF"
30.             android:textSize="16sp"
31.             app:layout_constraintEnd_toEndOf="parent"
32.             app:layout_constraintStart_toStartOf="parent"
33.             app:layout_constraintTop_toTopOf="parent" />
34.
35.         <!-- Pill-shaped badge for slot count -->
36.         <com.google.android.material.card.MaterialCardView
37.             android:id="@+id/cardSlotsBadge"
38.             android:layout_width="wrap_content"
39.             android:layout_height="wrap_content"
40.             android:layout_marginTop="12dp"
41.             app:cardBackgroundColor="#2010B981"
42.             app:cardCornerRadius="8dp"
43.             app:cardElevation="0dp"
44.             app:layout_constraintBottom_toBottomOf="parent"
45.             app:layout_constraintStart_toStartOf="parent"
46.             app:layout_constraintTop_toBottomOf="@id/txtCardStationName"
47.             app:strokeWidth="0dp">
48.
49.             <TextView
50.                 android:id="@+id/txtCardSlots"
51.                 android:layout_width="wrap_content"
52.                 android:layout_height="wrap_content"
53.                 android:fontFamily="sans-serif-medium"
54.                 android:paddingHorizontal="8dp"
55.                 android:paddingVertical="4dp"
56.                 android:text="32 slots"
57.                 android:textColor="#10B981"
58.                 android:textSize="12sp" />
59.         </com.google.android.material.card.MaterialCardView>
60.
61.         <!-- Right-aligned energy metric -->
62.         <TextView
63.             android:id="@+id/txtCardEnergy"
64.             android:layout_width="0dp"
65.             android:layout_height="wrap_content"
66.             android:layout_marginStart="8dp"
67.             android:fontFamily="sans-serif-medium"
68.             android:gravity="end"
69.             android:maxLines="1"
70.             android:text="1500 kWh"
71.             android:textColor="#F59E0B"
72.             android:textSize="14sp"
73.             app:layout_constraintBottom_toBottomOf="@id/cardSlotsBadge"
74.             app:layout_constraintEnd_toEndOf="parent"
75.             app:layout_constraintStart_toEndOf="@id/cardSlotsBadge"
76.             app:layout_constraintTop_toTopOf="@id/cardSlotsBadge" />
77.
78.         </androidx.constraintlayout.widget.ConstraintLayout>
79.     </com.google.android.material.card.MaterialCardView>
80.
81.

```


D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\menu\bottom_nav_menu.xml

```
1. <?xml version="1.0" encoding="utf-8"?>
2. <menu xmlns:android="http://schemas.android.com/apk/res/android">
3.
4.     <!-- ID must be nav_dashboard to match DashboardActivity.java -->
5.     <item
6.         android:id="@+id/nav_dashboard"
7.         android:icon="@drawable/ic_map"
8.         android:title="Dashboard" />
9.
10.    <!-- ID must be nav_booking to match MyBookingsActivity.java & CreateBookingActivity.java -->
11.    <item
12.        android:id="@+id/nav_booking"
13.        android:icon="@drawable/ic_book"
14.        android:title="Bookings" />
15.
16.    <item
17.        android:id="@+id/nav_profile"
18.        android:icon="@drawable/ic_profile"
19.        android:title="Profile" />
20. </menu>
21.
```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\mipmap-anydpi-v26\ic_launcher_round.xml

```
1. <?xml version="1.0" encoding="utf-8"?>
2. <adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
3.     <background android:drawable="@drawable/ic_launcher_background" />
4.     <foreground android:drawable="@drawable/ic_launcher_foreground" />
5.     <monochrome android:drawable="@drawable/ic_launcher_foreground" />
6. </adaptive-icon>
7.
8.
```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\mipmap-anydpi-v26\ic_launcher.xml

```
1. <?xml version="1.0" encoding="utf-8"?>
2. <adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
3.     <background android:drawable="@drawable/ic_launcher_background" />
4.     <foreground android:drawable="@drawable/ic_launcher_foreground" />
5.     <monochrome android:drawable="@drawable/ic_launcher_foreground" />
6. </adaptive-icon>
7.
8.
```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\navigation\nav_graph.xml

```
1. <?xml version="1.0" encoding="utf-8"?>
2. <navigation xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:app="http://schemas.android.com/apk/res-auto"
4.     xmlns:tools="http://schemas.android.com/tools"
5.     android:id="@+id/nav_graph"
6.     app:startDestination="@id/FirstFragment">
7.
8.     <fragment
9.         android:id="@+id/FirstFragment"
10.        android:name="com.team.smartsolar.FirstFragment"
11.        android:label="@string/first_fragment_label"
12.        tools:layout="@layout/fragment_first">
```

```

13.
14.         <action
15.             android:id="@+id/action_FirstFragment_to_SecondFragment"
16.             app:destination="@id/SecondFragment" />
17.     </fragment>
18.     <fragment
19.         android:id="@+id/SecondFragment"
20.         android:name="com.team.smartsolar.SecondFragment"
21.         android:label="@string/second_fragment_label"
22.         tools:layout="@layout/fragment_second">
23.
24.         <action
25.             android:id="@+id/action_SecondFragment_to_FirstFragment"
26.             app:destination="@id/FirstFragment" />
27.     </fragment>
28. </navigation>
29.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\values\colors.xml

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <resources>
3.     <color name="black">#FF000000</color>
4.     <color name="white">#FFFFFFF</color>
5. </resources>
6.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\values\themes.xml

```

1. <resources xmlns:tools="http://schemas.android.com/tools">
2.     <!-- Base application theme. -->
3.     <style name="Base.Theme.SmartSolar" parent="Theme.Material3.DayNight.NoActionBar">
4.         <!-- Customize your light theme here. -->
5.         <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
6.     </style>
7.
8.     <style name="Theme.SmartSolar" parent="Base.Theme.SmartSolar" />
9. </resources>
10.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\res\values-v23\themes.xml

```

1. <resources xmlns:tools="http://schemas.android.com/tools">
2.
3.     <style name="Theme.SmartSolar" parent="Base.Theme.SmartSolar">
4.         <!-- Transparent system bars for edge-to-edge. -->
5.         <item name="android:navigationBarColor">@android:color/transparent</item>
6.         <item name="android:statusBarColor">@android:color/transparent</item>
7.         <item name="android:windowLightStatusBar">?attr/isLightTheme</item>
8.     </style>
9. </resources>
10.

```

D:\Y4S2\EAD\Smart-Solar-Microgrid-Trading-System\android\app\src\main\AndroidManifest.xml

```

1. <?xml version="1.0" encoding="utf-8"?>
2. <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3.     xmlns:tools="http://schemas.android.com/tools">

```

```
4.
5.     <uses-permission android:name="android.permission.INTERNET" />
6.
7.     <application
8.         android:allowBackup="true"
9.         android:dataExtractionRules="@xml/data_extraction_rules"
10.        android:fullBackupContent="@xml/backup_rules"
11.        android:icon="@mipmap/ic_launcher"
12.        android:label="@string/app_name"
13.        android:roundIcon="@mipmap/ic_launcher_round"
14.        android:supportsRtl="true"
15.        android:theme="@style/Theme.SmartSolar"
16.        android:usesCleartextTraffic="true">
17.         <meta-data
18.             android:name="com.google.android.geo.API_KEY"
19.             android:value="AIzaSyB5Qa2DfyDoqGyY1fghJD1mZyNNNOrJl9E" />
20.
21.         <activity
22.             android:name=".MainActivity"
23.             android:exported="true"
24.             android:windowSoftInputMode="adjustResize">
25.             <intent-filter>
26.                 <action android:name="android.intent.action.MAIN" />
27.
28.                 <category android:name="android.intent.category.LAUNCHER" />
29.             </intent-filter>
30.         </activity>
31.         <activity
32.             android:name=".RegisterActivity"
33.             android:exported="false"
34.             android:windowSoftInputMode="adjustResize" />
35.         <activity
36.             android:name=".LoginActivity"
37.             android:exported="false"
38.             android:windowSoftInputMode="adjustResize" />
39.         <activity
40.             android:name=".DashboardActivity"
41.             android:exported="false"
42.             android:windowSoftInputMode="adjustResize" />
43.         <activity
44.             android:name=".ProfileActivity"
45.             android:exported="false"
46.             android:windowSoftInputMode="adjustResize" />
47.         <activity
48.             android:name=".CreateBookingActivity"
49.             android:exported="false"
50.             android:windowSoftInputMode="adjustResize" />
51.         <activity
52.             android:name=".BookingSummaryActivity"
53.             android:exported="false"
54.             android:windowSoftInputMode="adjustResize" />
55.         <activity
56.             android:name=".MyBookingsActivity"
57.             android:exported="false"
58.             android:windowSoftInputMode="adjustResize" />
59.         <activity
60.             android:name=".ViewQrActivity"
61.             android:exported="false"
62.             android:windowSoftInputMode="adjustResize" />
63.         <activity
64.             android:name=".ModifyBookingActivity"
65.             android:exported="false"
66.             android:windowSoftInputMode="adjustResize" />
67.         <activity
68.             android:name=".StationMapActivity"
```

```
69.         android:exported="false"
70.         android:theme="@style/Theme.SmartSolar"
71.         android:windowSoftInputMode="adjustResize" />
72.     </application>
73.
74. </manifest>
75.
```

4 All the references

- [1] Microsoft, “ASP.NET Core Documentation,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/>. [Accessed: Oct. 5, 2026].
- [2] MongoDB Inc., “MongoDB C#.NET Driver,” MongoDB Documentation. [Online]. Available: <https://mongodb.github.io/mongo-csharp-driver/>. [Accessed: Oct. 5, 2026].
- [3] Microsoft, “Host ASP.NET Core on Windows with IIS,” Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/iis/>. [Accessed: Oct. 5, 2026].
- [4] Meta Platforms, “React Documentation.” [Online]. Available: <https://react.dev/>. [Accessed: Oct. 5, 2026].
- [5] Vite, “Vite: Next Generation Frontend Tooling.” [Online]. Available: <https://vitejs.dev/>. [Accessed: Oct. 5, 2026].
- [6] Tailwind Labs, “Tailwind CSS Documentation.” [Online]. Available: <https://tailwindcss.com/>. [Accessed: Oct. 5, 2026].
- [7] Recharts, “Recharts: A Composable Charting Library Built on React Components.” [Online]. Available: <https://recharts.org/>. [Accessed: Oct. 5, 2026].
- [8] Axios, “Axios: Promise Based HTTP Client for the Browser and Node.js.” [Online]. Available: <https://axios-http.com/>. [Accessed: Oct. 5, 2026].
- [9] Google, “Android Developers Documentation.” [Online]. Available: <https://developer.android.com/>. [Accessed: Oct. 5, 2026].
- [10] Google, “Maps SDK for Android,” Google for Developers. [Online]. Available: <https://developers.google.com/maps/documentation/android-sdk/overview>. [Accessed: Oct. 5, 2026].
- [11] JourneyApps, “ZXing Android Embedded,” GitHub Repository. [Online]. Available: <https://github.com/journeyapps/zxing-android-embedded>. [Accessed: Oct. 5, 2026].
- [12] Square, Inc., “Retrofit: A Type-Safe HTTP Client for Android and Java.” [Online]. Available: <https://square.github.io/retrofit/>. [Accessed: Oct. 5, 2026].

5 Git Repository link
[Smart-Solar-Microgrid-Trading-System](#)

6 Individual Contributions

Member	Contribution
Nithika (@NIKKA vRULZ)	Mobile Application — Prosumer Core (Android UI/UX layouts, SQLite persistence, booking workflows, dashboard metrics, QR generation)
Sasmitha Kavindu (@sasmithaK)	Central Web Service + Mobile Hardware (C# API, MongoDB, IIS hosting, API business rules, Android Maps integration, Operator QR scanning)
Hiruni Chamathka (@chamahiru)	Web Application & QA (React Backoffice UI, user/prosumer/node management, API integration, API edge-case testing)
Desima Weerasinghe (@DesimaW)	Web Application & Docs (React Grid Operator UI, slot/booking UI, leading Project Report and Demo Video production)

7 Challenges

Challenge	Resolution	Outcome and Learning
Strictly Enforcing the FAT Service Architecture Ensuring that complex business rules, such as the 7-day future booking limit and 12-hour cancellation notice, did not leak into the React or Android frontends, which would violate the FAT Service requirement.	All business logic was centralized inside dedicated service classes, such as ReservationService.cs, within the .NET 8 Web API. The frontend clients were restricted to capturing user input and displaying HTTP error responses returned by the API.	Learned how to properly separate UI logic from business rules, resulting in a more secure and maintainable system where clients cannot bypass server-side validation.
Securely Managing Database Credentials for IIS Hosting The MongoDB connection string had to be integrated for IIS deployment without hardcoding sensitive credentials in appsettings.json or exposing them through version control.	.NET User Secrets were used for local development, as documented in SETUP.md, while environment variables were used on the production IIS server. The connection string was therefore injected securely at runtime.	Gained practical experience in application security and secret management, particularly when deploying .NET applications to a Windows/IIS server.
Integrating Google Maps with Local SQLite Caching The Android StationMapActivity needed to dynamically display microgrid nodes retrieved from the API. Slow or unavailable network connections could otherwise cause delays, UI freezing, or	A DatabaseHelper class was implemented to cache API node data in a local SQLite database. The Map Activity first loads cached markers, allowing the application to remain responsive even when network connectivity is poor or unavailable.	Learned the importance of local caching and offline-support strategies in mobile application development. Successfully integrated Google Maps, REST APIs, and SQLite caching.

failures.		
Generating Secure and Scannable QR Codes The Android ViewQrActivity needed to generate a reliable QR code containing sufficient information to verify a booking without making the QR code too dense and difficult to scan.	Instead of embedding the complete JSON booking object, a secure 12-character SHA-256 hash payload was generated on the backend during the ApproveAsync process. The ZXing-android-embedded library was then used to encode this short payload into the QR code.	Learned how to optimize QR-code payloads for reliable physical scanning and gained experience integrating camera/barcode-related functionality into a native Android application.